

Primeiros passos
com a linguagem

RUST



novatec

José Augusto N. G. Manzano

Primeiros passos com a linguagem **RUST**

José Augusto N. G. Manzano

Novatec

© Novatec Editora Ltda. 2018.

Todos os direitos reservados e protegidos pela Lei 9.610 de 19/02/1998. É proibida a reprodução desta obra, mesmo parcial, por qualquer processo, sem prévia autorização, por escrito, do autor e da Editora.

Editor: Rubens Prates

Revisão gramatical: Marta Almeida de Sá

Editoração eletrônica: Carolina Kuwabata

Capa: Carolina Kuwabata

ISBN: 978-85-7522-684-1

Histórico de edições impressas:

Junho/2018 Primeira edição

Novatec Editora Ltda.

Rua Luís Antônio dos Santos 110

02460-000 – São Paulo, SP – Brasil

Tel.: +55 11 2959-6529

Email: novatec@novatec.com.br

Site: www.novatec.com.br

Twitter: twitter.com/novateceditora

Facebook: facebook.com/novatec

LinkedIn: linkedin.com/in/novatec

Sumário

[Agradecimentos](#)

[Prefácio](#)

[Sobre o autor](#)

[Capítulo 1 ■ Linguagem Rust](#)

- [1.1 Breve histórico da linguagem Rust](#)
- [1.2 Instalação e teste da linguagem Rust](#)
- [1.3 Gerenciador cargo](#)
- [1.4 Notação utilizada](#)
- [1.5 Gerenciamento de memória](#)

[Capítulo 2 ■ Ação sequencial](#)

- [2.1 Tipos de dados](#)
 - [2.1.1 Inteiros](#)
 - [2.1.2 Reais](#)
 - [2.1.3 Cadeias e caracteres](#)
 - [2.1.4 Lógicos](#)
- [2.2 Variáveis](#)
- [2.3 Constantes](#)
- [2.4 Operadores aritméticos](#)
- [2.5 Expressões aritméticas](#)
- [2.6 Instruções da linguagem Rust](#)
- [2.7 Programas em Rust](#)
 - [2.7.1 Fluxos de entrada e saída](#)
 - [2.7.2 Utilização de valores numéricos](#)
 - [2.7.3 Outras operações numéricas](#)
 - [2.7.4 Definição de constantes](#)
 - [2.7.5 Valores lógicos](#)
 - [2.7.6 Operações matemáticas](#)
- [2.8 Linhas de comentários](#)
- [2.9 Mensagens de advertência](#)
- [2.10 Exercícios de fixação](#)

Capítulo 3 ■ Desvio condicional

- [3.1 Condição e decisão](#)
- [3.2 Desvios condicionais](#)
 - [3.2.1 Desvio condicional simples](#)
 - [3.2.2 Desvio condicional composto](#)
- [3.3 Operadores lógicos](#)
 - [3.3.1 Operador lógico de conjunção](#)
 - [3.3.2 Operador lógico de disjunção](#)
 - [3.3.3 Operador lógico de negação](#)
- [3.4 Outros desvios condicionais](#)
 - [3.4.1 Desvio condicional encadeado](#)
 - [3.4.2 Desvio condicional sequencial](#)
 - [3.4.3 Operador condicional](#)
- [3.5 Controle com múltipla escolha](#)
- [3.6 Divisibilidade](#)
- [3.7 Validação do fluxo de entrada](#)
- [3.8 Exercícios de fixação](#)

Capítulo 4 ■ Laços

- [4.1 Laço pré-teste](#)
- [4.2 Laço seletivo](#)
- [4.3 Laço iterativo](#)
- [4.4 Interrupção na execução de laços](#)
- [4.5 Jogo de adivinhação](#)
- [4.6 Exercícios de fixação](#)

Capítulo 5 ■ Sub-rotinas

- [5.1 Dividir para conquistar](#)
- [5.2 Paradigmas top-down e bottom-up](#)
- [5.3 Sub-rotinas em Rust](#)
- [5.4 Escopo e visibilidade de variáveis](#)
- [5.5 Sub-rotina: procedimento](#)
- [5.6 Passagem de parâmetro](#)
 - [5.6.1 Passagem de parâmetro por valor](#)
 - [5.6.2 Passagem de parâmetro por referência](#)
- [5.7 Sub-rotina: função](#)
 - [5.7.1 Função básica](#)
 - [5.7.2 Função recursiva](#)
 - [5.7.3 Função apontada](#)
- [5.8 Sub-rotina como parâmetro](#)

[5.9 Sub-rotina fechada](#)

[5.10 Exercícios de fixação](#)

Capítulo 6 ■ Complementos

[6.1 Conversão de tipos](#)

[6.2 Ponteiros](#)

[6.2.1 Ponteiro de referência](#)

[6.2.2 Ponteiro exclusivo](#)

[6.2.3 Ponteiro bruto](#)

[6.3 Terminal ANSI](#)

[6.4 Biblioteca definida pelo programador](#)

[6.5 Metaprogramação com macros](#)

[6.6 Laço iterativo compassado](#)

[6.7 Caracteres ASCII, Unicode e Byte](#)

Capítulo 7 ■ Estruturas homogêneas

[7.1 Matriz estática](#)

[7.1.1 Matriz unidimensional](#)

[7.1.2 Matriz bidimensional](#)

[7.1.3 Matriz interna](#)

[7.2 Matriz dinâmica](#)

[7.2.1 Matriz unidimensional](#)

[7.2.2 Matriz bidimensional](#)

[7.3 Fatiamento de matriz](#)

[7.4 Ordenação e pesquisa](#)

[7.4.1 Ordenação de elementos matriciais](#)

[7.4.2 Pesquisa de elementos matriciais](#)

[7.5 Iteradores](#)

[7.6 Exercícios de fixação](#)

Capítulo 8 ■ Estruturas heterogêneas

[8.1 Tuplas de dados](#)

[8.2 Estruturas](#)

[8.2.1 Estrutura clássica](#)

[8.2.2 Estrutura tupla](#)

[8.2.3 Estrutura-unidade](#)

[8.3 Enumerações](#)

[8.4 Agregações](#)

[8.5 Definições de traços](#)

[8.6 Exercícios de fixação](#)

Capítulo 9 ■ Suplementos

[9.1 Programação genérica](#)

[9.1.1 Funções](#)

[9.1.2 Estruturas](#)

[9.2 Programação funcional](#)

[9.3 Tratamento de erros e exceções](#)

[9.4 Data e hora](#)

[9.5 Coleções](#)

[9.6 Cadeias e caracteres](#)

[9.7 Criptografia](#)

[9.8 Armazenamento básico de dados](#)

Referências bibliográficas

Agradecimentos

À minha esposa Sandra e à minha filha Audrey, motivos de constante inspiração para o meu trabalho. Pela paciência nos momentos de ausência, quando estou absorto em escrever, dedicando-me ao ensino.

Ao amigo – professor do Instituto Federal do Estado do Piauí e colega de lida – Charleno Pires, por ter apresentado e me instigado a estudar a linguagem Rust, o que culminou na elaboração desta humilde obra.

A todos os alunos que passaram e passam pela minha vida, que acreditaram e acreditam em mim e seguiram e seguem as orientações que passo; por me incentivarem continuamente quando me questionam sobre temas que ainda não conheço, por me levarem a um patamar mais elevado, por exigirem, assim, que eu pesquise mais.

A você, amigo leitor, que já me conhece de outros trabalhos ou não, por investir de forma legal na obtenção deste livro. Obrigado por não fazer fotocópias ou uso de pirataria, e por se utilizar desta minha contribuição a você.

Vida longa e próspera.

Prefácio

Esta obra apresenta as bases da programação de computadores com uso da linguagem Rust para estudantes de programação de computadores e demais profissionais das áreas da computação que tenham interesse em iniciar um estudo preliminar e introdutório desta linguagem. É importante ter como pré-requisito sólidos conhecimentos de lógica de programação.

Este livro pode ser utilizado em sala de aula (laboratório com prática de programação) de cursos superiores na área da computação (tecnologia em análise e desenvolvimento de sistemas, ciência da computação, engenharia da computação e física computacional), cursos técnicos (ensino médio integrado ou concomitante) e em atividades de treinamento empresarial.

Objetiva-se com esta obra fornecer informações básicas de uso da linguagem Rust. Não é pretensão apresentar uma obra que seja completa e que represente a totalidade da linguagem, o que seria, além de pretensioso, impossível de ser alcançado nestas páginas. A linguagem Rust é extensa e permite resolver determinados problemas de diversas formas e por meio de alguns paradigmas de programação.

O objetivo foi escrever um livro voltado ao público brasileiro com base na documentação oficial disponibilizada da linguagem, o que não foi tarefa fácil. Pouco foi usado dos fóruns e das discussões aos quais temos acesso, e quando isso ocorreu foi indicado como referência no texto. No Brasil há um grupo de profissionais e entusiastas estudando e discutindo Rust a partir do endereço github.com/rust-br. Parte do que está neste texto foi encontrado em locais que não abordavam a linguagem em si. Há detalhes apresentados que foram descobertos depois de horas de tentativas e erros e maçantes pesquisas na grande rede. Após certo tempo de trabalho com a linguagem, foi possível chegar a esta simples e humilde obra. Sei que este

texto não é completo e não responde a certas questões, mas está em uma formatação que já permite apresentar a linguagem e iniciar a formação do conhecimento no país.

Ao final deste estudo, o leitor estará capacitado a entender e aplicar de forma básica as técnicas de programação abordadas, respeitando-se os limites deste texto. Nesta apresentação, o leitor terá contato com mais de 150 exemplos de programação que demonstram vários recursos da linguagem.

Pelo fato de seu conteúdo ser didático, a obra pode ser usada de maneira gradativa, a fim de acomodar e atender às necessidades do professor/instrutor, distribuindo o conteúdo de aula dentro do período letivo ou da carga horária necessária. Este livro apresenta um total de nove capítulos que cobrem diversos aspectos operacionais da linguagem.

Neste contexto é abordado o histórico preliminar do surgimento da linguagem de programação Rust. São relatados os tipos de dados primitivos; variáveis mutáveis e imutáveis; constantes; operadores aritméticos; expressões aritméticas; operações numéricas; uso de valores lógicos; instruções de entrada e de saída; noções sobre a estrutura de programas escritos em Rust; uso de ações de precisão numérica; ações de tomada de decisão; uso de condições; estruturas com decisões simples, compostas, encadeadas, sequenciais e seletivas; indicação sobre uso de operadores relacionais e lógicos; divisibilidade; técnicas de execução de laços para repetição de trechos de código de programa; interrupção na execução de laços; técnicas de divisão e conquista; escopo e visibilidade de variáveis; procedimentos; funções básicas, recursivas, apontadas e fechadas; passagens de parâmetros por valor e por referência; conversão de tipo; ponteiros; biblioteca definida pelo programador; metaprogramação com macros; matrizes estáticas e dinâmicas; ordenação e pesquisa de elementos; iteradores, tuplas, enumerações; estruturas; agregações; traços; programação genérica; programação funcional; tratamento de erros e exceções; manipulação de datas e horas; coleções; cadeias e caracteres; criptografia (com cifras e hashes); manipulação básica

e preliminar de arquivos.

Em alguns capítulos, uma lista de exercícios é fornecida com o objetivo de auxiliar a fixação dos detalhes apresentados no capítulo a respeito da linguagem. Os capítulos que não contêm exercícios de fixação apresentam detalhes complementares ou suplementares. O objetivo da aplicação dos exercícios não é propor dificuldade lógica, mas sim a prática na escrita de principais instruções a partir dos comandos da linguagem Rust.

A todos um grande abraço e um bom começo de aprendizado com a linguagem Rust. Espero que o investimento ora realizado na aquisição legal deste livro lhe seja útil e valha cada centavo usado em sua compra.

Augusto Manzano

Sobre o autor

José Augusto Navarro Garcia Manzano é professor, mestre em Análise e Desenvolvimento de Sistemas, Ciências Econômicas, e licenciado em Matemática. Atua na área de Tecnologia da Informação, Computação e Informática (desenvolvimento de software, ensino e treinamento) desde 1986. Participou do desenvolvimento de aplicações computacionais para áreas de telecomunicações e comércio. Iniciou sua carreira docente em cursos livres; trabalhou posteriormente em empresas de treinamento e leciona desde então nos ensinos técnico e superior.

Atualmente é professor, com dedicação exclusiva no IFSP (Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, antiga Escola Técnica Federal). Desenvolveu competências e habilidades para ministrar componentes curriculares de Lógica de Programação (Algoritmos), Estrutura de Dados, Microinformática, Informática, Linguagens de Programação Estruturada, Linguagens de Programação Orientada a Objetos, Engenharia de Software, Sistemas de Informação, Engenharia da Informação, Arquitetura de Computadores e Tecnologias Web. Conhece o uso de diversas linguagens de programação, tais como: Classic BASIC, COMAL, Logo, Assembly, Pascal, FORTRAN, C, C++, D, Java, Modula-2, Structured BASIC, C#, Lua, HTML, XHTML, Javascript, Python, VBA e Ada. Tem mais de uma centena de obras publicadas, além de artigos publicados no Brasil e no exterior.

CAPÍTULO 1

Linguagem Rust

O estudo deste capítulo é opcional. Pode-se partir diretamente para a leitura do próximo capítulo. No entanto este capítulo se destina a proporcionar visão geral sobre o surgimento e a evolução da linguagem de programação Rust. Orienta sobre o uso da formatação de escrita dos códigos de programa ao longo deste trabalho.

1.1 Breve histórico da linguagem Rust

Rust (*ferrugem* ou *oxidação* em inglês) é uma linguagem de programação desenvolvida sob a óptica de código aberto; sendo voltada às ações de concorrência (sem disputa de dados), velocidade, abstração (sem overhead) e segurança de memória (sem coletor de lixo), é uma linguagem que atende a alguns paradigmas de programação, como: programação genérica, programação funcional (declarativa), programação imperativa (estruturada e orientada a objetos) e programação concorrente.

Os programas escritos em Rust têm desempenho similar ao de programas escritos em linguagens de baixo nível e operam com abstrações encontradas em linguagens de alto nível (BALBAERT, 2015, p. 3). Rust é uma linguagem de sistemas, o que significa que é voltada a aplicações “pesadas” para programadores que seguem uma linha de produção *hardcore*, como o desenvolvimento de sistemas operacionais, mecanismos de simulação para aplicações de realidade virtual, engine de jogos, entre outras (MOZ://A, 2017).

Na linguagem de programação Rust, os paradigmas de programação suportados se misturam e interligam-se de forma natural. Graças a sua engenharia de desenvolvimento, a linguagem permite escrever programas

ambiciosos em alto nível, com desempenho similar, encontrados em programas escritos em assembly de forma rápida.

Quando o programador comete um deslize, o compilador informa o que está incorreto ou necessita de acertos para que o programa seja escrito de forma adequada. A maior parte das verificações de segurança de acesso e gerenciamento de memória é realizada em tempo de compilação, garantindo alto grau de desempenho em tempo de execução de um programa (RUST, 2017). Dessa forma, situações permitidas em outras linguagens em Rust são verificadas na compilação, e, se houver algo incorreto, o compilador indica o que deve ser corrigido para que o código esteja dentro dos parâmetros operacionais necessários. As poucas ações não verificadas na compilação resultam em algum tipo de erro de execução que, quando interrompem o programa, pode ser corrigido mudando-se a postura de programação. Os erros em Rust são chamados de *pânico*.

Rust é uma linguagem de programação que começou a ser desenvolvida em 2006 pelo canadense Graydon Hoane, um funcionário da Mozilla Research, que a produziu com influência da linguagem OCaml, utilizada em aplicações industriais com suporte à programação funcional, programação imperativa e programação orientada a objetos. Em 2009, a Mozilla interessou-se pela linguagem e passou a ser o seu primeiro investidor. Com a participação da Mozilla, a linguagem foi portada para o compilador LLVM, que se trata de uma coleção de tecnologias de compiladores com ferramentas modulares e reutilizáveis, e lançou Rust oficialmente em 2010 (DAN_ATILIO, 2015, GOMES, 2010 & MAFRA, 2014). Desde então, a linguagem vem sendo desenvolvida com a participação de um extenso time de desenvolvedores, além, é claro, de Graydon Hoane (<https://www.rust-lang.org/pt-BR/team.html>).

A linguagem Rust tem uma sintaxe semelhante à sintaxe encontrada nas linguagens de programação C e C++, com um certo tempero de estilo da linguagem Ada. Esta linguagem é extremamente versátil e permite escrever programas das mais diversas finalidades, como drivers e sistemas

operacionais, além de outras possibilidades já apontadas.

Embora seja uma linguagem recente, completando oito anos de vida em 2018, já ganhou grandes adeptos, destacando-se a própria empresa Mozilla no desenvolvimento do programa *Servo* e de partes importantes do programa de navegação Firefox. Rust vem sendo utilizada também em projetos como OpenDNS, Dropbox, Skylight, Autumn, Tessel, Chef Habitat, Neon, Xi editor, como destaca Tavares (2017), além do sistema operacional Redox OS, totalmente escrito em Rust (<https://www.redox-os.org/>).

A linguagem Rust não tem, ainda, uma documentação de padrões ao estilo ISO/IEC, como ocorre com outras linguagens já consagradas, como C, C++, Pascal, entre outras. Neste sentido, é mantido um conjunto de documentos a partir do endereço <https://doc.rust-lang.org/stable/reference/introduction.html>, os quais devem ser considerados por desenvolvedores de software.

Com Rust é possível desenvolver programas menos propensos a bugs e vulnerabilidades de segurança no acesso à memória. A linguagem inclui recursos poderosos, como abstrações de custo zero, gerenciamento seguro de memória, concorrência e muito mais (MOZ://A, 2017). Pode ser usada de forma imperativa ou declarativa, a critério do programador.

Uma curiosidade sobre a linguagem Rust é sua mascote, além do logotipo oficial (catraca de bicicleta com a letra R). Depois de o sistema operacional Linux usar um pinguim e um antigo gerenciador de banco de dados chamado FoxPro usar uma raposa como mascote, vários projetos da área da computação começaram a considerar o uso de animais para seus produtos como mascotes. No caso da linguagem Rust, sua mascote, não oficial, é o *Ferris*, um simpático caranguejo que pode ser conferido no site Rustocean no endereço: <http://www.rustacean.net/>.

1.2 Instalação e teste da linguagem Rust

Para desenvolver programas de computador em determinada linguagem de programação, é necessário instalar tal linguagem no computador em

uso. No caso da linguagem Rust, há algumas formas, segundo sua documentação, de efetuar sua instalação, sendo a mais comum feita por meio da ferramenta de instalação `rustup`, que gerencia de forma consistente as diversas versões lançadas da linguagem dentro do seu ciclo de lançamento para diversas plataformas computacionais. Para mais informações sobre o gerenciador `rustup`, consulte o endereço <https://github.com/rust-lang-nursery/rustup.rs/blob/master/README.md>.

Todas as ferramentas de operação da linguagem Rust são instaladas no diretório `%USERPROFILE%\cargo\bin`, onde ficam os programas `rustup`, `cargo` e `rustc`. Por isso é comum que esse diretório seja manualmente configurado junto à variável `PATH` caso o processo de instalação não surta este efeito automaticamente, o que pode ocorrer por causa da grande variedade de configurações e plataformas em que a linguagem pode ser usada. Para saber se o processo de instalação foi bem-sucedido, execute, no prompt de comando do terminal, modo texto do sistema operacional, a instrução `rustc --version`. Se esta apresentar falha na chamada do programa, talvez seja necessário configurar manualmente a variável `PATH`.

No caso da instalação do programa `rustup` para o sistema operacional Windows, é necessário ter previamente instaladas as ferramentas de compilação Microsoft C++ do Visual Studio edição 2013 ou posterior. Caso não tenha esse recurso instalado, é necessário fazê-lo primeiro. Para isso, acesse, por meio do programa de navegação, o endereço <http://landinghub.visualstudio.com/visual-cpp-build-tools>, acione o botão **Download Visual C++ Build Tools 2015**, copie o arquivo e execute-o por meio de um duplo clique.

Para instalar o programa `rustup` nos sistemas operacionais Linux ou Mac OS é necessário estar com uma janela de terminal aberta e informar a instrução `curl https://sh.rustup.rs -sSf | sh` para fazer o download do programa. Provavelmente será necessário informar sua senha de usuário. A instalação estará concluída quando for apresentada no terminal a mensagem "Rust is installed now. Great!" (DHINAKARAN, 2017, p. 9; KAIHLAVIRTA, V., 2017, p. 8). Para mais detalhes sobre a instalação Linux

ou Mac, acesse o endereço <https://doc.rust-lang.org/book/second-edition/ch01-01-installation.html>.

Depois de instalar as ferramentas de compilação C++ da Microsoft, acione a instalação do programa `rustup-init.exe`, que é realizada em uma tela do modo console (modo texto ou modo prompt). Informe a opção **1** (um), acione a tecla **<Enter>** e aguarde o processo de instalação ser encerrado, o que ocorrerá quando for apresentada a mensagem `Press the Enter key to continue`. Acione a tecla **<Enter>**.

Após a instalação, abra um terminal texto no sistema operacional em uso e execute a instrução `rustc --version`, que, se tudo estiver correto, apresentará uma mensagem semelhante à informação `rustc 1.24.0 (4d90ac38c 2018-02-12)` indicando que a linguagem pode ser usada em seu sistema.

O programa `rustup` é o programa de instalação da linguagem Rust, o qual faz o controle de versões a ser utilizadas, acompanhado dos programas `rustc`, que são os compiladores da linguagem, e do programa `cargo`, que permite gerenciar as dependências de pacotes (conjuntos de bibliotecas) a ser usados, possibilitando compilar tudo o que é necessário para um programa ser criado por meio de pacotes externos que são obtidos do repositório oficial chamado `creates.io`.

Como já é tradição em várias linguagens de programação, o primeiro programa a ser escrito em uma nova linguagem é o programa “*Alo, Mundo!*” (sem acentuação). Para isso, é adequado que se crie no sistema em uso um diretório ou uma pasta para a gravação dos projetos dos programas-fonte. Sugere-se então a criação da pasta ou do diretório `estudo`. Para o programa em questão, crie a pasta ou o diretório `alo`.

Após criar a pasta ou o diretório de projeto `alo` dentro da pasta ou diretório `estudo`, faça seu acesso via linha de comando de acordo com as regras de uso do sistema operacional em operação. Abra um editor de texto e escreva o código seguinte:

```
fn main() {  
    println!("Alo, Mundo!");  
}
```

```
}
```

Grave na sequência dentro de `estudo/alo` o código de programa com o nome `main.rs` (não use outro nome), e na sequência execute a instrução na linha de comando:

```
rustc main.rs
```

Em seguida, se não houver nenhum erro de codificação, o programa poderá ser executado. Na linha de comando do sistema operacional Windows, escreva a instrução `main` (ou `.\main`) e acione <Enter>. Nos sistemas operacionais padrão POSIX, como os sistemas Linux e Mac, escreva o comando `./main` e acione <Enter>. Observe a apresentação da mensagem “Alo, Mundo!”.

O código anterior, apesar de simples, mostra alguns elementos importantes na anatomia de um programa escrito em Rust. O trecho de bloco de código delimitado entre os símbolos de chaves após a indicação `fn main()` caracteriza-se por ser a função principal onde um programa é escrito. Dentro do bloco de código da função `main()` há o uso da macro `println!()` posicionada à direita com indentação de quatro espaços para apresentar a mensagem e, em seguida, saltar uma linha em branco. O indicativo `ln` da macro indica *line new* (nova linha). A indentação-padrão da linguagem Rust é operar com quatro espaços. A diferença estrutural entre uma macro e uma função é o uso do símbolo (!).

A mensagem “Alo, Mundo!” é usada como parâmetro (argumento) da macro `println`; esta mensagem é delimitada entre aspas inglesas, indicando ser uma string (uma cadeia de caracteres). Outro detalhe importante nesta instrução é o encerramento desta com um símbolo ponto e vírgula (;). A maior parte das instruções é sinalizada com o uso do símbolo de ponto e vírgula.

A macro `println!()` escreve no monitor de vídeo o conteúdo do parâmetro informado. Há uma versão que faz a escrita do conteúdo sem mover o cursor para a próxima linha. Trata-se da macro `print!()`.

A linguagem Rust tem um ciclo de atualização com média de *seis semanas* entre versões. Isso significa que há várias versões disponíveis da linguagem

para uso. Para manter-se atualizado é interessante, de vez em quando, fazer uso no modo terminal texto do sistema operacional do comando `rustup update stable`.

Os exemplos deste livro foram produzidos em um computador com sistema operacional Windows.

1.3 Gerenciador cargo

O programa `cargo` caracteriza-se por ser um sistema de compilação e de gerenciamento de pacotes da linguagem chamado *Rustaceans* que cuida da construção do código de programas na linguagem, baixando as bibliotecas e criando-as de acordo com sua necessidade e conforme são indicadas para uso. Uma biblioteca em Rust ocorre quando um código de programa requer dependências (DHINAKARAN, 2017, p.64).

O programa “Alo, Mundo!”, anteriormente demonstrado, não contém dependências, por ser um código de escopo simples. Dessa forma, sua compilação apenas usou a parte do programa `cargo` que efetuou a montagem do código. No entanto, programas mais complexos poderão usar dependências de bibliotecas. A inclusão dessas bibliotecas ocorre facilmente com o uso do programa `cargo` que foi instalado juntamente com o programa `rustup`.

Para fazer um teste sobre o uso do programa `cargo` que difere da forma anterior, deve ser criado um novo projeto dentro do diretório/pasta `estudo`. Neste caso será criado o projeto `alo2`. Para isso, dentro de `estudo` execute o comando:

```
cargo new alo2 --bin
```

O parâmetro `--bin` informa que será criado um arquivo binário do tipo aplicativo executável. O nome `alo2` caracteriza o nome do projeto e da pasta/diretório usada para criar os arquivos do projeto.

Ao acessar a pasta/diretório `alo2`, você terá acesso a dois arquivos: um denominado `Cargo.toml` (similar a um arquivo de inicialização do tipo *ini*) e outro denominado `.gitignore` (indicação de um repositório `git` a ser

usado com o sistema de controle de versão que poderá ser modificado ou desconsiderado com o uso do parâmetro `--vcs`). Haverá também uma pasta/diretório chamada `src` contendo um arquivo nomeado `main.rs`.

O arquivo `Cargo.toml` gerado automaticamente é composto das seguintes linhas:

```
[package]
name = "alo2"
version = "0.1.0"
authors = ["Fulano"]
```

```
[dependencies]
```

O arquivo TOML (*Tom's Obvious, Minimal Language*) é usado como formato de configuração do programa `cargo`. As linhas da seção `package` caracterizam-se em apresentar as informações de configuração usadas pelo programa para compilar determinado código. A seção `dependencies` mostra a lista dos pacotes necessários à compilação de um programa pelo `cargo`.

O arquivo `src/main.rs` gerado automaticamente é composto das seguintes linhas de instrução:

```
fn main() {
    println!("Hello, world!");
}
```

A mensagem "Hello, world" do programa do projeto `alo2` é semelhante à mensagem "Alo, Mundo!" do programa do projeto `alo`.

A compilação de um programa baseado em um projeto `cargo` (projeto com carga de dependência) é diferente da forma usada para compilar um projeto simples. Para isso, execute a instrução a partir da pasta/diretório `estudo/alo2`:

```
cargo build
```

Assim que a compilação foi realizada, o programa `cargo` gerou a criação do arquivo de carga chamado `Cargo.lock`, que tem uma estrutura de dados semelhante a:

```
[root]
```

```
name = "alo2"  
version = "0.1.0"
```

O arquivo `Cargo.lock` é usado pelo programa `cargo` para acompanhar as dependências de uma aplicação. Como este projeto não contém dependências, o arquivo tem uma estrutura simplificada.

Na sequência, para executar o programa gerado e ver seu resultado, informe a instrução:

```
./target/debug/alo2 (nos sistemas operacionais POSIX)  
.\target\debug\alo2.exe (no Sistema operacional Windows)
```

Outra forma de fazer a criação de um projeto com carga de dependência e colocá-lo em execução de forma automática é utilizar a instrução:

```
cargo run
```

O gerenciador `cargo` possibilita gerenciar melhor a distribuição e a construção de carga de um programa, deixando o código-fonte separado do código compilado posicionado em uma pasta ou um diretório `target/debug`. Outra vantagem é que os comandos de uso do programa `cargo` são os mesmos, independentemente do sistema operacional em uso. Por essa razão, a maior parte dos projetos tratados nesta obra o utilizará.

1.4 Notação utilizada

Procurou-se manter ao máximo a notação adotada para a linguagem Rust baseada na documentação fornecida para o compilador, mas, para facilitar a visualização dos códigos de programa apresentados como exemplos, foi necessário definir alguns parâmetros gráficos adicionais:

- todas as variáveis definidas e usadas são escritas em caracteres minúsculos (padrão da linguagem);
- na indicação das sintaxes, o leitor encontra alguns termos escritos entre os símbolos “<” (menor que) e “>” (maior que), que caracterizam pelo fato de ser informações obrigatórias que devem ser substituídas por algum elemento da linguagem;
- o que for mencionado entre os símbolos “[“ (colchete esquerdo) e “]”

(colchete direito) é informação opcional, que pode ser omitida para utilização do recurso indicado; exceção quando da definição e do uso de tabelas em memória;

- a definição de estruturas, tuplas, unidades, enumerações e traços seguirá a nomenclatura-padrão da linguagem.
- nesta obra, os programas apresentados são escritos com espaço quatro de indentação.

Todos os códigos de programas desenvolvidos e utilizados foram feitos para o ambiente de trabalho em modo *console*, ou seja, em modo texto, também chamado modo CLI (*Command-Line Interface*).

As estruturas sintáticas apresentadas no livro estão de acordo com as formas indicadas na documentação oficial da linguagem, principalmente no que tange à definição de blocos após instruções de decisão e laços que na linguagem é sempre apresentada com a definição de blocos explícitos, sendo esta estratégia obrigatória na linguagem Rust, diferenciando-a de outras linguagens da família C, onde tais blocos são indicados sem as chaves quando se trata da ação de apenas uma instrução.

1.5 Gerenciamento de memória

Pelo fato de Rust ser uma linguagem de sistemas, suas ações ocorrem em baixo nível. Desta forma, Rust vê a memória do computador dividida em duas áreas: a pilha (*stack*) e o monte (*heap*).

A área de memória *stack* é um local onde os dados operacionalizados pelo programa são alocados na memória de forma otimizada para uso, como: variáveis; parâmetro, chamadas de sub-rotinas e retornos a funções. Esta área é limitada e não pode ser usada com dados muito grandes, os quais devem ser tratados na área de memória *heap*. A *stack* é rápida, e por essa razão é a área alocada por padrão pela linguagem Rust. Os dados alocados na *stack* são dados de escopo local.

A área *heap* gerencia o uso de qualquer área lógica da memória e opera com dados que não podem ser operacionalizados na *stack*. A área *heap* é

limitada apenas pelo tamanho de memória virtual disponível no computador, sendo este espaço muito maior do que o espaço usado pela área *stack*. A área *heap* é lenta ao ser comparada com a área *stack*, mas apresenta a vantagem de ter maior espaço operacional de trabalho. Os dados alocados na área de memória *heap* são dados de escopo global.

A linguagem Rust não tem recurso de coleta de lixo (*garbage collection*) como outras linguagens. A ação de coleta de lixo é um recurso que remove da memória as referências definidas a acessos a objetos e/ou variáveis dinâmicas. Em Rust, a liberação da memória usada é realizada de forma automática, o que permite manter na memória os dados necessários, enquanto são necessários.

CAPÍTULO 2

Ação sequencial

Neste capítulo, o leitor terá contato com o processo de desenvolvimento da programação sequencial, utilizando-o para efetivar ações de entrada, processamento e saída. São vistos nesta etapa do estudo tipos de dados, variáveis (mutáveis e imutáveis), constantes, operadores aritméticos, expressões aritméticas, controle dos fluxos de entrada e de saída e formatação de valores, além do uso de linhas de comentários.

2.1 Tipos de dados

Todo computador tem como princípio básico a capacidade de processar os *dados* advindos do mundo externo e transformá-los em *informações* que sejam úteis para seus usuários. Os dados são representados por informações processadas por um computador que são armazenadas em memória. Para que isso seja possível, cada dado a ser manipulado precisa ser qualificado de acordo com uma categoria prévia, ou seja, cada dado deve essencialmente referir-se ao conjunto de valores a que pertence e ao qual pode assumir (WIRTH, 1989, p. 3).

A memória de um computador é organizada em bytes, menor dado que pode ser usado e gerenciado. Um byte pode representar 256 caracteres possíveis. No entanto, os computadores conseguem combinar mais de um byte, formando, assim, tipos complexos de dados, como valores inteiros longos, valores com casas decimais, entre outras possibilidades.

Rush opera por meio de um conjunto de tipos de dados incorporados na linguagem a partir de sua biblioteca `std` dividido em dois subconjuntos: *escalar* (para uso com valores simples) e *composto* (para uso com conjunto de valores), não havendo a necessidade de especificar de forma explícita

direta o tipo de dado a certa variável, como se faz nas linguagens C, C++, C#, Java, Pascal, entre outras. Apesar de a linguagem permitir o uso de variáveis sem que seja especificado de forma explícita seu tipo de dado, Rush é uma linguagem que opera com tipos de dados estáticos.

O subconjunto escalar é formado pelos tipos de dados primitivos: inteiro (*integer*), real (*floating-point*¹), lógico (*boolean*), cadeia (*str*) e caractere (*character*). Já o subconjunto composto é formado pelos tipos de dados primitivos: tuplas (*tuples*) e arranjos (*arrays*).

2.1.1 Inteiros

São inteiros os dados numéricos positivos e/ou negativos, excluindo qualquer número fracionário, ou seja, valores numéricos com casas decimais. Em linguagem Rush, esse tipo de dado pode ser referenciado pelos identificadores indicados na Tabela 2.1.

Tabela 2.1 – Tipos de dados inteiros

Sinalizado	Não sinalizado	Tamanho
i8 de -127 até 128	u8 de 0 até 255	8 bits (1 byte)
i16 de -32.768 até 32.767	u16 de 0 até 65.535	16 bits (2 bytes)
i32 de -2.147.483.648 até 2.147.483.647	u32 de 0 até 4.292.967.295	32 bits (4 bytes)
i64 de -9.223.372.036.854.775.808 até 9.223.372.036.854.775.807	u64 de 0 até 18.446.744.073.709.551.615	64 bits (8 bytes)
isize	usize	arch

Os tipos de dados *isize* e *usize* têm uma definição de faixa de valores que depende da arquitetura de computador em uso, podendo assumir os tamanhos de 32 ou 64 bits (KAYHLAVIRTA, 2017, p. 15). Esta categoria abrange todos os tipos inteiros da linguagem, alocando o mesmo espaço

de memória que é usado por um ponteiro. A faixa de abrangência de valores `isize` assemelha-se à mesma faixa do tipo `i64` e `usize` assemelha-se à mesma faixa do tipo `u64`. Os tipos `isize` e `usize` são usados, normalmente, nas ações de acesso às posições de `slots` de matrizes.

Os limites de abrangência dos valores numéricos inteiros podem ser detectados por meio dos recursos `min_value` e `max_value` associados a certo tipo de dado. Neste caso, se solicitada a apresentação do valor da referência `i8::max_value` com a instrução `println!("{}", i8::max_value)`, obtém-se o valor 127.

Valores inteiros podem ser representados na linguagem Rust na forma de literais, como indicado na Tabela 2.2. Uma forma literal estabelece um estilo formatado de apresentação do valor numérico inteiro.

Tabela 2.2 – Regras para formatação de literais

Literal numérica	Exemplo	Representação
Decimal	1_234	Indica o uso do valor decimal 1234
Hexadecimal	0xAB	Indica o uso do valor decimal 171
Octal	0o12	Indica o uso do valor decimal 10
Binário	0b1111_1010	Indica o uso do valor 250

Os tipos literais de valores inteiros aceitam a definição de sufixos de tipos de dados como a indicação do valor `26u8`, que mostra o valor 26 como um tipo de dado inteiro sinalizado de 8 bits.

Se não for especificado para uma variável do tipo inteiro, o tipo de dado a ser usado na linguagem Rust assume automaticamente o tipo `i32`.

2.1.2 Reais

São reais os dados numéricos positivos, negativos e os números fracionários e inteiros. Em linguagem Rust, esse tipo de dado pode ser referenciado pelos identificadores apresentados na Tabela 2.3.

Tabela 2.3 – Tipos de dados reais

Sinalizado	Tamanho
f32 de -3.438 até 3.438	32 bits (4 bytes)
f64 de -1.7308 até 1.7308	64 bits (8 bytes)

O tipo de dado `f32` é usado para definir números de ponto flutuante (números fracionários com seis dígitos decimais de precisão absoluta – precisão simples) de 32 bits, enquanto o tipo de dado `f64` é utilizado para representar números de ponto flutuante de 64 bits (com 15 dígitos decimais de precisão absoluta – precisão dupla). Valores desse tipo devem ser informados com o ponto decimal: `10.` (para 10.0), `5.2` (para 5.2), `8.0` (para 8.0).

O tipo de dado real padrão para a linguagem é o tipo `f64`, que tem desempenho idêntico ao tipo de dado `f32`, com maior precisão numérica. Em sistemas de 32 bits é possível fazer uso do tipo `f64`, mas esta opção terá um desempenho mais lento do que o uso do tipo de dado `f32`. A precisão de valores reais em Rust está de acordo com a norma IEEE 754. Na maior parte dos programas-exemplo desta obra, utiliza-se o tipo de dado `f64`.

Se não for especificado para uma variável do tipo real, o tipo de dado a ser usado na linguagem Rust assume automaticamente o tipo `f64`.

2.1.3 Cadeias e caracteres

Tanto strings (cadeias) quanto caracteres referem-se basicamente ao mesmo tipo de dado, tendo como diferença conceitual o fato de o tipo `char` (caractere) representar apenas um caractere e o tipo `str` (cadeia) referir-se a um conjunto de caracteres. Os dados `char` são delimitados entre aspas simples e os dados `str` são delimitados entre aspas inglesas.

Os caracteres usados com os tipos de dados `char` e `str` são considerados alfabéticos quando a sequência de caracteres se refere, por exemplo, às letras do alfabeto ocidental, ou são alfanuméricos quando usados em

conjunto com os caracteres alfabéticos, de forma que não são valores numéricos usados em cálculos matemáticos.

Os tipos de dados `char` e `str` representam valores escalares estáticos (imutáveis) no padrão Unicode. Dessa forma, a linguagem Rush pode operar com caracteres da tabela ASCII (American Standard Code Information Interchange), letras acentuadas, ideogramas chineses/japoneses/coreanos, emojis e espaços de largura zero. O padrão Unicode tem uma composição numérica diferente da encontrada na tabela ASCII, ocupando 2 bytes de memória, enquanto um código ASCII ocupa apenas 1 byte. O padrão Unicode permite trabalhar com alguns estilos de formatação como os UTF-7, UTF-8, UTF-16 e UTF-32. UTF significa *Unicode Transformation Format* (transformação de formato unicode). O caractere `'\u{2764}'` corresponde a um símbolo representado por um pequeno coração.

Dados `char` ocupam quatro bytes na memória.

O uso de dados alfabéticos ou alfanuméricos mutáveis é definido pelo tipo de dado `String`.

2.1.4 Lógicos

Os dados lógicos (booleanos) ocupam somente 1 byte (8 bits) de memória, têm apenas um de dois valores – `true` (verdadeiro) ou `false` (falso) – e são referenciados pela palavra-chave `bool`. Um tipo de dado `bool` serve, principalmente, para verificar o resultado de certa condição com o uso de tomadas de decisão ou laços, além de expressões lógicas.

2.2 Variáveis

Em programação, variável é uma região de memória do computador previamente identificada, que tem por finalidade armazenar temporariamente os dados de um programa. Uma variável armazena apenas um valor por vez. É considerado valor qualquer conteúdo que possa ser armazenado na memória de um computador. Um valor a ser armazenado e manipulado na memória está relacionado ao tipo de dado

de uma variável, podendo ser numérico, lógico ou um caractere.

O nome de uma variável possibilita sua identificação e seu posterior uso em um programa. Sendo assim, é necessário estabelecer algumas regras para escolhê-lo:

- nomes de variável podem ser atribuídos com um ou mais caractere;
- o primeiro caractere do nome de uma variável não pode ser um número; deve ser uma letra ou um “_” (*underline*);
- o caractere *underline* isoladamente pode ser usado como uma variável;
- o nome de uma variável não pode ter espaços em branco;
- não pode ser nome de variável a palavra reservada a uma instrução ou a um identificador da linguagem Rush.

Deve-se considerar, em um programa, que uma variável pode exercer dois papéis. Um de ação, quando é modificada ao longo da execução de um programa para apresentar determinado resultado, e outro de controle, quando pode ser “vigiada” e controlada durante a execução de um programa.

A linguagem Rush opera com duas categorias de variáveis: *mutáveis* e *imutáveis*. Uma variável mutável (definida pelos comandos `let` e `mut`) é aquela que possibilita alteração de seu valor ao longo da execução de um programa, e a imutável não permite a alteração de seu valor após a definição de um valor a ela que poderá ser feita a qualquer momento no programa a partir de uma única entrada. Uma variável imutável (definida pelo comando `let`) é uma variável que aceita apenas uma vez a definição de valor para a variável, sendo este o tipo padrão de variável usado na linguagem Rush, por ser um dos critérios de segurança e para uso dos paradigmas de programação concorrente e de programação funcional.

A definição de variável pode ser realizada dentro de um programa de duas maneiras: forma direta e indireta. Na forma direta se definem um rótulo de identificação como nome da variável, seu tipo de dado opcional e um valor associado a essa variável. Já na forma indireta se definem o rótulo e um tipo de dado opcional, mas não se associa a ela nenhum valor, pois o

valor será definido no momento da ação de uma entrada de dados, a qual o usuário fornecerá via teclado.

A definição de variáveis na linguagem Rush é realizada a partir da instrução:

```
let [mut] <rótulo>[:<tipo>][ = <valor>];
```

Veja alguns exemplos de definição de variáveis mutáveis e imutáveis de forma direta, que, apesar de expressarem o tipo *integer*, podem ser usadas para qualquer outro tipo de dado suportado pela linguagem:

let x = 10;	Valor inteiro decimal – imutável
let x = 0o12;	Valor inteiro octal – imutável
let x = 0xA;	Valor inteiro hexadecimal – imutável
let x: u8 = 10;	Valor inteiro decimal sem sinalização – imutável
let x: u8 = 0o12;	Valor inteiro octal sem sinalização – imutável
let x: u8 = 0xA;	Valor inteiro hexadecimal sem sinalização – imutável
let x: u8 = 0xA;	Valor inteiro hexadecimal sem sinalização – imutável
let x = 3.0f32;	Valor real decimal 3.0 – imutável
let mut x = 10;	Valor inteiro decimal – mutável
let mut x = 0o12;	Valor inteiro octal – mutável
let mut x = 0xA;	Valor inteiro hexadecimal – mutável
let mut x: u8 = 10;	Valor inteiro decimal sem sinalização – mutável
let mut x: u8 = 0o12;	Valor inteiro octal sem sinalização – mutável
let mut x: u8 = 0xA;	Valor inteiro hexadecimal sem sinalização – mutável

Veja alguns exemplos de definição de variáveis de forma indireta:

let x;	Valor inteiro decimal – imutável
let x: u8;	Valor inteiro decimal – imutável
let mut x;	Valor inteiro decimal – mutável
let mut x: u8;	Valor inteiro decimal – mutável

As variáveis indiretas terão seus valores definidos no momento da entrada

de dados. Assim sendo, não é possível usar os qualificadores à frente de certo valor numérico para orientar o programa compilador como usar o valor na memória. Nesse caso, no momento da entrada e da saída desses valores serão usados códigos de formatação que auxiliarão o comportamento do compilador.

A definição do tipo de dado de uma variável em Rust, como comentado, é opcional. Dessa forma, quando o tipo não é definido, a linguagem faz a inferência do tipo à variável a partir da análise do valor em uso (a instrução `let x = 5` diz que a variável `x` é do tipo inteiro sem inferência de tipo). No entanto é mais eficiente definir o tipo de dado a uma variável, e será esta a forma usada nesta obra (a instrução `let x: i32 = 5` diz que a variável `x` é do tipo inteiro com inferência de tipo ao mencionar `i32`).

As variáveis em Rust podem operar de maneira diferente de outras linguagens de programação, como, por exemplo, ao ser operado na forma de padrões (*patterns*). Um padrão é definido como:

```
let (a, b) = (1, 2);
```

Neste caso, as variáveis `A` e `B` são inicializadas respectivamente com os valores 1 e 2.

Como comentado, a definição de uma variável na linguagem Rust pode ser feita de forma mutável e imutável. O estilo imutável é a forma-padrão de definição de variáveis para a linguagem. Por exemplo, a definição da variável `x` de 32 bits sinalizada a seguir com um valor inicial e a alteração de seu valor em seguida – como

```
let x = 1;  
x = 2;
```

– ocasionaria um erro de compilação no programa, por ser a definição da variável `x` por padrão imutável. A definição de uma variável mutável deve ser considerada como:

```
let mut x = 1;  
x = 2;
```

Neste caso, a variável `x` inicializada como o valor 1 pode ter seu valor alterado para 2 sem a ocorrência de um erro de acesso indevido à

memória.

2.3 Constantes

Em relação a constantes, deve-se levar em consideração, por mais óbvio que seja, que constante é tudo aquilo que é fixo ou estável. Em vários momentos, esse conceito será utilizado em programação. Por exemplo, o valor 1.23 da expressão seguinte é uma constante; nesse caso, uma constante explícita: `RESULTADO = ENTRADA * 1.23`. É pertinente salientar que o valor de uma constante pode ser associado diretamente a uma variável, mas o inverso não é verdadeiro.

Uma constante pode ser referência a partir de um contexto implícito. Uma constante é implícita quando é declarada em um programa codificado em linguagem Rush, por meio do comando `const`, a partir da sintaxe:

```
const <rótulo>: <tipo> = <valor>;
```

Veja exemplos de definição de constantes com o comando `const`:

```
const X: i32 = 1;  
const X: f32 = 1.0;
```

Um ponto importante a ser considerado é que uma variável ocupa espaço na memória do computador, pois, quando é definida, segundo o tipo associado, o compilador aloca na memória espaço para que a variável possa armazenar o valor a ela associado. Já uma constante é definida de maneira diferente, pois não ocupa espaço na memória, por se tratar apenas de um rótulo associado a um valor que será usado no programa quando este for necessário, não sendo este valor armazenado em memória. Além das constantes que podem ser definidas explicitamente em um programa, há um conjunto de constantes matemáticas básicas internas da linguagem, como indicado na Tabela 2.4.

Tabela 2.4 – Constantes matemáticas básicas

Constante	Valor 32 bits	Valor 64 bits	Ação
E	2.7182817	2.718281828459045	Constante de Euler (e)
FRAC_1_PI	0.31830987	0.3183098861837907	Inverso de π , $1/\pi$

Constante	Valor 32 bits	Valor 64 bits	Ação
FRAC_1_SQRT_2	0.70710677	0.7071067811865476	Constante matemática $\sqrt{\pi}/2$
FRAC_2_PI	0.63661975	0.6366197723675814	Constante matemática $2/\pi$
FRAC_2_SQRT_PI	1.1283792	1.1283791670955126	Constante matemática $2/\sqrt{\pi}$
FRAC_PI_2	1.5707964	1.5707963267948966	Constante matemática $\pi/3$
FRAC_PI_3	1.0471976	1.0471975511965979	Constante matemática $\pi/3$
FRAC_PI_4	0.7853982	0.7853981633974483	Constante matemática $\pi/4$
FRAC_PI_6	0.5235988	0.5235987755982989	Constante matemática $\pi/6$
FRAC_PI_8	0.3926991	0.39269908169872414	Constante matemática $\pi/8$
LN_2	0.6931472	0.6931471805599453	Logaritmo natural na base 2
LN_10	2.3025851	2.302585092994046	Logaritmo natural na base 10
LOG10_E	0.4342945	0.4342944819032518	Logaritmo da constante de “e” na base 10
LOG2_E	1.442695	1.4426950408889634	Logaritmo da constante de e na base 2
PI	3.1415927	3.141592653589793	Constante matemática π (pi)
SQRT_2	1.4142135	1.4142135623730951	Raiz quadrada de 2

Para uso das constantes internas é necessário inserir no código de um programa antes da função `main()` o comando `use` com uma das seguintes sintaxes:

```
use std::f32::consts::<nome da constante>;
use std::f64::consts::<nome da constante>;
```

Na indicação nome da constante deve ser colocado o nome da constante indicado na Tabela 2.4. Com o detalhe de tipo `f32` e `f64`, faz-se a definição do formato de retorno do valor da constante em uso – se é respectivamente de 32 ou de 64 bits.

2.4 Operadores aritméticos

Tanto variáveis como constantes podem ser utilizadas na elaboração de cálculos matemáticos com operadores aritméticos, que podem ser

classificados em **unários** ou **binários**. São binários quando atuam em operações de exponenciação, multiplicação, divisão, adição e subtração, em que se utilizam dois componentes. São unários quando atuam na inversão de um valor, atribuindo a ele o sinal positivo ou negativo, ou seja, atuam diretamente em apenas um componente. A Tabela 2.5 apresenta um resumo dos operadores aritméticos utilizados na linguagem Rush.

Os operadores aritméticos indicados serão, em sua maioria, usados ao longo da apresentação da linguagem Rush, para que, ao final deste estudo, o leitor tenha o conhecimento necessário para sua utilização.

Tabela 2.5 – Operadores aritméticos

Operador	Operação	Tipo	Resultado
+	manutenção de sinal	unário	-
-	inversão de sinal	unário	-
%	resto de divisão (módulo)	binário	inteiro
/	divisão	binário	inteiro ou real
*	multiplicação	binário	inteiro ou real
+	adição	binário	inteiro ou real
-	subtração	binário	inteiro ou real
+=	adição cumulativa	binário	inteiro ou real
-=	subtração cumulativa	binário	inteiro ou real
*=	multiplicação cumulativa	binário	inteiro ou real
/=	divisão cumulativa	binário	inteiro ou real
%=	módulo cumulativo	binário	inteiro
=	atribuição	binário	qualquer tipo de dado
<code>base.pow(expoente)</code>	exponenciação	binário	real
<code>base.sqrt()</code>	raiz quadrada	unário	real

Cabe ressaltar que o uso do operador aritmético / (divisão) pode gerar quocientes inteiros (absolutos) ou reais (relativos) conforme o uso dos tipos de dados inteiros ou reais associados às variáveis em uso.

 Observação: Os operadores para **manutenção de sinal** e **inversão de sinal** são usados, respectivamente, para definir se um dado valor numérico é **positivo** ou **negativo**.

O operador de atribuição “=” é usado para vincular o resultado da operação processada do lado direito da expressão à variável indicada a sua esquerda.

Na tabela anterior, não são operadores aritméticos as funções `pow()` e `sqrt()`. São recursos que facilitam operações de exponenciação e radiciação, que são ações de processamento matemático. Por essa razão, estão citadas com os demais operadores aritméticos. Além dessas há ainda, para exponenciação, `powi()`, `powf()`, e para radiciação, `cbrt()`.

2.5 Expressões aritméticas

É muito comum a necessidade de trabalhar com expressões aritméticas, uma vez que, em sua maioria, todo trabalho computacional está relacionado a cálculos matemáticos. Essas expressões são definidas pela relação entre variáveis e constantes numéricas com a utilização dos operadores aritméticos.

Considere a fórmula $AREA = \pi \cdot RAI0^2$ para o cálculo da área de uma circunferência, em que estão presentes as variáveis `AREA` e `RAIO`, a constante π ($\pi = 3.14159 \dots$), os operadores aritméticos de multiplicação e, também, a operação de potência, elevando o valor da variável `RAIO` ao quadrado.

As expressões aritméticas escritas para ser executadas em um computador têm formato um pouco diferente do conhecido em matemática. Por exemplo, a expressão $x = \{ 43 \cdot [55 : (30 + 2)] \}$ será escrita em Rust como `x = (43 * (55 / (30 + 2) ) )`. Perceba que as chaves e os colchetes são abolidos, ficando em seu lugar apenas os parênteses. A fórmula para cálculo da área de uma circunferência seria escrita da seguinte maneira:
`AREA = 3.14159 * RAI0 * RAI0.`

E a fórmula usada para calcular a área de um triângulo, em que é necessário multiplicar a base pela altura e, em seguida, dividir pela constante 2, como ficaria? Observe a fórmula-padrão seguinte:

Ela deveria ser escrita como $A = (B * H) / 2$.

2.6 Instruções da linguagem Rush

As instruções são representadas pelo conjunto de *palavras-chave* (vocabulário) de uma linguagem de programação, com a finalidade de comandar, em um computador, o seu funcionamento e a maneira como os dados armazenados devem ser tratados. Na linguagem Rust as instruções devem ser escritas com letras minúsculas (a linguagem diferencia caracteres maiúsculos de minúsculos). Em virtude dessa característica, Rust é considerada *case-sensitive*.

A Tabela 2.6 mostra uma lista contendo as palavras-chave (os comandos, propriamente ditos) da linguagem de programação Rush. Nenhum rótulo definido pelo programador, como variáveis, constantes, nomes de procedimentos/funções externas, pode ser nomeado com uma dessas palavras, por serem palavras reservadas da linguagem, como indicado na Tabela 2.6.

Tabela 2.6 – Palavras-chave dos comandos da linguagem Rush

abstract	alignof	as	become	box
break	const	continue	crate	do
else	enum	extern	false	final
fn	for	if	impl	in
let	loop	macro	match	mod
move	mut	offsetof	override	priv
proc	pub	pure	ref	return
Self	self	sizeof	static	struct
super	trait	true	type	typeof
unsafe	unsized	use	virtual	where
while	yield	-	-	-

2.7 Programas em Rush

Este tópico trata dos diversos recursos para o desenvolvimento eficiente de programas de computador escritos na linguagem Rust. O leitor aprenderá a utilizar todos os recursos apresentados anteriormente e mais alguns novos em situações mais próximas da realizada cotidiana.

2.7.1 Fluxos de entrada e saída

É por meio dos fluxos de entrada e de saída da biblioteca `std::io` que as operações de acesso ao teclado e ao monitor de vídeo podem ser efetuadas. Para demonstrar operações com os fluxos de entrada e de saída, são apresentados alguns pequenos programas com diversas situações que ilustram operações de entrada e de saída de dados. Atente com muito cuidado a cada detalhe apresentado e observe cada explicação fornecida, pois este tópico é importante para a execução dos demais exemplos de programação apresentados no livro.

O programa seguinte mostra a mensagem "Programação na linguagem Rust." no modo CLI do sistema operacional em uso de forma semelhante à do exemplo apresentado do programa "Alo, mundo!", mas agora com a capacidade de efetuar uma pausa após a apresentação da mensagem "Tecle <Enter> para encerrar...", que necessitará de novos recursos da linguagem. Para isso, na linha de comando dentro da pasta/diretório `estudo`, execute as instruções:

```
cargo new c02ex01 --bin
cd c02ex01
```

Abra o arquivo de programa `main.rs` da pasta/diretório `src` de `c02ex01` em `estudo` criado automaticamente com um editor de texto de sua preferência e altere o código do arquivo do programa "Hello, world!" com o código seguinte:

```
use std::io;
use std::io::prelude::*;

fn main() {
    println!("Programação na linguagem Rust.\n");
    print!("Tecle <Enter> para encerrar...");
```

```

    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

O comando `use` é usado para fazer a chamada dos arquivos dos pacotes (bibliotecas) de auxílio da linguagem Rust. Neste caso, carregam-se os arquivos de pacotes `std::io` e `std::io::prelude::*`, que disponibilizam para o programa os recursos para uso da macro `println!()` e das funções `stdin()` para a entrada e `stdout()` para a saída da biblioteca `std::io` e das funções `flush()` e `read()` da biblioteca `std::io::prelude::*`.

A função `io` (*input/output*) é usada para auxiliar as operações de entrada e de saída pelo usuário na linha de comando, sendo esta uma das muitas funções existentes na biblioteca-padrão `std` (*standard*). A função de uma biblioteca é associar certo conjunto de módulos de operação, como é o caso de `prelude`, que tem por finalidade a característica de gerenciar as operações de entrada e *flush*. Para acesso às demais funcionalidades da biblioteca `std`, consulte o endereço <https://doc.rust-lang.org/std/index.html#modules>.

A biblioteca-padrão `std`, assim como outras bibliotecas-padrão, como `alloc` (fornece ponteiros e coleções de recursos para o gerenciamento de valores alocados no *heap* de memória), `core` (tem recursos básicos e mínimos para a linguagem), `proc_macro` (fornece interfaces para a definição e a derivação de macros) e `std_unicode` (fornece uma coleção de recursos para o tratamento de caracteres e *strings* de caracteres), são denominadas nessa linguagem como *crates* (caixas de recursos, ou seja, pacotes de recursos que podem ser referenciados em um projeto).

O uso do caractere especial de controle `\n` após a mensagem “Programação na linguagem Rust.” junto à macro `println!()` faz com que ocorra um salto de linha em branco após a apresentação da mensagem, além do salto comandado pelo `\n`. A macro `print!()`, pelo fato de não realizar o salto de linha, deixa o buffer de saída sobrecarregado. Dessa forma, é necessário fazer a limpeza do buffer com outra instrução de apoio. Neste caso, é realizada pela ação da instrução `io::stdout().flush().unwrap();`.

A instrução `io::stdout().flush().unwrap();` é formada por três recursos

distintos: o `io::stdout()` relacionado ao módulo `prelude` é responsável por definir e direcionar para a saída-padrão (neste caso, o monitor de vídeo) o processo atual de ação; `flush()` é um método responsável por garantir que a saída seja enviada imediatamente ao periférico-padrão de saída conectado ao computador; e a função `unwrap()` faz com que a ação da instrução seja executada da forma mais rápida possível e da maneira mais simples possível. A indicação de `stdout()` a `io` com `::` se caracteriza por ser `stdout()` uma função associada à `io`.

A instrução `io::stdin().read(&mut [0u8]).unwrap();` por meio do método `read()` efetua a leitura de apenas um caractere de acordo com o código de formatação de 8 bits `[0u8]` indicado como parâmetro do método. Antes do código de formatação é usada a indicação `&mut`, que implica no uso do retorno de uma referência mutável executada imediatamente pela função `unwrap()`. O recurso `io::stdin()` indica o uso do periférico-padrão de entrada, normalmente o teclado.

As funções `stdin()` para tratamento da entrada de dados e `stdout()` para o tratamento da saída de dados são recursos que retornam a instância de operação a que são destinadas em relação aos periféricos-padrão de entrada e saída associados ao terminal de uso do computador.

Após a escrita do código em `src` de `c02ex01` dentro de `estudo`, execute a partir da pasta/diretório `estudo` a instrução:

```
cargo run
```

Observe a apresentação das mensagens e a necessidade de acionar <Enter> para encerrar o programa.

O programa seguinte solicita o nome de uma pessoa e apresenta uma mensagem de saudação com o nome da pessoa à direita. Para isso, lembre-se de executar as instruções na linha de comando dentro da pasta/diretório `estudo`:

```
cargo new c02ex02 --bin  
cd c02ex02
```

Abra o arquivo de programa `main.rs` da pasta/diretório `src` de `c02ex02` em `estudo` criado automaticamente com um editor de texto de sua preferência

e altere o código do arquivo com o código seguinte:

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut nome = String::new();

    print!("Informe seu nome: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut nome).expect("Entrada incorreta");

    if let Some('\n') = nome.chars().next_back() {
        nome.pop();
    }

    if let Some('\r') = nome.chars().next_back() {
        nome.pop();
    }

    println!("Olá, {} \n", nome);
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Após a escrita do código em `src` de `c02ex02`, em `estudo`, execute a partir da pasta/diretório `estudo` a instrução:

```
cargo run
```

Observe a ação de interação do programa solicitando a entrada do nome e a apresentação da mensagem de saudação. Note que no programa anterior estão em uso novas instruções e a definição da variável `NOME`.

A variável `NOME` é definida como mutável e associada ao tipo de dado `String` da biblioteca-padrão da linguagem que é criada a partir da função `new()`, que estabelece em memória uma instância (reserva um espaço operacional) de trabalho para a variável.

A instrução `io::stdin().read_line(&mut nome).expect("Entrada incorreta");`

efetua a entrada dos caracteres informados no teclado junto à variável `NOME` passada como um parâmetro mutável (`&mut`) da forma mais simples possível. A leitura do dado é efetivada pelo método `read_line()` associado ao recurso `io::stdin()`, que recebe o dado informado pelo usuário no periférico-padrão de entrada, neste caso, o teclado. A função `expect()` deverá ser executada se a entrada fornecida para a função `read_line(&mut nome)` for inválida por algum motivo. O símbolo `&` (e comercial) à frente de `mut` tem por finalidade indicar que o parâmetro usado pelo método `read_line()` é do tipo referência para a variável mutável `NOME`, compatibilizando a entrada de um dado mutável como se fosse, neste momento, imutável, que é o padrão operacional da linguagem Rust.

A função `expected()` pode ser substituída pela função `unwrap()`. A diferença existente nessas funções relaciona-se ao fato de que a função `unwrap()` trata qualquer ocorrência de erro como um evento de pânico, indicando mensagem-padrão dessa ação. Nestes casos, é mais eficiente usar a função `expected()`, que permite a apresentação de algum código de tratamento de erro personalizado no sentido de facilitar sua posterior correção pela equipe de desenvolvimento, caso um erro não identificado na fase de desenvolvimento da aplicação não tenha sido previsto.

Após a efetivação da entrada, os trechos de código com o comando `if let` (se deixado) permitem usar padrões correspondentes dentro da condição de uma declaração indicada. Neste caso, efetua a verificação da ocorrência do acionamento da tecla `<Enter>`, que ao ser acionada efetua duas ações operacionais: o avanço de linha (`\n`) e o retorno do cursor (`\r`). Se os dois comandos `if let` resultarem como verdadeiros, será executada a instrução `nome.pop()`, que remove os caracteres `\n` e `\r` do *heap* de memória, mantendo apenas a parte que realmente interessa para ser atribuída à variável `nome`, que compõe os caracteres informados na entrada.

O trecho `Some('\código') = nome.chars().next_back()` nas instruções `if let` usa a função `Some()` para identificar a localização dos caracteres especiais `\n` ou `\r` indicados como `código`. `Some()` é uma função que serve para “pegar” algo ou alguma coisa que relaciona-se a próxima iteração (pega o

próximo caractere após localizar o caractere indicado como código). Assim sendo, a função `Some()` “pega” o caractere que estiver após `\n` e `\r` e recebe como atribuição a ordem de voltar uma posição por meio do trecho `nome.chars().next_back()`, onde `NOME` é a variável usada para a entrada do *string* e `chars()` é o método que pega na sequência de caracteres determinado caractere. O método `next_back()`, quando executado, avança seu contador de caracteres para trás, mantendo o ponteiro de deslocamento da *string* na posição dos caracteres especiais detectados `\n` e `\r`. No momento em que essas condições forem verdadeiras, as instruções `nome.pop()`, por meio do método `pop()` vinculado à variável `NOME`, faz a retirada do caractere da sequência de caracteres definido na *string*.

Além de utilizar os caracteres especiais `\n` e `\r`, é possível usar uma série de caracteres especiais de controle da formatação para a saída de dados. Observe a Tabela 2.7.

Tabela 2.7 – Caracteres especiais de controle

Caractere especial	Finalidade
<code>\"</code>	Apresenta o símbolo de aspas no ponto em que é indicado.
<code>\\</code>	Apresenta o símbolo de barra no ponto em que é indicado.
<code>\'</code>	Apresenta o apóstrofo no ponto em que é indicado.
<code>\0</code>	Gera um caractere nulo (zero).
<code>\n</code>	Gera uma linha a partir do ponto indicado.
<code>\r</code>	Gera um retorno de carro sem avanço de linha.
<code>\t</code>	Gera um espaço de tabulação do ponto indicado.

A Tabela 2.7 apresenta as sequências de escape que podem ser utilizadas na linguagem Rust para o envio de certos comportamentos ao monitor de vídeo (ecrã) em modo console.

2.7.2 Utilização de valores numéricos

Anteriormente, foram apresentados alguns recursos de uso com entrada e saída de mensagens como sequências de caracteres (*strings*). Nesta etapa do

aprendizado, são mostrados mais alguns exemplos com valores numéricos inteiros e reais, o que é feito de modo diferente do modo executado nas entradas alfanuméricas. Acompanhe atentamente cada exemplo a seguir.

O programa seguinte solicita a entrada de um valor inteiro de 32 bits desconhecido e depois efetua sua apresentação. Grave o projeto com o nome `c02ex03` (`cargo new c02ex03 --bin`) no diretório (pasta) `estudo` de acordo com as orientações já apresentadas, abra o arquivo de programa `main.rs`, criado automaticamente com um editor de texto de sua preferência, e altere o código do arquivo desta forma:

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut valor = String::new();
    let vlr: i64;

    print!("Informe um valor inteiro: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).unwrap();
    vlr = valor.trim().parse::<i64>().unwrap();

    println!();
    println!("Resultado = {}", vlr);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Depois de escrever o código em `src` de `c02ex03` em `estudo`, execute a partir da pasta/diretório `estudo` a instrução:

```
cargo run
```

No programa, informe inicialmente um valor inteiro e veja a resposta apresentada. Depois execute o programa novamente e efetue entradas com valores reais e dados alfanuméricos para ver a apresentação da ocorrência

de erro da entrada que ocasiona a interrupção do funcionamento do programa. O tratamento para este tipo de ocorrência será orientado no próximo capítulo.

O programa do projeto `c02ex03` tem novas funcionalidades em relação aos exemplos anteriores. Atente para as partes desconhecidas que são explanadas a seguir.

A instrução `let mut valor = String::new();` é usada para efetuar a entrada de dados no programa. A linguagem Rust só aceita a entrada de dados que sejam do tipo caractere alfanumérico. Dessa forma, os valores numéricos fornecidos na entrada precisam ser convertidos posteriormente na forma numérica correspondente. Por essa razão é que está sendo definida a variável `VALOR` como do tipo `String`. Os símbolos `::` do trecho `::new` indicam que `new` é uma função associada ao tipo `String`, tendo como finalidade criar na memória uma instância vazia para a variável em uso com a finalidade de receber uma *string*.

A instrução `let vlr: i64;` é usada para definir a variável do tipo numérica que será usada para a recepção do valor numérico após sua recepção e conversão.

A entrada do valor numérico atribuído à variável de nome `VALOR` é efetuada pela instrução já conhecida e comentada: `io::stdin().read_line(&mut valor).unwrap();`. Após a entrada, o valor na variável `VALOR` deve ser convertido para seu equivalente numérico por meio da instrução `vlr = valor.trim().parse::<i64>().unwrap();` junto à variável `VLR`.

A instrução `vlr = valor.trim().parse::<i64>().unwrap();` faz uso do método `trim()`, que retira qualquer espaço em branco que tenha sido eventualmente fornecido na entrada para a variável `VALOR`. Depois disso, o conteúdo é convertido por meio do método `parse::<formato>()`, em que `formato` é a indicação de um dos rótulos de definição dos tipos de dados numéricos aceitos pela linguagem.

O programa seguinte deve calcular o salário líquido de um trabalhador cuja base da remuneração seja calculada em horas. Para isso é necessário

obter alguns dados básicos, como valor da hora de trabalho, número de horas trabalhadas no mês e o percentual de desconto do INSS, para que seja possível apresentar os resultados do salário bruto, do valor descontado e do salário líquido. Grave o projeto com o nome `c02ex04` (cargo `new c02ex04 --bin`) em estudo seguindo as orientações já apresentadas, abra o arquivo de programa `main.rs`, criado automaticamente com um editor de texto de sua preferência, e altere o código do arquivo como segue:

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut hora_trab = String::new();
    let mut vlr_hora = String::new();
    let mut percent_desc = String::new();

    let ht: f64;
    let vh: f64;
    let pd: f64;
    let td: f64;
    let sb: f64;
    let sl: f64;

    print!("Entre a quantidade de horas trabalhadas ...: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut hora_trab).unwrap();
    ht = hora_trab.trim().parse::<f64>().unwrap();

    print!("Entre o valor da hora de trabalho .....: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut vlr_hora).unwrap();
    vh = vlr_hora.trim().parse::<f64>().unwrap();

    print!("Entre o valor do percentual de desconto ...: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut percent_desc).unwrap();
    pd = percent_desc.trim().parse::<f64>().unwrap();
```

```

    sb = ht * vh;
    td = (pd / 100.) * sb;
    sl = sb - td;

    println!();
    println!("Salário Bruto ....: {}", sb);
    println!("Desconto .....: {}", td);
    println!("Salário Liquido .: {}", sl);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Depois de escrever o código em `src` de `c02ex04` em `estudo`, execute a partir da pasta/diretório `estudo` a instrução:

```
cargo run
```

Ao executar o programa, insira valores com ponto decimal e observe como os valores de saída são apresentados. Experimente, respectivamente, os valores 10.33, 12.5 e 6.5. Veja que o programa apresenta como saída os valores 129.125, 8.393125 e 120.731875.

Apesar de os valores serem exibidos de forma correta, seu formato não é elegante. Para melhorar o aspecto da apresentação de valores numéricos com ponto flutuante, é necessário utilizar alguns recursos de formatação. Assim sendo, é necessário proceder a uma pequena alteração junto ao trecho de código indicado.

```

    println!("Salário Bruto ....: {}", sb);
    println!("Desconto .....: {}", td);
    println!("Salário Liquido .: {}", sl);

```

Que poderá ter como definição de formação os pontos marcados em **negrito**.

```

    println!("Salário Bruto ....: {:8.2}", sb);
    println!("Desconto .....: {:8.2}", td);
    println!("Salário Liquido .: {:8.2}", sl);

```

Para apresentar valores reais formatados, basta fazer uso da máscara `{:tamanho.decimais}`, onde `decimais` é a quantidade de casas decimais a ser definida para a mantissa do valor numérico e `tamanho` é a quantidade total de caracteres usados para a formatação. Dessa maneira, o código de formatação `{:8.2}` proporciona uma máscara de saída de dados numéricos como `99999.99`, onde a parte inteira do valor numérico (expoente) terá 5 posições e a parte decimal (mantissa) terá 2 posições.

2.7.3 Outras operações numéricas

Além dos exemplos relacionados ao uso de valores numéricos inteiros e reais com a linguagem Rust, é possível fazer uso de outros recursos relacionados a operações numéricas. Entre as várias possibilidades de cálculo numérico, é possível trabalhar com operações de divisão de valores com quociente inteiro, extração de resto de divisão, exponenciação, radiciação e apresentação de valores em bases numéricas diferentes da base decimal.

O programa seguinte exemplifica, com valores inteiros, a obtenção do quociente inteiro e do resto da divisão. Salve o projeto com o nome `c02ex05`. (`cargo new c02ex05 --bin`) em estudo seguindo as orientações já apresentadas; abra o arquivo de programa `main.rs`, criado automaticamente com um editor de texto de sua preferência, e altere o código do arquivo como segue:

```
use std::io;
use std::io::prelude::*;
fn main() {
    let mut dividendo = String::new();
    let mut divisor = String::new();

    let dvd: i64;
    let dvr: i64;
    let res: i64;
    let quo: i64;

    print!("Entre o valor do dividendo ...: ");
```

```

    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut dividendo).unwrap();
    dvd = dividendo.trim().parse::<i64>().unwrap();

    print!("Entre o valor do divisor .....: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut divisor).unwrap();
    dvr = divisor.trim().parse::<i64>().unwrap();

    quo = dvd / dvr;
    res = dvd % dvr;

    println!();
    println!("Quociente = {:8}", quo);
    println!("Resto = {:8}", res);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Após a escrita do código em `src` de `c02ex05` em `estudo`, execute a partir da pasta/diretório `estudo` a instrução:

```
cargo run
```

As variáveis `QUO`, `DVD`, `DVR` e `RES` estão definidas como `i64`. Isso faz com que o operador de divisão “/” gere como resultado, para a variável `QUO`, um valor também do tipo inteiro.

Outro detalhe é o operador `%` (módulo), que permite obter o resto da divisão de números inteiros. Esse recurso é muito útil quando há necessidade de verificar se determinado valor é múltiplo ou divisor de outro valor.

A apresentação dos resultados inteiros usam a máscara de formatação `{:8}` indicando o uso de oito espaços em branco para a apresentação de valores numéricos alinhados da direita para a esquerda.

O programa seguinte demonstra o uso de recursos de cálculo

complementares para obtenção de resultados de exponenciações e radiciações. A linguagem Rust apresenta internamente algumas funções para os cálculos matemáticos de potência, destacando-se as operações de `pow()` para exponenciação de valores numéricos inteiros com base positiva ou negativa com índice positivo, `powi()` para exponenciação de valores numéricos inteiros com base real e com índice inteiro positivo ou negativo, `powf()` para exponenciação de valores numéricos reais positivos ou negativos, `sqrt()` para raiz quadrada e `cbrt()` para raiz cúbica de valores reais.

Grave o projeto seguinte com o nome `c02ex06` (`cargo new c02ex06 --bin`) em `estudo` seguindo as orientações já apresentadas; abra o arquivo de programa `main.rs` criado automaticamente com um editor de texto de sua preferência e altere o código do arquivo como segue para em seguida efetuar a gravação e a execução do programa (`cargo run`):

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut base = String::new();
    let mut indice = String::new();

    let bas: f64;
    let ind: f64;

    print!("Entre o valor da base .....: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut base).unwrap();
    bas = base.trim().parse::<f64>().unwrap();

    print!("Entre o valor do índice ...: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut indice).unwrap();
    ind = indice.trim().parse::<f64>().unwrap();

    println!();
    println!("Exponenciação = {:.2}", bas.powf(ind));
}
```

```

println!("Raiz quadrada = {:.2}", bas.sqrt());
println!("Raiz cúbica = {:.2}", bas.cbrt());

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

O programa solicitará o valor da BASE e do ÍNDICE apresentando em seguida os valores da potência da BASE ao ÍNDICE e os valores das raízes quadrada e cúbica da BASE. As funções `powf()`, `sqrt()` e `cbrt()` são recursos que operam com valores, apenas, reais.

A partir deste ponto supõe-se que o leitor tenha se habituado à criação dos projetos de programas definidos para a linguagem Rust, além de ter aprendido como efetuar sua execução. Assim sendo, para os demais exemplos será indicado apenas o nome do projeto.

O programa seguinte efetua a entrada dos valores numéricos de base e índice inteiros e apresenta o resultado da potência dos valores informados. Para este programa é necessário, para fazer uso da função `pow()`, definir o valor do índice como sendo do tipo `u32`. Defina para uso o projeto com o nome `c02ex07`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let mut base = String::new();
    let mut indice = String::new();

    let bas: i64;
    let ind: u32;

    print!("Entre o valor da base .....: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut base).unwrap();
    bas = base.trim().parse::<i64>().unwrap();
}

```

```

    print!("Entre o valor do índice ...: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut indice).unwrap();
    ind = indice.trim().parse::<u32>().unwrap();

    println!();
    println!("Exponenciação = {:.2}", bas.pow(ind));

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

O programa anterior efetua o cálculo da exponenciação com valores inteiros. A base do cálculo pode ser efetuada com valores inteiros de 64 bits positivos e negativos (*i64*), mas o índice só pode ser operado com valores inteiros de 32 bits positivos (*u32*). Caso queira calcular potências de valores inteiros positivos e negativos de 32 bits de uma base real de 64 bits, será necessário usar a função `powi()`.

O programa seguinte efetua a entrada dos valores numéricos de base como real, e índice, como inteiro, e apresenta o resultado da potência dos valores informados a partir de uso do valor de índice positivo ou negativo. O uso da função `powi()` requer a base como real e aceita o índice como inteiro. Defina para uso o projeto com o nome `c02ex08`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let mut base = String::new();
    let mut indice = String::new();

    let bas: f64;
    let ind: i32;

    print!("Entre o valor da base .....: ");
    io::stdout().flush().unwrap();

```

```

    io::stdin().read_line(&mut base).unwrap();
    bas = base.trim().parse::<f64>().unwrap();

    print!("Entre o valor do índice ...: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut indice).unwrap();
    ind = indice.trim().parse::<i32>().unwrap();

    println!();
    println!("Exponenciação = {:.2}", bas.powi(ind));

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao executar o programa anterior, experimente informar um valor de índice negativo para ver a operacionalização da função `powi()`.

Para operar potências Rust, use as funções `pow()`, `powf()` e `powi()`, que, de acordo com a maneira utilizada, podem operar resultados a partir da entrada de valores positivos ou negativos inteiros e/ou reais. Para as operações de radiciação, pode-se fazer uso das funções `sqrt()` e `cbrt()` para valores reais. Se quiser calcular raízes de outros índices diferentes de quadrado e cubo, podem-se criar recursos para o cálculo e raízes enésimas por meio das variações das funcionalidades de exponenciação, sendo:

```

base.pow(1 / índice)
base.powf(1 / índice)
base.powi(1 / índice)

```

O próximo exemplo de programa visa apresentar o valor numérico inteiro 10 em seus equivalentes binário, octal e hexadecimal. Defina para uso o projeto com o nome `c02ex09`.

```

use std::io;
use std::io::prelude::*;

fn main() {

```

```

let valor = 10;

println!("Binário .....: {:b}", valor);
println!("Octal .....: {:o}", valor);
println!("Hexadecimal ...: {:X}", valor);
println!();
println!("ou");
println!();
println!("BIN: {:b} OCT: {:o} HEX: {:x}", valor, valor, valor);

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, o valor **10** é apresentado como **1010**, **12**, **A** (quando em uso a máscara **{:X}**) e **a** (quando em uso a máscara **{:x}**). Pode-se também usar as máscaras **{:e}** e **{:E}** para notação científica e **{:p}** para ponteiros.

2.7.4 Definição de constantes

Muitas vezes, é necessário em um programa fazer uso de constantes numéricas para a efetivação de ações de processamento matemático. As constantes podem estar definidas diretamente em uma equação ou expressão aritmética e podem ser definidas em um código de programa de forma indireta; neste caso, com o comando **const**.

Para exemplificar esta ocorrência, considere um programa que apresente o valor do cálculo da área de um cilindro a partir do fornecimento dos valores do raio do diâmetro da base e da altura do cilindro. Defina para uso o projeto com o nome **c02ex10**.

```

use std::io;
use std::io::prelude::*;

fn main() {
    const PI: f64 = 3.14159;

```

```

let mut altura = String::new();
let mut raio = String::new();

let h: f64;
let r: f64;
let a: f64;

print!("Entre a altura ...: ");
io::stdout().flush().unwrap();
io::stdin().read_line(&mut altura).unwrap();
h = altura.trim().parse::<f64>().unwrap();

print!("Entre o raio .....: ");
io::stdout().flush().unwrap();
io::stdin().read_line(&mut raio).unwrap();
r = raio.trim().parse::<f64>().unwrap();

a = 2.0 * PI * r * (r + h);

println!();
println!("Área do Cilindro .: {:.2}", a);

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Depois que o programa for executado, informe os valores solicitados e observe a apresentação do valor utilizando a constante definida. Note que os rótulos de declaração de constantes devem ser feitos com caracteres maiúsculos.

O programa seguinte é semelhante ao programa anterior, só que utiliza a constante `PI` definida na linguagem Rust. Defina para uso o projeto com o nome `c02ex11`.

```

use std::io;
use std::io::prelude::*;

```

```

use std::f64::consts::PI;

fn main() {
    let mut altura = String::new();
    let mut raio = String::new();

    let h: f64;
    let r: f64;
    let a: f64;

    print!("Entre a altura ...: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut altura).unwrap();
    h = altura.trim().parse::<f64>().unwrap();
    print!("Entre o raio .....: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut raio).unwrap();
    r = raio.trim().parse::<f64>().unwrap();

    a = 2.0 * PI * r * (r + h);

    println!();
    println!("Área do Cilindro .: {:8.2}", a);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Depois que o programa for executado, informe os valores solicitados e observe a apresentação do valor utilizando a constante definida. Note que nesta versão está sendo usada, no início do programa, a chamada da biblioteca de tratamento de constantes com a instrução `use std::f64::consts::PI;`, onde a constante `PI` está sendo ativada para uso.

2.7.5 Valores lógicos

Foi comentado na apresentação dos tipos de dados que o tipo `bool` opera

com o valor lógico `true` para verdadeiro e o valor `false` para falso. Com o intuito de corroborar esta informação, o próximo programa demonstra essa ocorrência. Defina para uso o projeto com o nome `c02ex12`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let x = true;
    let y: bool = false;

    println!("Valor logico verdadeiro de X ...: {}", x);
    println!("Valor logico falso de Y .....: {}", y);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao fazer a execução do programa são apresentadas as mensagens de saída com os valores `true` e `false`.

2.7.6 Operações matemáticas

A linguagem Rust, além das operações de exponenciação e radiciação, engloba um conjunto de operações matemáticas diversas, na forma de métodos como arredondamento, teto, piso, exponencial, logaritmo, radianos, máximos, mínimos, seno, cosseno etc.

O programa seguinte apresenta diversos valores obtidos por meio do uso de métodos matemáticos específicos da linguagem acessados com o uso de algumas constantes numéricas. Defina para uso o projeto com o nome `c02ex13`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    println!("abs(-3.5) .....: {}", -3.5f64.abs());
```



```

println!("2 ^ 3 .....: {}", 2i32.pow(3));
println!("sqrt 25 .....: {}", 25f64.sqrt());
println!("cbrt 27 .....: {}", 27f64.cbrt());
println!("round 2.45 .....: {}", 2.45f64.round());
println!("floor 2.10 .....: {}", 2.10f64.floor());
println!("ceiling 2.10 .....: {}", 2.10f64.ceil());
println!("e ^ 2 .....: {}", 2f64.exp());
println!("log(2) .....: {}", 2f64.ln());
println!("log10(2) .....: {}", 2f64.log10());
println!("90 to Radians .....: {}", 90f64.to_radians());
println!("PI to Degrees .....: {}", 3.14f64.to_degrees());
println!("max 4, 5 .....: {}", 4f64.max(5f64));
println!("min 4, 5 .....: {}", 4f64.min(5f64));
println!("sin 3.14 .....: {}", 3.14f64.sin());
println!("cos 3.14 .....: {}", 3.14f64.cos());
println!("tan 3.14 .....: {}", 3.14f64.tan());
println!("asin 3.14 .....: {}", 3.14f64.asin());
println!("acos 3.14 .....: {}", 3.14f64.acos());
println!("atan 3.14 .....: {}", 3.14f64.atan());
println!("sinh 3.14 .....: {}", 3.14f64.sinh());
println!("cosh 3.14 .....: {}", 3.14f64.cosh());
println!("tanh 3.14 .....: {}", 3.14f64.tanh());

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

apresentados como NaN indicam que o resultado do cálculo retornou um valor que não é um número (NaN = *Not a Number*).

2.8 Linhas de comentários

Um dos pontos importantes em programação de computadores é a possibilidade de definir, no código de um programa, linhas de comentários que descrevam e lembrem o programador de detalhes e/ou características para uma eventual manutenção do código-fonte escrito.

Nesse sentido, é possível lançar mão de três formas de comentário interno a ser usadas no código de um programa em linguagem Rust. Todas as linhas de comentários usadas no código de um programa são desconsideradas pelo compilador, ou seja, durante a compilação do código-fonte, elas (bem como os espaços em branco) são retiradas do código, que se torna o programa executável. Por isso, adquira, como programador, o hábito de usar linhas de comentários nos códigos de programa que são produzidos para fazer sua documentação interna. São formas usadas na linguagem Rust:

- `//` – forma usada para documentar trechos com uma linha – breve informação para a linha de instrução;
- `///` – forma usada para documentar trechos com uma linha – comentários gerais;
- `/* */` – forma usada para documentar trechos com mais de uma linha – comentários de trechos especiais/características.

Como exemplo de uso das linhas de comentários, considere o código seguinte do projeto `c02ex14`, que apresenta como resultado os limites mínimos e máximos dos tipos de dados numéricos inteiros suportados pela linguagem.

```
/// main.rs
```

```
/* Projeto .....: c02ex14  
* Autor(es) .....: Augusto Manzano  
* Data .....: 11/9/2017  
* Versao .....: 1.0  
* Revisao .....: 11/9/2017  
* */
```

```
use std::*;  
use std::io::prelude::*;
```

```
fn main() {
```

```
/// Trecho com apresentação de faixa mínima e máxima de valores
```

```

println!("i8 ....: de {:20} até {:20}", i8::MIN, i8::MAX);
println!("u8 ....: de {:20} até {:20}", u8::MIN, u8::MAX);
println!();
println!("i16 ....: de {:20} até {:20}", i16::MIN, i16::MAX);
println!("u16 ....: de {:20} até {:20}", u16::MIN, u16::MAX);
println!();
println!("i32 ....: de {:20} até {:20}", i32::MIN, i32::MAX);
println!("u32 ....: de {:20} até {:20}", u32::MIN, u32::MAX);
println!();
println!("i64 ....: de {:20} até {:20}", i64::MIN, i64::MAX);
println!("u64 ....: de {:20} até {:20}", u64::MIN, u64::MAX);
println!();
println!("isize .: de {:20} até {:20}", isize::MIN, isize::MAX);
println!("usize .: de {:20} até {:20}", usize::MIN, usize::MAX);

println!();
print!("Tecle <Enter> para encerrar..."); // Açione apenas <Enter>
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Note as três formas de uso das linhas de comentários. As que se iniciam com `///` são linhas de comentários de documentação geral; a iniciada por `/*` caracteriza-se como linha de comentário de certo bloco; e as iniciadas por `//` são linhas de comentários para o código do programa.

2.9 Mensagens de advertência

Quando se efetua a compilação de um programa com o comando `cargo run`, o compilador Rust, em virtude de seu critério de segurança, pode apresentar algumas mensagens de advertência avisando que algo no código está definido e não se encontra em uso ou sobre alguma outra ocorrência.

A apresentação das mensagens de advertência não impede a compilação do programa; indica apenas alguma ocorrência de algo incorreto no código do programa, como por exemplo a definição de variável que não está em uso no código.

O programa seguinte demonstra a definição de duas variáveis e faz uso apenas de uma variável; devido a esta ocorrência, uma mensagem de advertência é apresentada. Defina para uso o projeto com o nome `c02ex15`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let a: u8 = 1;
    let b: u8 = 2;

    println!("variável [A] = {}", a);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser compilado o programa, antes de este apresentar a mensagem `println!("variável [A] = {}", a);`, ocorre a indicação da mensagem de advertência:

```
warning: unused variable: `b`
--> src\main.rs:7:9
   |
 7 | let b: u8 = 2;
   | ^
   |
   = note: #[warn(unused_variables)] on by default
```

A mensagem de advertência `warning: unused variable: `b`` indica que a variável `B` definida na linha 7 a partir da coluna 9 é uma variável não usada e mostra o local na linha de código em que a ocorrência acontece. Além da advertência, é apresentada a mensagem `note: #[warn(unused_variables)] on by default`, indicando que a detecção da ocorrência de variável não utilizada (`unused_variables`) é uma ocorrência da definição-padrão do compilador a partir da ação interna da função `warn()`. A indicação de definição `#[<algo>]` estabelece que qualquer

atributo, exceto o indicado, permanece inalterado.

A definição-padrão de detecção de alguma ação que gera advertência pode ser desativada por meio da função `allow()`, que permite que certa ocorrência seja desativada. Neste sentido, para desativar a apresentação da mensagem de advertência relacionada à variável não usada, basta definir na primeira linha do código de programa a instrução de `#![allow(unused_variables)]`. O uso do atributo `#![<algo>]` estabelece que a permissão do atributo com a função `allow()` seja desconsiderada para `algo`.

O símbolo `!` (interrogação) após o símbolo `#` (tralha) indica que a permissão do atributo `unused_variables` deve ser desconsiderada, e dessa forma as mensagens de advertência no momento da compilação não são apresentadas. Observe a inserção da linha em negrito no código do programa do projeto `c02ex15`.

```
#![allow(unused_variables)]
use std::io;
use std::io::prelude::*;

fn main() {
    let a: u8 = 1;
    let b: u8 = 2;

    println!("variável [A] = {}", a);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

2.10 Exercícios de fixação

Desenvolver os programas em Rust dos problemas elencados de 1 até 13, ficando a cargo do professor selecionar a ordem e os problemas a serem resolvidos.

1. Ler uma temperatura em graus Celsius de 64 bits e apresentá-la convertida em graus Fahrenheit. A fórmula de conversão é $F = (9 * C + 160) / 5$, sendo F a temperatura em Fahrenheit e C a temperatura em Celsius.
2. Ler uma temperatura em graus Fahrenheit de 64 bits e apresentá-la convertida em graus Celsius. A fórmula de conversão é $C = ((F - 32) * 5) / 9$, sendo F a temperatura em Fahrenheit e C a temperatura em Celsius.
3. Ler dois valores inteiros de 32 bits para as variáveis A e B e efetuar a troca dos valores de forma que a variável A passe a ter o valor da variável B e a variável B passe a ter o valor da variável A. Apresentar os valores após a efetivação do processamento da troca.
4. Elaborar um programa que calcule e apresente o valor do volume de uma caixa retangular a partir do uso de valores reais de 32 bits, utilizando a fórmula $\text{volume} = \text{comprimento} \times \text{largura} \times \text{altura}$.
5. Efetuar a leitura de um valor numérico inteiro positivo de 64 bits e apresentar o resultado do valor lido também inteiro positivo de 32 bits elevado ao quadrado.
6. Ler dois valores numéricos inteiros sinalizados de 32 bits (representados pelas variáveis A e B) e apresentar o resultado do quadrado da diferença do primeiro valor (variável A) em relação ao segundo valor (variável B).
7. Desenvolver um programa que leia três valores numéricos inteiros sinalizados de 64 bits (representados a partir das variáveis A, B e C) e apresente como resultado o valor da soma dos quadrados dos valores lidos.
8. Desenvolver um programa que leia três valores numéricos inteiros sinalizados de 32 bits (representados a partir das variáveis A, B e C) e apresente como resultado o valor do quadrado da soma dos três valores lidos.
9. Elaborar programa que leia o valor numérico correspondente ao salário atual (variável SA) de um trabalhador e também faça a leitura do valor

do percentual de reajuste (variável PR) a ser atribuído. Apresentar o valor do novo salário (variável NS). Fazer uso de valores reais de 32 bits.

10. Elaborar um programa que calcule e apresente o valor do resultado da área de uma circunferência (variável VOL). O programa deve solicitar a entrada do valor do raio da circunferência (variável R). Para a execução deste problema, utilize a fórmula $VOL = 3.14159265 * R^2$. Considerando “^” como expoente. Usar valores de 64 bits.
11. Elaborar um programa que leia dois valores numéricos reais positivos de 64 bits desconhecidos representados pelas variáveis A e B. Calcular e apresentar os resultados das quatro operações aritméticas básicas.
12. Elaborar um programa que leia dois valores numéricos inteiros positivos de 32 bits, os quais devem representar respectivamente a base e o expoente de uma potência. Calcule a potência e apresente o resultado obtido.
13. Criar um programa que leia um valor numérico inteiro sinalizado de 32 bits e apresente como resultado os seus valores sucessor e antecessor.

¹ Ponto flutuante (notação científica) é a maneira usada em computadores para representar digitalmente números reais (formado por um expoente – parte inteira, um ponto de separação e uma mantissa – parte fracionária), desta forma é possível cobrir uma extensa faixa numérica em um pequeno espaço de memória.

CAPÍTULO 3

Desvio condicional

Este capítulo aborda a ação de tomada de decisão em um programa, utilizando estruturas de decisão com condição; operadores relacionais e lógicos; desvio condicional simples; desvio condicional composto; desvio condicional sequencial; desvio condicional encadeado; operador condicional (ternário), estrutura de controle com múltipla escolha, aplicação do conceito de divisibilidade e detecção da validade da entrada de dados.

3.1 Condição e decisão

Para que um programa de computador possa tomar decisões, é necessário imputar uma condição. Não só os computadores agem segundo esse critério. Os seres humanos também tomam decisões com base em condições apresentadas a eles. Assim sendo, para tomar uma decisão é necessário haver uma condição.

Do ponto de vista computacional pode-se considerar decisão a ação lógica que conduz a uma escolha a partir da definição de certa condição, sendo condição a relação lógica estabelecida entre dois e somente dois elementos passíveis de avaliação, podendo-se avaliar as relações de variáveis com variáveis e de variáveis com constantes, considerando-se que constante pode ser a definição de qualquer valor. A relação lógica entre os elementos de uma condição é obtida com a utilização dos operadores relacionais, de acordo com a Tabela 3.1.

Tabela 3.1 – Operadores relacionais

Símbolo	Significado	Prioridade
>	maior que	1
<	menor que	1
>=	maior ou igual que	1
<=	menor ou igual que	1
==	igual a	2
!=	diferente de (não igual)	2

A partir do exposto é possível considerar as relações como condições válidas entre variáveis e variáveis ou variáveis e constantes, como: $A == B$, $A != B$, $A > B$, $A < B$, $A >= B$, $A <= B$ ou $A == 5$, $A != 5$, $A > 5$, $A < 5$, $A >= 5$, $A <= 5$. Não há possibilidade de nem sentido em estabelecer uma relação lógica baseada apenas na definição de constantes.

3.2 Desvios condicionais

O desvio condicional está associado às decisões definidas em um programa de computador. Uma decisão a ser tomada pode ser verdadeira ou falsa. Se for verdadeira, é uma ação que pode ser executada; se for falsa, pode determinar outra ação a ser ou não executada. Portanto, um desvio condicional pode ser simples ou composto.

3.2.1 Desvio condicional simples

Um desvio condicional será simples quando houver uma condição que desvie a execução do programa caso o resultado lógico avaliado da condição seja verdadeiro. Se o resultado lógico for falso, nada acontecerá – o programa simplesmente seguirá o seu fluxo de execução.

Um desvio condicional simples é definido, na linguagem de programação Rust, por meio da instrução `if`, que tem a seguinte sintaxe:

```
if <condição> {
    <instrução 1 executada quando condição for verdadeira>;
    <instrução 2 executada quando condição for verdadeira>;
}
```

```
    <instrução N executada quando condição for verdadeira>;  
}  
<instrução executada após condição ser verdadeira e/ou falsa>;
```

Para exemplificar a utilização de decisão simples em um programa, considere a necessidade de desenvolver um programa que leia dois valores numéricos inteiros de 8 bits positivos, some-os e apresente o resultado, caso o valor somado seja maior que 10. Para isso, na linha de comando dentro da pasta/diretório `estudo`, crie o projeto `c03ex01`.

```
use std::io;  
use std::io::prelude::*;  
  
fn main() {  
    let mut valor_a = String::new();  
    let mut valor_b = String::new();  
  
    let a: u8;  
    let b: u8;  
    let r: u8;  
  
    print!("Entre valor <A>: ");  
    io::stdout().flush().unwrap();  
    io::stdin().read_line(&mut valor_a).unwrap();  
    a = valor_a.trim().parse::<u8>().unwrap();  
  
    print!("Entre valor <B>: ");  
    io::stdout().flush().unwrap();  
    io::stdin().read_line(&mut valor_b).unwrap();  
    b = valor_b.trim().parse::<u8>().unwrap();  
    r = a + b;  
  
    println!();  
  
    if r > 10 {  
        println!("Resultado = {}", r);  
    }  
    println!();  
    print!("Tecle <Enter> para encerrar...");
```

```

io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Após a definição dos tipos de variáveis, é solicitada a leitura dos dois valores implicados pelas variáveis A e B. Depois, é efetuada a adição dos valores das variáveis A e B, o que é implicado pela variável R. Nesse ponto, por meio de uma condição definida junto ao comando `if`, o programa verifica se é possível apresentar o resultado da adição; caso seja, o resultado será maior que 10; se não for, o programa não apresentará absolutamente nada.

3.2.2 Desvio condicional composto

Anteriormente, foi visto como usar a instrução `if` simples para avaliar determinada condição (desvio condicional simples). Agora, será explicado como usar a instrução `if... else`.

Numa instrução `If... else`, se a condição for *verdadeira*, será executada a instrução posicionada entre as instruções `if` e `else`. Se a condição for *falsa*, será executada a instrução que estiver logo depois de `else`, como é indicado na seguinte sintaxe:

```

if <condição> {
    <instrução 1 executada quando condição for verdadeira>;
    <instrução 2 executada quando condição for verdadeira>;
    <instrução N executada quando condição for verdadeira>;
} else {
    <instrução 1 executada quando condição for falsa>;
    <instrução 2 executada quando condição for falsa>;
    <instrução N executada quando condição for falsa>;
}
<instrução executada após condição ser verdadeira ou falsa>;

```

Como exemplo, considere um programa que faça a leitura de dois valores numéricos inteiros de 32 bits sinalizados, processe a multiplicação dos valores fornecidos e, caso o resultado obtido for maior ou igual a 20, apresente o valor da multiplicação somado com 5. Se o resultado da multiplicação não for maior ou igual a 20, este deverá ter 7 subtraído dele.

Para isso, crie o projeto c03ex02.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut valor_a = String::new();
    let mut valor_b = String::new();

    let a: i32;
    let b: i32;
    let r: i32;

    print!("Entre valor <A>: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor_a).unwrap();
    a = valor_a.trim().parse::<i32>().unwrap();

    print!("Entre valor <B>: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor_b).unwrap();
    b = valor_b.trim().parse::<i32>().unwrap();

    r = a * b;

    println!();

    if r >= 20 {
        println!("Resultado = {}", r + 5);
    } else {
        println!("Resultado = {}", r - 7);
    }
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao fazer um teste de execução do programa, se forem fornecidos os valores 10 e 11, o resultado apresentado será 26. Caso sejam fornecidos os valores

19 e 9, o resultado será 12.

3.3 Operadores lógicos

Às vezes, é necessário trabalhar com o relacionamento de duas ou mais condições ao mesmo tempo e na mesma instrução `if` para que seja tomada uma única decisão, usando-se testes condicionais múltiplos. Para esses casos, usam-se operadores lógicos, também conhecidos como *operadores booleanos*. Os operadores lógicos são três: `&&` (e – operador de conjunção), `||` (ou – operador de disjunção inclusiva) e `!` (não – operador de negação). Em alguns casos, o uso de operadores lógicos evita muitas instruções `if` encadeadas.

Os operadores lógicos `&&` e `||` permitem que seja usada mais de uma condição para a tomada de uma única decisão. Já o operador lógico `!` tem por finalidade a negação do estado lógico de uma condição.

Para usar adequadamente os operadores lógicos em expressões lógicas é necessário levar em consideração a sua ordem de precedência. A Tabela 3.2, de precedência de operadores lógicos, exhibe a ordem hierárquica de execução dos operadores lógicos que podem ser utilizados em uma expressão lógica.

Tabela 3.2 – Precedência de uso dos operadores lógicos

Operador	Operação	Precedência
<code>! (not)</code>	negação	1
<code>&& (and)</code>	conjunção	2
<code> (or)</code>	disjunção	3

A tabela de precedência dos operadores lógicos indica a prioridade de execução em uma expressão lógica. O operador `!` é o que tem maior nível de prioridade, portanto deve ser executado primeiro. Num segundo momento, tem-se o operador lógico `&&`, que tem médio nível de prioridade, e, por último, o operador `||`, com baixo nível de prioridade.

Por exemplo, a expressão lógica `(A == B) && ! (A > 5)` deve ser avaliada a

partir de ! (A > 5), e somente depois de se obter seu resultado é que se pode avaliar o restante da expressão lógica.

3.3.1 Operador lógico de conjunção

O operador lógico de conjunção **&&** é utilizado quando dois ou mais relacionamentos lógicos precisam ser verificados. A Tabela 3.3 mostra quando o resultado lógico é verdadeiro.

Tabela 3.3 – Tabela verdade para o operador lógico de conjunção

Condição 1	Condição 2	Resultado
falsa	falsa	falso
verdadeira	falsa	falso
falsa	verdadeira	falso
verdadeira	verdadeira	verdadeiro

O operador **&&** faz com que determinada operação seja executada somente se todas as condições mencionadas forem simultaneamente verdadeiras.

O programa seguinte mostra, com a utilização do operador de conjunção **&&**, que a mensagem **0 valor está entre 20 e 90** somente será apresentada se o valor numérico inteiro fornecido de 32 bits para a variável **NÚMERO** estiver entre 20 e 90. Qualquer valor fornecido fora da faixa definida apresenta a mensagem **"0 valor não está entre 20 e 90"**. Para isso, crie o projeto **c03ex03**.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut valor = String::new();

    let numero: i32;

    print!("Entre um valor numérico inteiro: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).unwrap();
```

```

    numero = valor.trim().parse::<i32>().unwrap();

    println!();

    if numero >= 20 && numero <= 90 {
        println!("O valor está entre 20 e 90.");
    } else {
        println!("O valor não está entre 20 e 90.");
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao executar o programa, efetue a entrada de valores dentro e fora da faixa para verificar a funcionalidade da tomada de decisão com o uso do operador lógico de conjunção.

3.3.2 Operador lógico de disjunção

O operador lógico de disjunção inclusiva `||` é utilizado quando dois ou mais relacionamentos lógicos precisam ser verificados. A Tabela 3.4 mostra quando o resultado lógico é verdadeiro.

Tabela 3.4 – Tabela verdade para o operador lógico de disjunção

Condição 1	Condição 2	Resultado
falsa	falsa	falso
verdadeira	falsa	verdadeiro
falsa	verdadeira	verdadeiro
verdadeira	verdadeira	verdadeiro

O operador `||` fará com que uma operação seja executada se pelo menos uma das condições mencionadas gerar um resultado lógico verdadeiro.

O programa seguinte lê um valor numérico inteiro de 8 bits referente aos

códigos 1, 2 ou 3. Qualquer outro valor deve apresentar a mensagem "Código inválido". Se o valor estiver correto, o valor do código deverá ser apresentado por extenso. Para isso, crie o projeto `c03ex04`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut cod = String::new();

    let codigo: i8;

    print!("Entre o código de acesso: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut cod).unwrap();
    codigo = cod.trim().parse::<i8>().unwrap();

    println!();

    if codigo == 1 || codigo == 2 || codigo == 3 {
        if codigo == 1 {
            println!("Foi acionado o código: um.");
        }
        if codigo == 2 {
            println!("Foi acionado o código: dois.");
        }
        if codigo == 3 {
            println!("Foi acionado o código: três.");
        }
    } else {
        println!("Código inválido.");
    }
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao executar o programa, insira os valores de códigos e faça diversas

entradas válidas e inválidas para verificar a funcionalidade da tomada de decisão com o operador lógico de disjunção.

3.3.3 Operador lógico de negação

O operador lógico de negação **!** é utilizado quando é necessário estabelecer a inversão do resultado de uma condição que deverá ser obrigatoriamente definida dentro de parênteses. O operador de negação **!** inverte o estado lógico da condição. Assim, se a condição for verdadeira, tornar-se-á falsa; e se a condição for falsa, tornar-se-á verdadeira. A Tabela 3.5 mostra quando o resultado lógico é verdadeiro.

Tabela 3.5 – Tabela verdade para o operador lógico de negação

Condição	Resultado
verdadeira	falso
falsa	verdadeiro

O exemplo seguinte mostra, com a utilização do operador de negação **!**, que o programa solicita a entrada de um valor numérico inteiro de 64 bits sinalizado e apresenta esse valor caso este não seja maior que 10. Se a condição for verdadeira, será apresentado o valor; se for falsa, será apresentada a mensagem "Valor inválido". Para isso, crie o projeto `c03ex05`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut valor = String::new();

    let vlr: i64;

    print!("Entre valor numérico inteiro: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).unwrap();
    vlr = valor.trim().parse::<i64>().unwrap();
```

```

println!();

if !(vlr >= 10) {
    println!("Valor informado = {}", vlr);
} else {
    println!("Valor inválido.");
}
println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao executar o programa, insira os valores anteriormente sugeridos para verificar a funcionalidade da tomada de decisão com o uso do operador lógico de negação.

3.4 Outros desvios condicionais

Desvios condicionais baseados em tomada de decisão possibilitam criar várias formas de desvios por meio dos conceitos anteriormente apresentados, podendo ser decisão simples e/ou decisão composta. Sendo assim, podem ser utilizadas relações de tomadas de decisão aninhadas ou sequenciais.

3.4.1 Desvio condicional encadeado

Para demonstrar a utilização dos operadores lógicos em um exemplo um pouco mais complexo, considere a necessidade de desenvolver um programa que leia três valores para os lados A, B e C de um triângulo. Esse programa deve verificar se os lados fornecidos formam realmente um triângulo. Se essa condição for verdadeira, deve ser indicado o tipo de triângulo formado: isósceles, escaleno ou equilátero.

É preciso considerar que triângulo é uma forma geométrica (polígono) composta de três lados, e cada lado é menor que a soma dos outros dois lados, ou seja, será um triângulo quando $A < B + C$, quando $B < A + C$ e quando

$C < A + B$.

Tendo certeza de que os valores informados para os três lados formam um triângulo, basta saber o seu tipo (isósceles, escaleno ou equilátero). Para isso é necessário considerar que um triângulo é isósceles quando tem dois lados iguais e um diferente, sendo $A=B$ ou $A=C$ ou $B=C$; é escaleno quando todos os lados são diferentes, sendo $A < B$ e $B < C$; e é equilátero quando todos os lados são iguais, sendo $A=B$ e $B=C$. Desse modo, crie o projeto c03ex06.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut lado_a = String::new();
    let mut lado_b = String::new();
    let mut lado_c = String::new();

    let a: f32;
    let b: f32;
    let c: f32;

    print!("Entre o lado <A>: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut lado_a).unwrap();
    a = lado_a.trim().parse::<f32>().unwrap();

    print!("Entre o lado <B>: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut lado_b).unwrap();
    b = lado_b.trim().parse::<f32>().unwrap();

    print!("Entre o lado <C>: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut lado_c).unwrap();
    c = lado_c.trim().parse::<f32>().unwrap();

    println!();
    if a < b + c && b < a + c && c < a + b {
```

```

    if a==b && b==c {
        println!("Triângulo equilátero.");
    } else {
        if a==b || a==c || c==b {
            println!("Triângulo isósceles.");
        } else {
            println!("Triângulo escaleno.");
        }
    }
} else {
    println!("Os valores fornecidos não formam um triângulo.");
}

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Esse exemplo demonstra, por meio do uso dos operadores lógicos, a capacidade de um programa definir se os valores fornecidos formam realmente um triângulo.

Se a condição for verdadeira, esse programa indicará o tipo de triângulo formado. Se forem fornecidos, respectivamente, os valores 8, 2 e 3 para os lados, representados pelas variáveis A, B e C, aparecerá a mensagem "Os valores fornecidos não formam um triângulo", pois a primeira condição `if a<b+c && b<a+c && c<a+b` resulta o valor lógico falso, uma vez que 8 é maior que a soma de 2 e 3.

Caso sejam fornecidos os valores 8, 8 e 2, a primeira instrução `if` será verdadeira, indicando que os lados fornecidos formam um triângulo. Apesar de o lado A ser igual ao lado B, mas este não ser igual ao lado C, o triângulo formado não é equilátero. A segunda instrução `if` tem seu resultado lógico como falso, sobrando a verificação da condição para a terceira instrução `if`, que indica o triângulo como isósceles, pois o lado A é igual ao lado B.

3.4.2 Desvio condicional sequencial

Para demonstrar o uso de desvios condicionais sequenciais, será criado um programa que simula uma simples calculadora e que faz a leitura de apenas dois valores reais. O usuário escolhe a operação e o programa apresenta o resultado da operação escolhida. Para isso, crie o projeto `c03ex07`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut valor_a = String::new();
    let mut valor_b = String::new();
    let mut escolha = String::new();

    let a: f32;
    let b: f32;
    let mut r: f32 = 0.;
    let opcao: u8;

    print!("Entre valor <A>: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor_a).unwrap();
    a = valor_a.trim().parse::<f32>().unwrap();

    print!("Entre valor <B>: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor_b).unwrap();
    b = valor_b.trim().parse::<f32>().unwrap();

    println!();
    println!("[1] - Adição");
    println!("[2] - Subtração");
    println!("[3] - Multiplicação");
    println!("[4] - Divisão");

    println!();
```

```

print!("Escolha uma opção: ");
io::stdout().flush().unwrap();
io::stdin().read_line(&mut escolha).unwrap();
opcao = escolha.trim().parse:::<u8>().unwrap();

if opcao == 1 {
    r = a + b;
}
if opcao == 2 {
    r = a - b;
}
if opcao == 3 {
    r = a * b;
}
if opcao == 4 {
    if b == 0. {
        r = 0.;
    } else {
        r = a / b;
    }
}
if opcao >= 1 && opcao <= 4 {
    println!("\n0 resultado da operação equivale a: {:.2}", r);
} else {
    println!("\n0 opção inválida.");
}
println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Execute alguns testes com valores inteiros e pontos flutuantes. Na decisão `if (opcao == 4)`, encontra-se uma segunda decisão encadeada, que verifica se o valor da variável `B` é `0` (zero). Sendo essa condição verdadeira, a variável `R` recebe o valor `0` (zero), que é apresentado como resposta. Essa decisão foi usada para evitar que uma mensagem de erro fosse exibida quando da divisão de um número por zero.

A definição da variável `R` é feita com a instrução `let mut r: f32 = 0.;`, que determina a variável `R` como sendo mutável e, dessa forma, pode receber diversos valores no mesmo programa. Uma variável mutável que receba diversos valores em um programa deve ser primeiro inicializada na linguagem Rust, por isso está sendo usado o valor `0.` (zero do tipo real; note a definição do ponto decimal após o valor zero). Caso a variável receba apenas um valor na execução do programa, esta não precisará ser inicializada como mutável, como foi feito em outros programas já apresentados.

O programa faz a verificação da validade da opção fornecida no menu. Nesse caso, se for fornecido um valor abaixo de 1 ou acima de 4 como ação de erro, será apresentada a mensagem "Opção inválida".

3.4.3 Operador condicional

A linguagem Rust não contém especificamente o recurso operador condicional, conhecido popularmente pela alcunha de *operador ternário*, como se encontra nas linguagens da família C. Apesar de Rust ter uma sintaxe próxima à sintaxe da família C, a linguagem se diferencia em diversos aspectos.

Muitos programadores habituados ao uso do recurso de operador ternário poderão franzir a testa achando que a linguagem não é tão interessante quanto se pensava. Se Rust não fosse interessante, não haveria com certeza despertado o interesse da Mozilla, e não teria sido usada para escrever um sistema operacional (Redox OS). O fato é que a linguagem Rust não precisa de um operador ternário quando efetua o mesmo tipo de ação diretamente com seu comando `if... else`. Em Rust é possível implicar em uma variável o valor tratado por uma instrução `if... else`.

Para demonstrar o uso de operação condicional, considere um programa que peça a leitura de três valores e apresente como resultado o maior valor fornecido. Para isso, crie o projeto `c03ex08`.

```
use std::io;
use std::io::prelude::*;
```

```

fn main() {
    let mut valor_a = String::new();
    let mut valor_b = String::new();

    let a: i32;
    let b: i32;
    let r: i32;

    print!("Entre valor <A>: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor_a).unwrap();
    a = valor_a.trim().parse::<i32>().unwrap();

    print!("Entre valor <B>: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor_b).unwrap();
    b = valor_b.trim().parse::<i32>().unwrap();

    r = if a > b {a} else {b};
    println!("\n0 maior valor informado é: {}", r);
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao executar o programa, experimente fornecer valores diferentes, como 7 para a variável A e 3 para a variável B, ou 2 para a variável A e 8 para a variável B. Experimente, também, fornecer valores iguais.

3.5 Controle com múltipla escolha

Ao trabalhar com grande quantidade de desvios condicionais encadeados ou sequenciais, pode-se dificultar a interpretação do código. Há uma forma mais adequada e eficiente de atender a esse tipo de situação, que é a estrutura de controle com múltipla escolha, denominada estrutura `match`, cuja sintaxe é:


```

match <variável> {
    1 => <operação específica 1>,
    2 => <operação específica 2>,
    3 => <operação específica 3>,
    4 | 5 => <operação específica 4>,
    6 .. 9 => <operação específica N>,
    _ => <operação padrão>
}

```

em que:

<variável> nome da variável a ser controlada na decisão;

1, 2, 3, _ conteúdo da variável a ser verificado;

<operação> execução de alguma ação específica.

O comando `match` é um padrão operacional que estabelece, por meio de uma seta para à direita (`=>`), chamada de *matcher*, a correspondência de determinada ação a ser executada por meio de uma expressão que ocorre entre a variável usada e o valor de cada linha do bloco do comando `match`. O conteúdo *variável* pode assumir algumas configurações de padrões válidos definidos e encontrados como 1, 2 e 3 para executar determinada ação; ou usar o operador `pipe` ao estabelecer que a ação seja executada caso um ou outro padrão seja usado, como 4 ou 5; ou ainda usar uma faixa de valores para a ação de um padrão, como indicar a seleção dos padrões de 6 a 9.

Para exemplificar essa estrutura, será criado um programa bem simples que pede como entrada um valor numérico inteiro de 8 bits positivo que esteja entre 1 e 12. Em seguida, o programa deve apresentar o nome do mês correspondente ao valor informado. Caso seja informado como mês um valor inexistente, o programa deve avisar ao usuário o erro cometido. Para isso, crie o projeto `c03ex09`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let mut mes = String::new();

```

```
let m: u8;

print!("Entre um numero equivalente a um MES: ");
io::stdout().flush().unwrap();
io::stdin().read_line(&mut mes).unwrap();
m = mes.trim().parse::<u8>().unwrap();

println!();

if m == 1 {
    println!("Janeiro.");
}
if m == 2 {
    println!("Fevereiro.");
}
if m == 3 {
    println!("Março.");
}
if m == 4 {
    println!("Abril.");
}
if m == 5 {
    println!("Maio.");
}
if m == 6 {
    println!("Junho.");
}
if m == 7 {
    println!("Julho.");
}
if m == 8 {
    println!("Agosto.");
}
if m == 9 {
    println!("Setembro.");
}
if m == 10 {
    println!("Outubro.");
}
```

```

    if m == 11 {
        println!("Novembro.");
    }
    if m == 12 {
        println!("Dezembro.");
    }
    if m < 1 || m > 12 {
        println!("Mês inválido.");
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Após inserir os valores dos meses, a cada execução do programa, e entrar valores fora da faixa de 1 a 12 e ver os resultados apresentados, considere a próxima versão do programa utilizando o comando `match`. Para isso, crie o projeto `c03ex10`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let mut mes = String::new();
    let m: u8;
    print!("Entre um numero equivalente a um MES: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut mes).unwrap();
    m = mes.trim().parse::<u8>().unwrap();

    println!();

    match m {
        1 => println!("Janeiro."),
        2 => println!("Fevereiro."),
        3 => println!("Março."),
        4 => println!("Abril."),
        5 => println!("Maio."),
    }
}

```

```

        6 => println!("Junho."),
        7 => println!("Julho."),
        8 => println!("Agosto."),
        9 => println!("Setembro."),
    10 => println!("Outubro."),
    11 => println!("Novembro."),
    12 => println!("Dezembro."),
    _ => println!("Mês inválido."),
}
println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Perceba que as linhas de instrução que continham as instruções `if` foram substituídas pelas linhas de instrução formadas pelo comando `match`.

O comando `match` permite que sejam usados mais de um valor para a escolha da ação a ser efetivada. Neste sentido, considere um programa que, ao receber os valores pares 2, 4, 6 ou 8, apresente a mensagem “Valor par”, e ao receber os valores 1, 3, 5, 7 ou 9, apresente a mensagem “Valor ímpar”. Para isso, crie o projeto `c03ex11`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let mut valor = String::new();
    let vlr: u8;

    print!("Entre um valor numérico: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).unwrap();
    vlr = valor.trim().parse::<u8>().unwrap();

    match vlr {
        1 | 3 | 5 | 7 | 9 => println!("Valor ímpar."),
        2 | 4 | 6 | 8 => println!("Valor par."),
        _ => println!("Valor inválido."),
    }
}

```

```

    }
    println!();
    print!("\nTecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao executar o programa, insira um valor de unidade entre 1 e 9. Observe que o acesso à apresentação das mensagens dependerá do valor fornecido.

O comando `match` para o controle com múltipla escolha pode ser operado a partir de faixas de valores estabelecidas com o operador (...) três pontos. Sendo assim, observe o seguinte trecho de código:

```

match vlr {
    1 ... 5 => println!("Menor ou igual a 5."),
    6 ... 9 => println!("Maior que 5."),
    _ => println!("Valor inválido."),
}

```

Neste caso, o valor da variável `vlr` será avaliado e, se fornecido um valor entre a faixa de 1 a 5, surgirá a mensagem `Menor ou igual a 5`; se fornecidos valores entre 6 e 9, será apresentada a mensagem `Maior que 5`. Para visualizar esta ação sugere-se alterar o programa do projeto `c03ex11` com esse trecho de código no mesmo local do trecho de código definido para a apresentação das mensagens de identificação de valores pares ou ímpares.

Outro detalhe a ser considerado no uso do comando `match` é a possibilidade de estabelecer valores para checagem do tipo `char` desde que sejam delimitados entre aspas simples.

3.6 Divisibilidade

Divisibilidade é a qualidade do que é divisível. Nesse contexto, dois conceitos devem ser conhecidos e entendidos pelos programadores de computador: múltiplos e divisores de números naturais. Entende-se por número natural um valor numérico que seja inteiro e positivo.

Múltiplos são os resultados obtidos da multiplicação de dois números naturais; e divisores são números naturais que dividem outros números

naturais com o objetivo de gerar um resultado exato de divisão, ou seja, obter resto de divisão sempre zero. Quando o resto de uma divisão de números naturais é igual a zero, tem-se divisibilidade, ou seja, resultado de divisão exata. Perceba que valores do tipo f32 ou f64 não entram nessa abordagem, apenas os valores de tipo inteiro.

Pelo fato de todo número natural ser múltiplo de si mesmo, uma maneira de trabalhar esses valores pode se dar por meio das operações de cálculo com divisores. Em uma divisão, existem alguns termos que precisam ser conhecidos: dividendo (valor que será dividido), divisor (valor que divide o dividendo), quociente (resultado da divisão do dividendo pelo divisor) e resto (valor que sobra da divisão). A Figura 3.1 mostra o processo de divisão de dois números naturais.

De acordo com a Figura 3.1, fica fácil deduzir a forma de obter a divisibilidade de um dividendo (valor 5) por um divisor (valor 2). Use a expressão matemática $\text{Resto} = \text{Dividendo} - \text{Divisor} \cdot \text{Quociente}$ para obter a divisibilidade, na qual as variáveis *Dividendo*, *Divisor* e *Quociente* serão substituídas por valores inteiros positivos. Sendo assim, considere os valores de dividendo 5 e divisor 2 na expressão matemática $\text{Resto} = 5 - 2 \cdot 2$. Basta, primeiro, multiplicar $2 \cdot 2$ para obter o valor 4, que será subtraído do valor 5 e resultará no valor 1 para a variável *Resto*, ou seja, *Resto* é diferente de zero. Nesse caso, não ocorreu a divisibilidade do valor 5 pelo valor 2 porque o valor 2 não é múltiplo do valor 5.

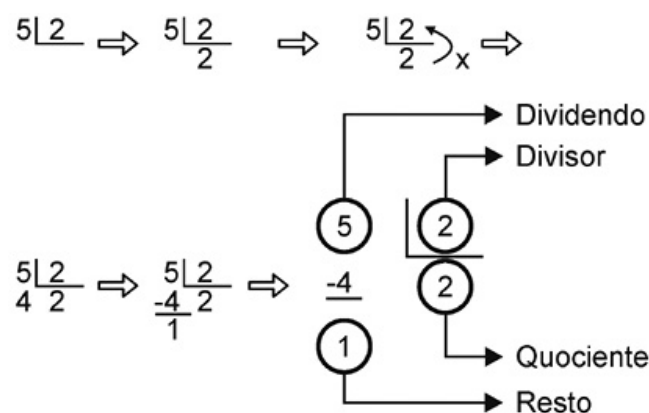


Figura 3.1 – Exemplo de divisão de dois números naturais.

A expressão matemática $\text{Resto} = \text{Dividendo} - \text{Divisor} \cdot \text{Quociente}$ é obtida

a partir da equação matemática:

$$r = a - n \left\lfloor \frac{a}{n} \right\rfloor$$

Em que a variável R representa o resto da divisão; a variável A , o dividendo; e a variável N , o divisor (KNUTH, 1972). Observe o cálculo do quociente com o uso da função *parte inteira inferior*, que obtém o resultado inteiro da divisão do dividendo a pelo divisor N . A função *parte inteira* tem o objetivo de retornar a parte inteira (o expoente) de um número real, desconsiderando as casas decimais (a mantissa) desse número.

A função *parte inteira* pode ser representada de duas formas: *parte inteira inferior*, também denominada *chão* ou *piso*, e *parte inteira superior*, também chamada de *teto*. Na forma *inferior*, ela retorna o expoente do valor, desconsiderando toda a mantissa desse valor. Já na forma *superior*, ela retorna o arredondamento do valor inteiro mais próximo. As funções *parte inteira inferior* e *parte inteira superior* são graficamente representadas como:

$\lceil x \rceil$ Parte inteira superior

$\lfloor x \rfloor$ Parte inteira inferior

Do ponto de vista computacional, a expressão matemática para obter a divisibilidade dos valores deve ser convertida em uma expressão aritmética. Assim sendo, a expressão matemática $\text{Resto} = \text{Dividendo} - \text{Divisor} \cdot \text{Quociente}$ será escrita na linguagem Rust como `resto = dividendo - divisor * (dividendo / divisor)`, levando-se em consideração que as variáveis `RESTO`, `DIVIDENDO` e `DIVISOR` são do tipo de dado inteiro. Nesse caso, o operador aritmético de divisão “/” gera automaticamente um quociente do tipo inteiro. Considerando um programa que verifique se o valor 5 é divisível pelo valor 2, este seria escrito em linguagem Rust como `resto = 5 - 2 * (5 / 2);`.

A expressão aritmética $\text{Resto} = \text{Dividendo} - \text{Divisor} \cdot (\text{Dividendo} / \text{Divisor})$ é uma operação algorítmica genérica que pode ser facilmente adaptada a qualquer linguagem de programação. Caso queira simplificar

esse tipo de operação em linguagem Rust, é possível usar o operador de resto de divisão (operador de módulo “%”). Nesse caso, a expressão `resto = 5 - 2 * (5 / 2);` pode ser escrita como `resto = 5 % 2;`.

A título de demonstração do processo de divisibilidade, considere um programa que efetue a leitura de um valor numérico inteiro e apresente esse valor caso seja divisível por 4 e 5. Não sendo divisível por 4 e 5, o programa deve apresentar a mensagem "Valor não é divisível por 4 e 5". Para resolver esse problema, é necessário, além do operador lógico `&&`, verificar se a condição do valor lido é ou não divisível por 4 e 5.

A seguir, são apresentadas duas soluções para o problema proposto. A primeira usa a forma genérica de solução do problema; a segunda, o operador de módulo de divisão.

Para resolver o problema de forma genérica, crie o projeto do primeiro exemplo com o nome `c03ex12`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut valor = String::new();

    let n: i64;
    let r4: i64;
    let r5: i64;

    print!("Entre um valor numérico: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).unwrap();
    n = valor.trim().parse::<i64>().unwrap();

    r4 = n - 4 * (n / 4);
    r5 = n - 5 * (n / 5);

    if r4 == 0 && r5 == 0 {
        println!("{}", n);
    } else {
```



```

        println!("Valor não é divisível por 4 e 5.");
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Observe nesse primeiro exemplo as expressões aritméticas de cunho genérico $r4 = n - 4 * (n / 4)$ e $r5 = n - 5 * (n / 5)$, para obtenção do valor da divisibilidade e do resto da divisão do valor da variável n pelos valores dos divisores 4 e 5, respectivamente. A instrução de tomada de decisão usa a condição $r4 == 0 \ \&\& \ r5 == 0$, que será verdadeira se ambas as condições forem verdadeiras.

Para resolver o problema de forma a aplicar o operador de módulo, crie o projeto do primeiro exemplo com o nome `c03ex13`.

```

use std::io;
use std::io::prelude::*;
fn main() {
    let mut valor = String::new();

    let n: i64;
    let r4: i64;
    let r5: i64;

    print!("Entre um valor numérico: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).unwrap();
    n = valor.trim().parse::<i64>().unwrap();

    r4 = n % 4;
    r5 = n % 5;

    if r4 == 0 && r5 == 0 {
        println!("{}", n);
    } else {
        println!("Valor não é divisível por 4 e 5.");
    }
}

```

```

    }
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Observe nesse segundo exemplo o operador de módulo de divisão $r4 = n \% 4$ e $r5 = n \% 5$, para obtenção do valor da divisibilidade e do resto da divisão do valor da variável n pelos valores dos divisores 4 e 5, respectivamente.

3.7 Validação do fluxo de entrada

No capítulo anterior foi usado para a entrada de dados alfanumérico o comando `if let` (se deixado), que é uma maneira de tomada de decisão que mistura, de certa forma, as ações do comando `if` com as ações do comando `match` para realizar uma ação imediatamente após uma ocorrência anterior. No capítulo 2 o comando `if let` é usado para realizar uma ação ao se detectar numa sequência alfanumérica a ocorrência dos caracteres especiais `\n` (salta linha) e `\r` (retorno do cursor) quando a tecla `<Enter>` é acionada.

Se essa verificação não for feita, a entrada alfanumérica manterá os caracteres especiais `\n` e `\r` na sequência alfanumérica informada, ocasionando um salto de linha e um retorno de cursor a mais do que normalmente se deseja.

Para fazer uso do comando `match` na validação do fluxo de entrada de dados, considere um programa que deva receber a entrada e a apresentação de um valor inteiro sinalizado de 8 bits, ou seja, a entrada de valores numéricos entre a faixa de -128 até 127. Para isso, crie o projeto `c03ex14`.

```

use std::io;
use std::io::prelude::*;

fn main() {

```

```

let mut valor = String::new();
let vlr: i8;

println!("Informe um valor inteiro: ");
io::stdout().flush().unwrap();
io::stdin().read_line(&mut valor).unwrap();
vlr = valor.trim().parse::<i8>().unwrap();

println!("Valor informado: {}", vlr);

println!();
println!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Execute o programa algumas vezes e entre valores na faixa de -128 a 127. Depois entre valores fora dessa faixa, valores reais e valores alfanuméricos. Quando uma entrada é feita de forma incorreta, ocorre um erro na recepção do valor, o programa é abortado da memória e uma mensagem de erro é indicada, como por exemplo a tentativa da entrada de um valor real:

```

thread 'main' panicked at 'called `Result::unwrap()` on an `Err` value:
ParseInt
Error { kind: InvalidDigit }', src\libcore\result.rs:860:4
note: Run with `RUST_BACKTRACE=1` for a backtrace.

```

Esta mensagem informa que a thread, ou seja, a função `main()`, entrou em pânico ao chamar a estrutura de enumeração (assunto que será tratado mais adiante nesta obra) interna de tipo de erro `Result::unwrap()` por ocorrer um erro (`Err`) em um dos caracteres do valor informado ao ser convertido este valor para um valor inteiro (`ParseInt`). Este erro ocorre ao se localizar o ponto decimal como caractere inválido (`InvalidDigit`). A mensagem de erro apresenta outras informações que não são, por ora, importantes.

Com pequenas alterações no código do programa anterior é possível, por meio do comando `match`, fazer a apresentação de uma mensagem de erro

mais elegante e manter a execução do programa sem que este seja abortado da memória. Para isso, crie o projeto `c03ex15`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut valor = String::new();

    print!("Informe um valor inteiro: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).unwrap();

    match valor.trim().parse::<u8>() {
        Ok(vlr) => println!("Valor informado: {}", vlr),
        Err(..) => println!("Não entrou valor inteiro.")
    };

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Execute o programa algumas vezes e entre valores na faixa de -128 a 127. Depois entre valores fora dessa faixa, valores reais e valores alfanuméricos, e veja a apresentação da mensagem `Não entrou valor inteiro`. Note que, depois da apresentação da mensagem, o programa se mantém em execução, sem ser abortado.

O comando `match` (baseado em uma estrutura da linguagem `Result` que é uma *pattern matching* – correspondência de padrões) usa o trecho `valor.trim().parse::<u8>()` para retornar o conteúdo da variável `VALOR`. Se o valor for correto, a função `Ok()` será executada. Caso contrário, se ocorrer algum erro, a função `Err()` será executada. As funções `Ok()` e `Err()` são ações executadas a partir do tipo `Result`, que é associado às instruções apontadas pelo comando `match`.

A função `Ok()` usa como parâmetro a indicação de uma variável – neste

caso, variável `vlr` (que não necessita ser declarada) – para receber o conteúdo validado. Sendo assim, se o conteúdo informado é correto a função `Ok()` pega o valor armazenado na função `Ok()` e faz sua devolução junto à variável indicada, transferindo na sequência por meio do operador `=>` o fluxo de ação para a instrução a sua direita; neste caso, `println!("Valor informado: {}", vlr)`.

A função `Err()` é disparada quando a entrada é incorreta, e pode ser usada sem a indicação de parâmetro (ação sinalizada com os símbolos `..` – que indica que não há retorno de valor; pode também ser usado *underline*), direcionando o tratamento de sua ação para a instrução a sua direita; neste caso, identificada como `println!("Não entrou valor inteiro,")`.

O programa anterior faz uso da forma mais simples possível de validar ou não a entrada de um valor numérico. Apesar de funcional, não é um programa muito promissor para a entrada de valores que serão efetivados em cálculos. Para que o programa anterior se torne mais funcional é preciso considerar uma pequena alteração junto ao comando `match`.

O programa seguinte efetua a leitura de um valor inteiro sinalizado de 8 bits e apresenta este valor multiplicado por 2. Para isso, crie o projeto `c03ex16`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut valor = String::new();
    let vlr: u8;
    let r: u8;

    print!("Informe um valor inteiro: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).unwrap();

    vlr = match valor.trim().parse:::<u8>() {
        Ok(vlr) => vlr,
        Err(_) => 0
    };
    r = (vlr * 2) as u8;
    println!("Resultado: {}", r);
}
```

```

};

r = vlr * 2;

println!("Resultado = {}", r);

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Execute o programa algumas vezes e efetue a entrada de valores inteiros, reais e alfanuméricos para visualizar o comportamento do comando `match` atribuído à variável `vlr`. Quando um valor incorreto for fornecido, o resultado apresentado será `0`, lembrando que o programa poderia apresentar uma mensagem de erro em vez de fornecer como resposta o valor `0` para uma entrada inválida capturada pela função `Err()` que está sinalizada com o uso de um símbolo de *underline*. O *underline* indica que não há interesse em saber qual tipo de erro foi retornado. Dessa forma, o próximo programa efetua este requinte de exigência a partir da entrada de um valor e da apresentação do seu quadrado. Para isso, crie o projeto `c03ex17`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let mut valor = String::new();
    let r: u8;

    print!("Informe um valor inteiro: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).unwrap();

    if let Ok(vlr) = valor.trim().parse::<u8>() {
        r = vlr * 2;
        println!("Resultado = {}", r);
    }
}

```

```

    } else {
        println!("Não entrou valor inteiro.")
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Execute o programa e efetue a entrada de valores inteiros, reais e alfanuméricos. Observe a apresentação do resultado quando a entrada for válida e a apresentação da mensagem `Não entrou valor inteiro` quando a entrada for inválida.

3.8 Exercícios de fixação

Desenvolver os programas em Rust dos problemas elencados de 1 até 9, ficando a cargo do professor selecionar a ordem e os problemas a serem resolvidos.

1. Efetuar a leitura de dois valores numéricos inteiros sinalizados de 64 bits representados pelas variáveis A e B e apresentar o resultado da diferença do maior valor pelo menor valor.
2. Fazer a leitura dos valores de quatro médias escolares bimestrais de um aluno representadas pelas variáveis N1, N2, N3 e N4, todas de 32 bits. Calcular a média aritmética (variável MD) desse aluno e apresentar a mensagem “aprovado com média” se a média obtida for maior ou igual a 5; caso contrário, apresentar a mensagem “reprovado com média”. Informar também, após a apresentação das mensagens, o valor da média obtida pelo aluno.
3. Ler os valores de quatro médias escolares bimestrais de um aluno representadas pelas variáveis N1, N2, N3 e N4. Calcular a média aritmética (variável MD1) desse aluno e apresentar a mensagem “aprovado com média” se a média obtida for maior ou igual a 7; caso contrário, o programa deve solicitar a quinta nota (nota de exame,

representada pela variável NE) do aluno e calcular uma nova média aritmética (variável MD2) entre a nota de exame e a primeira média aritmética. Se o valor da nova média for maior ou igual a 5, apresentar a mensagem “aprovado em exame com média”; caso contrário, apresentar a mensagem “reprovado com média”. Informar também, após a apresentação das mensagens, o valor da média obtida pelo aluno.

4. Ler três valores inteiros não sinalizados de 32 bits representados pelas variáveis A, B e C e apresentar os valores lidos dispostos em ordem crescente.
5. Efetuar a leitura de quatro valores numéricos inteiros sinalizados de 64 bits representados pelas variáveis A, B, C e D. Apresentar apenas os valores que sejam divisíveis por 2 e 3.
6. Efetuar a leitura de quatro valores numéricos inteiros sinalizados de 64 bits representados pelas variáveis A, B, C e D. Apresentar apenas os valores que sejam divisíveis por 2 ou 3.
7. Ler cinco valores numéricos inteiros sinalizados de 32 bits (variáveis A, B, C, D e E), identificar e apresentar o maior e o menor valores informados, sem que seja feita a ordenação dos valores.
8. Fazer a leitura de um valor numérico inteiro sinalizado de 64 bits qualquer e apresentá-lo caso *não seja maior que 3*. Dica: para a solução deste problema, utilize apenas o operador lógico de negação.
9. Ler um número inteiro não sinalizado de 32 bits e multiplicá-lo por dois. Apresentar o resultado somente se o resultado for maior que 30.

CAPÍTULO 4

Laços

Este capítulo aborda um dos recursos mais importantes da atividade de programação para computadores que corresponde à capacidade de trechos de programas de poder repeti-los de forma iterativa ou interativa. Os laços (também conhecidos pelos termos *loopings*, *malhas* ou *ciclos*) possibilitam repetir determinado trecho de código de programa várias vezes. A ideia geral do conceito de laço decorre do fato de um laço ser um nó correção facilmente desatável a partir de uma ou mais alças. Dessa forma, é possível definir quantidades variadas de repetições, com laços determinísticos (quando se sabe o número de vezes que o laço deve ser executado – laço iterativo) e laços indeterminados (quando não se sabe quantas vezes o laço deve ser executado, dependendo de uma resposta positiva do usuário para fazê-lo – laços interativos). São vistos laços do tipo pré-teste com fluxo verdadeiro, pré-teste com fluxo falso, iterativo, seletivo, além de orientar o uso da técnica de consistência de entrada de dados para validar a entrada de dados fornecidos por um usuário.

4.1 Laço pré-teste

O laço pré-teste pode ser executado com fluxo condicional falso ou verdadeiro. Na linguagem Rust, esse tipo de laço é manipulado pelo comando `while`, que faz um teste lógico no início do laço, verificando se é possível executar o trecho de instruções subordinadas a ele. A forma-padrão para o laço `while` é a execução com fluxo condicional verdadeiro, e caso necessite efetuar um laço condicional com fluxo falso, torna-se necessário o uso do operador lógico `!`. Este tipo de laço, devido a sua característica operacional, pode ser usado para criar laços iterativos e

iterativos.

A estrutura de laço `while` tem seu funcionamento controlado por condição. É possível executar um conjunto de instruções enquanto a condição verificada permanecer válida. No momento em que essa condição não for válida, o processamento da rotina será desviado para fora do laço. A sintaxe dessa estrutura de laço corresponde a:

```
while [!(<condição>)] {  
    <instrução 1 enquanto condição for válida>;  
    <instrução 2 enquanto condição for válida>;  
    <instrução 3 enquanto condição for válida>;  
    <instrução N enquanto condição for válida>;  
}
```

O operador lógico de negação (!) é opcional e deve ser usado quando se quiser simular um laço do tipo pré-teste com fluxo condicional falso.

Como exemplo de uso de laço pré-teste com fluxo condicional verdadeiro, considere o programa seguinte, que mostra o resultado da tabuada de um número qualquer. Nesse caso, é necessário definir um contador para controlar a apresentação dos resultados da tabuada (um contador de 1 até 10), e, cada vez que o trecho almejado do programa for executado, esse contador deverá ser incrementado com 1. Para isso, crie o projeto `c04ex01`.

```
use std::io;  
use std::io::prelude::*;  
  
fn main() {  
    let mut tabuada = String::new();  
  
    let n: u8;  
    let mut i: u8;  
    let mut r: u8;  
  
    print!("Entre o valor da tabuada: ");  
    io::stdout().flush().unwrap();  
    io::stdin().read_line(&mut tabuada).unwrap();  
    n = tabuada.trim().parse::<u8>().unwrap();
```

```
println!();

i = 1;
while i <= 10 {
    r = n * i;
    println!("{:2} x {:2} = {:3}", n, i, r);
    i = i + 1;
}

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}
```

Assim que o programa é inicializado, solicita-se o valor para cálculo da tabuada, representado pela variável *N*. Em seguida, ajusta-se a apresentação dos dados da direita para a esquerda, que é a forma mais adequada de exibir os dados numéricos.

Na sequência, a variável utilizada para controle do número de execuções do laço é inicializada com o valor 1 (*i* = 1). Então, a instrução **while** (*i* <= 10) verifica a condição estabelecida, deduzindo que ela é verdadeira, pois o valor da variável *I*, que no momento é 1, é realmente menor ou igual a 10, e, enquanto for assim, deve processar o laço. Dessa forma, inicia-se a execução da rotina de instruções contidas no laço de repetição delimitado com a instrução **while**.

Dentro do laço, faz-se o cálculo da operação *r* = *n* * *i* e a posterior apresentação de cada valor de forma tabuada (a partir da direita). Assim que os valores são exibidos em formato de tabuada, o programa muda de linha. Em seguida, o programa encontra a linha com a instrução *i* = *i* + 1, indicando o acréscimo de 1 à variável contador. Observe que a variável *I* tem, nesse momento, o valor 1, e, somado a mais 1, ela passa a ter o valor 2, ou seja, *i* = *i* + 1. Sendo assim, *i* = 1 + 1, que resulta em *i* = 2.

Após a variável *I* estar carregada com o valor 2, o processamento do programa volta para a instrução **while** (*I* <= 10), que verifica a condição

da variável *i* novamente. Sendo a condição *verdadeira*, o bloco de código contido no laço será executado novamente. Quando o processamento do programa chegar à instrução *i = i + 1*, a variável *i* passará a ter valor 3. O programa vai processar novamente a rotina de instruções, passando o valor da variável *i* para 4, que será verificado, e, sendo menor ou igual a 10, a rotina será executada mais algumas vezes até atingir o valor 11, quando o laço será automaticamente encerrado.

 **Observação:** A programação de laços em linguagem Rust permite fazer uso de contadores na forma *i = i + 1*. No entanto é possível fazer uso do mesmo efeito com o estilo *i += 1*. Nos próximos exemplos, a forma *i = i + 1* não será mais utilizada.

Imagine que o usuário queira calcular a tabuada mais de uma vez. É difícil prever quantas vezes o usuário vai solicitar essa ação. Nesse caso, é necessário definir um segundo laço que seja executado enquanto a resposta do usuário para a execução do programa for SIM. Para isso, crie o projeto `c04ex02`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut tabuada = String::new();
    let mut resposta = String::new();

    let mut n: u8;
    let mut i: u8;
    let mut r: u8;
    let mut resp: char;

    resp = 'S';
    while resp == 'S' || resp == 's' {

        print!("Entre o valor da tabuada: ");
        io::stdout().flush().unwrap();
        io::stdin().read_line(&mut tabuada).unwrap();
        n = tabuada.trim().parse::<u8>().unwrap();
```

```

println!();

i = 1;
while i <= 10 {
    r = n * i;
    println!("{:2} x {:2} = {:3}", n, i, r);
    i += 1;
}

println!();
println!("Deseja continuar?");
print!("Tecle [S] para SIM / qualquer caractere para NAO: ");
io::stdout().flush().unwrap();
io::stdin().read_line(&mut resposta).unwrap();
resp = resposta.trim().parse::<char>().unwrap();

println!();
tabuada.clear();
resposta.clear();

}

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

No programa anterior, foi inserido um segundo laço que é controlado pela variável RESP, que, enquanto tiver valor igual a “S” ou “s”, executa o bloco de instruções subordinado a `while`. Nesse caso, o número de repetições da rotina é controlado pelo usuário e depende da informação fornecida para a variável RESP. Se o usuário responder “S” ou “s”, o programa vai continuar; se responder qualquer caractere, ele será encerrado.

Uma ação importante a ser considerada quando se opera com variáveis dentro de laços que serão usadas sucessivamente para a recepção de

valores, como é o caso das variáveis `TABUADA` e `RESPOSTA`: é necessário usar com essas variáveis a função `clear()` para fazer a limpeza de seu conteúdo antes de elas serem novamente usadas.

A codificação de programas de computador em linguagem Rust permite simplificar a escrita ao inicializar o valor de uma variável. No programa anterior, a variável `RESP` é definida como do tipo `char` e, antes de iniciar o laço, ela é atribuída com o valor `S` para dar início ao processamento do laço de repetição. Veja, em seguida, uma sugestão para simplificar a escrita do código do programa com o adendo da conversão para maiúsculo do caractere dado como resposta à continuidade ou não dos cálculos das tabuadas. Para isso, crie o projeto `c04ex0`:

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut tabuada = String::new();
    let mut resposta = String::new();

    let mut n: u8;
    let mut i: u8;
    let mut r: u8;
    let mut resp: char = 'S';
    while resp == 'S' {

        print!("Entre o valor da tabuada: ");
        io::stdout().flush().unwrap();
        io::stdin().read_line(&mut tabuada).unwrap();
        n = tabuada.trim().parse::<u8>().unwrap();

        println!();

        i = 1;
        while i <= 10 {
            r = n * i;
            println!("{:2} x {:2} = {:3}", n, i, r);
            i += 1;
        }
    }
}
```

```

    }

    println!();
    println!("Deseja continuar?");
    print!("Tecle [S] para SIM / qualquer caractere para NAO: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut resposta).unwrap();
    resp = resposta.to_uppercase().trim().parse:::<char>().unwrap();

    println!();
    tabuada.clear();
    resposta.clear();

}

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Após a inicialização da variável `RESP` com o seu tipo, simultaneamente é atribuído o valor da variável, que, nesse caso, é “s”. Essa forma de trabalho permite reduzir um pouco o número de linhas de código dos programas desenvolvidos.

A instrução `while resp == 'S'` manterá o laço em execução caso o conteúdo da variável `RESP` seja “s”. Note a função `to_uppercase()` junto à instrução `resp = resposta.to_uppercase().trim().parse:::<char>().unwrap();`, onde a resposta dada à variável `RESP` é, antes de ser atribuída à variável `RESP`, convertida para o formato de caractere maiúsculo. Na sequência, as demais funções são os métodos para remover espaços em branco, conversão da entrada de *string* para caractere (`char`). É aconselhável sempre usar a função `to_uppercase()` logo após a variável a ser convertida.

Como exemplo de uso de laço pré-teste com fluxo condicional falso, considere o programa seguinte, que mostra o resultado da tabuada de um

número qualquer. Para isso, crie o projeto `c04ex04`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut tabuada = String::new();

    let n: u8;
    let mut i: u8;
    let mut r: u8;

    print!("Entre o valor da tabuada: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut tabuada).unwrap();
    n = tabuada.trim().parse::<u8>().unwrap();

    println!();

    i = 1;
    while !(i > 10) {
        r = n * i;
        println!("{:2} x {:2} = {:3}", n, i, r);
        i += 1;
    }
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Neste exemplo, perceba o uso do operador `!` negando a condição (`i > 10`), simulando o uso de um laço pré-teste com fluxo executado para a condição falsa. No momento em que a condição se torna verdadeira, o laço é encerrado.

4.2 Laço seletivo

A linguagem Rust não contém na sua estrutura comando para a execução

de laço pós-teste com fluxo falso ou verdadeiro, como há em outras linguagens. A linguagem Pascal tem como laço pós-teste (do tipo fluxo falso) a instrução `repeat... until`, e as linguagens C e C++ dispõem do laço pós-teste (do tipo fluxo verdadeiro) a partir da instrução `do {... } while`. Isso pode preocupar alguns programadores. No entanto, Rust tem um dos melhores comandos para a execução de laços – o comando `loop`, que possibilita escrever laços com a definição do ponto de saída do laço de forma seletiva. Este requinte é encontrado nas linguagens *Structured BASIC* e *ADA*.

Em um laço do tipo seletivo, é possível escolher o local do programa para verificar a condição de encerramento do laço, a qual pode ser definida em qualquer posição de seu bloco de ação. Assim como os laços pré-teste e pós-teste, este laço permite a criação de ações iterativas e interativas.

A estrutura de laço `loop` tem seu funcionamento controlado por um condição que pode ser colocada em qualquer linha do bloco de ação do laço. É possível executar um conjunto de instruções até que a condição verificada seja verdadeira. No momento em que essa condição é verdadeira, o processamento do laço é encerrado. O laço `loop` pode ser usado para efetivação de ações iterativas e interativas. A sintaxe dessa estrutura de laço corresponde a:

```
loop {  
    <instrução 1 enquanto condição for válida>;  
    <instrução 2 enquanto condição for válida>;  
    <instrução 3 enquanto condição for válida>;  
    <instrução N enquanto condição for válida>;  
    if [!( <condição> )] { break; }  
}
```

A instrução `if condição { break; }` pode ser colocada em qualquer uma das linhas dentro do bloco de ação do comando `loop`.

O programa seguinte tem por finalidade apresentar o resultado de uma tabuada de um número qualquer gerenciado pela ação de laço do tipo seletivo. Neste sentido, são apresentados dois códigos que mostram o controle do laço na parte superior deste laço (simulando laço pré-teste) e

o controle inferior do laço (simulando laço pós-teste).

O exemplo a seguir caracteriza-se por fazer uso da ação de laço pós-teste com fluxo falso similar ao comando `repeat... until` da linguagem Pascal. Para isso, crie o projeto `c04ex05`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut tabuada = String::new();

    let n: u8;
    let mut i: u8;
    let mut r: u8;

    print!("Entre o valor da tabuada: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut tabuada).unwrap();
    n = tabuada.trim().parse::<u8>().unwrap();

    println!();

    i = 1;
    loop {
        r = n * i;
        println!("{:2} x {:2} = {:3}", n, i, r);
        i += 1;
        if i > 10 { break; }
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Depois que o programa for executado, o efeito de sua funcionalidade será idêntico ao dos programas exemplificados anteriormente. No entanto, nesta versão está em uso um estilo de laço pós-teste quando a decisão de

saída do laço está na parte inferior do laço.

O exemplo a seguir caracteriza-se por fazer uso da ação de laço pré-teste com fluxo verdadeiro similar ao comando `while` das linguagens C e C++. Para isso, crie o projeto `c04ex06`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut tabuada = String::new();

    let n: u8;
    let mut i: u8;
    let mut r: u8;

    print!("Entre o valor da tabuada: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut tabuada).unwrap();
    n = tabuada.trim().parse::<u8>().unwrap();

    println!();

    i = 1;
    loop {
        if !(i <= 10) { break; }
        r = n * i;
        println!("{:2} x {:2} = {:3}", n, i, r);
        i += 1;
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Depois que o programa for executado, o efeito de sua funcionalidade será idêntico ao dos programas exemplificados anteriormente. No entanto, nesta versão, está em uso um estilo de laço pré-teste quando a decisão de

saída do laço está na parte superior do laço.

De um ponto de vista mais prático, não há, na linguagem Rust, necessidade de fazer uso do laço `while`, uma vez que o laço `loop` atende a diversas necessidades de operação de trechos de programas que precisam ter suas instruções repetidas um determinado número de vezes.

4.3 Laço iterativo

Foram apresentadas duas formas de elaborar laços: uma usando o comando `while` e outra usando o comando `loop`. Com essas duas técnicas é possível elaborar rotinas que executem um laço determinado número de vezes com o uso de um contador (variáveis que executam algo determinado número de vezes) ou de uma variável que aguarde a resposta do usuário. Independentemente da forma escolhida, ela será conhecida como variável de controle.

Existe outra forma de facilitar o uso de contadores finitos sem as estruturas anteriores. Dessa forma, os laços `while` e `loop` passam a ser utilizados em *loopings* nos quais não se sabe de antemão o número de vezes que uma sequência de instruções deve ser executada. Os laços que têm um número finito de execuções podem ser processados por meio do laço `for`, cujo funcionamento é controlado pela variável de controle contador. O uso do comando `for` na linguagem Rust segue o estilo de sintaxe:

```
for <variável> in <início> .. <fim> {  
    <instrução 1>  
    <instrução 2>  
    <instrução N>  
}
```

O laço `for` é executado a partir da definição de uma *variável* (de tipo inteiro) que sofrerá a contagem na faixa de valores estabelecida entre o *início* e o *fim* da contagem. A variável do contador para o laço `for` não precisa ser qualificada antes de seu uso; basta apenas citar uma variável para ela ser aceita. Há um ponto de atenção: caso queira efetuar um laço de dez passagens, não use a definição da faixa de `1 .. 10`; use a definição 1

.. 11.

Para exemplificar o uso dessa estrutura de laço, será considerado como exemplo o programa de tabuada de um número qualquer utilizado anteriormente neste capítulo. Para isso, crie o projeto `c04ex07`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut tabuada = String::new();

    let n: u8;
    let mut r: u8;

    print!("Entre o valor da tabuada: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut tabuada).unwrap();
    n = tabuada.trim().parse::<u8>().unwrap();

    println!();

    for i in 1 .. 11 {
        r = n * i;
        println!("{:2} x {:2} = {:3}", n, i, r);
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Quando executado o programa, o conjunto de instruções situado dentro do laço `for` é executado dez vezes, pois a variável `i` (variável de controle inteira) é inicializada com valor 1 e é incrementada automaticamente com mais 1 até atingir o valor 11.

O laço `for` na linguagem Rust opera apenas laços crescentes executados com passo de crescimento sempre de 1 em 1. Caso queira realizar laços

decrecentes, use, preferencialmente, o laço `while` para essa finalidade. Mas, caso queira usar o laço iterativo `for` com contador decrescente, será necessário fazer uso do método `rev()` associado à definição da faixa de valor de iteração. Assim sendo, o programa seguinte apresenta os valores de 5 até 1. Para isso, crie o projeto `c04ex08`.

```
use std::io;
use std::io::prelude::*;
fn main() {
    for i in (1 .. 6).rev() {
        println!("{}", i);
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, serão apresentados no monitor de vídeos os valores 5, 4, 3, 3, 3 e 1.

Observe que para uso do método `rev()` é necessário especificar a faixa de valores entre os símbolos de parênteses. O método `rev()` é um mecanismo de iteração da linguagem Rust que permite inverter a ordem de acesso a uma lista de valores.

 **Observação:** Caso queira usar o valor da variável `I` ou outra variável definida para um laço `for` em operações de cálculo, além do uso feito para controlar as iterações, será necessário fazer a conversão da variável `I` para um tipo de dado inteiro, como `u8`, `i8`, `u16`, `i16`, `u32`, `i32`, `u64` e `i64`, por meio do comando `as`. Por exemplo, para apresentar os quadrados de 1 a 5 dentro do laço `for` basta usar uma instrução como `println!("{}", (i as u8).pow(2))`. No capítulo 6 há mais informações sobre conversão de dados.

4.4 Interrupção na execução de laços

Laços são estruturas operacionais que podem ser interrompidas em sua execução conforme determinada condição. Esta ocorrência pode ser

efetivada com o uso das instruções `break` e `continue`, sendo a instrução `break` mais radical, uma vez que faz o término da execução do laço. Já a instrução `continue` faz uma interrupção temporária no laço, mantendo o controle operacional dentro do laço em execução.

Como exemplo de uso da instrução `break`, considere o programa seguinte, que apresenta uma sequência de valores num laço que efetua a contagem de 1 a 10. Para isso, crie o projeto `c04ex09`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    for i in 1 .. 11 {
        if i == 6 { break; }
        println!("{:2}", i);
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao executar o programa serão apresentados os valores de 1 a 5 e o laço será interrompido e encerrado, pois, quando a variável `i` estiver com o valor 6, a instrução `break` será executada, fazendo com que forçosamente o laço seja encerrado, não chegando a variável `i` a atingir o valor 10.

Como exemplo de uso da instrução `continue`, considere o programa seguinte, que apresenta uma sequência de valores num laço que efetua a contagem de 1 a 10. Para isso, crie o projeto `c04ex10`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    for i in 1 .. 11 {
        if i == 6 { continue; }
        println!("{:2}", i);
    }
}
```

```

    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao executar o programa, serão apresentados os valores de 1 a 5 e de 7 a 10; o valor 6 será omitido. A omissão da apresentação do valor 6 decorre da ação da instrução `continue`, que faz uma interrupção temporária na ação do programa. Isso posto, quando a variável `i` estiver com o valor 6, sua apresentação será omitida, pois a instrução `continue` fará com que o laço em execução seja interrompido na iteração atual e que passe para a próxima interação da repetição a ser efetivada.

4.5 Jogo de adivinhação

A partir da visão apresentada das ações básicas de programação, que são a programação sequencial (Capítulo 2), a programação com decisões (Capítulo 3) e a programação com laços (este capítulo), é possível escrever um programa mais elaborado e contextualizado.

Dessa forma, o programa a seguir é um jogo de adivinhação de números inteiros entre 1 e 10 adaptado a partir de código idêntico, definido no Capítulo 2 do livro *The Rust Programming Language*, 2ª edição, produzido pela equipe de desenvolvimento da linguagem Rust e disponibilizado no site oficial da linguagem no endereço <https://doc.rust-lang.org/book/second-edition/ch02-00-guessing-game-tutorial.html>. Para isso, crie o projeto `c04ex11`.

De maneira diferente dos programas apresentados antes para este código, será necessário fazer uso da biblioteca de geração de valores aleatórios. Para isso, não basta apenas colocar no código-fonte a chamada da biblioteca com a instrução `use rand::Rng;`, pois é necessário configurar o projeto junto ao arquivo `cargo.toml` na pasta/diretório `c04ex10`. Assim sendo, abra o arquivo `Cargo.toml` do projeto `c04ex11`, e será apresentado o

seguinte trecho de texto:

```
[package]
name = "c04e10"
version = "0.1.0"
authors = ["Fulano"]
```

```
[dependencies]
```

As bibliotecas de apoio a linguagens Rust são referenciadas como *crates*. Uma *crate* é uma “caixa” que contém recursos adicionais a um programa. Neste caso, é necessário indicar o uso de recursos para a geração de números aleatórios. No arquivo `Cargo.toml`, acrescente após a indicação `[dependencies]` o trecho com a instrução `rand = "x.y.zz"`, em que `x.y.zz` é o número da versão do recurso a ser usado, como demonstrado a seguir (em **negrito**) a partir da indicação da versão **0.3.14**.

```
[package]
name = "c04e10"
version = "0.1.0"
authors = ["Fulano"]
```

```
[dependencies]
rand = "0.3.14"
```

Na sequência, é necessário, na janela do terminal, após gravar a alteração do arquivo `Cargo.toml`, fazer a construção do recurso. Para isso, escreva no terminal a instrução:

```
cargo build
```

O processo levará certo tempo e apresentará, durante o procedimento, mensagens de orientação informando a atualização do registro e o download da biblioteca. Na data em que este recurso foi testado, setembro de 2017, a versão do *crate* `rand` disponível era **0.3.16**, o que levou à apresentação das mensagens:

```
Updating registry `https://github.com/rust-lang/crates.io-index`
Downloading rand v0.3.16
error: unable to get packages from source
```

Caso ocorra algo semelhante, é necessário atualizar o número da versão

no arquivo `cargo.toml`. Use o número de versão apresentado; neste caso, `0.3.16`, e altere o arquivo `cargo.toml` com o trecho em negrito.

```
[package]
name = "c04e10"
version = "0.1.0"
authors = ["Fulano"]
```

```
[dependencies]
rand = "0.3.16"
```

Na sequência, execute a ação de construção `cargo build` na janela do terminal com a instrução.

Nesta fase a apresentação das mensagens no terminal deverá ser diferente da apresentação das mensagens de erro, como:

```
Downloading rand v0.3.16
Downloading libc v0.2.30
  Compiling libc v0.2.30
  Compiling rand v0.3.16
  Compiling c04e10 v0.1.0 (file:///C:/Users/Fulano/Estudo/c04ex10)
  Finished dev [unoptimized + debuginfo] target(s) in 12.86 secs
```

No exemplo de configuração do arquivo `cargo.toml`, foi definida na seção `[dependencies]` a *crate* externa adequada para uso de determinado recurso. Neste caso, o recurso da *crate* `rand` da versão semântica `0.3.16` que determina para o compilador Rust a versão da *crate* pública que tenha uma API (Application Programming Interface) compatível.

Na sequência, abra o arquivo de programa `main.rs` da pasta/diretório `src` de `c04ex11` em `estudo` criado automaticamente com um editor de texto de sua preferência e altere o código do arquivo com o código seguinte:

```
extern crate rand;

use std::io;
use std::io::prelude::*;
use rand::Rng;
use std::cmp::Ordering;
```

```

fn main() {
    println!("\nAdivinhe um número!");
    let numero_secreto = rand::thread_rng().gen_range(1, 11);

    loop {
        print!("\nEntre um número: ");
        io::stdout().flush().unwrap();

        let mut numero_informado = String::new();

        io::stdin().read_line(&mut numero_informado)
            .expect("Erro na entrada de dados.");

        let numero_informado: u32 = match
            numero_informado.trim().parse() {
                Ok(numero) => numero,
                Err(_) => continue
            };
        println!("Você informou o valor: {}", numero_informado);

        match numero_informado.cmp(&numero_secreto) {
            Ordering::Less => println!("Valor é baixo!"),
            Ordering::Greater => println!("Valor é alto!"),
            Ordering::Equal => {
                println!("Você acertou!");
                break;
            }
        }
    }
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Após a escrita do código em src de c04ex10 em estudo, execute a partir da pasta/diretório estudo a instrução:

```
cargo run
```

Ao executar o programa, insira um valor e observe a mensagem

apresentada. Se um valor for indicado como alto, tente um valor mais baixo. Faça a entrada até que o valor seja o valor sorteado.

Um diferente detalhe usado no programa anterior é o local de definição das variáveis. Note que neste exemplo as variáveis estão sendo definidas de forma diferente da dos exemplos anteriores. A linguagem Rust não requer que as variáveis sejam obrigatoriamente definidas antes do uso, todas juntas, como foi anteriormente realizado. No entanto a prática de definir todas as variáveis antes de seu uso torna o código do programa mais elegante. A decisão de usar uma ou outra forma dependerá de diversas circunstâncias a serem avaliadas pelo programador que não cabem ser discutidas aqui.

No início do programa, a instrução `extern crate rand;` indica o uso de um módulo externo de recursos, neste caso, `rand`. Dessa forma, o compilador Rust é informado de que é necessário para compilar um programa fazer uso de um recurso de dependência externa. A partir desta definição é possível fazer a chamada de qualquer recurso da biblioteca `rand` por meio do prefixo `rand::`, como `rand::Rng` e `rand::thread_rng()`.

A instrução `use rand::Rng;` estabelece a característica `Rng`, que define e implementa os métodos para a geração de valores aleatórios estabelecidos na instrução `let numero_secreto = rand::thread_rng().gen_range(1, 11);`, em que a variável `NUMERO_SECRETO` é atribuída por meio da ação do método `thread_rng()`, que fornece o gerador de números aleatórios semeado pelo sistema operacional, a menos que se estabeleça o contrário quando associado a um método que interfira na semente de geração de números aleatórios, como é o caso do método `gen_range()`. O método `gen_range()` advém da biblioteca `rand::Rng` e requer, para ser operado, dois parâmetros: o primeiro parâmetro estabelece o valor inicial e o segundo parâmetro estabelece o valor final para gerar um valor aleatório entre eles. Observe que, para gerar valores aleatórios inteiros entre 1 e 10, definem-se os parâmetros 1 e 11. Isso se faz necessário, uma vez que o limite inferior é inclusivo e o limite superior é exclusivo.

Em relação à instrução `use std::cmp::Ordering;`, tem-se o estabelecimento

de uso da biblioteca `std::cmp`, que se utiliza da enumeração `Ordering`, que tem como variação de ação `Less` (menor), `Greater` (maior) e `Equal` (igual), que são os resultados possíveis encontrados na comparação entre dois valores quando do uso do método `cmp()`, que efetua por meio do trecho `numero_informado.cmp(&numero_secreto)` a comparação entre os valores informados e gerados secretamente, fornecendo para o comando `match` a expressão de correspondência que será usada para a apresentação das mensagens de resultado de cada uma das ações validadas.

O comando `match` nesta versão do programa não faz uso da ação de exclusão quando nenhuma das opções é validada e processada. Isso não é necessário, pois as opções que devem ser executadas sempre serão neste programa uma das três possibilidades. No entanto a ausência dessa ação de exclusão exige que se use o comando `break` para sair do bloco do comando `match` sem que haja ocorrência de erro na compilação do programa.

Outra pequena diferença aplicada neste projeto em relação aos demais projetos apresentados se encontra em relação ao uso da instrução `let numero_informado: u32 = match numero_informado.trim().parse()` com o comando `match`. Observe que no método `parse()` este não está sendo usado com a forma `parse::<u32>()`, pois a conversão da entrada de dados na forma de *string* para o inteiro de 32 bits positivo está definido no trecho `let numero_informado: u32`. Perceba que neste momento o que está sendo realizado é a redefinição do tipo da variável `NUMERO_INFORMADO`, que, definido como `String`, passa neste momento a ser uma variável do tipo de dado inteiro. Além do detalhe observado há, no bloco de ação desta instrução, a validação do `NUMERO_INFORMADO` com o método `Ok()` e a indicação pelo método `Err()` direcionando o erro para o comando `continue`, que dará andamento na ação do comando `loop` se alguma entrada for incorreta, sendo esta uma forma de estabelecer a consistência de validade da entrada de dados de um usuário dentro de um laço.

4.6 Exercícios de fixação

Desenvolver os programas em Rust dos problemas elencados de 1 até 7 para os exercícios A, B, C e D, ficando a cargo do professor selecionar a ordem e os problemas a serem resolvidos.

A. Laço condicional pré-teste verdadeiro (`while`).

B. Laço condicional pré-teste falso (`while` com o operador de negação `!`).

C. Laço condicional seletivo (`loop`) com comando `if` definido ao final do laço.

D. Laço iterativo (`for`).

É importante levar em consideração que talvez um ou outro problema não possa ser resolvido com um determinado tipo de laço. Fica a critério do professor escolher os laços e os exercícios que devem ser feitos pelo aluno.

Atente para os seguintes problemas:

1. Elaborar um programa que apresente como resultado os quadrados calculados a partir dos números inteiros não sinalizados de 16 bits na faixa de valores de 15 a 200.
2. Construir um programa que apresente a soma dos cem primeiros números naturais ($1+2+3+\dots+98+99+100$). Use um tipo de dado que seja compatível com o formato 64 bits sinalizado.
3. Elaborar um programa que apresente o somatório dos valores pares existentes na faixa de 1 até 500.
4. Elaborar um programa que apresente todos os valores numéricos inteiros ímpares situados na faixa de 0 a 20.
5. Construir um programa que apresente todos os valores numéricos inteiros divisíveis por 4 e menores que 200. A variável que controla o contador do laço deve ser obrigatoriamente iniciada com valor 1.
6. Escrever um programa que calcule e apresente o somatório do número de grãos de trigo que se pode obter num tabuleiro de xadrez, obedecendo à seguinte regra: colocar um grão de trigo no primeiro quadro e nos quadros seguintes o dobro do colocado no quadro

anterior. Ou seja, no primeiro quadro coloca-se um grão, no segundo quadro colocam-se dois grãos (neste momento têm-se três grãos), no terceiro quadro colocam-se quatro grãos (tendo neste momento sete grãos), no quarto quadro colocam-se oito grãos (tendo-se então quinze grãos) até atingir o 64º quadro (*este exercício foi baseado em uma situação exposta no capítulo 16 do livro O homem que calculava, de Malba Tahan, publicado pela Editora Record*). Para o controle do laço, utilize uma variável não sinalizada inteira de 8 bits, e para os cálculos do programa, utilize variáveis reais de 64 bits.

7. Elaborar um programa que leia dez valores numéricos reais de 64 bits e apresente no final o somatório e a média dos valores lidos. Para controlar o laço, use uma variável inteira não sinalizada de 8 bits.

CAPÍTULO 5

Sub-rotinas

Neste capítulo o leitor terá contato com o uso de sub-rotinas. Este capítulo apresenta o uso de sub-rotinas criadas pelo programador, sendo indicados as definições de funções, os procedimentos, o escopo (a visibilidade) de variáveis e passagens de parâmetro por valor e por referência. No tocante ao uso de módulos, o leitor conhecerá a estratégia de definir um conjunto de funcionalidades que podem ser consideradas públicas ou privadas.

5.1 Dividir para conquistar

É sabido que problemas complexos necessitam, para sua solução, de algoritmos também complexos. Assim sendo, é possível dividir um problema grande e complexo em problemas menores e mais simples (dividir para conquistar). Na medida em que se resolvem os problemas menores, obtém-se a solução do problema maior com mais facilidade. Cada parte do problema maior passa a ter um algoritmo mais simples e facilita mais o caminho para uma grande solução. O conceito de sub-rotinas tem sido adotado desde meados da década de 1950 (PRESSMAN, 1995, p. 427).

Uma sub-rotina é um bloco de programa que pode realizar operações computacionais de entrada, de processamento e de saída. Ao dividir um problema complexo em partes menores, automaticamente é utilizada a ideia de abstração. Abstrair um algoritmo significa considerar isoladamente um ou mais elementos de seu todo; significa, de forma geral, separar o todo em partes.

O uso de sub-rotinas proporciona duas vantagens. A primeira é a

possibilidade de dividir um problema complexo em partes menores, mantendo essas partes mais bem organizadas. A segunda é permitir criar rotinas de trechos de programa que se repetem ao longo do programa, proporcionando certo grau de economia de trabalho e reaproveitamento de código.

A operação de uso de sub-rotinas em um programa de computador proporciona três características operacionais (adaptado de SEBESTA, 2003, p. 330):

- Cada sub-rotina tem um único ponto de entrada.
- Toda unidade de programa chamadora (unidade-mestra) é suspensa durante a execução da unidade de módulo chamado (unidade-escravo), o que implica a existência de somente uma sub-rotina em execução em um determinado momento, exceto quando se trabalha com paralelismo ou corrotinas.
- Quando a execução da unidade-escravo é encerrada, o controle do fluxo de execução do programa volta para a primeira linha de instrução após a chamada na unidade-mestra, quando se trata da chamada de um procedimento. No caso de chamada de uma função, o retorno ocorre exatamente na mesma linha de código que efetuou a chamada.

Uma sub-rotina pode ser usada para se fazer diversas operações computacionais (entrada, processamento e saída). Ao se fazer uso dessa técnica, pode-se deparar-se com a necessidade de dividir uma sub-rotina em outras tantas sub-rotinas, quantas forem necessárias, buscando uma solução mais simples de uma parte do problema maior. O processo de dividir sub-rotinas em outras é denominado *Método de Refinamento Sucessivo* (dividir para conquistar).

5.2 Paradigmas top-down e bottom-up

Os métodos *top-down* (de cima para baixo) e *bottom-up* (de baixo para cima) são procedimentos de trabalho que podem facilitar a construção de

programas de computador. O método *top-down* descreve de forma resumida as ações de um programa sem considerar detalhes específicos. Já o método *bottom-up* descreve de forma detalhada as operações de um programa de computador com o objetivo de apresentá-lo detalhadamente como um todo.

O projeto do programa deve ser baseado no método *top-down*. Já o desenvolvimento do programa em si é feito com o método *bottom-up*. Os paradigmas *top-down* e *bottom-up* possibilitam grande facilidade na aplicação das etapas de trabalho da programação estruturada e orientada a objetos, mesmo quando a linguagem de programação não exige que tal abordagem seja implementada.

O método *top-down* caracteriza-se basicamente pelos seguintes fatores:

- Antes de iniciar a construção de um programa de computador, deve-se ter em mente as tarefas principais que o programa deve executar. Não é necessário saber como as tarefas vão funcionar; apenas, saber quantas tarefas são.
- Conhecidas todas as tarefas a serem executadas, deve-se especificar como deve ser o programa principal, que vai controlar todas as outras tarefas distribuídas em suas sub-rotinas.
- Definido o programa principal, é iniciado o processo de detalhamento estrutural para cada sub-rotina. Dessa forma, se definem vários algoritmos, um para cada rotina em separado, para que se tenha uma visão do que deve ser executado em cada módulo de programa. Há programadores que estabelecem o número máximo de linhas de programa que uma rotina deve ter. Se o número de linhas ultrapassa o limite preestabelecido, a rotina em desenvolvimento é dividida em outra sub-rotina.

A utilização do paradigma de cima para baixo (*top-down*) permite uma visão detalhada da estrutura de cada parte do programa de forma separada e a integração que essas partes têm no programa. Com o projeto de baixo para cima (*bottom-up*), cada sub-rotina do programa pode ser testada separadamente, garantindo que o programa esteja sem erro ao seu

término.

A utilização de sub-rotinas na programação de computadores permite elaborar rotinas exclusivas. Por exemplo, uma rotina somente para entrada, outra para o processamento e outra para a saída dos dados.

O uso do método *bottom-up* sugere que o programa seja escrito a partir de suas sub-rotinas inferiores, deixando-se por último as rotinas de nível mais alto. A linguagem Rust aceita que as sub-rotinas sejam escritas em qualquer parte do programa, ficando a critério do programador fazer uso do paradigma *bottom-up*. Os programas apresentados nesta obra se utilizam do paradigma *bottom-up* por ser esta uma forma bem organizada de montagem de um programa e por garantir melhor legibilidade do código.

5.3 Sub-rotinas em Rust

As formas mais comuns de sub-rotinas em linguagens de programação são os procedimentos e as funções. Os procedimentos são usados em operações genéricas, normalmente em ações de entrada e saída, podendo ser usados em ações de processamento; e as funções são usadas em ações específicas de processamento matemático e lógico, por terem a característica de sempre retornar um valor como resposta.

O desenvolvimento de sub-rotinas segue os mesmos cuidados de programação já apresentados e aplicados nos capítulos anteriores. A forma sintática de definição na linguagem Rust é obtida a partir de uma estrutura sintática bastante flexível:

```
fn <nome> ([<parâmetro>: <tipo do parâmetro>, ...]) [-> <tipo de
retorno>] {
    [<variáveis>];
    [<instruções>];
    [[<return>] <valor de retorno[;]>]
}
```

em que:

<tipo de retorno>	tipo de dado a que a sub-rotina dará retorno
-------------------	--

<i><nome></i>	nome atribuído à sub-rotina
<i><tipo de parâmetro></i>	quando existente, é preciso declarar tipo
<i><parâmetro></i>	informação opcional
<i><variáveis></i>	lista de variáveis locais (opcionais)
<i><instruções></i>	processamento do corpo da função
<i><return> <valor de retorno></i>	definição do retorno quando for função

A linguagem Rust permite que uma sub-rotina definida assuma três tipos de comportamento. O primeiro, quando a sub-rotina é executada e não retorna valor ao módulo de programa que fez sua chamada, simulando, assim, a ação de um procedimento; o segundo, quando uma sub-rotina tem a capacidade de retornar apenas um valor ao módulo de programa que fez sua chamada, executada como uma função; o terceiro, quando a sub-rotina trabalha com passagem de parâmetros por valor e por referência, podendo ser definida como um procedimento ou uma função.

Uma sub-rotina, seja ela um procedimento ou uma função, permite, além de dividir um programa em partes, fazer o aproveitamento de trechos de códigos que se repetem durante a execução do programa, proporcionando, assim, uma economia em número de linhas. Um programa menor é sempre mais eficaz que um programa maior para efetuar a mesma tarefa.

5.4 Escopo e visibilidade de variáveis

O escopo e a visibilidade estão associados à forma como determinado recurso encontra-se definido no código de um programa em relação a sua existência na memória do computador. A definição de escopo (visibilidade) é uma característica das linguagens de programação estruturada, conseqüentemente herdada das linguagens orientadas a objeto e usada em diversos outros paradigmas de programação, pois facilita a organização interna do código e dos dados, no sentido de economizar ao máximo o espaço de memória utilizado, desenvolvendo-se programas mais organizados e de melhor legibilidade.

Dessa forma, pode-se concluir que o escopo de um recurso (sua visibilidade na memória) é a relação de alcance que se tem do local onde se encontra definido, de modo que possa ser visualizado pelas várias partes do código de um programa. Em Rust, as constantes devem ser declaradas com escopo global e as variáveis devem ser declaradas com escopo local. Dessa forma, as constantes são declaradas antes do estabelecimento das sub-rotinas, e as variáveis, dentro das sub-rotinas, junto ao bloco de programa delimitado pelos símbolos { (abre chave) e } (fecha chave), normalmente, usados junto a `main()` ou na definição de uso de comandos como `if`, `else`, `while`, `loop` e `for`. É possível definir, dentro de um bloco, outros blocos internos (adjacentes) para utilização de variáveis. O exemplo seguinte mostra o uso de duas variáveis que são definidas em blocos que estabelecem o nível de escopo e visibilidade dessas variáveis. Para isso, crie o projeto `c05ex0.`:

```
use std::io;
use std::io::prelude::*;

fn main() {
    let x: i8 = 1;
    {
        let y: i8 = 2;
        println!("O valor de x é {} e o valor de y é {}.", x, y);
    }
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, a mensagem processada indica a apresentação dos valores das variáveis `x` e `y`. A variável `x` tem seu escopo de visibilidade em estilo global, seu valor é válido dentro do bloco (primeiro bloco) da função `main()` e do segundo bloco (bloco encadeado, uma forma de estabelecer o conceito de closure – fechamento). No entanto a variável `y` é uma variável de escopo local (escopo fechado) que tem seu

valor válido apenas dentro do segundo bloco; ela não é encontrada no primeiro bloco.

Quando se definem variáveis com escopo global e local é necessário tomar cuidado com o efeito de sombreamento que acontece quando se usam variáveis com mesmo nome em escopos locais e globais encadeados. O programa seguinte demonstra o uso de sombreamento de variáveis. Para isso, crie o projeto `c05ex02`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let x: i8 = 1;
    {
        println!("x = {}. ", x);
        let x: i8 = 2;
        println!("x = {}. ", x);
    }

    println!("x = {}. ", x);
    let x: i8 = 3;
    println!("x = {}. ", x);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, são apresentados os valores da variável `x` como sendo 1, 2, 1 e 3. O primeiro valor como 1 advém da instrução `let x: i8 = 1;`. A variável `x` assume o valor 2 quando executa a instrução `let x: i8 = 2;` definida no segundo bloco encadeado ao primeiro bloco, tendo a partir de então a variável `x` com um efeito de sombra (sombreada). Ao sair do bloco encadeado, a variável `x` mantém seu valor original – neste caso, valor 1 – e depois assume o valor 3.

5.5 Sub-rotina: procedimento

Um procedimento é um tipo de sub-rotina utilizado na organização estrutural de um programa que permite separar as partes operacionais do código de maneira mais organizada, desde que o desenvolvedor (programador e/ou analista) seja também organizado. No texto, a chamada de um procedimento é definida com uma palavra sublinhada.

O programa seguinte apresenta de maneira simples o uso de uma sub-rotina do tipo procedimento, a qual efetua a leitura de um valor inteiro positivo e apresenta o resultado da fatorial do valor fornecido. Neste código são usados dois procedimentos: um para o cálculo em si e outro para a apresentação da mensagem de pausa do programa. Para isso, crie o projeto c05ex03:

```
use std::io;
use std::io::prelude::*;

fn pausa() {
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

fn fatorial() {
    let mut n = String::new();
    let mut f: u64 = 1;

    print!("Entre o valor da fatorial: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut n).expect("Erro na entrada.");
    let n: u64 = match n.trim().parse() {
        Ok(numero) => numero,
        Err(_) => 0
    };

    for i in 1 .. n + 1 {
        f *= i;
    }
}
```

```

    }

    println!("Resultado = {}", f);
    pausa();
}

fn main() {
    fatorial();
}

```

Ao ser executado o programa, é solicitada a entrada de um valor para o cálculo da fatorial. Após efetuar a entrada, o programa mostra o resultado. Se fornecido o valor 5, a resposta do programa será 120.

Observe no código do programa a distribuição das sub-rotinas `pausa()`, `fatorial()` e `main()` nessa ordem. Esta forma de escrita de código está de acordo com a forma de desenvolvimento baseada no modelo *bottom-up*. A sub-rotina `pausa()` vem à frente da sub-rotina que fará seu uso, neste caso, `fatorial()` que vem à frente da rotina que fará seu uso, sendo `main()`. No modelo *bottom-up* as sub-rotinas de níveis inferiores são escritas a frente das sub-rotinas de níveis mais altos, caracterizando uma estrutura hierárquica de ação e de acesso aos recursos estabelecidos.

A estrutura do código utiliza diversos elementos computacionais já apresentados na linguagem Rust. Um detalhe que merece atenção é o comando `for` com a instrução `for i in 1..n + 1`, que acrescenta mais 1 ao valor da variável `n` para que o ciclo de iteração que define o limite da variável `n` possa ser realizado de forma adequada na linguagem, pois, se não houver a soma de mais 1, após `1..n` ao valor-limite de `n`, o cálculo será realizado um passo a menos na contagem de 1 a `n` devido a característica operacional executada pelo comando `for`.

Como comentado antes, a linguagem Rust aceita que o código de um programa com sub-rotinas possa ser escrito fora da ordem estabelecida pelo paradigma *bottom-up*. Assim, a versão do programa `c05ex03`, mostrado a seguir, também seria executada, apesar de não ser o modelo adotado para este livro.


```

use std::io;
use std::io::prelude::*;

fn main() {
    fatorial();
}

fn fatorial() {

    let mut n = String::new();
    let mut f: u64 = 1;

    print!("Entre o valor da fatorial: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut n).expect("Erro na entrada.");
    let n: u64 = match n.trim().parse() {
        Ok(numero) => numero,
        Err(_) => 0
    };

    for i in 1 .. n + 1 {
        f *= i;
    }

    println!("Resultado = {}", f);
    pausa();
}

fn pausa() {
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

5.6 Passagem de parâmetro

O uso de passagem de parâmetros em uma sub-rotina proporciona uma

comunicação bidirecional entre as rotinas do programa, principalmente quando se utilizam variáveis de escopo local. Quando se trabalha com sub-rotinas (sejam procedimentos ou funções que serão apresentadas mais adiante) na linguagem de programação Rust, é possível passar valores de uma sub-rotina para outra. Dessa forma, pode-se passar valores de uma sub-rotina ou rotina chamadora a outra sub-rotina, e vice-versa. A passagem de parâmetro entre sub-rotinas poderá ser definida sob a óptica de dois aspectos na linguagem Rust: *passagem de parâmetro por valor* (quando uma sub-rotina envia um conteúdo para outra sub-rotina), *passagem de parâmetro por referência* (quando uma sub-rotina envia um conteúdo para outra sub-rotina e recebe pelo mesmo parâmetro um resultado como resposta a sua chamada)

5.6.1 Passagem de parâmetro por valor

A passagem de parâmetro por valor é usada quando há necessidade de enviar algum valor para uma sub-rotina chamada. Qualquer modificação feita no parâmetro da sub-rotina será mantida dentro do escopo da sub-rotina.

Para ilustrar o uso de *passagem de parâmetro por valor*, considere o exemplo de programa fatorial de um número inteiro positivo qualquer. Para isso, crie o projeto `c05ex04`.

Abra o arquivo de programa `main.rs` da pasta/diretório `src` de `c05ex04` em estudo criado automaticamente com um editor de texto de sua preferência e altere o código do arquivo com o código seguinte:

```
use std::io;
use std::io::prelude::*;

fn pausa() {
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

```

fn fatorial(n: u64) {
    let mut f: u64 = 1;

    for i in 1 .. n + 1 {
        f *= i;
    }

    println!("Resultado = {}", f);
    pausa();
}

fn main() {
    let mut valor = String::new();

    print!("Entre o valor da fatorial: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).expect("Erro na entrada.");
    let valor: u64 = match valor.trim().parse() {
        Ok(numero) => numero,
        Err(_) => 0
    };

    fatorial(valor);
}

```

Ao ser executado o programa, é solicitada a entrada de um valor para o cálculo da fatorial. Após efetuar a entrada, o programa mostra o resultado. Se fornecido o valor 5, a resposta do programa será 120.

Usa-se a passagem de parâmetro por valor quando há necessidade de passar algum conteúdo à sub-rotina chamada. O conteúdo passado pelo parâmetro na chamada da sub-rotina é copiado a partir da chamada para a sub-rotina chamada. Se houver alguma modificação na variável local da sub-rotina, isso não afetará o valor do parâmetro passado, ou seja, o processamento será executado apenas na sub-rotina, ficando, assim, o resultado obtido “preso” na área de abrangência da sub-rotina chamada.

O programa anterior recebe na entrada um conteúdo atribuído à variável VALOR e transfere o valor recebido à variável N da sub-rotina `fatorial()` via

passagem de parâmetro por valor. Na área interna de código na sub-rotina `fatorial()`, encontra-se a definição da variável local `F`, que será calculada com a execução do laço `for`, tendo, ao final, o valor da fatorial informado (transferido) para o parâmetro `N`. Ao término, o valor calculado será apresentado por meio da variável `F`, que somente é válida na função `fatorial()`. Por essa razão, o valor fica “preso” na sub-rotina.

5.6.2 Passagem de parâmetro por referência

A passagem de parâmetro por referência é usada quando há necessidade de receber algum valor como retorno de uma sub-rotina chamada por meio de um ou mais de seus parâmetros. Qualquer modificação feita no parâmetro da sub-rotina será refletida no ponto de chamada da sub-rotina.

Para ilustrar o uso de *passagem de parâmetro por referência*, considere o exemplo de programa fatorial de um número inteiro positivo qualquer. Para isso, crie o projeto `c05ex05`.

```
use std::io;
use std::io::prelude::*;
fn pausa() {
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

fn fatorial(n: u64, f: &mut u64) {
    for i in 1 .. n + 1 {
        *f *= i;
    }
}

fn main() {
    let mut valor = String::new();
    let mut result: u64 = 1;
```

```

    print!("Entre o valor da fatorial: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).expect("Erro na entrada.");
    let valor: u64 = match valor.trim().parse() {
        Ok(numero) => numero,
        Err(_) => 0
    };

    fatorial(valor, &mut result);

    println!("Resultado = {}", result);
    pausa();
}

```

Ao ser executado o programa, é solicitada a entrada de um valor para o cálculo da fatorial. Após efetuar a entrada, o programa mostra o resultado. Se fornecido o valor 5, a resposta do programa será 120.

Observe a chamada da sub-rotina `fatorial(valor, &mut result)` no trecho principal do programa. Note que a sub-rotina `fatorial()` recebe a passagem de dois parâmetros: um parâmetro por valor, no caso, `VALOR` com o fornecimento do valor a ser calculado, e outro parâmetro por referência, no caso, `RESULT` com o valor 1 definido a partir da instrução `let mut result: u64 = 1;`.

Após a chamada da sub-rotina `fatorial()`, o conteúdo da variável `VALOR` é passado como passagem de parâmetro por valor à variável `N`, e o conteúdo da variável `RESULT` é passado como passagem de parâmetro por referência à variável `FAT`. Na medida em que o valor de `F` é alterado, a variável `RESULT` passa a ter copiado automaticamente o valor da variável `F`. A ação da passagem de parâmetro por valor estabelece um canal de envio de valor entre a chamada da sub-rotina e a sub-rotina na forma de via de mão única. Já a passagem de parâmetro por referência estabelece uma canal de envio e recepção de valor na forma de via de duas mãos.

A passagem de parâmetro por referência, na maioria das linguagens de programação, precisa ser indicada de forma diferente da usada na passagem de parâmetro por valor. No caso da linguagem Rust, a referência

de um parâmetro é definida a partir do uso do qualificador `&mut` após o nome do parâmetro (`f: &mut u64`). Para que o parâmetro – neste caso, `F` – seja operacionalizado dentro do bloco de ação da sub-rotina, este deve ser precedido do símbolo (*) asterisco.

A obtenção do conteúdo de retorno na chamada da sub-rotina deve ser feita com o uso do qualificador `&mut`, como indicado na instrução `fatorial(valor, &mut result)`; na sub-rotina `main()` junto à variável `RESULT`.

5.7 Sub-rotina: função

Uma função é um tipo de sub-rotina especial que sempre retorna um valor como resposta, sendo útil para o estabelecimento de ações de processamento em um código de programa.

Uma sub-rotina do tipo função é uma estrutura de modularidade que tem como característica operacional a capacidade de sempre retornar um conteúdo como resposta para a chamada da função (RAVICHANDRAN, 1996, p. 73), independentemente de a função fazer uso de passagem de parâmetros por valor ou por referência.

Para cumprir o requisito de retornar uma resposta, uma sub-rotina do tipo função faz, na linguagem Rust, uso opcional do comando `return` com a indicação de algum conteúdo de resposta associado a própria função. Uma típica sub-rotina de função tem por finalidade, sempre, retornar um conteúdo como resposta. Desta forma, uma função necessita obrigatoriamente estar definida de acordo com um dos tipos de dados suportados pela linguagem.

5.7.1 Função básica

Com base nos exemplos anteriores, será demonstrada uma função de cálculo de fatorial com passagem de parâmetro por valor com a finalidade de receber a entrada de um conteúdo e retornar o resultado na própria função. De forma geral, uma função pode receber conteúdo de um tipo de dado e retornar como resposta um conteúdo de outro tipo. Para isso, crie o projeto `c05ex06`.

```

use std::io;
use std::io::prelude::*;

fn pausa() {
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

fn fatorial(n: u64) -> u64 {
    let mut f: u64 = 1;
    for i in 1 .. n + 1 {
        f *= i;
    }
    return f;
}

fn main() {
    let mut valor = String::new();

    print!("Entre o valor da fatorial: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).expect("Erro na entrada.");
    let valor: u64 = match valor.trim().parse() {
        Ok(numero) => numero,
        Err(_) => 0
    };

    println!("Resultado = {}", fatorial(valor));
    pausa();
}

```

Ao ser executado o programa, é solicitada a entrada de um valor para o cálculo da fatorial. Após efetuar a entrada, o programa mostra o resultado. Se fornecido o valor 5, a resposta do programa será 120.

A função `fatorial()`, por meio da instrução `return f`, retorna o conteúdo final da variável `f`, devolvendo-o ao programa principal, que fez a

chamada à função `fatorial()`. Esse recurso devolve um valor de retorno. Se houver mais valores a serem retornados, torna-se necessário utilizar passagem de parâmetro por referência. O retorno da função é estabelecido na sua definição com a instrução `fn fatorial(n: u64) -> u64`, onde a indicação `->` define o tipo de retorno a ser efetivado com o comando `return`, neste caso, o retorno de um valor do tipo `u64`.

O uso do comando `return` em uma sub-rotina do tipo função, como comentado, é opcional, e neste caso o trecho de código da sub-rotina deve ser codificado como:

```
fn fatorial(n: u64) -> u64 {  
    let mut f: u64 = 1;  
    for i in 1 .. n + 1 {  
        f *= i;  
    }  
    f  
}
```

Observe que, ao ser retirado o comando `return`, deixa-se apenas a variável `f` sem o uso do símbolo `(;)` ponto e vírgula. Apesar dessa possibilidade, os programas deste livro seguirão uma estrutura mais ortodoxa, mantendo-se a definição do comando `return`. As funções que omitem o comando `return` são funções com a ação de retorno antecipado (*early returns*). Já as funções com uso de `return` são consideradas funções em estilo pobre (*poor style*).

Como segundo exemplo, será desenvolvida uma função que compara dois valores fornecidos e retorna esses valores trocados se o primeiro valor for maior que o segundo. Essa função recebe como parâmetros dois valores do tipo inteiro e retorna como resultado o valor `true`, se o primeiro valor for maior que o segundo, ou `false`, se os parâmetros fornecidos forem respectivamente o menor e o maior valor. Para isso, crie o projeto `c05ex07`.

```
use std::io;  
use std::io::prelude::*;  
  
fn pausa() {  
    println!();  
}
```



```

    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

fn troca(a: &mut u64, b: &mut u64) -> bool {
    let x: u64;
    if *a > *b {
        x = *a;
        *a = *b;
        *b = x;
        return true;
    } else {
        return false;
    }
}

fn saida(a: u64, b: u64) {
    println!("N1 = {}", a);
    println!("N2 = {}", b);
}

fn main() {
    let mut n1 = String::new();
    let mut n2 = String::new();

    println!("Ordena valores\n");

    print!("Entre N1: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut n1).expect("Erro na entrada.");
    let mut n1: u64 = match n1.trim().parse() {
        Ok(numero) => numero,
        Err(_) => 0
    };

    print!("Entre N2: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut n2).expect("Erro na entrada.");
}

```

```

let mut n2: u64 = match n2.trim().parse() {
    Ok(numero) => numero,
    Err(_) => 0
};

if troca(&mut n1, &mut n2) {
    saida(n1, n2);
    println!("\nHouve troca de valores.");
} else {
    saida(n1, n2);
    println!("\nNão houve troca de valores.");
}

pausa();
}

```

Neste exemplo, a sub-rotina de função `troca()` recebe, por passagem de parâmetros, por referência, as variáveis `A` e `B`. Se o conteúdo da variável `A` for maior que o conteúdo da variável `B`, seus valores serão trocados e retornados nas variáveis `A` e `B`. Sendo assim, a função `troca()` retorna à sua chamada os valores das variáveis `A` e `B` trocados ou não, de acordo com os valores fornecidos, além de devolver seu retorno como `true` quando ocorrer a troca ou `false` quando a troca não for processada. Note que neste exemplo a função `troca()` retorna três conteúdos, sendo os conteúdos das variáveis `A` e `B` e o retorno em seu nome, que pode ser `true` ou `false`.

5.7.2 Função recursiva

Devido à característica operacional de retorno de conteúdo que uma função tem, é possível desenvolver funções que façam chamadas a si mesmas. Esse efeito denomina-se *recursividade* e permite executar o processamento da função sem laço. O uso de recursividade proporciona a escrita de um código mais elegante com alto grau de abstração.

Como exemplo de função recursiva, considere o programa seguinte, que efetua o cálculo da fatorial de um número qualquer e apresenta o resultado obtido. Para isso, crie o projeto `c05ex08`.

```

use std::io;
use std::io::prelude::*;

fn pausa() {
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

fn fatorial(n: u64) -> u64 {
    if n == 0 {
        return 1;
    } else {
        return n * fatorial(n - 1);
    }
}

fn main() {
    let mut valor = String::new();

    print!("Entre o valor da fatorial: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).expect("Erro na entrada.");
    let valor: u64 = match valor.trim().parse() {
        Ok(numero) => numero,
        Err(_) => 0
    };
    println!("Resultado = {}", fatorial(valor));
    pausa();
}

```

Ao ser executado o programa, é solicitada a entrada de um valor para o cálculo da fatorial. Após efetuar a entrada, o programa mostra o resultado. Se fornecido o valor 5, a resposta do programa será 120.

Ao observar o trecho de função recursiva `fatorial(n: u64)`, nota-se no bloco da sub-rotina o uso da condição `n == 0`, que, se verdadeira, retorna como resposta o valor 1. Se a condição for falsa, ocorrerá o retorno do

cálculo matemático `return n * fatorial(n - 1)`, em que `fatorial` é atribuído pelo resultado da operação `fatorial(N - 1) * N`. O parâmetro `N` determina o número de vezes que a operação deve ser efetuada, e `fatorial(n - 1)` é uma nova instância de chamada da função `fatorial` a si mesma, com o valor do parâmetro `N` menos 1.

O processo de recursividade é considerado muito elegante em programação, pois facilita a abstração e a modularidade no desenvolvimento de funções que podem ser complexas. No entanto, devido ao efeito de empilhamento, essa estratégia pode consumir grande espaço de memória.

5.7.3 Função apontada

Uma função apontada se refere a um tipo de sub-rotina que pode ser atribuído a uma variável de modo que a variável apontada possa ser usada como sinônimo da sub-rotina. Há duas formas de uso de funções apontadas: com inferência e sem inferência.

Como exemplo de função apontada com inferência, considere o programa seguinte, que efetua o cálculo da fatorial de um número qualquer e apresenta o resultado obtido. Para isso, crie o projeto `c05ex09`.

```
use std::io;
use std::io::prelude::*;

fn pausa() {
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

fn fatorial(n: u64) -> u64 {
    if n == 0 {
        return 1;
    } else {
        return n * fatorial(n - 1);
    }
}
```

```

    }
}

fn main() {
    let mut valor = String::new();
    let resp = fatorial;

    print!("Entre o valor da fatorial: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).expect("Erro na entrada.");
    let valor: u64 = match valor.trim().parse() {
        Ok(numero) => numero,
        Err(_) => 0
    };

    println!("Resultado = {}", resp(valor));
    pausa();
}

```

Ao ser executado o programa, é solicitada a entrada de um valor para o cálculo da fatorial. Após efetuar a entrada, o programa mostra o resultado. Se fornecido o valor 5, a resposta do programa será 120.

A instrução `let resp = fatorial;` faz o apontamento da sub-rotina `fatorial()` à variável `RESP` com inferência. O uso de uma função apontada sobre uma variável permite que a variável se comporte como se fosse a própria sub-rotina.

Como exemplo de função apontada sem inferência, considere o programa seguinte, que efetua o cálculo da fatorial de um número qualquer e apresenta o resultado obtido. Para isso, crie o projeto `c05ex10`.

```

use std::io;
use std::io::prelude::*;

fn pausa() {
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

```

}

fn fatorial(n: u64) -> u64 {
    if n == 0 {
        return 1;
    } else {
        return n * fatorial(n - 1);
    }
}

fn main() {
    let mut valor = String::new();
    let resp: fn(u64) -> u64 = fatorial;

    print!("Entre o valor da fatorial: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).expect("Erro na entrada.");
    let valor: u64 = match valor.trim().parse() {
        Ok(numero) => numero,
        Err(_) => 0
    };

    println!("Resultado = {}", resp(valor));
    pausa();
}

```

Ao ser executado o programa, é solicitada a entrada de um valor para o cálculo da fatorial. Após efetuar a entrada, o programa mostra o resultado. Se fornecido o valor 5, a resposta do programa será 120.

A instrução `let resp = fatorial;` faz o apontamento da sub-rotina `fatorial()` à variável `RESP` com inferência. O uso de uma função apontada sobre uma variável permite que a variável se comporte como se fosse a própria sub-rotina.

5.8 Sub-rotina como parâmetro

O recurso de sub-rotinas da linguagem Rust permite que sejam passadas para uma sub-rotina outras sub-rotinas como parâmetro. Como exemplo

de função como passagem de parâmetro de outra função, considere o programa seguinte, que efetua o cálculo da fatorial de um número qualquer e apresenta o resultado obtido. Para isso, crie o projeto `c05ex11`.

```
use std::io;
use std::io::prelude::*;

fn pausa() {
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

fn fatorial(n: u64) -> u64 {
    if n == 0 {
        return 1;
    } else {
        return n * fatorial(n - 1);
    }
}

fn escrevafat(vlr: u64, subrotina: fn(u64) -> u64) -> u64 {
    println!("Resultado = {}", subrotina(vlr));
    return vlr;
}

fn main() {
    let mut valor = String::new();

    print!("Entre o valor da fatorial: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).expect("Erro na entrada.");
    let valor: u64 = match valor.trim().parse() {
        Ok(numero) => numero,
        Err(_) => 0
    };

    escrevafat(valor, fatorial);
}
```

```
    pausa();  
}
```

Ao ser executado o programa, é solicitada a entrada de um valor para o cálculo da fatorial. Após efetuar a entrada, o programa mostra o resultado. Se fornecido o valor 5, a resposta do programa será 120.

Neste projeto, a sub-rotina `escrevafat()` recebe dois parâmetros. O primeiro parâmetro representado pela variável `VLR` do tipo `u64` é usado como valor para a chamada da sub-rotina identificada como `SUBROTINA`, que é o segundo parâmetro do tipo `fn (u64) -> u64`. A indicação `fn` é usada para acessar a sub-rotina passada como parâmetro, a indicação `(u64)` é usada para receber o parâmetro passado para a sub-rotina como parâmetro e a parte `-> u64` determina o tipo de retorno da sub-rotina função passada como parâmetro.

5.9 Sub-rotina fechada

Uma sub-rotina fechada, também chamada de *closure* (clausura), é um recurso válido dentro do escopo para o qual é definido, assemelhando-se a um escopo local. Esse efeito ocorre quando se definem dentro de uma sub-rotina variáveis livres no contexto léxico ou quando uma ou mais sub-rotinas são definidas dentro de outra sub-rotina.

No caso da linguagem Rust, a implementação de sub-rotinas fechadas é realizada de modo diferente do adotado em outras linguagens de programação. Uma sub-rotina fechada não pode ser declarada fora de outra sub-rotina, apenas dentro de outras sub-rotinas. Sendo assim, para ser usadas dentro de programas simples, devem ser declaradas dentro da sub-rotina `main()`.

Uma das formas de declaração de uma ação de sub-rotina fechada ocorre a partir da sintaxe:

```
let <nome> = |<parâmetro>: [<tipo>]| {[return] <ação sobre o parâmetro>  
[;]};
```

Onde *nome* é a identificação do nome de chamada da sub-rotina fechada, *parâmetro* e *tipo* são definições do dado que a sub-rotina recebe, em que o

tipo pode ser omitido se internamente a sub-rotina for tratada e *ação sobre o parâmetro* refere-se ao processamento a ser executado pela sub-rotina. O bloco definido entre (`{ }`) chaves é opcional quando se trata de apenas uma ação expressa em uma só linha, mas obrigatório quando há um conjunto de instruções. Os parâmetros de fechamento são declarados dentro do conjunto de símbolos (`| |`) denominados canos (*pipes*). O uso do comando *return* é opcional, por isso pode ser omitido.

O programa seguinte demonstra o uso de uma sub-rotina fechada que, ao receber o valor de um parâmetro numérico inteiro sinalizado de 64 bits, apresenta seu valor sucessor. Para isso, crie o projeto `c05ex12`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut valor = String::new();

    let sucessor = |x: i64| {x + 1;}; // ou let sucessor = |x: i64| x +
1;

    print!("Entre um valor numérico inteiro: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).expect("Erro na entrada.");
    let valor: i64 = match valor.trim().parse() {
        Ok(numero) => numero,
        Err(_) => 0
    };

    println!("Sucessor de {} igual a {:?}.", valor, sucessor(valor));

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, informe um valor numérico e note a apresentação do valor sucessor do valor numérico informado. Para

apresentar o valor de uma sub-rotina fechada, usa-se o código de formatação `{:?}`. Outro detalhe a ser considerado é o fato de que a variável usada no parâmetro, neste caso a variável `x`, não pode ser mutável, sendo esta uma variável fechada; por isso, não pode ter seu valor sobrecarregado.

Outra forma de declaração de uma ação de sub-rotina fechada pode ser definida a partir da sintaxe, lembrando que o uso do comando `return` dentro do bloco da ação sobre o parâmetro é opcional:

```
let <nome> = |<parâmetro>: [<tipo>]| -> <tipo> {<ação sobre o
parâmetro>[;]};
```

No sentido de demonstrar o uso desta segunda forma de definição, considere um programa que mostre o valor antecessor de um valor numérico inteiro sinalizado de 64 *bits* fornecido assim que o programa solicitar a entrada de um valor numérico inteiro. Para isso, crie o projeto `c05ex13`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut valor = String::new();

    let antecessor = |x: i64| -> i64 {x - 1};

    print!("Entre um valor numérico inteiro: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).expect("Erro na entrada.");
    let valor: i64 = match valor.trim().parse() {
        Ok(numero) => numero,
        Err(_) => 0
    };

    println!("Antecessor de {} igual a {:?}.", valor,
    antecessor(valor));

    println!();
    print!("Tecle <Enter> para encerrar...");
```

```

    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, informe um valor numérico e note a apresentação do valor antecessor do valor numérico informado a partir da segunda forma de definição de uma sub-rotina fechada.

Os exemplos anteriores mostram a definição de sub-rotinas fechadas com a utilização de apenas uma instrução interna. No entanto pode haver a definição de sub-rotina fechada, a qual utiliza mais de uma instrução e opera com variáveis mutáveis.

O programa seguinte mostra a definição de uma sub-rotina fechada com retorno de valor. A sub-rotina apresenta o valor da fatorial de um número inteiro sinalizado de 64 bits. Para isso, crie o projeto `c05ex14`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let mut valor = String::new();

    let fatorial = |n| {
        let mut f: u64 = 1;
        let limite: u64 = n;
        for i in 1 .. limite + 1 {
            f *= i;
        };
        return f; // ou f; (sem o 'return')
    };

    print!("Entre um valor numérico inteiro: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).expect("Erro na entrada.");
    let valor: u64 = match valor.trim().parse() {
        Ok(numero) => numero,
        Err(_) => 0
    };
}

```

```
println!("Fatorial de {} igual a {:?}.", valor, fatorial(valor));

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, informe um valor numérico e note a apresentação do resultado da fatorial do valor fornecido. Nesta sub-rotina fechada estão sendo definidas internamente as variáveis necessárias para a execução da rotina.

Os exemplos anteriores mostram o uso de um parâmetro de recepção de valores para uma sub-rotina fechada. No entanto é possível operar ações com mais de um parâmetro.

O programa seguinte recebe a entrada de dois valores inteiros sinalizados de 8 bits e informa se os valores fornecidos são iguais ou diferentes. Para isso, crie o projeto `c05ex15`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut a = String::new();
    let mut b = String::new();

    let compara = |v1: i8, v2: i8| -> bool {
        if v1 == v2 {
            return true;
        } else {
            return false;
        }
    };

    print!("Entre o 1o. valor numérico inteiro: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut a).expect("Erro na entrada.");
    let a: i8 = match a.trim().parse() {
        Ok(numero) => numero,
```

```

        Err(_) => 0
    };
    println!();

    print!("Entre o 2o. valor numérico inteiro: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut b).expect("Erro na entrada.");
    let b: i8 = match b.trim().parse() {
        Ok(numero) => numero,
        Err(_) => 0
    };
    println!();

    if compara(a, b) {
        println!("Valores fornecidos são iguais.");
    } else {
        println!("Valores fornecidos são diferentes.");
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, informe os dois valores e observe a resposta fornecida. Nesta versão, a sub-rotina fechada recebe como parâmetro dois valores sinalizados de 8 bits e retorna como resposta um valor `bool`. Por isso o uso da definição de retorno da sub-rotina é obrigatório, como indicado na instrução `let compara = |v1r1: i8, v1r2: i8| -> bool`.

5.10 Exercícios de fixação

Desenvolver os programas em Rust dos problemas elencados de 1 até 4, ficando a cargo do professor selecionar a ordem e os problemas a serem resolvidos. Utilize sub-rotinas de procedimento com passagem de parâmetro por valor.

1. Elaborar um programa de computador que calcule e apresente o valor

do somatório dos N primeiros números inteiros, definidos por um operador ($1 + 2 + 3 + 4 + 5 + 6 + 7 + \dots + N$). Fazer uso de valores positivos sinalizados de 32 bits

2. Desenvolver um algoritmo de programa de computador que calcule e apresente o valor de uma potência inteira de um número qualquer elevado a um expoente qualquer. Ao informar para a sub-rotina o número da base e do expoente, deve apresentar o seu resultado da potência. Por exemplo, se for usado no programa principal o procedimento POTÊNCIA(2,3), deve ser apresentado o valor 8. Resolva a exponenciação com uso de laço. Não use o operador de exponenciação. Use valores positivos não sinalizados de 64 bits.
3. Elaborar um programa que leia um número inteiro e apresente uma mensagem informando se o número é par ou ímpar. Usar valores positivos não sinalizados de 64 bits.
4. Elaborar um programa que apresente como resultado um número positivo, mesmo que a entrada tenha sido feita com valor negativo. Usar valores positivos sinalizados de 64 bits.

Desenvolver os programas dos problemas computacionais elencados de 1 até 3 com base no uso de sub-rotinas de procedimento com passagem de parâmetro por referência.

1. Elaborar um programa de computador que calcule e apresente o valor do somatório dos N primeiros números inteiros, definidos por um operador ($1 + 2 + 3 + 4 + 5 + 6 + 7 + \dots + N$). Fazer uso de valores positivos sinalizados de 32 bits.
2. Desenvolver um algoritmo de programa de computador que calcule e apresente o valor de uma potência inteira de um número qualquer elevado a um expoente qualquer. Ao informar para a sub-rotina o número da base e do expoente, deve apresentar o seu resultado da potência. Por exemplo, se for usado no programa principal o procedimento POTÊNCIA(2,3), deve ser apresentado o valor 8. Resolva a exponenciação com uso de laço. Não use o operador de exponenciação.

Use valores positivos não sinalizados de 64 bits.

3. Elaborar um programa que apresente como resultado um número positivo, mesmo que a entrada tenha sido feita com valor negativo. Usar valores positivos sinalizados de 64 bits.

Desenvolver os programas dos problemas computacionais elencados de 1 até 3 com base no uso de sub-rotinas de funções básicas que façam uso, quando necessário, de passagem de parâmetro por valor.

1. Elaborar um programa de computador que calcule e apresente o valor do somatório dos N primeiros números inteiros, definidos por um operador ($1 + 2 + 3 + 4 + 5 + 6 + 7 + \dots + N$). Fazer uso de valores positivos sinalizados de 32 bits.
2. Desenvolver um algoritmo de programa de computador que calcule e apresente o valor de uma potência inteira de um número qualquer elevado a um expoente qualquer. Ao informar para a sub-rotina o número da base e do expoente, deve apresentar o seu resultado da potência. Por exemplo, se for usado no programa principal o procedimento POTÊNCIA(2,3), deve ser apresentado o valor 8. Resolva a exponenciação com uso de laço. Não use o operador de exponenciação. Use valores positivos não sinalizados de 64 bits.
3. Elaborar um programa que apresente como resultado um número positivo, mesmo que a entrada tenha sido feita com valor negativo. Usar valores positivos sinalizados de 64 bits.

Desenvolver os programas dos problemas computacionais elencados de 1 até 3 com base no uso de sub-rotinas de funções apontadas que façam uso quando necessário de passagem de parâmetro por valor.

1. Elaborar um programa de computador que calcule e apresente o valor do somatório dos N primeiros números inteiros, definidos por um operador ($1 + 2 + 3 + 4 + 5 + 6 + 7 + \dots + N$). Fazer uso de valores positivos sinalizados de 32 bits.
2. Desenvolver um algoritmo de programa de computador que calcule e apresente o valor de uma potência inteira de um número qualquer

elevado a um expoente qualquer. Ao informar para a sub-rotina o número da base e do expoente, deve apresentar o seu resultado da potência. Por exemplo, se for usado no programa principal o procedimento POTÊNCIA(2,3), deve ser apresentado o valor 8. Resolva a exponenciação com uso de laço. Não use o operador de exponenciação. Usar valores positivos não sinalizados de 64 bits.

3. Elaborar um programa que apresente como resultado um número positivo, mesmo que a entrada tenha sido feita com valor negativo. Usar valores positivos sinalizados de 64 bits.

Desenvolver os programas dos problemas computacionais elencados de 1 até 3 com base no uso de sub-rotinas de funções fechadas que façam uso quando necessário de passagem de parâmetro por valor.

1. Elaborar um programa de computador que calcule e apresente o valor do somatório dos N primeiros números inteiros, definidos por um operador ($1 + 2 + 3 + 4 + 5 + 6 + 7 + \dots + N$). Fazer uso de valores positivos sinalizados de 32 bits.
2. Desenvolver um algoritmo de programa de computador que calcule e apresente o valor de uma potência inteira de um número qualquer elevado a um expoente qualquer. Ao informar para a sub-rotina o número da base e do expoente, deve apresentar o seu resultado da potência. Por exemplo, se for usado no programa principal o procedimento POTÊNCIA(2,3), deve ser apresentado o valor 8. Resolva a exponenciação com uso de laço. Não use o operador de exponenciação. Use valores positivos não sinalizados de 64 bits.
3. Elaborar um programa que apresente como resultado um número positivo, mesmo que a entrada tenha sido feita com valor negativo. Usar valores positivos sinalizados de 64 bits.

CAPÍTULO 6

Complementos

Este capítulo apresenta, de forma despretensiosa, alguns recursos complementares necessários ao uso da linguagem Rust que não foram apresentados anteriormente. Tais recursos são importantes ferramentas operacionais que auxiliam diversas ações da linguagem. Assim sendo, são aqui apresentados os conceitos de conversão de tipos, ponteiros, uso de funções em modo terminal ANSI, biblioteca definida pelo programador e macros.

6.1 Conversão de tipos

Pode ocorrer, no uso da linguagem Rust, a necessidade de converter dados de um tipo para outro tipo de dado. A ação de conversão de tipos de dados recebe o nome de *casting* (conversão), podendo ser feita por meio de uma ação de coerção de dados, de modelagem de tipos numéricos ou de modelagem de ponteiros.

As ações de *casting* em diversas linguagens de programação podem ser de dois tipos: implícitas (controladas pelo compilador) e explícitas (controladas pelo programador). Em virtude de seu contexto de segurança, a linguagem Rust requer que as conversões de dados sejam controladas pelo programador. Dessa forma, não se aplica o conceito de conversões implícitas ou explícitas, pois, do ponto de vista da linguagem Rust, toda conversão é explícita.

A linguagem Rust não tem uma sintaxe específica para a ação de conversão de dados, mas aceita que essa ação seja realizada com o comando `as` (como) quando efetuadas conversões por coerção ou modelagem de tipos numéricos.

A conversão de dados por coerção é obtida a partir da sintaxe:

```
let <variável1>: <tipo> = <valor>;  
let <variável2> = <variável1> as <tipo>;
```

O programa seguinte estabelece o valor 5 para uma variável *x* do tipo inteiro positivo de 8 bits e converte coercitivamente o valor de 8 bits para um valor de 16 bits junto à variável *y*, apresentando o resultado de *y*. Para isso, crie o projeto `c06ex01`.

```
use std::io;  
use std::io::prelude::*;  
  
fn main() {  
    let x: u8 = 5;  
    let y = x as i64;  
  
    println!("{}", y);  
  
    println();  
    print!("Tecle <Enter> para encerrar...");  
    io::stdout().flush().unwrap();  
    io::stdin().read(&mut [0u8]).unwrap();  
}
```

Ao ser executado o programa, o conteúdo de 8 bits não sinalizado da variável *x* é transferido para a variável *y* de 64 bits como um dado sinalizado que é obrigado a receber o valor da variável *x*. O valor transferido para a variável *y* é então apresentado. Neste caso, se a instrução de conversão `let y = x as i64;` fosse escrita como `y = x;`, seria apresentada uma mensagem de erro no momento da compilação do programa.

O uso da ação de modelagem entre tipos numéricos de dados é obtido a partir da sintaxe:

```
let <variável> = <valor> as <tipo>;
```

Para fazer esse tipo de conversão é necessário ter a atenção voltada aos seguintes pontos:

- Converter inteiro de mesmo tamanho de sinalizado para não sinalizado produz um valor que é a diferença do limite máximo do valor informado em relação ao limite operacional do tipo de dados em uso. Por exemplo, converter o valor `-5i8` em `u8` produzirá o valor 251 (que é a diferença de 256 menos o valor 5 informado).
- Converter inteiro maior em inteiro menor produz um valor que é a diferença do limite máximo do valor informado em relação ao limite operacional do tipo de dados em uso. Por exemplo, converter o valor `513u32` em `u8` produz o valor 1 (que é a diferença de 513 menos 512 informado).
- Converter inteiro menor em inteiro maior mantém o valor usado. Por exemplo, converter o valor `987u32` em `u64` produz o valor 987. Se usado para converter o valor `-9i8` para `i16`, o sinal será mantido, bem como o valor.
- Converter dado de ponto flutuante (valor do tipo real) em inteiro fará o arredondamento da mantissa do valor real para zero, mantendo apenas o expoente do valor. Por exemplo, converter o valor `10.99f32` para `i8` resultará no valor 10.
- Converter inteiro em ponto flutuante fará com que o valor inteiro seja armazenado na memória ou utilizado como se fosse um valor real.
- Converter ponto flutuante de 32 bits em ponto flutuante de 64 bits não produz nenhuma perda do valor.
- Converter ponto flutuante de 64 bits em ponto flutuante de 32 bits acarreta perda do valor, produzindo um valor mais próximo do possível.

O programa seguinte demonstra alguns efeitos de conversão de dados com modelagem de tipos numéricos. Para isso, crie o projeto `c06ex01`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let valor1 = true as u8; // mostra 1
```

```

let valor2 = false as u8; // mostra 0
let valor3 = 65u8 as char; // mostra A
let valor4 = -5i8 as u8; // mostra 251
let valor5 = 10.99f32 as i8; // mostra 10
let valor6 = 513u32 as u8; // mostra 1
let valor7 = 987u32 as u64; // mostra 987
let valor8 = -9i8 as i16; // mostra -9

println!("{}", valor1);
println!("{}", valor2);
println!("{}", valor3);
println!("{}", valor4);
println!("{}", valor5);
println!("{}", valor6);
println!("{}", valor7);
println!("{}", valor8);
println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, serão apresentados os valores indicados nas linhas de comentário do código, demonstrando as principais conversões da modelagem de tipos numéricos.

A conversão baseada em modelagem por ponteiro será abordada após a seção sobre ponteiros (a seguir), neste capítulo.

6.2 Ponteiros

Ponteiro é uma maneira de se fazer referência a determinado endereço de memória por meio de uma variável definida para esta finalidade. Neste sentido, a linguagem Rust contém alguns tipos de dados para esta ação. O uso de ponteiros ocorre na linguagem Rust de forma mais segura que em outras linguagens, e, por conseguinte, estes são definidos de forma mais simples. A linguagem Rust utiliza os tipos de ponteiro: *reference pointers* (ponteiros de referência), *boxes pointers* (ponteiros de caixa, também

conhecidos como ponteiros exclusivos) e *raw pointers* (ponteiros brutos, também conhecidos como ponteiros inseguros) (BLANDY & ORENDORFF, 2016, p. 52-53). Apesar de haver autores que qualificam outros tipos de ponteiros em Rust, neste trabalho será mantido o que é exposto por Blandy & Orendorff.

No capítulo anterior, o conceito de *reference pointers* foi aplicado no uso da passagem de parâmetro por referência. O tipo ponteiro de referência é uma forma básica utilizada na linguagem Rust. Uma referência é uma maneira de apontar para qualquer valor em qualquer lugar da memória por meio dos operadores (&) e comercial e (*) asterisco. Enquanto o operador & estabelece o acesso ao conteúdo do endereço apontado, o operador * faz com que seja possível manipular o conteúdo do endereço referenciado por &.

Os ponteiros tipo *boxes pointers* obtidos a partir do uso do recurso `Box::new()` são a forma mais simples e segura de fazer uso de ponteiros na linguagem Rust. Este tipo de ponteiro, quando em uso, aloca o espaço de memória suficiente para seu uso e libera o espaço reservado de memória quando o ponteiro deixa de ser usado.

Já os ponteiros tipo *raw pointers* são ponteiros brutos que não garantem que seu apontamento seja feito para um endereço válido de memória. O uso desse recurso obriga o programador a retirar a referência da memória, uma vez que a ação não é realizada de forma automática pela linguagem.

Os exemplos de ponteiros neste tópico são fornecidos a título de ilustração. Sendo assim, os exemplos demonstram o uso de cada uma das formas da maneira mais simples possível. O uso mais apurado de ponteiros ocorrerá de forma contextualizada nos próximos capítulos na medida em que for necessário.

6.2.1 Ponteiro de referência

Para fazer uso de ponteiros em Rust é importante saber a diferença entre desestruturação e desreferência¹ e não confundir tais conceitos com os conceitos usados nas linguagens C e C++: a desestruturação de um

ponteiro é estabelecida com o uso do `&` (e comercial), de `ref` e `ref mut`, enquanto a desreferência é estabelecida com o uso do `*` (asterisco).

O programa seguinte demonstra o uso de ponteiros de referência com desreferência e desestruturação sobre um valor inteiro sinalizado de 32 bits. Para isso, crie o projeto `c06ex03`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let valor = &10i32;

    match valor {
        &val => println!("Valor por desestruturação (&) ... {:?}",
val),
    }

    match *valor {
        val => println!("Valor por desreferência (*) ... {:?}", val),
    }
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, serão apresentadas as mensagens com a indicação do valor `10` tanto para a ação de desestruturação como para ação de desreferência.

A instrução `let valor = &10i32;` estabelece o uso de uma referência de 32 bits. O símbolo `&` indica que uma referência ao valor `10` está sendo atribuída à variável `VALOR`.

O trecho de código que sinaliza o uso do primeiro bloco do comando `match` faz a comparação do conteúdo entre a referência da variável `VALOR` e a referência da variável `VAL` (variável definida dentro do bloco do primeiro `match` com uso do símbolo `&`). Se a referência de `&val` for compatível com a referência de 32 bits da variável `VALOR` (), então ter-se-á a desestruturação

da referência do `&10i32` da variável `VALOR`, atribuindo-se este valor à variável `VAL`, que será apresentada junto à instrução `println!("Valor por desestruturação (&) ...: {:?}", val)`.

O trecho de código que sinaliza o uso do segundo bloco do comando `match` faz a desreferência do símbolo `&` ao estabelecer o uso do símbolo `*` (asterisco) antes do nome da variável `VALOR` por meio do trecho de comando `match *valor`. A ocorrência em definir o uso do `*` faz com que não seja necessário usar o símbolo `&` junto à instrução de atribuição `val => println!("Valor por desreferência (*) ...: {:?}", val)`.

Veja que há duas maneiras de obter o valor referenciado e associado a uma variável com ponteiro de referência: uma maneira por desestruturação com o uso de `&`, e outra, por desreferência com o uso de `*` antes da variável com valor referenciado.

Além de estabelecer a referência a um valor por meio de ponteiro de referência, há a possibilidade de fazer referência a uma variável. Neste caso é necessário usar o comando `ref` antes da definição de uma variável após o uso do comando `let`.

O programa seguinte demonstra o uso de ponteiro de referência associado a uma variável que armazena um valor inteiro sinalizado de 32 bits. Para isso, crie o projeto `c06ex04`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let ref valor = 10i32;

    match valor {
        &val => println!("Valor por desestruturação (&) ...: {:?}",
val),
    }

    match *valor {
        val => println!("Valor por desreferência (*) ...: {:?}", val),
    }
}
```

```
println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}
```

A instrução `let ref valor = 10i32`; estabelece o uso de uma referência de 32 bits. O símbolo `&` é omitido da definição do valor por estar em uso o comando `ref` antes da variável `valor`.

Observe que os projetos `c06ex03` e `c06ex04` têm funcionalidades idênticas, tendo como diferença apenas a forma de definir os ponteiros de referência, sendo `&` para um valor e `ref` para uma variável.

Outro detalhe a ser considerado é o fato de se poder recuperar a referência de valores ou variáveis que inicialmente não foram referenciadas.

O programa seguinte demonstra o uso do ponteiro de referência associado a uma variável não referenciada que utiliza um valor inteiro sinalizado de 32 bits. Para isso, crie o projeto `c06ex05`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let valor = 10i32;

    match valor {
        ref val => println!("Valor obtido por referência: {:?}", val),
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

A instrução `let valor = 10i32`; estabelece o uso de uma variável de 32 bits com valor 10. A instrução `ref val => println!("Valor obtido por referência: {:?}", val)` estabelece a referência definida junto à variável `VAL`

advinda da variável `VALOR`.

Atente para o fato de que, se estiver em uso uma variável mutável como `let mut valor = 10i32;`, haverá necessidade de estabelecer também uma referência mutável. Neste caso, com uma instrução dentro do bloco `match` similar a `ref mut val => println!("Valor obtido por referência: {:?}", val)`.

6.2.2 Ponteiro exclusivo

Para fazer uso de ponteiro exclusivo é necessário solicitar, por meio do programa, a alocação no *heap* de memória e inicializar o espaço alocado com determinado valor. A alocação de memória é realizada pelo recurso `Box::new(valor)` associado a uma variável ponteiro.

A variável associada por `Box::new()` é o ponteiro que será usado para apontar ao valor definido na memória. Apesar de não ser exigência da linguagem, é interessante manter uma forma de identificar visualmente variáveis comuns de variáveis ponteiros. Neste sentido, será considerado o uso da letra *pe* antes do nome da variável para indicar o uso de uma variável ponteiro.

O programa seguinte mostra a definição de uma variável do tipo ponteiro chamada `PA` e associada ao valor `10`. Neste caso, o valor `10` será alocado em uma posição de memória que será associada à variável `PA`. Para isso, crie o projeto `c06ex06`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let pa = Box::new(10);
    println!("Ponteiro [pa] = {}", *pa);
    println!("Endereço [pa] = {:p}", pa);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

```
}
```

A variável `PA` é um ponteiro para um local na pilha que contém o valor `10` armazenado. A instrução `println!("Ponteiro [a] = {}", *pa);` mostra o resultado `10` e a instrução `println!("Endereço [a] = {:p}", pa);` mostra, por meio de `{:p}`, o valor do endereço de memória onde o conteúdo `10` está armazenado.

O programa seguinte efetua a entrada de um valor inteiro sinalizado de 64 bits e apresenta o valor informado e o local da memória onde o valor foi armazenado. Para isso, crie o projeto `c06ex07`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut valor = String::new();
    let vlr: i64;
    let pvlr;

    print!("Informe um valor inteiro: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).unwrap();
    vlr = valor.trim().parse::<i64>().unwrap();

    pvlr = Box::new(vlr);

    println!("Ponteiro [pvlr] = {}", pvlr);
    println!("Endereço [pvlr] = {:p}", pvlr);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Nesta versão o programa mostra, por meio da instrução `pvlr = Box::new(vlr);`, a atribuição do valor informado na variável `VLR` colocando seu conteúdo na área de memória apontada pelo ponteiro exclusivo `PVLR`.

A definição de ponteiros e os valores associados são, por padrão, imutáveis. Não é possível definir ponteiro mutável com valor imutável e vice-versa. É preciso estabelecer de forma sempre coerente os tipos de comportamento das variáveis e dos ponteiros com o uso ou não de mutabilidade/imutabilidade.

A alocação de memória para ponteiro exclusivo é feita por meio da instrução `let pa = Box::new(valor);` ou pela instrução `let mut pa = Box::new(valor);`, onde o ponteiro `PA` é o ponteiro e `VALOR` pode ser um valor definido (`c05ex01`) para a instrução ou a definição de uma variável que contenha um valor (`c03ex02`). No entanto, após usar a definição do ponteiro, é possível redefinir o ponteiro em uso.

O programa seguinte mostra a definição de um ponteiro associado a uma variável que sofre sobreposições de valores no código marcado em negrito. Para isso, crie o projeto `c06ex08`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let a = 3;
    let mut pb = Box::new(a);

    println!("Variável [a] = {}", a);
    println!("Ponteiro [b] = {}", pb);

    *pb = 45;

    println!();
    println!("Variável [a] = {}", a);
    println!("Ponteiro [b] = {}", pb);
    pb = Box::new(9);

    println!();
    println!("Variável [a] = {}", a);
    println!("Ponteiro [b] = {}", pb);
    println!();
    print!("Tecle <Enter> para encerrar...");
```

```

        io::stdout().flush().unwrap();
        io::stdin().read(&mut [0u8]).unwrap();
    }

```

Na execução do programa ocorre primeiro a apresentação dos valores 3 e 3. Depois a instrução `*pb = 45;` faz com que o valor 3 do ponteiro `PB` seja alterado para 45. Outra alteração ocorre com a instrução `pb = Box::new(9);`, que estabelece o valor 9 para o ponteiro `PB`. Observe que há duas maneiras de redefinir o valor de um ponteiro mutável exclusivo: uma usando `Box::new()` e outra usando o símbolo `(*)` asterisco.

6.2.3 Ponteiro bruto

Os ponteiros brutos (*raw pointers*) são uma forma especial de utilizar ponteiros na linguagem Rust. No entanto é fundamental saber que o acesso a essa forma de ponteiro não executa as ações de segurança que a linguagem normalmente garante a outros tipos de ponteiro. Portanto, são ponteiros que permitem acessos inseguros à memória, levando o programador a ter de tomar muito mais cuidado ao utilizar ponteiros brutos e na liberação da área de memória apontada. Os ponteiros brutos na linguagem Rust se assemelham à forma de ponteiros usados nas linguagens C e C++, por isso proporcionam a mesma liberdade que se tem em C e C++. O uso de ponteiros brutos é feito a partir do trecho de sintaxe:

```

*const <tipo>
*mut <tipo>

```

O uso do ponteiro `*const` estabelece uma variável do tipo ponteiro bruto como imutável, e o uso do ponteiro `*mut` estabelece uma variável do tipo ponteiro bruto como mutável. Para fazer uso dos especificadores de ponteiros brutos, recomenda-se usá-los em ações de conversão de tipos de dados associadas às variáveis que os representem.

Um ponteiro do tipo *raw pointer* pode ser apontado para uma área inválida de memória (o que pode ser um problema), por exemplo, uma área de memória que anteriormente tenha sido liberada, uma área de memória que contenha um valor nulo ou uma área de memória que contenha um

valor de um tipo diferente de dado. Essas ocorrências são ações inseguras e podem ocasionar problemas e falhas na execução de um programa.

O problema de segurança no uso de ponteiros brutos não está na sua definição, mas sim no momento de seu uso. Um detalhe a ser considerado no uso de ponteiros brutos é a necessidade de fazer a desreferência do ponteiro de tipo bruto dentro de um bloco inseguro definido com o comando `unsafe` (BALBAERT, 2015, p. 146-147). Um ponteiro bruto pode ser facilmente usado, mas não pode ser desreferenciado como os demais ponteiros, por isso essa operação é executada dentro de um bloco inseguro (bloco `unsafe`).

O programa do exemplo seguinte, adaptado de Balbaert (2017, p. 147), demonstra a definição de um ponteiro bruto associado a uma variável imutável e sua desreferência obrigatória dentro de um bloco `unsafe`. Para isso, crie o projeto `c06ex09`:

```
use std::io;
use std::io::prelude::*;
fn main() {
    let vlr = 10;
    let imutavel = &vlr as *const i32;

    unsafe {
        let valor = *imutavel;
        println!("Valor do ponteiro bruto imutável: {}", valor);
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, o valor `10` associado à variável imutável denominada `IMUTAVEL` é atribuído à variável `VALOR` por desreferência (`let valor = *imutavel;`) dentro do bloco `unsafe`, sendo este apresentado junto à instrução `println!("Valor do ponteiro bruto imutável: {}", valor);`. Se o

bloco `unsafe` não for definido, ocorrerá um erro no momento da compilação do programa.

Pelo fato de o programa do projeto `c06ex09` fazer uso de um ponteiro bruto imutável (`*const`), a definição da variável `VLR` junto à instrução `let vlr = 10;` é também imutável. Se o ponteiro bruto em uso fosse mutável, a variável em uso deveria também ser mutável.

O programa seguinte demonstra o uso de dois ponteiros brutos – um mutável e outro imutável – vinculados a uma mesma variável. Para isso, crie o projeto `c06ex10`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut vlr = 10;
    let pb1 = &vlr as *const i32;
    let pb2 = &mut vlr as *mut i32;

    unsafe {
        println!("Ponteiro bruto 1: {}", *pb1);
        println!("Ponteiro bruto 2: {}", *pb2);
    }
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, o valor `10` é mostrado para o relacionamento estabelecido junto aos ponteiros mutável e imutável. A ação de estabelecer um ponteiro mutável e um ponteiro imutável para a mesma área de memória é ilegal (ou a posição é mutável ou é imutável), a menos que estejam em uso ponteiros brutos conforme indicados.

6.3 Terminal ANSI

O modo *Terminal ANSI* oferece controle padronizado das ações vinculadas

ao uso do terminal de vídeo e teclado. Terminal é um equipamento que oferece ao usuário uma interface de acesso ao computador, como o teclado e o monitor de vídeo.

O uso do equipamento chamado *terminal* é comum em computadores de grande porte, em que se tem um console de acesso às funções do computador que estejam distantes do terminal, diferentemente de um mini ou microcomputador, cujas funções de entrada e saída em vídeo (terminal) e de processamento (CPU) são conectadas próximo umas das outras, quando não no mesmo console, como é o caso dos notebooks.

O modelo mais famoso de *terminal* para computadores de grande e médio porte foi o VT100, da empresa DEC. Esse terminal foi o primeiro a ter um microprocessador e um dos primeiros a incorporar o controle por sequência de *escape ANSI*. Como a sequência de *escape* foi uma proposta de padronização do órgão ANSI (*American National Standards Institute* – Instituto Nacional Americano de Padrões) para os recursos de acesso a terminais da época, o padrão do modelo VT100 acabou por prevalecer até os dias atuais em diversos sistemas operacionais, como ocorre com o Unix (incluindo-se os muitos sabores Linux e Mac OS). Em relação aos sistemas operacionais da empresa Microsoft, o recurso está disponível, de forma indireta, nos antigos sistemas MS-DOS e em todas as versões do Windows de 32 bits até a versão 10.

Para usar os recursos do modo Terminal ANSI nas versões de 64 bits (e também 32 bits, se for o caso) do Windows e do Windows 10 é necessário obter ferramentas desenvolvidas por terceiros, como é o caso do programa ConEmu, ferramenta adequada para ser usada com a linguagem Rust.

O programa ConEmu pode ser obtido no endereço <https://conemu.github.io/downloads>. Baixe a última versão compatível com o sistema de 32 ou 64 bits, a partir do botão **Download** no link **Download ConEmu Preview, Installer (32-bit, 64-bit) - 6.35 MB**.

Proceda com a instalação e, em seguida, por meio do arquivo copiado com um duplo clique do ponteiro do mouse, acione o programa e selecione o tipo de instalação **x64** para um sistema operacional Windows

de 64 bits ou x86 para um sistema operacional Windows de 32 bits. Siga as instruções apresentadas. Ao final da instalação será criado o ícone do programa na área de trabalho.

Adicione o ícone do programa ConEmu, confirme a execução e, quando apresentada a tela de uso, semelhante à tela do *prompt de comando* (CMD), selecione na barra de ferramenta o último ícone à direita. Ao ser apresentado o menu de opções, selecione a opção **Settings**. Na caixa de diálogo apresentada, selecione **Colors** em **Features** ao lado esquerdo da tela e em **Schemes** selecione a opção **Terminal.app** e acione o botão **Save settings**.

O uso do programa ConEmu ocorre de forma idêntica à do programa CMD. Sendo assim, utilize-o normalmente como o *prompt de comando*.

Para controlar o vídeo ou o teclado por meio dos recursos do modo *Terminal ANSI*, é necessário usar comandos específicos, precedidos do código de controle de *escape*, que, na linguagem Rust, consiste no uso do código numérico decimal 27 de acesso à função da tecla <Esc>.

Os comandos (sequência de escape) são divididos em dois grupos de ação (vídeo e teclado). No grupo *vídeo*, é possível manipular vídeo, controlar e movimentar o cursor e os atributos de tela. No grupo *teclado*, é possível redefinir a ação de qualquer tecla.

A seguir, é apresentada apenas uma parte da tabela com os códigos de controle de vídeo. As funções do teclado não serão usadas, por isso são omitidas. Para mais detalhes sobre códigos de controle de *escape*, consulte o site MS-DOS 6 - Device Drivers, no endereço <http://bluesock.org/~willg/dev/ansi.html>.

As tabelas de 6.1 até 6.4 apresentam alguns dos códigos de controle ANSI para limpar vídeo, movimentar cursor, determinar a posição do cursor, alterar a apresentação de cores de fundo e de texto. A indicação ESC deve ser substituída pelo código numérico decimal 27.

Tabela 6.1 – Código ANSI para limpar vídeo

Limpeza do vídeo		
Sequência ESC	Função	Descrição
ESC[2J	limpar vídeo	Efetua a limpeza do vídeo e posiciona o cursor no topo esquerdo da tela.
ESC[K	limpar linha	Efetua a limpeza da linha a partir da posição em que o cursor estiver parado.

Tabela 6.2 – Código ANSI para manipular cursor

Posição do cursor		
Sequência ESC	Função	Descrição
ESC[s	salva posição	Salva a posição atual do cursor para posterior restauração.
ESC[u	restaura posição	Restaura a posição do cursor previamente salvo com a sequência de salvamento.

Tabela 6.3 – Código ANSI para movimentar cursor

Movimentação do cursor		
Sequência ESC	Função	Descrição
ESC[valor A	cursor para cima	Move o cursor para cima o número de vezes estabelecido no parâmetro valor.
ESC[valor B	cursor para baixo	Move o cursor para baixo o número de vezes estabelecido no parâmetro valor.
ESC[valor C	cursor para direita	Move o cursor para a direita o número de vezes estabelecido no parâmetro valor.
ESC[valor D	cursor para esquerda	Move o cursor para a esquerda o número de vezes estabelecido no parâmetro valor.
ESC[lin;col H	posiciona cursor	Move o cursor para a lin (linha 1 a 25) e a col (coluna 1 a 80) especificadas.

É preciso considerar que o recurso de modo *Terminal ANSI* é utilizado diretamente pelo sistema operacional e não pelo programa em si ou pela linguagem de programação. O programa apenas o ativa por intermédio de suas instruções. Na linguagem Rust, o uso desses recursos é utilizado a partir da instrução:

```
print!("{}", <código ANSI>, 27 as char);
```

Em que *código ANSI* é a definição de um código válido do modo *Terminal ANSI*. O valor 27 é convertido para o tipo de dado *char* por intermédio do trecho de código *as char*, que é definido na *string* junto ao código de formato { }.

Tabela 6.4 – Código ANSI para definição de cores

Modo caractere		
Sequência ESC	Função	Descrição
ESC[atr;...;atr m	Alteração de modo	Altera, por meio do parâmetro atr (atributo), as configurações indicadas na lista seguinte. Cor de texto: 30 - preto 31 - vermelho 32 - verde 33 - amarelo 34 - azul 35 - magenta 36 - ciano 37 - branco Cor de fundo: 40 - preto 41 - vermelho 42 - verde 43 - marrom 44 - azul 45 - magenta 46 - ciano 47 - cinza-claro

Em seguida, é exibido um programa exemplo que faz o tratamento de

cores de fundo (segundo plano) e cores de texto (primeiro plano) por meio do modo *Terminal ANSI*. É possível executá-lo nos sistemas operacionais Linux, OS X e Windows com o uso do programa ComEmu. Para isso, crie o projeto `c06ex11`.

```
use std::io;
use std::io::prelude::*;

// Constantes para as cores de texto e fundo

const BLACK: u8 = 0; // Preto
const BLUE: u8 = 1; // Azul
const GREEN: u8 = 2; // Verde
const CYAN: u8 = 3; // Ciano
const RED: u8 = 4; // Vermelho
const MAGENTA: u8 = 5; // Magenta
const BROWN: u8 = 6; // Marrom
const LGRAY: u8 = 7; // Cinza claro
// Constantes para as cores de texto

const DGRAY: u8 = 8; // Cinza escuro
const LBLUE: u8 = 9; // Azul claro
const LGREEN: u8 = 10; // Verde claro
const LCYAN: u8 = 11; // Ciano claro
const LRED: u8 = 12; // Vermelho claro
const LMAGENTA: u8 = 13; // Magenta claro
const YELLOW: u8 = 14; // Amarelo
const WHITE: u8 = 15; // Branco

fn normal() {
    print!("{}", 0m, 27 as char);
}

fn clear() {
    print!("{}", 2J, 27 as char);
}

fn background(cor: u8) {
    match cor {
```

```

    BLACK => print!("{}", 40m, 27 as char), // Preto
    BLUE => print!("{}", 44m, 27 as char), // Azul
    GREEN => print!("{}", 42m, 27 as char), // Verde
    CYAN => print!("{}", 46m, 27 as char), // Ciano
    RED => print!("{}", 41m, 27 as char), // Vermelho
    MAGENTA => print!("{}", 45m, 27 as char), // Magenta
    BROWN => print!("{}", 43m, 27 as char), // Marron
    LGRAY => print!("{}", 47m, 27 as char), // Cinza claro
    _ => {},
}
}

fn text(cor: u8) {
    match cor {
        BLACK => print!("{}", 30m, 27 as char), // Preto
        BLUE => print!("{}", 34m, 27 as char), // Azul
        GREEN => print!("{}", 32m, 27 as char), // Verde
        CYAN => print!("{}", 36m, 27 as char), // Ciano
        RED => print!("{}", 31m, 27 as char), // Vermelho
        MAGENTA => print!("{}", 35m, 27 as char), // Magenta
        BROWN => print!("{}", 33m, 27 as char), // Marron
        LGRAY => print!("{}", 37m, 27 as char), // Cinza claro
        DGRAY => print!("{}", 1;30m, 27 as char), // Cinza claro
        LBLUE => print!("{}", 1;34m, 27 as char), // Azul claro
        LGREEN => print!("{}", 1;32m, 27 as char), // Verde claro
        LCYAN => print!("{}", 1;36m, 27 as char), // Ciano claro
        LRED => print!("{}", 1;31m, 27 as char), // Vermelho claro
        LMAGENTA => print!("{}", 1;35m, 27 as char), // Magenta claro
        YELLOW => print!("{}", 1;33m, 27 as char), // Amarelo
        WHITE => print!("{}", 1;37m, 27 as char), // Branco
        _ => {},
    }
}

fn position(linha: u8, coluna: u8) {
    if coluna >= 1 && coluna <= 80 && linha >= 1 && linha <= 24 {
        print!("{}", {}, {}, 27 as char, linha, coluna);
    }
}

```

```
fn main() {  
    clear();  
    println!("Teste de padrão de Cores com Código Numérico\n\n");  
  
    background(1);  
    text(14);  
    println!("Fundo: Azul // Texto: Amarelo");  
    println!();  
    normal();  
  
    background(1);  
    text(7);  
    println!("Fundo: Azul // Texto: Cinza claro");  
    println!();  
    normal();  
  
    background(4);  
    text(14);  
    println!("Fundo: Vermelho // Texto: Amarelo");  
    println!();  
    normal();  
  
    background(0);  
    text(2);  
    println!("Fundo: Preto // Texto: Verde");  
    println!();  
    normal();  
  
    background(7);  
    text(12);  
    println!("Fundo: Cinza claro // Texto: Vermelho claro");  
    println!();  
    normal();  
  
    background(BROWN);  
    text(WHITE);  
    println!("Fundo: Marrom // Texto: Branco");  
    println!();  
}
```

```

    normal();

    println!();
    position(24,1);
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Durante a ação de compilação do programa aparecem notificações de advertência indicando que no código as sub-rotinas `lower()` e `high()` não estão sendo usadas. Para desabilitar a mensagem de advertência neste projeto, basta definir como primeira linha do código a instrução `#![allow(dead_code)]`, em que `dead_code` é o atributo que se refere à definição de detecção de sub-rotinas não usadas.

O atributo `dead_code` tem funcionalidade similar ao atributo `unused_variables`, relacionado às variáveis não utilizadas. Se for preciso estabelecer mais de um atributo de exceção, pode ser usada a instrução `#![allow(unused_variables, dead_code)]`.

Ao ser executado o programa, ocorre o tratamento das cores de fundo (segundo plano) e de texto (primeiro plano) por meio das sub-rotinas `background()` e `text()`. No exemplo, são utilizadas para as cores de fundo apenas oito tonalidades.

Observe o trecho do código com o uso do comando `const` para estabelecer diversas constantes para o programa. Cada constante tem um código de cor associado. Além de fornecer o número da cor para as funções externas `background()` e `text()`, é possível informar o nome da cor, como demonstrado.

As sub-rotinas `backgroud()` e `text()` para controle de seus recursos utilizam o comando `match` com a definição da última ação, controlada pelo símbolo *underline* em branco, uma vez que não há a necessidade de indicar uma ação de erro para o conjunto de valores usados.

6.4 Biblioteca definida pelo programador

As sub-rotinas definidas no projeto `c06ex11` podem ser utilizadas em outros programas. Para evitar o inconveniente de sempre ter de escrevê-las no código de um novo programa, é possível guardá-las em um arquivo pessoal de biblioteca externa.

Com base no exposto, será definido um novo projeto que agregue o código do programa em si e o código da biblioteca. Além dos arquivos do projeto será necessário configurar o arquivo `cargo.toml`. Para isso, crie o projeto `c06ex12`.

Na pasta/diretório `src` de `c06ex12` em `estudo` é necessário definir o arquivo da biblioteca. As funcionalidades de uma biblioteca, para serem usadas externamente, devem ser declaradas com o comando `pub`, sejam sub-rotinas, constantes ou outro recurso. Se `pub` não for definido à frente de um recurso, esse recurso só poderá ser usado dentro do escopo do arquivo da biblioteca. Para isso, defina este arquivo com o nome `ecra.rs` e escreva o seguinte código:

```
pub const BLACK: u8 = 0; // Preto
pub const BLUE: u8 = 1; // Azul
pub const GREEN: u8 = 2; // Verde
pub const CYAN: u8 = 3; // Ciano
pub const RED: u8 = 4; // Vermelho
pub const MAGENTA: u8 = 5; // Magenta
pub const BROWN: u8 = 6; // Marrom
pub const LGRAY: u8 = 7; // Cinza claro
pub const DGRAY: u8 = 8; // Cinza escuro
pub const LBLUE: u8 = 9; // Azul claro
pub const LGREEN: u8 = 10; // Verde claro
pub const LCYAN: u8 = 11; // Ciano claro
pub const LRED: u8 = 12; // Vermelho claro
pub const LMAGENTA: u8 = 13; // Magenta claro
pub const YELLOW: u8 = 14; // Amarelo
pub const WHITE: u8 = 15; // Branco

pub fn normal() {
    print!("{}", [0m", 27 as char);
}
```

```

pub fn clear() {
    print!("{}", 27 as char);
}
pub fn background(cor: u8) {
    match cor {
        BLACK => print!("{}", 27 as char), // Preto
        BLUE => print!("{}", 27 as char), // Azul
        GREEN => print!("{}", 27 as char), // Verde
        CYAN => print!("{}", 27 as char), // Ciano
        RED => print!("{}", 27 as char), // Vermelho
        MAGENTA => print!("{}", 27 as char), // Magenta
        BROWN => print!("{}", 27 as char), // Marron
        LGRAY => print!("{}", 27 as char), // Cinza claro
        _ => {},
    }
}

pub fn text(cor: u8) {
    match cor {
        BLACK => print!("{}", 27 as char), // Preto
        BLUE => print!("{}", 27 as char), // Azul
        GREEN => print!("{}", 27 as char), // Verde
        CYAN => print!("{}", 27 as char), // Ciano
        RED => print!("{}", 27 as char), // Vermelho
        MAGENTA => print!("{}", 27 as char), // Magenta
        BROWN => print!("{}", 27 as char), // Marron
        LGRAY => print!("{}", 27 as char), // Cinza claro
        DGRAY => print!("{}", 27 as char), // Cinza claro
        LBLUE => print!("{}", 27 as char), // Azul claro
        LGREEN => print!("{}", 27 as char), // Verde claro
        LCYAN => print!("{}", 27 as char), // Ciano claro
        LRED => print!("{}", 27 as char), // Vermelho claro
        LMAGENTA => print!("{}", 27 as char), // Magenta claro
        YELLOW => print!("{}", 27 as char), // Amarelo
        WHITE => print!("{}", 27 as char), // Branco
        _ => {},
    }
}

```



```
pub fn position(linha: u8, coluna: u8) {
    if coluna >= 1 && coluna <= 80 && linha >= 1 && linha <= 24 {
        print!("{}", "{}[{}];{}H", 27 as char, linha, coluna);
    }
}
```

Após definir o arquivo da biblioteca, será preparado o código do programa. Neste caso, abra o arquivo de programa `main.rs` da pasta/diretório `src` de `c06ex12` em estudo e altere o conteúdo do arquivo com o código seguinte:

```
extern crate ecralib;

use ecralib::*;

use std::io;
use std::io::prelude::*;

fn main() {
    clear();
    println!("Teste de padrão de Cores com Código Numérico\n\n");

    background(1);
    text(14);
    println!("Fundo: Azul // Texto: Amarelo");
    println!();
    normal();

    background(1);
    text(7);
    println!("Fundo: Azul // Texto: Cinza claro");
    println!();
    normal();

    background(4);
    text(14);
    println!("Fundo: Vermelho // Texto: Amarelo");
    println!();
    normal();
}
```

```

background(0);
text(2);
println!("Fundo: Preto // Texto: Verde");
println!();
normal();

background(7);
text(12);
println!("Fundo: Cinza claro // Texto: Vermelho claro");
println!();
normal();

background(BROWN);
text(WHITE);
println!("Fundo: Marrom // Texto: Branco");
println!();
normal();

println!();
position(24,1);
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Antes de executar o programa é necessário definir a configuração do arquivo `cargo.toml`. Assim, é necessário acrescentar no arquivo `cargo.toml` a seção `lib` conforme a parte marcada em negrito:

```

[package]
name = "c06ex12"
version = "0.1.0"
authors = ["Fulano"]

[dependencies]

[lib]
name = "ecralib"
path = "src/ecra.rs"

```

A seção `lib` no campo `name` tem o nome da *crate* (o módulo) usada no programa principal, sendo esta acionada com a instrução `extern crate ecralib;`, e o campo identifica o local e o nome do arquivo que contém o código da biblioteca (`ecra.rs`).

Após a escrita dos códigos em `src` de `c06ex12` e a definição do arquivo `cargo.toml` em `estudo`, execute a partir da pasta/diretório `estudo` a instrução:

```
cargo run
```

Ao ser executado o programa, o efeito visual apresentado do projeto `c06ex12` é idêntico ao resultado do programa do projeto `c06ex11`. Observe que no código do programa `main.rs` é utilizada a instrução `extern crate ecralib;`, que faz a chamada do módulo *crate* externo `ecralib`. A instrução `use ecralib::*` é usada para colocar em uso os recursos públicos que estão disponíveis na *crate* `ecralib`.

6.5 Metaprogramação com macros

A linguagem Rust possibilita definir macros que são, em essência, uma forma de metaprogramação. A metaprogramação é uma maneira de escrever programas que manipulam ou escrevem outros programas, bem como seus dados. As macros não são novidade nesta obra, pois o uso de instruções com o recurso `println!()` se caracteriza por ser o uso de uma macro. O símbolo `!` exclamação é usado na nomenclatura do nome de uma macro, diferenciando-a de uma estrutura de sub-rotina.

A definição básica de uma macro é realizada em Rust a partir de uma macro específica para essa finalidade, chamada `macro_rules!` com sintaxe:

```
macro_rules! <nome> {  
    (padrão1) => (ação);  
    (padrão2) => (ação);  
    ...  
    (padrãoN) => (ação);  
}
```

ou

```

macro_rules! <nome> {
    (padrão1) => {
        ação1;
        ação2;
        ...
        açãoN;
    };
    (padrão2) => {
        ação1;
        ação2;
        ...
        açãoN;
    };
    ...
    (padrãoN) => {
        ação1;
        ação2;
        ...
        açãoN;
    };
}

```

A definição de uma macro é realizada a partir da macro `macro_rules!`, na qual se estabelece o conjunto de regras a ser utilizadas. Uma regra é estabelecida a partir da indicação de um padrão, do uso do operador `=>` (`matcher`) e das ações que serão substituídas no momento da compilação do programa. No trecho das ações é possível estabelecer mais de uma instrução finalizada com (;) ponto e vírgula (BALBAERT, 2015, p. 122 & RUNGE, 2016, p. 21).

Como exemplo básico de uso de uma macro, o programa seguinte, por meio da macro `escreva!()`, apresenta uma mensagem de saudação (adaptado de Kaihlavirta, 2017, p. 157). Para isso, crie o projeto `c06ex13`.

```

use std::io;
use std::io::prelude::*;

macro_rules! escreva {
    ($esc:expr) => (print!(concat!($esc, "\n")));
}

```

```

    ($esc:expr, $($param:tt)*) => (print!(concat!($esc, "\n"),
    $($param)*));
}

fn main() {
    escreva!("Estudo da linguagem Rust");

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, a instrução `escreva!("Estudo da linguagem Rust");` efetua sua ação apresentando a mensagem indicada.

No programa do projeto `c06ex13`, a macro `escreva!()` faz uso dos padrões `($esc:expr)` e `($esc:expr, $($param:tt)*)`, em que os elementos indicados após o símbolo (\$) cifrão são os padrões definidos pela macro, como se fossem os parâmetros das sub-rotinas de função e/ou procedimento.

O padrão `($esc:expr)` é usado se apenas uma expressão for indicada, e o padrão `($esc:expr, $($param:tt)*)` é usado se for informada uma expressão com um conjunto de parâmetros. A expressão (que no caso é a mensagem `Estudo da linguagem Rust`) é recebida pela variável `ESC` (em `esc:expr`), e o argumento é recebido pela variável `PARAM` (em `arg:tt`). As variáveis podem ser definidas a partir de qualquer nome: `ESC` vem de `escreva` e `PARAM` vem de `parâmetro`.

Os fragmentos especiais `exp` e `tt` são siglas que correspondem a `expression` (expressão) e `token tree` (árvore de token), respectivamente, e devem ser indicados como estão escritos. Além de `exp` e `tt` existem outros fragmentos especiais que podem ser utilizados, como `ident`, `block`, `stmt`, `pat`, `ty`, `path`, `meta`, `item` e `vis`.

A macro `concat!()` tem por finalidade concatenar todos os elementos usados como parâmetros da macro `escreva!()` em um *string* estático.

6.6 Laço iterativo compassado

A linguagem Rust não tem uma maneira direta de executar laços iterativos de maneira compassada, como, por exemplo, contar de 1 a 10 de 2 em 2. Esse tipo de ação pode ser facilmente implementado com os laços do tipo `while` ou `loop`, mas não é possível ser executado pelo laço `for`.

Para a execução de um laço compassado iterativo pode ser usada a macro `cfor!()`, que tem por finalidade fornecer uma forma de execução similar às das linguagens de programação da família C. No entanto, para que essa macro possa ser utilizada, é necessário comprimir certos requisitos, como a configuração do arquivo `Cargo.toml`, e fazer uso das instruções `#[macro_use]` e `extern crate cfor;`.

O programa a seguir apresenta os valores de 1 a 10 de 2 em 2. Para isso, crie o projeto `c06ex14`.

Abra o arquivo `Cargo.toml`, acrescente a linha mostrada em negrito, salvando o arquivo na sequência:

```
[package]
name = "c06ex14"
version = "0.1.0"
authors = ["Manzano"]
```

```
[dependencies]
cfor = "1.1.0"
```

Então, execute o comando:

```
cargo build
```

Depois, abra o arquivo `main.rs` da pasta/diretório `src` de `c06ex14` no diretório (pasta) `estudo` e altere o código do arquivo com o conteúdo seguinte:

```
#[macro_use]
extern crate cfor;

use std::io;
use std::io::prelude::*;

fn main() {
```

```

    cfor!(let mut i = 1; i <= 10; i += 2; {
        println!("{}", i);
    });

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, serão apresentados os valores 1, 3, 5, 7 e 9 como se fosse a instrução `for()` da família de linguagens C.

Para usar a macro `for!()` é necessário, além de configurar o arquivo `Cargo.toml`, fazer uso da instrução `#[macro_use]`, que informa que uma macro será usada a partir de uma biblioteca diferente da biblioteca-padrão; e a instrução `extern crate cfor;` informa que é para considerar o uso do módulo `cfor` na compilação do programa.

6.7 Caracteres ASCII, Unicode e Byte

A linguagem Rust opera com uma extensa quantidade de caracteres, sendo compatível com os seguintes padrões: ASCII, que representa os tradicionais 127 símbolos-padrão americanos, incorporando os sinais operacionais de um computador, números, símbolos e letras sem acento; e Unicode, que representa os símbolos de escrita de qualquer idioma, chegando, de acordo com o site *Space Science & Engineering Center da University of Wisconsin*, a um total de 65.533 caracteres de vários idiomas (<http://www.ssec.wisc.edu/~tomw/java/unicode.html>). Os 256 caracteres-padrão ASCII fazem parte do conjunto de caracteres Unicode.

Para o uso correto de caracteres Unicode em certo idioma, é fundamental que o computador esteja configurado para representar seus caracteres. Caso contrário, o efeito visual de certo caractere acentuado ou especial não será devidamente apresentado.

O programa seguinte apresenta alguns caracteres textuais e seus respectivos códigos de identificação interna do computador tanto para o

formato ASCII como para o formato Unicode. Para isso, crie o projeto c06ex15.

```
use std::io;
use std::io::prelude::*;
use std::char::from_u32;

fn main() {
    // Estilo: Caractere para ASCII

    println!("{}", 'A' as u8);
    println!("{}", 'B' as u8);
    println!("{}", 'C' as u8);

    // Estilo: ASCII para Caractere

    println!("{}", 65 as char);
    println!("{}", 66 as char);
    println!("{}", 67 as char);

    // Estilo: Caractere para Unicode

    println!("{}", 'Á' as u32);
    println!("{}", 'É' as u32);
    println!("{}", '©' as u32);

    // Estilo: Unicode para Caractere

    println!("{}", from_u32(193).unwrap());
    println!("{}", from_u32(201).unwrap());
    println!("{}", from_u32(169).unwrap());

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, serão apresentados três valores referentes aos três caracteres no formato ASCII para os símbolos A, B e C,

representados pelo tipo `char`, ou seja, um valor de tipo `u8`. Na sequência, o programa mostra o código de três caracteres Unicode e seus respectivos valores representados pelo tipo `u32`.

Em particular, para a conversão do valor Unicode em caractere, é necessário fazer uso do método `from_u32()`, uma vez que dados no formato Unicode são representados pelo tipo de dado `u32`. O método `from_u32()` é responsável por permitir a conversão do valor numérico do caractere Unicode interpretando-o como o caractere comum no formato ASCII equivalente. Para operar o método `from_u32()` é preciso usar a biblioteca `std::char::from_u32`. Se o caractere não puder ser impresso, será indicado em seu lugar (C).

Qualquer forma de representação de caracteres utilizados em um computador pode ser traduzida em bytes. Em particular, para operação com caracteres ASCII, a linguagem Rust possibilita o uso de outra forma de obter o valor do byte do caractere, diferentemente de se fazer uso da conversão com `as u8` após a indicação do caractere.

O programa seguinte demonstra a conversão de caractere ASCII em formato numérico a partir da definição de conversão com código byte, e não com a forma de conversão `as u8`. Para isso, crie o projeto `c06ex16`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    // Estilo: Caractere para ASCII

    println!("{}", b'A');
    println!("{}", b'B');
    println!("{}", b'C');
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, são apresentados os valores numéricos dos

caracteres indicados. Nesta versão está sendo usado, antes da definição do caractere entre aspas simples, o símbolo (`b`), que efetua a conversão do dado a sua frente no seu respectivo código numérico da tabela ASCII. O código byte (`b`) só pode ser usado para substituir a forma de conversão de dados com tipo `u8`.

No entanto é necessário levar em consideração que o código (`b''`) não converte o dado em si para seu valor ASCII. Esta ação converte o código numérico do caractere na forma bruta no momento em que o código numérico é usado, não deixando o código convertido memorizado.

Além de o código (`b`) ser usado com caracteres delimitados entre aspas simples, há a possibilidade de usá-lo com *strings* delimitadas com aspas inglesas, desde que se faça em conjunto o uso do código de formatação `{:?}`.

O programa seguinte demonstra o uso do código (`b`) com a definição de uma *string*. Para isso, crie o projeto `c06ex17`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    println!("{:?}", b"Linguagem Rust");
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, a mensagem definida como "Linguagem Rust" é apresentada na sua forma convertida como [76, 105, 110, 103, 117, 97, 103, 101, 109, 32, 82, 117, 115, 116], demonstrando o uso do efeito do código (`b""`).

Os exemplos de programas demonstrados neste tópico referem-se à apresentação de caracteres isolados, mas e se for preciso mostrar uma faixa de caracteres com base em seus valores ASCII?

Neste sentido, o próximo programa mostra os caracteres alfabéticos

maiúsculos A, B, C, D, E obtidos a partir dos valores da tabela ASCII: 65, 66, 67, 68 e 69. Para isso, crie o projeto `c06ex18`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    for i in 65 .. 70 {
        println!("{}", (i as u8) as char);
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, os caracteres referentes à sequência numérica são apresentados. Para a apresentação dos caracteres é necessário o uso de dois níveis de conversão.

O primeiro nível de conversão é transformar o tipo de dado da variável `i` do laço `for` de `i32` para `u8`, daí o uso do trecho `(i as u8)`. Em seguida, passa-se à fase de conversão para o tipo `as char`, como indicado junto à instrução `println!("{}", (i as u8) as char);`. O uso dos parênteses na definição da conversão `i as u8` é opcional, e foi aqui colocado para melhor legibilidade. Portanto, poderá ser retirado sem nenhuma ocorrência de erro.

¹ O termo “desreferência” é uma tradução livre do termo em inglês “desreference”, e é muito empregado na área de computação. Significa a ação que permite acessar determinado dado que se encontra armazenado em um endereço de memória.

CAPÍTULO 7

Estruturas homogêneas

Uma variável simples ocupa um bloco de memória onde um valor será armazenado, enquanto uma variável composta ocupa um conjunto de blocos sequencialmente distribuídos de memória para armazenar mais de um valor. Neste capítulo é demonstrado o uso de agrupamento de dados em forma de matrizes (unidimensionais e bidimensionais) a partir de estruturas de dados estáticas (imutáveis) e dinâmicas (mutáveis) – os vetores. Uma variável pode conter apenas um valor por vez. No caso de uma matriz, ela pode armazenar mais de um valor por vez, pois é dimensionada exatamente para essa finalidade. A manipulação dos elementos de uma matriz ocorre de forma individualizada.

7.1 Matriz estática

Uma matriz estática (arranjo em Rust) se caracteriza por ser uma estrutura de dados finita onde se conhece de antemão o número de elementos que esta contém, sendo então este conjunto de dados delimitado e imutável. Assim, é possível criar matrizes estáticas fechadas (internas) e/ou abertas (externas): fechadas, quando se define internamente tal estrutura no código de um programa com a definição de valores na forma de tabelas internas; abertas, quando o usuário de forma interativa fornece os valores de preenchimento da matriz em uso.

7.1.1 Matriz unidimensional

Esse tipo de estrutura de dados é conhecido como *matriz de uma dimensão* e é usado comumente na criação de listas simples de dados. A matriz usa uma única variável, que tem determinado tamanho e pode armazenar

mais de um valor (elemento) do mesmo tipo de dado. O tamanho de uma matriz é conhecido como dimensão, constituído por constantes numéricas inteiras e positivas. Os nomes atribuídos a matrizes seguem as mesmas regras de nomes atribuídos a variáveis simples. Uma matriz unidimensional estática em Rust pode ser definida como:

```
let a = [0; 8]; // 8 inteiros de 32 bits para A não mutável
let mut a = [0.; 10]; // 10 inteiros de 32 bits para A mutável
let a: [f64; 3] = [0.; 3]; // 3 reais de 64 bits para A não mutável
let mut a: [i32; 5] = [0; 5]; // 5 inteiros de 32 bits para A mutável
```

O programa seguinte tem como objetivo ler e armazenar dez elementos inteiros de 32 bits em uma matriz A do tipo vetor. O programa deverá construir a matriz B de mesmo tipo que a matriz A, observando a seguinte lei de formação: se o valor de um elemento da matriz A estiver numa posição de índice par, ele deve ser multiplicado por 5; sendo o valor de índice ímpar, o elemento de A deve ser somado a 5. Ao final, os conteúdos das duas matrizes devem ser exibidos. Para isso, crie o projeto `c07ex01`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut a = [0; 10];
    let mut b = [0; 10];

    let mut valor = String::new();

    println!("Exemplo de checagem de índice\n");

    // Entrada de dados

    for i in 0 .. 10 {
        print!("Entre o {:2}o. valor: ", i + 1);
        io::stdout().flush().unwrap();
        io::stdin().read_line(&mut valor).unwrap();
        a[i] = valor.trim().parse::<i32>().unwrap();
        valor.clear();
    }
}
```

```

println!();

// Processamento par ou impar

for i in 0 .. 10 {
    if i % 2 == 0 {
        b[i] = a[i] * 5;
    } else {
        b[i] = a[i] + 5;
    }
}

// Apresentacao das matrizes

for i in 0 .. 10 {
    println!("A[{:2}] = {:4} na posição {}.", i + 1, a[i], i);
}
println!();

for i in 0 .. 10 {
    println!("B[{:2}] = {:4} na posição {}.", i + 1, b[i], i);
}
println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

São utilizados no programa três laços do tipo `for`. O primeiro controla a entrada dos dados, o segundo verifica se cada índice da matriz A é par ou ímpar e faz as operações, implicando os elementos calculados na matriz B, e o terceiro apresenta as duas matrizes.

Antes da definição dos laços de operação do programa, as instruções `let mut a = [0; 10];` e `let mut b = [0; 10];` são usadas para definir as matrizes de uma dimensão a serem usadas no programa. A indicação `[0, 10]` estabelece que as variáveis A e B são matrizes. O valor 10 determina a quantidade de elementos para as matrizes e o valor 0 determina o valor definido para as dez posições de memória usadas pelas matrizes A e B. A

indicação do valor `0` (pode ser outro valor inteiro) determina que o tipo de dado a ser usado na matriz é `i32`.

No trecho de entrada do programa, após inserir o valor informado dentro da posição da matriz `A[i]` por meio da instrução `a[i] = valor.trim().parse::<i32>().unwrap();`, é necessário fazer a limpeza do dado informado na variável `VALOR` para que a variável possa receber a próxima entrada. Esta ação é executada pela instrução `valor.clear();`.

No laço destinado ao processamento `for i in 0 .. 10`, é utilizada a instrução `if i % 2 == 0`, sendo `%` um operador aritmético da linguagem Rust que possibilita extrair o resto de uma divisão de números inteiros. Essa operação recebe o nome de módulo. No caso apresentado, qualquer valor existente na variável `I` que, dividido por 2, resultar zero será par. Se o resto da divisão for diferente de zero, o valor da variável `I` será ímpar. O valor `10` do laço pode ser substituído por `a.len()`, onde o método `len()` associado à variável `A` estabelece para o laço a quantidade de elementos a ser usada.

O programa seguinte lê cinco elementos numéricos inteiros de 32 bits para uma matriz `A` de uma dimensão e no final apresenta a soma de todos os elementos ímpares que estiverem armazenados na matriz. Para isso, crie o projeto `c07ex02`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut a: [i32; 5] = [0; 5];
    let mut s: i32 = 0;

    let mut valor = String::new();

    println!("Somatório de elementos ímpares\n");

    // Entrada de dados

    for i in 0 .. 5 { // ou use: for i in 0 .. a.len()
```

```

        print!("Entre o {:2}o. valor: ", i + 1);
        io::stdout().flush().unwrap();
        io::stdin().read_line(&mut valor).unwrap();
        a[i] = valor.trim().parse::<i32>().unwrap();
        valor.clear();
    }
    println!();

    // Processamento par ou impar

    for i in 0 .. 5 {
        if a[i] % 2 != 0 {
            s += a[i];
        }
    }

    // Apresentacao das matrizes

    println!("Soma elementos impares = {}", s);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

A definição da matriz A é feita no projeto `c07ex02` com a instrução `let mut a: [i32; 5] = [0; 5];`, diferentemente do projeto `c07ex01`, que definiu as matrizes A e B com uso da instrução `let mut a = [0; 10];` e `let mut a = [0; 10];`. Note que há duas formas de fazer definição de matrizes estáticas de uma dimensão. No entanto a forma do projeto `c07ex02` é mais explícita, pois determina que a matriz A terá 5 elementos inteiros de 32 bits (`[i32; 5]`).

Observe que no laço destinado ao processamento foi utilizada a instrução `if a[i] % 2 != 0` para verificar se o elemento informado pelo teclado é um valor ímpar; se for, este valor ímpar é acumulado na variável `SOMA`, que, ao final, é usada para apresentar o somatório dos elementos ímpares

informados durante a execução do programa.

O programa a seguir efetua a entrada de cinco valores numéricos reais de 32 bits e mostra o somatório dos valores informados. Para isso, crie o projeto c07ex03.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut a: [f32; 5] = [0.; 5]; // ou: let mut a = [0.; 5];
    let mut s: f32 = 0.;

    let mut valor = String::new();

    println!("Somatório de elementos reais\n");

    // Entrada de dados
    for i in 0 .. 5 {
        print!("Entre o {:2}o. valor: ", i + 1);
        io::stdout().flush().unwrap();
        io::stdin().read_line(&mut valor).unwrap();
        a[i] = valor.trim().parse::<f32>().unwrap();
        valor.clear();
    }
    println!();

    // Somatório

    for i in 0 .. 5 {
        s += a[i];
    }

    // Apresentacao das matrizes

    println!("Soma = {}. ", s);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
```

```
io::stdin().read(&mut [0u8]).unwrap();
}
```

Informe cinco valores reais e observe a apresentação do resultado da soma dos valores informados. Ao fornecer os valores numérico, entre-os com expoente e mantissa para efetivação do somatório. Note, neste projeto, que, para definir matrizes com dados reais, é usada a instrução `let mut a: [f32; 5] = [0.; 5];`, em que `[f32; 5]` determina a quantidade de 5 posições do tipo `f32` atribuídas a partir de `[0.; 5]` com valores `0.` (zero real) nas 5 posições.

7.1.2 Matriz bidimensional

Uma matriz bidimensional se assemelha a uma tabela composta de linhas e colunas. Em matrizes com mais de uma dimensão, os elementos também são manipulados individualmente, sendo a referência feita sempre por meio de dois índices: o primeiro para indicar a linha; o segundo, a coluna.

A linguagem de programação C++ não trabalha diretamente com matrizes que tenham mais de uma dimensão, como outras linguagens fazem, mas permite a simulação perfeita desse efeito. Isso ocorre porque essa linguagem trabalha com matrizes de matrizes, ou seja, uma matriz considerada de duas dimensões é, na verdade, uma matriz cujos elementos são outra matriz. Uma matriz bidimensional estática em Rust pode ser definida como:

```
let a = [[0; 5]; 4];
let mut a = [[0.; 5]; 4];
let a: [[f64; 3]; 2] = [[0.; 3]; 2];
let mut a: [[f64; 3]; 2] = [[0.; 3]; 2];
```

Uma matriz de duas dimensões é semelhante à de uma dimensão, sendo representada por nome e tamanho (dimensão de linhas e colunas). O trecho `[[f64; 3]; 2]` representa uma matriz de 3 linhas com 2 colunas para o tipo de dado `f64` (valores numéricos reais de 64 bits), e o trecho `[[0.; 3]; 2]` indica que a matriz é inicializada pelo valor `0.` em todas as suas posições.

O programa seguinte apresenta uma matriz com duas dimensões, e efetua a entrada e a saída das quatro notas bimestrais de dez alunos. Para isso, crie o projeto `c07ex04`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut nota: [[f32; 10]; 4] = [[0.; 10]; 4];

    let mut valor = String::new();

    println!("Leitura e apresentação de notas escolares.\n");

    println!("Entrada dos dados.\n");

    // Entrada das notas

    for i in 0 .. 10 {
        println!("Informa as notas do {:2}o. aluno:\n", i + 1);
        for j in 0 .. 4 {
            print!("Nota {}: ", j + 1);
            io::stdout().flush().unwrap();
            io::stdin().read_line(&mut valor).unwrap();
            nota[j][i] = valor.trim().parse::<f32>().unwrap();
            valor.clear();
        }
        println!();
    }

    println!("Saída dos dados.\n");

    // Apresentação das notas

    for i in 0 .. 10 {
        println!("As notas do {:2}o. aluno são:\n", i + 1);
        for j in 0 .. 4 {
            println!("Nota {} = {:.6.2}", j + 1, nota[j][i]);
        }
    }
}
```

```

        println!();
    }
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Observe o uso das variáveis `I` e `J` para controlar as posições dos elementos na matriz. A variável `I` controla o posicionamento do elemento em relação à linha da tabela, e a variável `J` controla o posicionamento do elemento em relação à coluna da tabela. As variáveis `I` e `J` em conjunto possibilitam o acesso a um único elemento da tabela.

Analisando o programa, tem-se a inicialização das variáveis `I` e `J` como 0 por meio dos laços `for`, ou seja, a leitura é feita na primeira linha da primeira coluna (0,0). Em seguida, é iniciado, em primeiro lugar, o laço da variável `I` para controlar a posição em relação às linhas; depois, é iniciado o laço da variável `J` para controlar a posição em relação às colunas.

Os valores inseridos na tabela são colocados na posição `NOTA[0][0]` (primeira coluna e primeira linha). O primeiro valor dentro dos colchetes representa a coluna; o segundo, a linha. Assim, para o primeiro aluno é informada a sua primeira nota. Depois, acrescenta-se 1 à coluna e coloca-se na entrada a posição `NOTA[1][0]` (segunda coluna e primeira linha) da tabela. Para o primeiro aluno é informada a segunda nota, e assim por diante.

Quando o contador de coluna, o laço da variável `J`, atingir o valor 3, ele será encerrado. Em seguida, o contador da variável `I` será incrementado com 1, tornando-se 1. Então, será inicializado novamente o contador `J` em 0, permitindo informar um novo dado.

O mecanismo de preenchimento se estende até que o contador de linhas atinja o seu último valor; no caso, 9. Esse laço de repetição é o principal, tendo a função de controlar o posicionamento na tabela por aluno. O segundo laço de repetição, mais interno, controla o posicionamento das notas. Em seguida, é apresentada a relação dos alunos e suas respectivas

notas.

7.1.3 Matriz interna

Os exemplos de matrizes usados até o presente momento referem-se à forma de acesso aberta, pois os programas dependem da entrada de valores nas tabelas pelo usuário do programa. No entanto há outra maneira de fazer a manipulação de dados organizados em tabelas, que são tabelas definidas internamente no código do programa. As matrizes definidas de forma fechada podem, em Rust, ser inicializadas na própria declaração dessas matrizes, como:

```
let a: = [1, 2, 3];
let mut a: = [1, 2, 3];
let a = ["a1", "a2", "a3"];
let a: [&str; 3] = ["a1", "a2", "a3"];
let a: [f64; 3] = [0., 1., 2.];
```

A definição de matrizes internas é útil quando o programa precisa ter internamente tabelas de valores para ser consultadas pelo próprio programa para a efetivação de certos processamentos sem a participação do usuário do programa para que isso aconteça.

Com o objetivo de demonstrar a técnica de uso de tabelas internas com valores numéricos inteiros de 32 *bits* em uma matriz unidimensional, considere o programa seguinte, que apresentará os valores da matriz **VALORES**, o tamanho que essa matriz ocupa na memória, a quantidade de elementos armazenados, e indicará a apresentação do primeiro e do décimo elemento da matriz. Para isso, crie o projeto **c07ex05**.

```
use std::io;
use std::io::prelude::*;
use std::mem;

fn main() {
    let valores: [i32; 10] = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9];
    let tamanho = valores.len();

    for i in 0 .. tamanho {
```

```

        println!("VALORES[{:2}] = {}", i + 1, valores[i]);
    }
    println();
    println!("1o. elemento = {}", valores[0]);
    println!("10o. elemento = {}", valores[9]);

    println();
    println!("A matriz possui {} elementos", tamanho);

    println();
    println!("A matriz ocupa {} bytes", mem::size_of_val(&valores));

    println();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

O programa apresenta a coleção de valores definidos para a matriz VALORES, mostra o primeiro e o décimo elemento, indica a quantidade de elementos da matriz e o total de bytes ocupado.

A definição da matriz interna é efetuada por meio da instrução `let valores: [i32; 10] = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9];`, mas poderia ser definida por meio da instrução `let valores = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9];`.

Para indicar a quantidade de bytes ocupados na memória o programa usa o recurso `mem::size_of_val(&valores)`, definido a partir da biblioteca `std::mem`, que fornece funções para consultar o tamanho dos dados manipulados na memória. O recurso `mem::size_of_val()` é responsável por retornar o tamanho do valor em bytes do dado fornecido como seu parâmetro.

Com o objetivo de demonstrar a técnica de uso de tabelas de valores em uma matriz bidimensional com tamanho explícito, considere o programa seguinte, que apresentará os valores da matriz VALORES, o tamanho que essa matriz ocupa na memória e a quantidade de elementos armazenados. Para isso, crie o projeto `c07ex06`.

```

use std::io;
use std::io::prelude::*;
use std::mem;

fn main() {
    let valores: [[i32; 5]; 3] = [
        [ 0, 1, 2, 3, 4],
        [ 5, 6, 7, 8, 9],
        [10, 11, 12, 13, 14]
    ];

    for i in 0 .. 3 {
        for j in 0 .. 5 {
            println!("VALORES[{}][{}] = {}", i + 1, j + 1, valores[i]
[j]);
        }
    }

    println!();
    println!("A matriz ocupa {} bytes", mem::size_of_val(&valores));

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

O programa anterior tem a definição interna de uma matriz de duas dimensões com três linhas e cinco colunas, perfazendo um total de quinze elementos.

Um detalhe a ser observado em relação a uma matriz interna e externa de duas dimensões é a disposição dos índices que controlam a linha e a coluna. Nas matrizes externas, o formato usado foi [J][I], e nas matrizes internas o formato a ser usado é [I][J].

7.2 Matriz dinâmica

Uma matriz dinâmica é uma estrutura de dados cujo tamanho se encontra indefinido no início do programa e será definido em tempo de

execução do programa a partir, e na medida, da interação de seu usuário, sendo então uma estrutura de dados de tamanho mutável. Na linguagem Rust, este tipo de ocorrência é chamado de vetor (vector), sendo administrado pelos recursos de macro `vec!()` nos casos de matrizes dinâmicas externas e de uso do ponteiro tipo *boxes pointers*: `Box::new()` nos casos de matrizes dinâmicas internas. O recurso `Box::new()` será demonstrado nos tópicos relacionados ao gerenciamento de pilhas e filas.

A principal vantagem no uso de matrizes e mesmo variáveis dinâmicas é a possibilidade de gerenciar melhor o espaço de memória disponível. Muitas vezes, o espaço de memória torna-se subutilizado, principalmente quando uma variável é declarada e somente usada em uma parte do programa, como uma variável temporária. Nesse caso, pode-se definir essa variável temporária de forma dinâmica. Após o uso, ela é destruída, e seu espaço pode ser utilizado por outra variável dinâmica.

Ao declarar uma matriz como estática interna ou externa ou variável simples, o compilador reserva a ela espaço suficiente de memória, liberando o espaço reservado na memória somente ao término do programa.

7.2.1 Matriz unidimensional

Para exemplificar o uso de uma matriz unidimensional dinâmica externa, considere um programa que solicite o número de elementos do tipo real de 32 bits que a matriz deve conter e apresente na sequência os valores informados. Não se sabe ao certo quantos elementos o usuário do programa vai querer inserir, portanto não é possível saber antecipadamente quantos elementos serão inseridos na tabela. Para isso, crie o projeto `c07ex07`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let tamanho: usize;
    let mut valor = String::new();
```



```

let mut quant = String::new();

println!("Matriz unidimensional dinâmica\n");

// Entrada dos elementos

print!("Entre o tamanho ...: ");
io::stdout().flush().unwrap();
io::stdin().read_line(&mut quant).unwrap();
tamanho = quant.trim().parse::<usize>().unwrap();
println!();

let mut a = vec![0.; tamanho]; // matriz dinâmica

for i in 0 .. tamanho {
    print!("Entre o {:2}o. valor: ", i + 1);
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut valor).unwrap();
    a[i] = valor.trim().parse::<f32>().unwrap();
    valor.clear();
}
println!();

// Apresentação dos elementos

for i in 0 .. tamanho {
    println!("A[{:2}] = {:8.2} na posição {}.", i + 1, a[i], i);
}

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao executar o programa, não insira valores muito altos para a quantidade de elementos a ser definida, pois será necessário informar a quantidade de valores indicados.

Um detalhe importante, exigido pela linguagem Rust, é o uso do tipo de

dado `usize`, definido para a variável `TAMANHO`. Nenhum outro tipo de dado pode ser usado para estabelecer o tamanho de uma matriz, visto que uma matriz tem como tamanho de posições os valores inteiros e positivos. Assim que a variável `TAMANHO` é devidamente definida, esta pode ser usada junto à instrução `let mut a = vec![0.; tamanho];` para estabelecer o tamanho de memória a ser usado para o armazenamento dos dados informados. A macro `vec!()` é um recurso que permite definir uma estrutura de memória dinâmica para o gerenciamento de uma lista de valores na forma de matriz.

7.2.2 Matriz bidimensional

O programa a seguir exemplifica a criação de uma matriz bidimensional dinâmica a partir da inserção da quantidade de linhas e colunas que a matriz deverá conter.

O programa seguinte efetua, após a definição da quantidade de linhas e colunas, a entrada e a apresentação de um conjunto de valores inteiros de 32 bits. Para isso, crie o projeto `c07ex08`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let linhas: usize;
    let colunas: usize;

    let mut valor = String::new();
    let mut lin = String::new();
    let mut col = String::new();

    println!("Matriz unidimensional dinâmica\n");

    // Entrada dos elementos

    print!("Entre a quantidade de linhas ...: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut lin).unwrap();
```

```

    linhas = lin.trim().parse::<usize>().unwrap();

    print!("Entre a quantidade de colunas ..: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut col).unwrap();
    colunas = col.trim().parse::<usize>().unwrap();
    println!();

    let mut a = vec![vec![0; linhas]; colunas]; // matriz dinâmica

    for i in 0 .. linhas {
        for j in 0 .. colunas {
            print!("Informe posição [{},{}]: ", j + 1, i + 1);
            io::stdout().flush().unwrap();
            io::stdin().read_line(&mut valor).unwrap();
            a[j][i] = valor.trim().parse::<i32>().unwrap();
            valor.clear();
        }
        println!();
    }

    // Apresentação das notas

    println!();
    for i in 0 .. linhas {
        for j in 0 .. colunas {
            println!("Posição [{},{}] = {}", j + 1, i + 1, a[j][i]);
        }
        println!();
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Execute o programa entre a quantidade de linhas e colunas da matriz e em seguida efetue a entrada dos elementos inteiros. Note, após a inserção

do último valor, a apresentação de todos os valores informados.

Nas matrizes de duas dimensões dinâmicas foi usada a macro `vec!()` de forma encadeada em dois níveis por meio da instrução `let mut a = vec![vec![0; linhas]; colunas];`. A macro `vec!()` mais interna manipula a quantidade de memória `[0; linhas]` ao estabelecer o espaço necessário para o gerenciamento da quantidade de linhas, servindo como base para a definição da quantidade de memória necessária para a segunda dimensão com a macro `vec!()` externa, por meio do trecho `[vec![0; linhas]; colunas]`. Esta forma de encadeamentos de macros `vec!()` permite definir matrizes dinâmicas com mais dimensões, conforme a necessidade.

7.3 Fatiamento de matriz

A partir de uma visão geral sobre a manipulação da coleção de dados na forma de matrizes, tanto de uma como de duas dimensões, cabe ressaltar um recurso da linguagem Rust normalmente não encontrado em outras linguagens de programação. Trata-se do efeito de fatiamento dos elementos de uma matriz, ou seja, as *slices*.

Uma *slice* assemelha-se a um *array*, tendo como característica operacional o fato de não ter seu tamanho conhecido em tempo de compilação, sendo estabelecida a partir de um tipo de dado genérico denominado `&[T]` (em que `T` refere-se ao tipo de dado real a ser substituído automaticamente pelo tipo de dado em uso – detalhes sobre o tipo `T` são apresentados junto ao tema programação genérica). Uma *slice*, ou uma fatia, é um recurso composto de duas *words* (que podem ser uma palavra de 32 bits em uma arquitetura x86 ou uma palavra de 64 bits em uma arquitetura x86-64): a primeira *word* é um ponteiro que aponta para os dados da matriz, e a segunda indica o comprimento da fatia a ser considerada.

A *slice* é uma maneira segura de obter uma parte de uma matriz, sendo aplicável apenas sobre uma matriz previamente definida. Esse recurso pode operar com o formato mutável ou não mutável, comportando-se como se fosse um *array*. Dessa forma, seria uma *slide* válida a seguinte situação (adaptado de THE RUST TEAM, 2015, p. 129):

```

let valores = [1, 2, 3, 4, 5, 6];
let parte = &valores [1..4]; // Consideram-se apenas os valores 2, 3 e
4
let total = &valores[..]; // Consideram-se todos os valores

```

O programa a seguir demonstra o uso do fatiamento estabelecido sobre um conjunto de dados de uma matriz. Para isso, crie o projeto `c07ex09`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let valores = [1.1, 2.2, 3.3, 4.4, 5.5, 6.6];
    let parte = &valores [1..4];
    let total = &valores[..];

    println!("Parte .....: {:?}", parte);
    println!("Total .....: {:?}", total);
    println!("2o. elemento ..: {:?}", &valores [1..2]);
    println!("3o. elemento ..: {:?}", &valores [2..3]);
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, o formato `{:?}` possibilita apresentar valores de qualquer tipo. O símbolo `(?)` interrogação funciona como uma espécie de curinga. Os valores apresentados são formas de fatiamento da matriz `VALORES`.

7.4 Ordenação e pesquisa

A linguagem Rust, como diversas outras linguagens de programação, não contém uma biblioteca específica com a disponibilidade de recursos de ordenação e pesquisa de valores em matrizes (a menos quando se usa o recurso *coleções*, a ser apresentado), ficando a cargo dos programadores implementar tais ocorrências. No entanto há disponibilidade de bibliotecas de terceiros para que se efetuem as operações que não são

tratadas nesta obra.

Tanto para a ação de ordenação como para a ação de pesquisa, serão apresentados neste tópico dois métodos de operação para cada uma dessas tarefas. Sendo assim, são vistos, para a classificação, os métodos de classificação distributiva e de classificação por seleção, e para a ação de pesquisa, são apresentados os métodos sequencial e binário.

Os métodos de ordenação e pesquisa apresentados não terão seus algoritmos discutidos, por ser este tema foco de outro tipo de literatura: estruturas de dados. As análises de otimalidade dos algoritmos também não serão discutidas.

7.4.1 Ordenação de elementos matriciais

O processo de ordenação visa colocar os elementos de determinado conjunto a partir de determinada ordem. No caso de dados numéricos, estes podem ser crescentes (de 0 até 9) ou decrescentes (de 9 até 0); já para os dados alfabéticos ou alfanuméricos, podem ser ascendentes (de A até Z) ou descendentes (de Z até A).

O programa seguinte demonstra a ação de ordenação de valores inteiros de 32 bits não sinalizados sob o método distributivo a partir de uma matriz unidimensional interna. Para isso, crie o projeto `c07ex10`.

```
use std::io;
use std::io::prelude::*;

fn ordendistr(matriz: &mut [u32]) {
    let tamanho = matriz.len();

    for i in 0 .. tamanho - 1 {
        for j in i + 1 .. tamanho {
            if matriz[i] > matriz[j] {
                matriz.swap(i, j);
            }
        }
    }
}
```

```

}

fn main() {
    let mut valores: [u32; 10] = [9, 8, 7, 6, 5, 4, 3, 2, 1, 0];

    ordendistr(&mut valores);

    println!("{:?}", valores);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, os valores desordenados 9, 8, 7, 6, 5, 4, 3, 2, 1 e 0 são apresentados na forma ordenada crescente 0, 1, 2, 3, 4, 5, 6, 7, 8 e 9 por meio do método de classificação distributivo.

Atente em especial para a linha de instrução `if matriz[i] > matriz[j]` definida dentro da sub-rotina `ordendistr()`, que se utiliza do operador relacional (`>`) “maior que” para colocar os dados da matriz em ordem crescente. Se quiser efetuar ordem decrescente, utilize o operador relacional “menor que” (`<`).

Outro detalhe que merece atenção no código é o uso do trecho `matriz: &mut [u32]`, definido como parâmetro da sub-rotina `ordendistr()`, que demonstra como fazer a passagem de parâmetro de uma matriz, neste caso, definida como `&mut`, por precisar apresentar fora do escopo da sub-rotina os elementos mudados de posição.

Atente também para a instrução `matriz.swap(i, j)`; definida dentro da sub-rotina `ordendistr()`, que utiliza o método `swap()` para efetivar a troca dos elementos entre as posições controladas pelas variáveis `i` e `j`.

O programa seguinte demonstra a ação de ordenação de valores inteiros de 32 bits não sinalizados sob o método por seleção a partir de uma matriz unidimensional interna. Para isso, crie o projeto `c07ex11`.

```

use std::io;

```

```

use std::io::prelude::*;

fn ordenselec(matriz: &mut [u32]) {
    let tamanho = matriz.len();
    let mut aux;

    for i in 0 .. tamanho {
        aux = i;
        for j in i + 1 .. tamanho {
            if matriz[aux] > matriz[j] {
                aux = j;
            }
        }
        matriz.swap(i, aux);
    }
}

fn main() {
    let mut valores: [u32; 10] = [9, 8, 7, 6, 5, 4, 3, 2, 1, 0];

    ordenselec(&mut valores);

    println!("{:?}", valores);
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, os valores desordenados 9, 8, 7, 6, 5, 4, 3, 2, 1 e 0 são apresentados na forma ordenada crescente 0, 1, 2, 3, 4, 5, 6, 7, 8 e 9 por meio do método de classificação por seleção.

Observe que, com exceção do método de ordenação, os demais recursos usados no programa do projeto c07ex11 são os mesmo usados no programa do projeto c07ex11.

7.4.2 Pesquisa de elementos matriciais

O uso de matrizes pode gerar grandes estruturas de dados, dificultando a localização rápida de determinado elemento. Para solucionar esse problema, você pode fazer pesquisas em matrizes por meio de programação. Nesse sentido, dois métodos podem ser utilizados: pesquisa sequencial e pesquisa binária, cujos algoritmos são apresentados.

O primeiro método algorítmico busca a informação requerida sequencialmente, desde o primeiro elemento até o último. Localizando a informação no caminho, ela é apresentada. Esse método de pesquisa é lento, mas eficiente nos casos em que os elementos de uma matriz encontram-se desordenados.

Com o objetivo de demonstrar a técnica de pesquisa sequencial, considere o programa seguinte, que recebe dez valores inteiros e concede ao usuário a possibilidade de pesquisar os valores armazenados. Para isso, crie o projeto `c07ex12`.

```
use std::io;
use std::io::prelude::*;

fn pesqseq(matriz: &mut [u32]) {
    let tamanho = matriz.len();
    let mut resposta = String::new();
    let mut resp: char;
    let mut i: usize;
    let mut acha: bool;
    let mut pesq = String::new();

    loop {

        i = 0;
        acha = false;
        pesq.clear();

        print!("\nEntre o valor a ser pesquisado: ");
        io::stdout().flush().unwrap();
        io::stdin().read_line(&mut pesq).expect("Erro na entrada.");
        let pesq: u32 = match pesq.trim().parse() {
            Ok(numero) => numero,
```

```

        Err(_) => 0
    };

    while i <= tamanho - 1 && acha == false {
        if pesq == matriz[i] {
            acha = true;
        } else {
            i += 1;
        }
    }

    if acha == true {
        println!("{}", foi localizado na {}. posição.", pesq, i +
1);
    } else {
        println!("{}", não foi localizado.", pesq);
    }

    println!();
    println!("Deseja continuar?");
    print!("Tecle [S] para SIM / qualquer caractere para NAO: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut resposta).unwrap();
    resp = resposta.to_uppercase().trim().parse:::<char>().unwrap();
    resposta.clear();

    if resp != 'S' { break; }
}

}

fn main() {
    let mut valores: [u32; 10] = [0; 10];
    let mut valor = String::new();

    for i in 0 .. 10 {
        print!("Entre o {:2}o. valor: ", i + 1);
        io::stdout().flush().unwrap();
        io::stdin().read_line(&mut valor).unwrap();
        valores[i] = valor.trim().parse:::<u32>().unwrap();
    }
}

```

```

        valor.clear();
    }

    pesqseq(&mut valores);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, forneça dez valores inteiros e, em seguida, teste algumas possibilidades de pesquisa. Forneça para a pesquisa tanto os valores fornecidos como os valores não fornecidos, para ver a apresentação das mensagens de localização ou não do conteúdo pesquisado.

O programa do projeto `c07ex12` faz, de certa forma, uso de diversas técnicas apresentadas e demonstradas ao longo dos capítulos anteriores, proporcionando, dessa forma, uma noção mais contextualizada de um programa com determinado grau de interatividade produzido na linguagem Rust.

O programa a seguir demonstra o uso do método de pesquisa binário em um programa que solicita a entrada de dez nomes de pessoas, apresenta os nomes e então solicita a ação de pesquisa. Para isso, crie o projeto `c07ex13`.

```

use std::io;
use std::io::prelude::*;

fn ordenselec(matriz: &mut [String]) {
    let tamanho = matriz.len();
    let mut aux;

    for i in 0 .. tamanho {
        aux = i;
        for j in i + 1 .. tamanho {
            if matriz[aux] > matriz[j] {

```

```

        aux = j;
    }
}
matriz.swap(i, aux);
}

}

fn pesqbin(matriz: &mut [String]) {
    let tamanho = matriz.len();
    let mut resposta = String::new();
    let mut resp: char;
    let mut inicio: usize;
    let mut fim: usize;
    let mut meio: usize;
    let mut acha: bool;
    let mut pesq = String::new();

    loop {

        inicio = 0;
        meio = 0;
        fim = tamanho - 1;
        acha = false;
        pesq.clear();

        print!("\nEntre o nome a ser pesquisado: ");
        io::stdout().flush().unwrap();
        io::stdin().read_line(&mut pesq).expect("Erro na entrada.");
        if let Some('\n') = pesq.chars().next_back() {
            pesq.pop();
        }
        if let Some('\r') = pesq.chars().next_back() {
            pesq.pop();
        }

        while inicio <= fim && acha == false {
            meio = (inicio + fim) / 2;
            if pesq == matriz[meio] {

```

```

        acha = true;
    } else {
        if pesq < matriz[meio] {
            fim = meio - 1;
        } else {
            inicio = meio + 1;
        }
    }
}

if acha == true {
    println!("{}", foi localizado na {}a. posição.", pesq, meio +
1);
} else {
    println!("{}", não foi localizado.", pesq);
}

println!();
println!("Deseja continuar?");
print!("Tecle [S] para SIM / qualquer caractere para NAO: ");
io::stdout().flush().unwrap();
io::stdin().read_line(&mut resposta).unwrap();
resp = resposta.to_uppercase().trim().parse:::<char>().unwrap();
resposta.clear();

if resp != 'S' { break; }
}

}

fn main() {
    let mut nomes: [String; 10] = Default::default();

    for i in 0 .. 10 {
        print!("Entre o {:2}o. nome: ", i + 1);
        io::stdout().flush().unwrap();
        io::stdin().read_line(&mut nomes[i]).expect("Entrada
incorreta");
        if let Some('\n') = nomes[i].chars().next_back() {

```

```

        nomes[i].pop();
    }
    if let Some('\r') = nomes[i].chars().next_back() {
        nomes[i].pop();
    }
}

ordenselec(&mut nomes);

println!();
for i in 0 .. 10 {
    println!("{:2}o. nome = {}", i + 1, nomes[i]);
}

pesqbin(&mut nomes);

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, informe dez nomes e teste as possibilidades de pesquisa. Atente para o fato de que o uso do algoritmo de pesquisa binário exige que a lista de valores esteja previamente ordenada, daí o uso da rotina de ordenação antes da ação de pesquisa.

Basicamente, todos os recursos usados no código do programa foram anteriormente apresentados no livro, exceto a instrução `let mut nomes: [String; 10] = Default::default();`, que, ao definir a matriz `NOMES` com 10 posições do tipo `String` (`[String; 10]`), faz uso do trecho `Default::default()`, que efetua a inicialização da matriz na memória. O recurso `Default` é usado no estabelecimento de valores-padrão relacionados ao tipo de dado em uso, e `default()` é o método usado para retornar um valor-padrão a ser estabelecido junto de `Default`.

 **Observação:** Apesar de não ter sido exemplificado, é oportuno esclarecer que operações executadas sobre elementos alfanuméricos de uma matriz podem requerer o uso do método `clone()`, que permite retornar a cópia do elemento

alfanumérico apontado para que este possa ser atribuído a uma variável simples ou mesmo composta. Por exemplo, considere o uso de duas matrizes A e B, em que a entrada dos dados é feita na matriz A e devem ser posteriormente transferidos os elementos da matriz A para a matriz B. A ação de transferência deve ser processada dentro de um laço como `a[i] = b[i].clone()`. A tentativa de realizar a ação sem o método `clone()` resulta em uma notificação de erro: **cannot move out of indexed content**.

7.5 Iteradores

Os iteradores são recursos encontrados em diversas linguagens orientadas a objetos, como Java, C#, C++, entre outras, servindo para administrar listas de valores contidos em faixas de valores estabelecidas ou definidas como elementos de matrizes ou de um conjunto de dados. Por meio desses recursos é possível percorrer o conjunto de elementos de uma matriz ou dos valores de uma lista, a partir do primeiro elemento, sem ter de se preocupar com os detalhes técnicos para executar essas ações e seu efetivo controle.

Na linguagem Rust, iteradores são naturalmente definidos quando o comando `for` é usado junto à faixa (lista) de valores de variação estabelecida para a variável em operação, sendo esta a maneira mais simples de fazer uso desse recurso. Por exemplo, a instrução `for i in 1 .. 6` (que pode ser escrita como `for i in 1..5`) é um iterador simples que conta de 1 até 5 de 1 em 1 para a variável `i`. Quando um iterador é definido, ele é iniciado no primeiro valor da faixa estabelecida e encerra-se um valor a menos dessa faixa.

O recurso de manipulação de iteradores já vinha sendo usado havia algum tempo em diversos outros exemplos por meio do comando `for`, que itera para a variável indicada cada um dos valores isolados da faixa estabelecida. Dessa forma, o trecho seguinte apresenta no monitor de vídeo os valores de 1 a 5 a partir da execução simplista de um iterador:

```
for i in 1 .. 6 {  
    println!("{}", i);  
}
```

O programa seguinte mostra a apresentação no monitor de vídeo de

valores de 1 a 5 por meio de um iterador com auxílio do método `next()` associado a uma faixa de valores e manipulado pelo comando `loop`. Para isso, crie o projeto `c07ex14` (código adaptado e baseado no exemplo usado em <https://doc.rust-lang.org/1.8.0/book/iterators.html>).

```
use std::io;
use std::io::prelude::*;

fn main() {

    let mut faixa = 1..6;

    loop {
        match faixa.next() {
            Some(i) => {
                println!("{}", i);
            },
            None => break
        }
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, os valores de 1 a 5 são apresentados a partir da faixa de iteração de 1 .. 6 definida e associada à variável FAIXA.

O laço `loop` será executado enquanto os valores a serem apresentados estiverem dentro da faixa estabelecida, sendo esta ação controlada pelo comando `match`. Quando o fluxo do programa entra no laço `loop`, o iterador do laço não está inicializado; ele é inicializado quando ocorre o uso do comando `match` por meio do método `next()` do iterador FAIXA que inicialmente coloca o ponteiro interno de controle da iteração em 1. O método `next()` fornece uma referência ao próximo valor do iterador a cada ação controlada por `loop` e retorna o próximo valor até chegar a 6.

A cada valor da iteração entre a faixa de 1 a 5, o comando `match` executa o método `Some()`, que, tendo na variável `I` um valor dentro da FAIXA é apresentado no monitor de vídeo. No momento em que o método `next()` avançar para a posição de valor 6, será encerrada a iteração detectada pela opção `None` do comando `match`, que desviará o fluxo do programa para o comando `break`, que interromperá a execução do laço `loop`, forçando sua parada.

Além de poder ser usados no estabelecimento de faixas de valores, os iteradores também podem ser definidos nas matrizes estáticas e/ou dinâmicas por meio do estabelecimento de coleções de valores na forma de estruturas internas. Neste sentido, passam a ser usados outros métodos de operação.

Quando um modo de iteração é aplicado sobre uma matriz, pode-se acessar o elemento da matriz apenas pela indicação da variável que controla o índice. Dessa forma, para apresentar um elemento, não é necessário usar o formato `A[I]`, basta apenas indicar `I`.

O programa seguinte demonstra a apresentação dos valores de uma matriz interna do tipo arranjo, chamada `VALORES`, utilizando-se a forma tradicional e a forma iterativa. Para isso, crie o projeto `c07ex15`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let valores: [i32; 10] = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9];

    for i in 0 .. valores.len() {
        println!("{}", valores[i]);
    }

    println!();

    for i in &valores {
        println!("{}", i);
    }
}
```

```

        println!();
        print!("Tecle <Enter> para encerrar...");
        io::stdout().flush().unwrap();
        io::stdin().read(&mut [0u8]).unwrap();
    }

```

Ao ser executado o programa, a sequência de valores de 0 a 9 é apresentada duas vezes. Na primeira, é usada a forma tradicional de acesso aos elementos de uma matriz; na segunda, demonstram-se os valores por meio do acesso aos elementos usando-se o recurso de iteração.

Apesar de as duas saídas serem semelhantes internamente, elas são executadas de modos muito diferentes. A segunda maneira de acesso aos elementos é a mais indicada e melhor, pois age de forma direta sem requerer a verificação dos limites inferior e superior operados pelo trecho de código `0 .. valores.len()` e, conseqüentemente, sem a necessidade de controlar índices de acesso como `[I]`. Veja que bastou informar a referência de acesso aos elementos da matriz com a indicação `&valores` para que a operação ocorresse de forma mais segura, confortável e mais simplificada em relação à escrita do código.

O trecho do programa do projeto `c07ex15` codificado como

```

for i in &valores {
    println!("{}", i);
}

```

pode ser codificado como

```

for i in valores.iter() {
    println!("{}", i);
}

```

As duas formas resultam na mesma ação. Na segunda, o uso do método `iter()` possibilita definir um iterador vinculado à variável `VALORES`, que terá uma referência a cada elemento da matriz. Escrever `&valores` ou escrever `valores.iter()` tem o mesmo efeito; ambas as formas definem um iterador para percorrer os elementos. No entanto o método `iter()` pode ser usado de maneira mais ampla.

O programa seguinte apresenta o somatório dos valores definidos na matriz `VALORES` formada pelos elementos 1, 2, 3, 4 e 5. Para tanto, crie o projeto com o nome `c07ex16`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let valores: [i32; 5] = [1, 2, 3, 4, 5];

    let total: i32 = valores.iter().sum();

    println!("{}", total);
    println();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, é apresentado o resultado de `TOTAL` com valor 15 obtido a partir do método `sum()` associado ao iterador `valores.iter()`.

No programa do projeto `c07ex16`, além do método iterador `iter()`, é usado o método de apoio `sum()`, que efetua a soma dos elementos da variável associada; neste caso, `VALORES`. Para que o método `sum()` seja adequadamente executado é importante que ele esteja associado ao método `iter()`.

O programa do projeto `c07ex16` pode fazer uso do trecho `valores.iter().sum()` para apresentar o total da soma diretamente com a macro `println!()` sem auxílio de uma variável, como ocorre com `TOTAL`. No entanto a instrução `println!("{}", total);` não poderá ser simplesmente substituída por `println!("{}", valores.iter().sum());`; é necessário qualificar o retorno do tipo do valor efetuado pelo método `sum()` no mesmo tipo de dado da coleção de valores. Dessa forma, a instrução direta de saída deve ser `println!("{}", valores.iter().sum::<i32>());`. Perceba o uso do trecho `sum::<i32>` qualificando o retorno do cálculo como sendo

do tipo `i32`, mesmo tipo usado na matriz `let valores: [i32; 5] = [1, 2, 3, 4, 5]`; na indicação do trecho `[i32, 5]`. Assim, o trecho de código

```
let valores: [i32; 5] = [1, 2, 3, 4, 5];
let total: i32 = valores.iter().sum();
println!("{}", total);
```

poderia ser escrito como

```
let valores: [i32; 5] = [1, 2, 3, 4, 5];
println!("{}", valores.iter().sum::<i32>());
```

O uso de iteradores proporciona uma série de ações predefinidas por meio de uma coleção de recursos internos da linguagem Rust que podem ser conhecidos em seu manual de referência online disponibilizado no endereço <https://doc.rust-lang.org/std/iter/trait.Iterator.html>.

O programa seguinte demonstra a aplicação de alguns iteradores, além dos iteradores já mostrados `iter()`, `sum()` e `ver()` no Capítulo 4. Para ser usados, alguns iteradores requerem que a lista seja mutável. Por isso, quando houver essa condição obrigatória, a lista de valores será definida como `mut`. Para isso, crie o projeto `c07ex17`.

```
use std::io;
use std::io::prelude::*;
```

```
fn main() {
    let valores1: [i32; 5] = [1, 2, 3, 4, 5];
    let valores2: [i32; 5] = [6, 7, 8, 9, 0];
    let valores3: [i32; 5] = [3, 2, 8, 5, 4];

    let lista = valores1.iter();
    println!("Produto da matriz .....: {:4}", lista.product::
<i32>());
    println!();

    let lista = valores1.iter();
    println!("Quantidade de elementos .....: {:4}", lista.count());
    println!();

    let lista = valores3.iter();
```

```

println!("Último elementos da lista ...: {:?}", lista.last());
println!();

let mut lista = valores3.iter();
println!("Mostra o 3o. elemento .....: {:?}", lista.nth(2));
println!();

let mut lista = valores1.iter().chain(valores2.iter()).skip(5);
for i in 0 .. 6 {
    println!("Elemento {:2} .....: {:?}", i,
lista.next());
}
println!();

let mut lista = valores1.iter().zip(valores2.iter());
for i in 0 .. 5 {
    println!("Elemento {:2} .....: {:?}", i,
lista.next());
}
println!();

let lista = valores3.iter();
println!("Maior valor da lista .....: {:?}", lista.max());
let lista = valores3.iter();
println!("Menor valor da lista .....: {:?}", lista.min());
println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, é apresentada uma lista com diversas informações obtidas a partir do uso de alguns recursos iteradores da linguagem.

Para a apresentação dos detalhes são usadas três matrizes denominadas VALORES1, VALORES2 e VALORES3, cada uma com cinco elementos inteiros de 32 bits cada um sinalizados.

Antes do uso de cada iterador apresentado executam-se instruções do tipo

`let lista = valores1.iter();` com o objetivo de inicializar o iterador. Quando uma ação de iteração é realizada, o ponteiro interno do recurso se desloca ao longo dos elementos da matriz e, ao chegar ao seu final, lá permanece. O retorno ao primeiro elemento ocorre com a execução de uma instrução como `let lista = valores1.iter();`.

O trecho de código `lista.product::<i32>()` gera a multiplicação dos elementos da matriz entre si. Pelo fato de a matriz `VALORES1` conter os valores de 1 a 5, é apresentado como resultado do produto o valor 120 (1 x 2 x 3 x 4 x 5). Se fosse realizada a ação com a matriz `VALORES3`, o resultado apresentado seria 960.

O trecho de código `lista.count()` possibilita a visualização da quantidade de elementos definidos dentro da variável `VALORES1`; neste caso, valor 5.

O trecho de código `lista.last()` proporciona a apresentação do último elemento definido dentro da variável `VALORES3`; neste caso, valor 4.

O trecho de código `lista.nth()` proporciona a visualização do elemento indicado como posição de `VALORES3`; neste caso, valor 8.

O trecho de código com a instrução `let mut lista = valores1.iter().chain(valores2.iter()).skip(5);` faz uso de dois iteradores: `chain()`, que vincula fisicamente duas matrizes, tratando-as como uma; e `skip()`, que estabelece a quantidade de saltos que devem ser dados a partir do início da lista sobre a posição indicada. Neste caso, o trecho de código controlado por `for i in 0 .. 6` faz a apresentação dos valores 6, 7, 8, 9 e 5.

O trecho de código com a instrução `let mut lista = valores1.iter().zip(valores2.iter());` faz, por meio do iterador `zip()`, a junção de pares entre duas matrizes indicadas. Neste caso, o recurso permite o uso par a par dos elementos vinculados de cada uma das matrizes.

Os iteradores `max()` e `min()`, quando associados a uma matriz, permitem que sejam obtidos o maior e o menor valor contidos na lista, não importando a ordem em que estejam dispostos.

7.6 Exercícios de fixação

Desenvolver os programas em Rust dos problemas elencados de 1 até 8, ficando a cargo do professor selecionar a ordem e os problemas a serem resolvidos.

1. Elaborar um programa que efetue a leitura de dez nomes de pessoas em uma matriz A do tipo vetor e apresente-os em seguida.
2. Elaborar um programa que leia oito elementos inteiros sinalizados de 32 bits em uma matriz A do tipo vetor. Construir uma matriz B de mesma dimensão com os elementos da matriz A multiplicados por 3. Apresentar os elementos da matriz B.
3. Escrever um programa que leia duas matrizes (denominadas A e B) do tipo vetor com cinco elementos reais de 64 bits. Construir uma matriz C, sendo cada elemento da matriz C a subtração de um elemento correspondente da matriz A com um elemento correspondente da matriz B. Ao final, apresentar os elementos da matriz C a partir da máscara :9.3.
4. Elaborar um programa que leia oito elementos inteiros não sinalizados de 32 bits de uma matriz A do tipo vetor. Construir uma matriz B de mesmo tipo, observando a seguinte lei de formação: “todo elemento da matriz B deve ser o quadrado do elemento da matriz A correspondente”. Apresentar os elementos das matrizes A e B a partir da máscara :3.
5. Elaborar um programa que leia duas matrizes do tipo vetor para o armazenamento de nomes de pessoas, sendo a matriz A com quatro elementos e a matriz B com três elementos. Construir uma matriz C, sendo esta a junção das matrizes A e B. Dessa forma, a matriz C deve ter a capacidade de armazenar sete elementos. Apresentar os elementos da matriz C.
6. Elaborar um programa que leia oito elementos do tipo real de 64 bits em uma matriz A unidimensional e construir uma matriz B de mesma dimensão com os mesmos elementos armazenados na matriz A, porém de forma invertida. Ou seja, o primeiro elemento da matriz A passa a ser

o último da matriz B; o segundo elemento da matriz A passa a ser o penúltimo da matriz B; e assim por diante. Apresentar os elementos das matrizes A e B de acordo com a máscara :5.2 na mesma linha.

7. Elaborar um programa que leia duas matrizes A e B de uma dimensão com seis elementos inteiros não sinalizados de 64 bits. A matriz A deve aceitar apenas a entrada de valores pares, enquanto a matriz B deve aceitar apenas a entrada de valores ímpares. A entrada das matrizes deve ser validada pelo programa, e não pelo usuário. Construir uma matriz C que seja o resultado da junção das matrizes A e B, de modo que a matriz C contenha doze elementos. Apresentar os elementos da matriz C.
8. Construir um programa que leia uma matriz A de uma dimensão do tipo vetor com doze elementos inteiros não sinalizados de 32 bits. Ao final do programa, apresentar a quantidade de valores pares e ímpares contidos na referida matriz.

Desenvolver os programas em Rust dos problemas elencados de 1 até 5, ficando a cargo do professor selecionar a ordem e os problemas a serem resolvidos.

1. Elaborar um programa que leia oito elementos numéricos inteiros sinalizados em uma matriz A do tipo vetor. Coloque-os em ordem decrescente pelo método distributivo e apresente os elementos ordenados. A ordenação deve ser executada a partir de uma sub-rotina específica para esta finalidade. Utilize para a saída a máscara :4.
2. Elaborar um programa que leia cinco elementos numéricos reais de 64 bits em uma matriz A do tipo vetor. Colocá-los em ordem crescente pelo método por seleção e apresentar os elementos ordenados sem utilizar a ordenação dentro de uma sub-rotina. Utilize para a saída a máscara :4.
3. Elaborar um programa que leia oito elementos numéricos inteiros não sinalizados de 32 bits em uma matriz A de uma dimensão do tipo vetor. Construir uma matriz B de mesma dimensão e mesmo tipo com os elementos da matriz A multiplicados por 5. Em seguida, definir uma sub-rotina de pesquisa sequencial, para pesquisar os elementos

armazenados na matriz B, utilizando um laço do tipo `loop`.

4. Elaborar um programa que leia dez elementos numéricos inteiros sinalizados de 32 bits em uma matriz A de uma dimensão do tipo vetor. Construir uma matriz B de mesma dimensão e mesmo tipo com os elementos da matriz A elevados à segunda potência. Em seguida, definir uma sub-rotina de pesquisa sequencial para pesquisar os elementos armazenados na matriz B utilizando um laço do tipo `while`.
5. Elaborar um programa que leia dez elementos do tipo *string* em uma matriz A e fazer a apresentação dos valores a partir da ação de pesquisa binária que deverá controlar internamente todos os requisitos operacionais necessários para que o método de pesquisa possa ser efetivamente utilizado.

Desenvolver os programas em Rust dos problemas elencados de 1 até 5, ficando a cargo do professor selecionar a ordem e os problemas a serem resolvidos.

1. Elaborar um programa que leia duas matrizes A e B, cada uma de duas dimensões com cinco linhas e três colunas para valores inteiros sinalizados de 32 bits. Construir uma matriz C de mesma dimensão que seja formada pela soma dos elementos da matriz A com os elementos da matriz B. Apresentar os elementos da matriz C.
2. Elaborar um programa que leia uma matriz A do tipo inteira com valores de 32 bits não sinalizados de duas dimensões com cinco linhas e cinco colunas. Construir uma matriz B de mesma dimensão, em que cada elemento seja o dobro de cada elemento correspondente da matriz A, com exceção dos valores situados na diagonal principal (posições B[1,1], B[2,2], B[3,3], B[4,4] e B[5,5]), os quais devem ser o triplo de cada elemento correspondente da matriz A. Apresentar ao final a matriz B.
3. Elaborar um programa que leia uma matriz A do tipo real com valores de 32 bits para duas dimensões com oito linhas e oito colunas. Apresentar o somatório dos elementos situados na diagonal principal (posições A[1,1], A[2,2], A[3,3], A[4,4], A[5,5], A[6,6], A[7,7] e A[8,8]) da

referida matriz.

4. Elaborar um programa que leia duas matrizes A e B, cada uma de duas dimensões com cinco linhas e seis colunas com valores sinalizados de 32 bits. A matriz A deve aceitar a entrada de valores pares, enquanto a matriz B deve aceitar a entrada de valores ímpares. As entradas dos valores nas matrizes A e B devem ser validadas pelo programa, e não pelo usuário. Construir a matriz C de mesma dimensão, que seja formada pela soma dos elementos da matriz A com os elementos da matriz B. Apresentar os elementos da matriz C.
5. Elaborar um programa que leia uma matriz A do tipo inteira de duas dimensões com cinco linhas e cinco colunas para valores sinalizados de 64 bits. Construir uma matriz B de mesma dimensão, em que cada elemento seja o dobro de cada elemento correspondente da matriz A, com exceção dos valores situados na diagonal inversa (posições B[1,5], B[2,4], B[3,3], B[4,2] e B[5,1]), os quais devem ser o triplo de cada elemento correspondente da matriz A. Apresentar ao final a matriz B.

CAPÍTULO 8

Estruturas heterogêneas

Este capítulo mostra o uso de algumas técnicas adicionais de estruturação de dados que complementam o estudo já apresentado sobre matrizes. Serão indicadas as técnicas de uso de tuplas de dados, estruturas heterogêneas, enumerações, agregações, traços e implementações. Temos aqui uma introdução aos detalhes básicos para uso da programação orientada a objetos em Rust.

8.1 Tuplas de dados

O capítulo anterior apresenta o conceito de matrizes internas e externas com uma e duas dimensões. Uma matriz é um conjunto de dados de mesmo tipo; portanto, é por sua natureza uma estrutura de dado homogênea. Já uma tupla é uma estrutura de dados que se utiliza de um conjunto de dados de tipos diferentes, sendo esta uma estrutura heterogênea. Os dados de uma tupla não precisam ter entre si uma relação contextual; podem ser definidos de forma aleatória sem que haja algum sentido entre eles.

Os elementos de uma matriz (de uma ou mais dimensões) são, na linguagem Rust, delimitados entre colchetes, podendo ser operados de modo estático ou dinâmico. No entanto os elementos de uma tupla são delimitados entre parênteses e são operados estaticamente. Como as variáveis simples e as matrizes, as tuplas devem ser declaradas dentro das sub-rotinas. As tuplas são úteis quando é necessário processar um conjunto de valores integrados, mas de tipos diferentes (DHINAKARAN, 2017, p.31).

Como exemplo inicial, considere o programa seguinte, que faz uso de uma

tupla contendo três valores de tipos diferentes, sendo uma *string*, um valor inteiro e um valor real. Para isso, crie o projeto `c08ex01`.

```
use std::io;
use std::io::prelude::*;
fn main() {
    let tupla = ("Laranja", 1000, 23.98);

    println!("1o. elemento da tupla {}", tupla.0);
    println!("2o. elemento da tupla {}", tupla.1);
    println!("3o. elemento da tupla {}", tupla.2);

    println!();
    println!("Todos elementos da tupla {:?}", tupla);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa serão apresentados separadamente cada valor da tupla definida junto a variável `TUPLA` e na sequência são apresentados o valor da tupla como um todo. Observe que para obter a apresentação de cada valor isolado usa-se o nome da tupla com um ponto e a indicação do índice de posição do dado acessado. Quando usado o nome da tupla sem a indicação do índice ocorre acesso a todo o conjunto de dados. Perceba o uso do código de formatação `{:?}` que possibilita apresentar de uma única vez todo o conjunto de dados definido na variável `TUPLA`.

No programa do projeto `c08ex01` a variável `TUPLA` opera com três dados distintos, sendo o primeiro dado um *string* do tipo `&str` (um dos dois tipos de dados para manipulação de cadeias de caracteres usadas pela linguagem Rust, sendo o outro o tipo `String`). O tipo primitivo `&str` é usado na definição de *strings* estáticos (`&'static str`, onde o símbolo `&` representa a ação estática) como é o caso da definição do *string* `"Laranja"` e o tipo derivado `String` é usado para a definição de sequências de

caracteres dinâmicas como ações de entrada pelo teclado quando não se tem a princípio a definição do tamanho de caracteres a serem usados. O segundo dado da tupla é um dado do tipo `i32` e o terceiro dado é do tipo `f32`.

Em relação a definição de *strings* imutáveis de forma direta com o tipo de dado primitivo `str` é pertinente abrir um parênteses e comentar que é possível definir variáveis com conteúdo baseado em sequências de caracteres a partir das formas:

```
let palavra = "Olá";  
let palavra: &'static str = "Olá";
```

Nestes dois casos o efeito de definição da variável `PALAVRA` é o mesmo, sendo a primeira forma mais popular em seu uso devido a sua simplicidade de escrita.

A definição de tuplas pode ser usada com sub-rotinas que retornem mais de um valor. Neste sentido, o programa seguinte estabelece a possibilidade de retorno dos valores das proporções: numérica definida pela relação entre o perímetro de uma circunferência e seu diâmetro e área. Para isso, crie o projeto `c08ex02`.

```
use std::io;  
use std::io::prelude::*;  
  
fn proporcao() -> (f32, f32) {  
    return (3.1415926535, 1.6180339887)  
}  
  
fn main() {  
    let valor = proporcao();  
  
    println!("Valor do pi ...: {:.2}", valor.0);  
    println!("Valor do phi ..: {:.2}", valor.1);  
    println!();  
    print!("Tecle <Enter> para encerrar...");  
    io::stdout().flush().unwrap();  
    io::stdin().read(&mut [0u8]).unwrap();  
}
```

Ao serem executados, os valores do pi e do phi são apresentados, advindos da sub-rotina função `proporção()`, que retorna dois valores definidos da tupla (3.1415926535, 1.6180339887). O retorno de mais de um valor para uma sub-rotina função foi possível a partir da definição da tupla.

A definição de tuplas pode ser feita por meio do acesso ao padrão `matching`, em virtude de esse tipo de estrutura de dado ser um conjunto de elementos incorporados. Sendo assim, no sentido de demonstrar o uso do comando `match` com a definição de tuplas, considere um programa que indique a posição apontada dos quadrantes de um ciclo trigonométrico. Para isso, crie o projeto `c08ex03`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let coordenada = (-1, 1);

    match coordenada {
        ( 1, 1) => println!("Primeiro quadrante"),
        (-1, 1) => println!("Segundo quadrante"),
        (-1,-1) => println!("Terceiro quadrante"),
        ( 1,-1) => println!("Quarto quadrante"),
        _ => {},
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, a mensagem `Segundo quadrante` é apresentada devido aos valores definidos na instrução `let coordenada = (-1, 1);`. Altere os valores do par de coordenada para o conjunto de valores definidos em `match` e veja as demais notificações apresentadas. Se for definido algum valor fora dos valores estabelecidos em `match`, nenhuma notificação será apresentada, por causa do trecho “`_ => {}`,”.

8.2 Estruturas

Estrutura, na linguagem Rust, é um modelo para a definição de um conjunto de dados encapsulados que serve de base para a definição de um novo tipo de dado unificado (tipo derivado). Os dados de uma estrutura devem ter entre si uma relação contextual.

As estruturas definidas na linguagem Rust podem ser classificadas em três variações: estruturas clássicas (forma tradicional de definição ao estilo da linguagem C, assemelhando-se à definição de classes sem o uso de sub-rotinas-membros); estruturas com o uso de tuplas; e estruturas de unidades, sem a definição de membros – semelhantes a uma tupla vazia, útil para definir genéricos.

As estruturas podem ser declaradas dentro (estruturas internas) e fora (estruturas externas) da sub-rotina `main()`: dentro de `main()` são estruturas de escopo local e válidas dentro do limite de operação delimitado pelo bloco definido com as chaves; fora da sub-rotina `main()`, terão escopo global no código de um programa. A definição de uma estrutura é feita com o primeiro caractere em maiúsculo; os demais caracteres seguem a nomenclatura da notação camelo. Por questões de legibilidade estrutural e respeito à nomenclatura *bottom-up*, as estruturas, neste trabalho, serão declaradas externamente e antes da sub-rotina `main()`.

8.2.1 Estrutura clássica

As estruturas clássicas, ao estilo da linguagem C, são definidas a partir da seguinte configuração:

```
struct [<NomeIdentificador>] {  
    <campo1>: <tipo>,  
    <campo2>: <tipo>,  
    ...  
    <campoN>: <tipo>,  
}
```

NomeIdentificador é o nome da estrutura, *campo* e *tipo* formam a relação de variáveis (campos em caracteres minúsculos) internas qualificadas por

meio do tipo de dado que representam, sendo sempre imutáveis. O nome de identificação de uma estrutura é em Rust definido com o uso de um caractere maiúsculo para o primeiro caractere e os demais minúsculos, exceto para nomes compostos, que devem ser grafados com base na notação camelo.

A estrutura definida é usada para estabelecer o tipo de dado de uma variável a ela associada. Uma variável definida a partir de uma estrutura recebe uma instância na memória dessa estrutura. Isso é feito por meio do comando `let`.

A definição de uma estrutura estabelece um formato de dado que será usado para modelar o comportamento de uma variável na memória. Por isso toda `struct` é imutável. Para fazer uso de uma variável com comportamento mutável utiliza-se o comando `mut` associado ao comando `let` quando há definição da variável a partir de uma `struct`.

O acesso aos campos de uma estrutura instanciada sobre uma variável é feito com o uso do símbolo `(.)` ponto como se fosse um operador de escopo.

O programa a seguir demonstra o uso de uma estrutura de dados `struct` modelada para armazenar o nome, as notas bimestrais e a média de um aluno. Após receber a entrada dos dados, o programa efetua o cálculo da média e, ao final, indica os dados fornecidos na entrada e informa se o aluno foi aprovado ou reprovado com base na verificação de sua média. A média de aprovação considerada deve ser maior ou igual a 6. Para isso, crie o projeto `c08ex04`.

```
use std::io;
use std::io::prelude::*;

struct CadAluno {
    nome: String,
    notas: Vec<f32>,
    media: f32,
}
```



```

fn main() {
    let mut aluno = CadAluno {
        nome: "".to_string(),
        notas: Vec::new(),
        media: 0.
    };
    let mut nota = String::new();
    let mut soma: f32 = 0.;

    print!("Entre nome do aluno ..: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut aluno.nome).unwrap();

    aluno.notas.resize(4, 0.);
    for i in 0 .. 4 {
        print!("Entre a {}a. nota .....: ", i + 1);
        io::stdout().flush().unwrap();
        io::stdin().read_line(&mut nota).unwrap();
        aluno.notas[i] = nota.trim().parse::<f32>().unwrap();
        soma += aluno.notas[i];
        nota.clear();
    }
    aluno.media = soma / 4.0;

    println!();
    println!("Nome do aluno .....: {}", aluno.nome);

    for i in 0 .. 4 {
        println!("{}", aluno.notas[i]);
    }
    println!();
    print!("Média .....: {:.2}", aluno.media);

    if aluno.media >= 6. {
        println!(" => Aprovado.");
    } else {
        println!(" => Reprovado.");
    }
}

```

```

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, efetue alguns testes informando diversos valores de notas para verificar as notificações de aprovação e reprovação.

Apesar de simples, o programa do projeto `c08ex01` tem muitos recursos importantes que devem ser conhecidos quando se utiliza a definição de estruturas de dados com `struct`, a começar pela definição dos campos da estrutura. A estrutura de dados `CadAluno` se compõe em três campos: `NOME` do tipo `String` para a manipulação de dados alfanuméricos; `NOTAS` do tipo `Vec<f32>` na definição de uma matriz unidimensional do tipo vetor para o armazenamento da nota, que serão definidas em quatro posições mais adiante no código; e `MEDIA` do tipo `f32` para o armazenamento de um valor real de 32 bits.

Dentro da sub-rotina `main()` é feita a inicialização das variáveis que são usadas no programa: `SOMA` do tipo real de 32 bits inicializada com valor zero para a soma das notas do aluno para o posterior cômputo da média; `NOTA` variável de apoio, que receberá a nota fornecida e fará seu tratamento e a transferência para a posição `NOTAS` da estrutura; e a variável `ALUNO` do tipo `CadAluno`, que precisa ter suas posições inicializadas antes do uso.

Observe que a variável `ALUNO` do tipo `CadAluno` faz uso na sua definição da indicação de chaves que determinam o conjunto de campos que devem ser inicializados. Para cada campo há uma forma de inicialização, compatível com sua estrutura. Para um campo do tipo `String` está sendo definido o conteúdo de inicialização `""` (representa um campo vazio) associado ao método `to_string()`, que determina ser o dado a ele associado compatível com o campo do tipo `String`. Para o campo `NOTAS` está sendo definido o conteúdo `Vec::new()`, que cria na memória a área de vetor que será utilizada. No caso do campo `MEDIA`, por ser um campo para armazenamento de um dado numérico, basta inicializá-lo com um valor

também numérico, como zero.

Antes de efetivar a entrada das notas, o programa faz uso da instrução `aluno.notas.resize(4, 0.);`, que estabelece a quantidade de dados que serão fornecidos para a matriz definida dentro da estrutura. O método `resize()` define a quantidade de 4 elementos com valor 0. para o campo `NOTAS`.

Atente também para a forma de acesso realizada a cada um dos membros (campos) da estrutura em uso, sendo estes efetivados por meio do *operador de ligação de membro* (“.” - ponto). Assim, são acessos válidos aos membros da estrutura `CadAluno` as referências feitas como `ALUNO.NOME`, `ALUNO.NOTA[0]`, `ALUNO.NOTA[1]`, `ALUNO.NOTA[2]`, `ALUNO.NOTA[3]` e `ALUNO.MEDIA`. Essa forma de referência é usada em operações de entrada, processamento e saída.

8.2.2 Estrutura tupla

A estrutura tupla é semelhante a uma estrutura clássica, tendo como diferença o fato de não ter nomes definidos para seus campos, sendo esta outra forma de estabelecer um conjunto de dados heterogêneos a partir de campos anônimos. Assim, uma estrutura tupla tem um nome de identificação com campos não nomeados, tendo indicados apenas os tipos de dados que representam cada campo anônimo da estrutura a partir da configuração:

```
struct [<NomeIdentificador>] (  
    <tipo1>,  
    <tipo2>,  
    ...  
    <tipoN>,  
)
```

NomeIdentificador é o nome da estrutura tupla, e *tipo* é a definição do tipo de dado que representa o campo anônimo associado a ela.

Apesar de uma estrutura tupla permitir o uso de mais de um tipo de dado como campo anônimo, ela é conveniente para a definição de um único campo. Para mais de um campo, a estrutura clássica se torna mais

conveniente. No caso de se usar um único campo, tem-se o que se chama em Rust de definição de um *newtype* (novo tipo, como um padrão de dado simples). Um tipo *newtype* permite a definição de um novo tipo de dado (tipo de dado derivado).

Para demonstrar o uso de uma estrutura tupla, o programa a seguir solicita a inserção do ano de nascimento e de um ano após o nascimento, como o ano atual, e apresenta a idade calculada com base nos valores dos anos indicados. Para isso, crie o projeto `c08ex05`.

```
use std::io;
use std::io::prelude::*;

struct Idade(u16);

fn main() {
    let mut ano_nasci = String::new();
    let mut ano_atual = String::new();

    let an: u16;
    let aa: u16;

    print!("Entre ano de nascimento ... ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut ano_nasci).unwrap();
    an = ano_nasci.trim().parse::<u16>().unwrap();

    print!("Entre ano atual ..... ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut ano_atual).unwrap();
    aa = ano_atual.trim().parse::<u16>().unwrap();

    let idade = Idade(aa - an);

    let Idade(tempo_de_vida) = idade;

    println!();
    println!("Sua idade é {} anos.", tempo_de_vida);
}
```

```
println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, efetue a entrada dos anos de nascimento e atual e observe a apresentação da idade calculada a partir dos não fornecidos. Observe a definição da variável `IDADE` qualificada a partir da estrutura de tupla `Idade()` com a definição da subtração dos anos de nascimento (`AN`) e do ano atual (`AA`) passados como se fosse o parâmetro de uma sub-rotina a estrutura tupla `Idade()` com a instrução `let idade = Idade(aa - an);`. Na sequência, a instrução `let Idade(tempo_de_vida) = idade;` atribui o valor da variável `IDADE` à variável `TEMPO_DE_VIDA` (variável `newtype` definida a partir da estrutura de tupla em uso), que é usada para mostrar a idade junto à instrução `println!("Sua idade é {} anos.", tempo_de_vida);`.

8.2.3 Estrutura-unidade

Uma estrutura-unidade é um tipo de dado definido cuja estrutura não é definida de forma concreta, podendo ser usada como a definição de tipos de dados abstratos. A estrutura-unidade, conhecida como estrutura opaca, é uma forma de representação de dados mais rara usada na linguagem Rust. Essa estrutura pode ser considerada na utilização de dados que não serão armazenados na memória, dados que não se sabe de antemão de que tipo são, ou mesmo fazer uso da estrutura sem saber o que ela, de fato, deve conter.

Uma questão que pode elucidar o uso desse tipo de recurso é a possibilidade de se fazer na linguagem Rust o acesso a algum recurso como uma sub-rotina de uma biblioteca de terceiros que retorne algum conteúdo do qual não se tenha acesso completo a seus campos, mas que seja necessário usar o resultado da ação da sub-rotina em diversos pontos do programa que recebe a incorporação do recurso, como recursos da biblioteca GTK+ (GOMEZ, 2017).

Quando se utiliza a definição de uma estrutura-unidade, torna-se necessário estabelecer no código do programa ações concretas para que a definição possa ser efetivamente usada. Sendo assim, o programa a seguir mostra ação realizada com a definição de estrutura-unidade em um projeto que apresenta de forma descritiva o som de um bicho de estimação. Para isso, crie o projeto `c08ex06`.

```
#![allow(unused_variables)]
use std::io;
use std::io::prelude::*;

struct Animal;
struct Gato;
struct Cao;
impl Animal {
    fn miar(&self, som: &Gato) -> () {
        println!("Miau, miau, miau!");
    }
}

impl Animal {
    fn latir(&self, som: &Cao) -> () {
        println!("Au au, au au, au au!");
    }
}

fn main () {
    let bichinho = Animal{};
    let gato = Gato{};
    let cao = Cao{};

    print!("O gato faz: "); bichinho.miar(&gato);
    print!("O cão faz: "); bichinho.latir(&cao);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

}

Ao ser executado o programa, os sons do gato e do cão serão apresentados. Note que há a definição de três estruturas-unidades, sendo `Animal()`, `Gato()` e `Cao()`. No caso da estrutura-unidade `Animal()` estão sendo definidas duas sub-rotinas com a implementação do som emitido por cada animal em uso.

As ações implementadas da estrutura-unidade `Animal()` junto às sub-rotinas `miar()` e `latir()` são feitas por meio do comando `impl`, que permite associar a sub-rotina definida à estrutura indicada; neste caso, `Animal`. Nas sub-rotinas há a definição do parâmetro `self` definido como referência de retorno para `&Gato` e `&Cao`, o que é recebido pela execução dos trechos `bichinho.miar(&gato)` e `bichinho.latir(&cao)`, respectivamente.

O comando `impl` usado junto de uma `struct` define a sub-rotina a ser vinculada à estrutura em estilo método, como ocorre com a programação orientada a objetos. O uso de `&self` antes dos parâmetros das sub-rotinas `miar()` e `latir()` possibilita que as sub-rotinas tenham acesso aos dados definidos para uma estrutura; neste caso, o acesso à mensagem escrita quando há chamada de cada uma das sub-rotinas.

8.3 Enumerações

Entre as estruturas de dados utilizadas na linguagem Rust, existe a possibilidade de estabelecer um conjunto de dados de vários tipos que contenha constantes inteiras relacionadas às posições que representam. Esse conjunto deve ter uma relação de valores válidos de acesso na lista enumerada de dados, sendo útil para a criação de listas de dados fechadas com configurações diferentes das que são conseguidas com matrizes, tuplas e estruturas.

A definição das enumerações é parecida com a forma de definição das estruturas, sendo possível estabelecer enumerações sem dados, como uma estrutura-unidade, além de poder definir enumerações com dados nomeados e não identificados como estruturas tuplas, sendo declarada a partir da sintaxe

```
enum [<NomeIdentificador>] {
    <elemento1> [= valor],
    <elemento2> [= valor],
    ...
    <elemento3> [= valor],
}
```

NomeIdentificador é o nome da enumeração, *elemento* é a definição do conteúdo da enumeração e *valor* é a definição do valor opcional da constante enumerada. Tanto o nome de identificação da enumeração como seus elementos devem seguir a forma de notação camelo com o primeiro caractere do nome em letra maiúscula.

Para exemplificar o uso de uma enumeração simples, considere o programa que efetua uma pesquisa na lista com base em um valor válido (calendário mensal) predeterminado na enumeração e atribuído ao código do programa. Para isso, crie o projeto `c08ex07`.

```
#![allow(unused_variables, dead_code)]
```

```
use std::io;
```

```
use std::io::prelude::*;
```

```
enum Meses {
    Janeiro,
    Fevereiro,
    Marco,
    Abril,
    Maio,
    Junho,
    Julho,
    Agosto,
    Setembro,
    Outubro,
    Novembro,
    Dezembro,
}
```

```
fn main() {
    let mes = Meses::Janeiro;
```



```

println!("Teste de acesso a lista enumerada");
println!();
print!("Mês {} é definido como ");

match mes {
    Meses::Janeiro => println!("Janeiro"),
    Meses::Fevereiro => println!("Fevereiro"),
    Meses::Marco => println!("Março"),
    Meses::Abril => println!("Abril"),
    Meses::Maio => println!("Maio"),
    Meses::Junho => println!("Junho"),
    Meses::Julho => println!("Julho"),
    Meses::Agosto => println!("Agosto"),
    Meses::Setembro => println!("Setembro"),
    Meses::Outubro => println!("Outubro"),
    Meses::Novembro => println!("Novembro"),
    Meses::Dezembro => println!("Dezembro"),
}
println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, é apresentado o nome por extenso do mês que se encontra atribuído à variável `MES` pela instrução `let mes = Meses::Janeiro;`.

O programa a princípio executa a configuração de compilação `#![allow(unused_variables, dead_code)]` que desabilita as mensagens de advertência pela falta de uso de alguma variável (`unused_variables`) e da ausência de uso de algum elemento interno (`dead_code`) do código. Esta ação é necessária porque várias mensagens de advertência são apresentadas na compilação do programa relacionadas ao não uso de todos os campos da enumeração `Meses`, exceto ao campo `Janeiro`.

O comando do padrão `match`, quando associado diretamente a uma enumeração, dispensa o uso da última opção, que efetua a ação de controle quando um erro ocorre nos valores da lista que `match` controla.

Esta característica decorre do fato de o padrão `match` receber valores que já se encontram definidos em uma enumeração, e não serão informados de forma interativa durante seu uso.

No programa do projeto `c08ex07`, a variável `MES` contém internamente o valor de referência de posição do campo da enumeração atribuído. O campo `Janeiro` da enumeração `Meses` tem na enumeração seu valor de posição como zero. Dessa forma, `Fevereiro` tem o valor 1, e assim por diante. Assim como uma matriz, a enumeração opera, por padrão, com posicionamento cardinal. Porém, diferentemente das matrizes, os valores do posicionamento das enumerações podem ser alterados.

O programa seguinte demonstra o uso de enumerações com a definição-padrão de posicionamento de campos e com a definição alterada dos valores de posicionamento dos campos. Para isso, crie o projeto `c08ex08`.

```
use std::io;
use std::io::prelude::*;
```

```
#[derive(Debug)]
enum Lista1 {
    Um,
    Dois,
    Tres,
}
```

```
#[derive(Debug)]
enum Lista2 {
    Um = 1,
    Dois,
    Tres,
}
```

```
#[derive(Debug)]
enum Lista3 {
    Um = 1,
    Tres = 3,
    Cinco = 5,
}
```

```

fn main() {
    println!("{:?} = {}", Lista1::Um, Lista1::Um as u8);
    println!("{:?} = {}", Lista1::Dois, Lista1::Dois as u8);
    println!("{:?} = {}", Lista1::Tres, Lista1::Tres as u8);
    println!();

    println!("{:?} = {}", Lista2::Um, Lista2::Um as u8);
    println!("{:?} = {}", Lista2::Dois, Lista2::Dois as u8);
    println!("{:?} = {}", Lista2::Tres, Lista2::Tres as u8);
    println!();

    println!("{:?} = {}", Lista3::Um, Lista3::Um as u8);
    println!("{:?} = {}", Lista3::Tres, Lista3::Tres as u8);
    println!("{:?} = {}", Lista3::Cinco, Lista3::Cinco as u8);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, são apresentados os conteúdos de cada uma das enumerações bem como seus valores de referência padrão, alterados de acordo com a especificação definida.

Na enumeração `Lista1`, os seus campos são definidos e mantidos com seus valores de posicionamento padrão 0, 1 e 2. Já na enumeração `Lista2`, ao se definir o primeiro campo com valor 1, fez-se automaticamente a definição dos demais campos numericamente para 2 e 3. No entanto, na enumeração `Lista3`, a alteração manual estabelecida foi 1, 3 e 5 para os campos da `Lista3`, pois, se após o campo definido como 5 houvesse mais campos, os demais seriam 6, 7, 8 e assim por diante.

Um detalhe que chama atenção é a configuração do processo de compilação definido antes de cada enumeração `#[derive(Debug)]`, que deve ser usada quando se quiser apresentar em uma saída o nome do campo por meio do código de formatação `{:?}`. Dessa forma, se usado, por exemplo, `Lista1::Um`, será apresentado `Um`, e para o código de formatação `{}`

será apresentado `Um` para a mesma indicação `Lista1::Um`, desde que seja usado o comando `as` de operação de *casting* para o tipo de dado `u8`.

Para demonstrar o uso de enumerações com elementos numerados, considere um programa que solicite a entrada de um valor numérico correspondente a um mês do calendário e indique o nome do mês por extenso. Para isso, crie o projeto `c08ex09`.

```
#![allow(unused_variables, unused_assignments)]
```

```
use std::io;
```

```
use std::io::prelude::*;
```

```
#[derive(Debug)]
```

```
enum Meses {
```

```
    Janeiro = 1,
```

```
    Fevereiro,
```

```
    Marco,
```

```
    Abril,
```

```
    Maio,
```

```
    Junho,
```

```
    Julho,
```

```
    Agosto,
```

```
    Setembro,
```

```
    Outubro,
```

```
    Novembro,
```

```
    Dezembro,
```

```
}
```

```
impl Meses {
```

```
    fn pega_mes(valor: u8) -> Meses {
```

```
        match valor {
```

```
            1 => Meses::Janeiro,
```

```
            2 => Meses::Fevereiro,
```

```
            3 => Meses::Marco,
```

```
            4 => Meses::Abril,
```

```
            5 => Meses::Maio,
```

```
            6 => Meses::Junho,
```

```
            7 => Meses::Julho,
```

```
            8 => Meses::Agosto,
```

```

        9 => Meses::Setembro,
        10 => Meses::Outubro,
        11 => Meses::Novembro,
        12 => Meses::Dezembro,
        _ => panic!(""),
    }
}
}

fn main() {
    let mut entra_mes = String::new();
    let mut m: u8;
    let mes: Meses;

    println!("Teste de acesso a lista enumerada");
    println!();
    loop {
        print!("Entre valor numérico entre 1 e 12: ");
        io::stdout().flush().unwrap();
        io::stdin().read_line(&mut entra_mes).unwrap();
        m = entra_mes.trim().parse::<u8>().unwrap();
        if m >= 1 && m <= 12 { break; }
        entra_mes.clear();
    }

    mes = Meses::pega_mes(m);

    println!();
    print!("Mês {} igual a {:?}\n", m, Meses::pega_mes(m));

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, será solicitada a entrada de um valor numérico entre 1 e 12 referente a um dos meses do calendário. Qualquer valor fornecido de forma inválida fará com que o laço `loop` exija que nova

entrada seja efetuada. Ao ser fornecido um valor válido, serão apresentados o valor e o nome do mês correspondente.

O programa do projeto `c08ex09` faz uso, na primeira linha de código junto à sub-rotina `allow()` para tratamento da apresentação, das mensagens de advertência do parâmetro `unused_assignments`, que ocorre durante a ação da instrução `mes = Meses::pega_mes(m);`, indicando que a variável `MES` não é lida pelo programa.

Na definição da enumeração `Meses`, é definido junto ao campo `Janeiro` o valor 1, o que faz com que os demais campos sejam seus sucessivos valores, como 2, 3, até 12.

O comando `impl` define para a enumeração `Meses` uma sub-rotina como um método de acesso chamado `pega_mes()`, que recebe a entrada de um parâmetro `valor` do tipo inteiro sinalizado de 8 bits e devolve como resposta um valor do tipo `Meses`, controlado por um padrão `match`, que, por receber o valor de avaliação a partir da passagem de parâmetro `valor`, exige que haja a definição da opção de erro indicada com o uso da macro `panic!()`. Os demais valores controlados pelo comando `match` devolvem o campo de referência ao valor do mês passado em `valor`.

A macro `panic!()` é usada para apresentar mensagens de erro quando estes ocorrem de forma semelhante à das macros `print!()` e `println!()`. Neste caso de uso, a macro `panic!()` está definida sem mensagem com o uso de um *string* vazio, pois, mesmo que um valor inválido seja fornecido, este não ocasionará uma ocorrência de pânico (erro de ação no programa), por estar sendo controlada esta ocorrência pelo laço `loop` ao impedir que uma entrada válida seja processada.

Outro detalhe no programa é o uso da instrução `mes = Meses::pega_mes(m);`, que atribui à variável `MES` o campo do mês advindo da enumeração `Meses`, capturado a partir da sub-rotina de função método `pega_mes()`, que recebe a passagem de parâmetro de um valor numérico representado pela variável `m` e associado à posição do campo da enumeração `Meses`.

Além do exposto anteriormente, a linguagem Rust permite que sejam

definidas, como componentes de enumerações, as tuplas ou a mescla de campos e tuplas. Assim sendo, o programa seguinte demonstra o uso de tuplas como componentes de uma enumeração que mostra os valores calculados das áreas de uma circunferência, de um quadrado e um trapézio. Para isso, crie o projeto `c08ex10`.

```
use std::io;
use std::io::prelude::*;
use std::f64::consts::PI;
enum FiguraGeometrica {
    Cilindro(f64, f64),
    Quadrado(f64),
    Trapezio(f64, f64, f64),
}

fn main() {
    let area_cil = FiguraGeometrica::Cilindro(15.0, 7.0);
    let area_qua = FiguraGeometrica::Quadrado(2.5);
    let area_tra = FiguraGeometrica::Trapezio(1.5, 2.5, 10.0);

    if let FiguraGeometrica::Cilindro(raio, altura) = area_cil {
        let area: f64;
        area = 2.0 * PI * raio * (raio + altura);
        println!("Área da circunferência ...: {:.2}", area);
    }

    if let FiguraGeometrica::Quadrado(lado) = area_qua {
        println!("Área do quadrado .....: {:.2}", lado.powf(2.0));
    }

    if let FiguraGeometrica::Trapezio(bs_maior, bs_menor, altura) =
area_tra {
        let area: f64;
        area = ((bs_maior + bs_menor) * altura) / 2.0;
        println!("Área do trapézio .....: {:.2}", area);
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
}
```

```

    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, os valores das áreas de cada figura geométrica são apresentados. Atente para a enumeração `FiguraGeometrica` com a definição de três tuplas como componente da estrutura.

O tratamento de operação de cada tupla da enumeração está sendo definido com o uso dos comandos `if let`, que proporcionam uma forma alternativa ao uso do padrão `match`.

8.4 Agregações

As estruturas de dados `struct` e `enum` podem ser usadas em ações de agregação (encadeamento), em que uma forma de estrutura pode ser base para a definição de campos ou enumerações em outras estruturas, assemelhando-se ao conceito de herança encontrado em linguagens orientadas a objetos.

O programa seguinte demonstra o uso de encadeamento de estruturas sobre o uso das estruturas `Lado` e `Retangulo` que se relacionam para dar acesso aos seus campos. Para isso, crie o projeto `c08ex11`.

```

use std::io;
use std::io::prelude::*;

struct Lado {
    base: f32,
    altura: f32,
}

struct Retangulo {
    lateral: Lado,
}

fn main() {
    let lado: Lado;
    let retangulo: Retangulo;
}

```



```

let area: f32;

lado = Lado {base: 4.789, altura: 2.975};

let Lado {base: bas, altura: alt} = lado;

retangulo = Retangulo {lateral: Lado {base: bas, altura: alt}};
area = retangulo.lateral.altura * retangulo.lateral.base;

println!("Retângulo");
println!("");
println!("Lado da base ...: {:8.2}", retangulo.lateral.base);
println!("Lado da altura .: {:8.2}", retangulo.lateral.altura);
println!("");
println!("Área .....: {:8.2}", area);

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, os valores definidos para os campos **BASE** e **ALTURA** são apresentados, bem como é apresentado o valor da área calculada de um retângulo.

Na definição da estrutura **Lado** são estabelecidos os campos **base** e **altura** do tipo primitivo **f32**, e na estrutura **Retangulo** há o estabelecimento do campo **Later** (lateral) do tipo derivado **Lado**, que é uma estrutura definida. Dessa forma, os campos, para serem acessados, precisam ser mencionados como **later.base** e **later.altura**.

Na sub-rotina de função **main()** são inicialmente definidas as variáveis de operação do programa, sendo as variáveis **LADO** e **RETANGULO** instâncias respectivas das estruturas **Lado** e **Retangulo**. A variável **AREA** do tipo **f32** é usada para o cálculo da área do retângulo a ser apresentada.

A instrução **lado = Lado {base: 4.789, altura: 2.975};** inicializa a variável **LADO** com os valores dos campos **base** e **altura** indicados junto à estrutura

Lado. Na sequência, a instrução `let Lado {base: bas, altura: alt} = lado;` pega os valores da variável `LADO` e transfere esses valores para as variáveis `BAS` e `ALT`, desestruturando a ligação feita pelo comando `let`.

A instrução `retangulo = Retangulo {Later: lado {base: bas, altura: alt}};` estabelece para a variável `RETANGULO` a estrutura `Retangulo` definida a partir do campo `base`, relacionada ao campo `later` de `Retangulo`.

Na sequência, a instrução `area = retangulo.later.altura * retangulo.later.base;` efetua o cálculo da área a partir da indicação dos campos `later.altura` e `later.base` relacionados à variável `RETANGULO`. Note que para a apresentação dos valores da base e da altura são usadas as mesmas definições de `retangulo.later.altura` e `retangulo.later.base`.

Assim como estruturas podem ser agregadas, pode-se fazer o mesmo com enumerações. O programa seguinte mostra a agregação de uma enumeração dentro de uma estrutura. A enumeração descreve os graus de escolaridade e a estrutura descreve os dados de uma pessoa. Para isso, crie o projeto `c08ex12`.

```
#![allow(unused_variables, dead_code)]
use std::io;
use std::io::prelude::*;
```

```
#[derive(Debug)]
enum Formacao {
    Basico,
    Fundamental,
    Medio,
    Superior,
}
```

```
#[derive(Debug)]
struct Pessoa {
    nome: String,
    nascimento: u16,
    escolaridade: Formacao,
}
```

```

fn main() {
    let nome = "Augusto Manzano".to_string();
    let nascimento = 1965;
    let escolaridade = Formacao::Superior;

    let pessoa = Pessoa {nome, nascimento, escolaridade};

    println!("Nome .....: {:?}", pessoa.nome);
    println!("Nascimento ....: {:?}", pessoa.nascimento);
    println!("Escolaridade ...: {:?}", pessoa.escolaridade);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado, o programa apresenta os dados definidos no código. Note na estrutura `Pessoa` o estabelecimento do campo `ESCOLARIDADE` do tipo `Formacao`, que é uma enumeração definida com os graus escolares. Note atentamente a instrução `let escolaridade = Formacao::Superior;`, que atribui à variável `ESCOLARIDADE` o conteúdo `Superior` da enumeração `Formacao`.

8.5 Definições de traços

Os recursos `struct` e `enum` se assemelham à definição de classes nas linguagens orientadas a objetos, por agregarem componentes idênticos aos campos-membros (atributos). Para a definição das sub-rotinas-membros (métodos), a linguagem Rust tem o comando `impl`, que relaciona as sub-rotinas com as estruturas ou enumerações, caracterizando a definição de sub-rotinas-membros relacionadas aos campos-membros.

Os recursos `struct`, `enum` e `impl` foram anteriormente apresentados. No entanto o uso desses recursos não estabelece por si só a aplicação do paradigma de programação orientada a objetos, conhecido como POO (programação orientada a objetos), pois, para que este modelo de programação seja aplicado, é necessário fazer uso do recurso `trait` (traço).

O comando `trait` permite que seja definida uma interface que agrega intimamente um conjunto de sub-rotinas vinculadas a uma estrutura ou enumeração. Sendo assim, `trait` se parece com a mesma definição de interface encontrada nas linguagens Java, Go e C#. O estilo de definição de sub-rotinas- membros (métodos) na linguagem Rust dentro de `trait` se assemelha à definição de sub-rotinas-membros externas, como ocorre na linguagem C++.

O paradigma da programação orientada a objetos é uma maneira de levar a estruturação de dados e seu controle em memória mais a sério. Este paradigma de programação surgiu durante a década de 1960, tornando-se popular nas décadas de 1980 e 1990, respectivamente, com as linguagens de programação SmallTalk (Xerox) e C++ (Bell).

A linguagem Rust, do ponto de vista ortodoxo, pode não ser considerada orientada a objetos, mas, de um ponto de vista mais despojado, fica clara a sua relação com a POO. As estruturas e enumerações vinculadas às implementações, ao instanciarem uma variável, podem não ser objetos, mas funcionam como tal. Rust pode não ser orientada a objetos de forma direta, ou pode não conter certas operações, em relação à forma escrita, mas torna-se conceitualmente orientada a objetos no que se refere ao seu comportamento. Não tem diretamente ações de herança, como se conhece, apesar de abranger polimorfismo.

O paradigma da programação orientada a objetos é, normalmente, focado nas linguagens de programação com suporte a esse modelo, e não tanto no comportamento da programação em si. Com Rust, o foco se mantém no comportamento da programação, como poderá ser constatado, pois alguns conceitos de uso da orientação a objetos são apresentados a partir de recursos essenciais da linguagem que aparentemente não têm relação direta com a orientação a objetos. Algo semelhante ocorre com a orientação a objetos na linguagem Lua.

A nomenclatura usada para a representação dos elementos operacionalizados nas classes pelas linguagens orientadas a objetos não são consistentes, exceto os conceitos de classes e objetos. Dependendo da

linguagem orientada a objetos em uso, certos conceitos internos a uma classe são diferentemente chamados. Por exemplo, o conceito de atributo (em Java) é chamado de campo-membro em Object Pascal e C++. Já na linguagem C# é chamado de *fields*: o conceito de método (em Java) é chamado de procedimento-membro ou função-membro em Object Pascal, função-membro em C# e algumas vezes em C++. Na linguagem Rust não há uma maneira-padrão de referenciar os componentes de uma classe, devido suas características operacionais. Com o objetivo de ter uma definição operacional, a nomenclatura usada nesta obra será: classe, objeto, campo-membro (para atributos) e sub-rotina-membro de procedimento (para um método que não retorne valor) e sub-rotina-membro de função (para um método que retorna valor).

O uso de classes e objetos em Rust é sutil, pois, a partir da definição das estruturas de dados `struct` e/ou `enum`, se tem a ideia de uma classe contendo campos-membros, e o objeto passa a ser a variável qualificada a partir da estrutura definida. Por exemplo, a classe `ClsPessoa`, usada para instanciar um objeto `PESSOA`, pode ser definida como:

```
struct ClsPessoa {  
    nome : String,  
    idade : i16,  
}  
  
fn main() {  
    let pessoa = ClsPessoa { nome: "".to_string(), idade: 0 };  
}
```

A instrução `let pessoa = ClsPessoa { nome: "".to_string(), idade: 0 };` pode ser considerada a definição da instância do objeto `PESSOA` a partir da classe `ClsPessoa` com a inicialização dos campos-membros (atributos) `NOME` e `IDADE`, como se fosse (ainda não é) o uso do método construtor `ClsPessoa{}`. Neste exemplo, faz-se a definição mais simples do conceito de classe e objetos em Rust.

Os elementos definidos dentro das estruturas e enumerações são sempre privados, assim como é tudo o que é definido na linguagem Rust, sendo

possível o uso no próprio código, a menos que se utilize o comando `pub` (público) à frente da definição de módulos, sub-rotinas, tipos, enumerações e estruturas, para que possam ser usadas por outro código. Se uma estrutura for definida como `pub`, esta será pública, mas não necessariamente seus campos, que deverão ser qualificados com `pub` separadamente.

Considerando, como exemplo, a classe `ClsPessoa`, tanto a própria classe como seus campos são por padrão privados, pois nenhum desses componentes está qualificado com o comando `pub`. Agora, a definição

```
pub struct ClsPessoa {  
    nome : String,  
    idade : i16,  
}  
  
fn main() {  
    let pessoa = ClsPessoa { nome: "".to_string(), idade: 0 };  
}
```

faz com que a classe seja pública ao usar `pub struct ClsPessoa {}`. No entanto, seus campos continuam sendo privados, pelo fato de não terem sido qualificados com `pub`. Dessa forma, a definição

```
pub struct ClsPessoa {  
    pub nome : String,  
    pub idade : i16,  
}  
  
fn main() {  
    let pessoa = ClsPessoa { nome: "".to_string(), idade: 0 };  
}
```

faz com que todos os componentes da classe, bem como a classe, sejam públicos. Contudo é pertinente lembrar que o objetivo da orientação a objetos é operar seus recursos sempre de forma privada.

Além dos campos de uma classe, a estrutura da orientação a objetos prevê o uso de sub-rotinas-membros para acesso e controle dos campos privados. Normalmente as sub-rotinas-membros são públicas e dão

acesso aos campos privados definidos para um objeto. Anteriormente as sub-rotinas-membros usadas foram definidas com o comando `impl` e relacionadas às estruturas e à enumeração de forma simplista, sem levar em conta o paradigma da programação orientada a objetos, utilizando-se sub-rotinas em estilo *getters* (que pegam o conteúdo da classe para um objeto) e *setters* (que colocam o conteúdo do objeto na classe), como na linguagem Java. As sub-rotinas em estilo *getter* são, em essência, do tipo função, pois retornam um valor, e as sub-rotinas em estilo *setter* são, em essência, do tipo procedimento.

Considerando, como exemplo de uso, a classe `ClsPessoa` com os campos `NOME` e `IDADE`, as sub-rotinas-membros (método) para acesso dos campos podem ser definidas como:

```
pub fn peganome(&self) -> String {
    return format!("{}", self.nome)
}

pub fn pegaidade(&self) -> i8 {
    return self.idade
}

pub fn poenome(&mut self, nome: String) {
    self.nome = nome;
}

pub fn poeidade(&mut self, idade: i8) {
    self.idade = idade;
}
```

As sub-rotinas apresentadas estão qualificadas por `pub`, que indica que são públicas e podem ser usadas por outros códigos além do código em que são definidas. No entanto, para o código em que são definidas, o uso do comando `pub` não altera seu comportamento. Usar ou não `pub` é uma maneira de estabelecer em Rust o nível de encapsulamento de recursos.

As sub-rotinas precedidas do termo `pega` caracterizam-se por serem sub-rotinas *getters*, e por essa razão devolvem, com o uso opcional do comando

`return`, o conteúdo do campo mencionado. A diferença entre elas é a forma do tratamento do retorno para o tipo de dado `String`, que é formatado com a macro `format!()`. O uso do comando `self` dentro do bloco de cada uma das sub-rotinas se assemelha ao uso do comando `this` em outras linguagens orientadas a objetos, servindo para referenciar o acesso ao campo da classe. As sub-rotinas *getters* em Rust têm em sua assinatura a definição do tipo de retorno por meio do operador seta para direita, como `pega_nome(&self) -> String` e `pega_idade(&self) -> i8`.

As sub-rotinas precedidas do termo `poe` caracterizam-se por serem sub-rotinas *setters* que recebem parâmetros mutáveis e os atribuem ao campo da classe em uso. A indicação do parâmetro `&mut self`, antes da definição da variável, funciona como o recebimento do parâmetro invisível `this` de outras linguagens orientadas a objetos. As sub-rotinas *getters* em Rust têm em sua assinatura formatação mais simples que as sub-rotinas *setters*, por não necessitarem retornar nenhuma resposta, como `poe_nome(&mut self, nome: String)` e `poe_idade(&mut self, idade: i8)`.

O comando `self` definido como parâmetro é uma forma especial de referenciar o tipo de passagem de parâmetro, sendo possível usar três formas diferentes: `self`, `&self` e `&mut self`. O parâmetro `self` se refere ao uso de um parâmetro imutável sem referência que não será apontado para nada, obtendo-se, assim, uma cópia do objeto em uso; `&self` se refere ao uso de um parâmetro imutável com referência, por ser necessário apontá-lo para alguma variável mutável; e `&mut self` se refere ao uso de um parâmetro mutável com referência que é preciso ser passado a sua chamada.

As sub-rotinas `pega` usam o parâmetro `&self` por “necessitarem” passar a sua chamada os dados coletados da posição que apontam. Os dados capturados precisam ser devolvidos a sua chamada, por isso devem ser definidos com a passagem de parâmetro por referência. Já as sub-rotinas `poe` usam o parâmetro `&mut self` para indicar que a ação de alteração dos campos da classe indicados como `self.nome` e `self.idade` seja efetivada pelos parâmetros apontados ao lado direito da indicação `&mut self`.

Com o objetivo de fazer uso contextualizado do conceito de programação orientada a objetos, considere um programa que efetue a entrada do nome e da idade de uma pessoa e apresente os dados informados em seguida. Para isso, crie o projeto c08ex13.

```
use std::io;
use std::io::prelude::*;

struct ClsPessoa {
    nome : String,
    idade : i8,
}

impl ClsPessoa {
    pub fn new() -> ClsPessoa {
        ClsPessoa {
            nome : "".to_string(),
            idade : 0,
        }
    }

    pub fn peganome(&self) -> String {
        return format!("{}", self.nome)
    }

    pub fn pegaidade(&self) -> i8 {
        return self.idade
    }

    pub fn poenome(&mut self, nome: String) {
        self.nome = nome;
    }

    pub fn poeidade(&mut self, idade: i8) {
        self.idade = idade;
    }
}

fn main() {
```

```

let mut pessoa = ClsPessoa::new();

let mut entra_nm = String::new();
let mut entra_id = String::new();
let id: i8;

print!("Entre o nome ...: ");
io::stdout().flush().unwrap();
io::stdin().read_line(&mut entra_nm).expect("Entrada incorreta");

if let Some('\n') = entra_nm.chars().next_back() {
    entra_nm.pop();
}
if let Some('\r') = entra_nm.chars().next_back() {
    entra_nm.pop();
}

print!("Entre a idade ...: ");
io::stdout().flush().unwrap();
io::stdin().read_line(&mut entra_id).unwrap();
id = entra_id.trim().parse::<i8>().unwrap();

pessoa.poenome(entra_nm);
pessoa.poeidade(id);

println!();
println!("Nome .....: {}", pessoa.penganome());
println!("Idade .....: {}", pessoa.pegaidade());

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, forneça o nome e a idade solicitados, e os dados serão, na sequência, apresentados. Se não fosse por um pequeno detalhe, esta versão do programa seria idêntica às versões apresentadas no capítulo anterior, e, assim, este não seria orientado a objetos.

Um objeto é uma variável especial que agrega em sua estrutura os campos e as sub-rotinas que controlam o acesso a seus dados. Dessa forma, um objeto deve ser instanciado na memória, e isso é normalmente feito nas linguagens orientadas a objetos com o uso de uma função de inicialização que, no programa do projeto `c09ex01`, é utilizada pela execução da instrução `let mut pessoa = ClsPessoa::new();`, que faz a instância para o objeto `PESSOA` da estrutura de classe `ClsPessoa` com sua inicialização realizada pela função associada `new()` que foi definida pelo trecho de código:

```
pub fn new() -> ClsPessoa {  
    ClsPessoa {  
        nome : "".to_string(),  
        idade : 0,  
    }  
}
```

A função associada `new()` é uma sub-rotina que, quando chamada, devolve como resposta a estrutura `ClsPessoa`, bem como inicializa na memória os campos `NOME` e `IDADE` da estrutura `ClsPessoa`, definindo então a instância de um objeto na memória. Assim como a sub-rotina de função associada `new()`, as demais sub-rotinas de acesso ao conteúdo da classe (`struct ClsPessoa`) foram definidas dentro da implementação `ClsPessoa`. Ao se definir a implementação, utiliza-se o mesmo nome da estrutura, indicando que são dois recursos separados que pertencem à mesma entidade. Neste caso, considera-se entidade a classe `ClsPessoa`, formada pelos campos definidos em seu escopo, e as sub-rotinas-membros, definidas no escopo da implementação.

A classe em Rust é a junção de uma estrutura com uma implementação que tenha definida uma função associada `new()` para sua definição em memória. A inserção dos dados na memória junto à instância do objeto `PESSOA` da classe `ClsPessoa` é feita por meio das instruções `pessoa.poenome(entra_nm);` e `pessoa.poeidade(id);`. A saída dos dados armazenados na memória junto ao objeto `PESSOA` é efetivada pelos trechos de código `pessoa.peganome()` e `pessoa.pegaidade()`.

A definição de código de programa escrito com o paradigma da orientação a objetos em Rust pode, por meio de `trait` (traço), ser aprimorada. Uma estrutura `trait` assemelha-se ao conceito de interface utilizado pelas linguagens Go, C# e Java, podendo ser usada para definir a funcionalidade que certo tipo deve fornecer ou mesmo implementar padrões de ação para sub-rotinas-membros. As interfaces definidas com `trait` podem ser usadas para especificar uma relação de assinaturas de sub-rotinas (semelhante à definição de prótipos de sub-rotinas em arquivos de cabeçalho da linguagem C/C++).

Considerando o mesmo exemplo do programa anterior, esta versão utiliza `trait` para especificar as assinaturas das sub-rotinas em uso. Para isso, crie o projeto `c08ex14`.

```
use std::io;
use std::io::prelude::*;

struct ClsPessoa {
    nome : String,
    idade : i8,
}

trait Acoes {
    fn new() -> ClsPessoa;
    fn peganome(&self) -> String;
    fn pegaidade(&self) -> i8;
    fn poenome(&mut self, nome: String);
    fn poeidade(&mut self, idade: i8);
}

impl Acoes for ClsPessoa {
    fn new() -> ClsPessoa {
        ClsPessoa {
            nome : "".to_string(),
            idade : 0,
        }
    }
}
```

```

fn peganome(&self) -> String {
    return format!("{}", self.nome)
}

fn pegaidade(&self) -> i8 {
    return self.idade
}

fn poenome(&mut self, nome: String) {
    self.nome = nome;
}

fn poeidade(&mut self, idade: i8) {
    self.idade = idade;
}

}

fn main() {
    let mut pessoa = ClsPessoa::new();

    let mut entra_nm = String::new();
    let mut entra_id = String::new();
    let id: i8;

    print!("Entre o nome ...: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut entra_nm).expect("Entrada incorreta");

    if let Some('\n') = entra_nm.chars().next_back() {
        entra_nm.pop();
    }
    if let Some('\r') = entra_nm.chars().next_back() {
        entra_nm.pop();
    }

    print!("Entre a idade ..: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut entra_id).unwrap();

```

```

    id = entra_id.trim().parse::<i8>().unwrap();

    pessoa.poenome(entra_nm);
    pessoa.poeidade(id);

    println!();
    println!("Nome .....: {}", pessoa.peganome());
    println!("Idade .....: {}", pessoa.pegaidade());

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, o comportamento será idêntico ao exemplo do programa anterior. No entanto esta versão tem alguns detalhes diferentes, como a retirada dos qualificadores `pub`, que não são permitidos quando se usa `trait`.

O bloco `trait` relaciona as assinaturas de todas as sub-rotinas usadas no programa que estão definidas dentro de `impl`. A implementação é definida, agora, pelo trecho de instrução `impl Acoes for ClsPessoa`, que indica que o traço `Acoes` é definido para `ClsPessoa`, sendo esta forma de definição mais sugestiva para a definição da programação orientada a objetos em Rust.

Além do estabelecimento de interfaces com o uso de `trait` para uma programação em estilo de orientação a objetos, é possível com esse comando especificar outras formas de traços, como a definição de sub-rotinas dentro do bloco `trait` e o uso de ações polimórficas.

Polimorfismo é a capacidade de interagir com campos e sub-rotinas-membros relacionados a determinado objeto sem a necessidade de conhecer a princípio seu tipo de uso. É a capacidade que um objeto tem de mudar sua forma na medida de uso de suas sub-rotinas-membros. Em Rust, polimorfismo é definido com o uso do comando `trait`.

O programa seguinte demonstra o uso do recurso `trait` na configuração

de uma interface que estabelece uma assinatura de sub-rotina que será definida de duas formas diferentes no estilo de polimorfismo universal paramétrico (adaptado de Boiethios, 2017). Para isso, crie o projeto c08ex15.

```
use std::io;
use std::io::prelude::*;

trait CalcArea {
    fn area(&self) -> f32;
}

struct Circunferencia {
    raio: f32,
} impl CalcArea for Circunferencia {
    fn area(&self) -> f32 {
        return self.raio.powi(2) * std::f32::consts::PI
    }
}

struct Quadrado {
    lado: f32,
} impl CalcArea for Quadrado {
    fn area(&self) -> f32 {
        return self.lado.powi(2)
    }
}

fn calcarea(figura: &CalcArea) {
    println!("Área calculada = {}", figura.area())
}

fn main() {
    let mut entra_lado = String::new();
    let mut entra_raio = String::new();

    let raio_ent: f32;
    let lado_ent: f32;

    print!("Entre o valor do lado de um quadrado: ");
    io::stdout().flush().unwrap();
```

```

    io::stdin().read_line(&mut entra_lado).unwrap();
    lado_ent = entra_lado.trim().parse::<f32>().unwrap();
    calcarea(&Quadrado{lado: lado_ent});

    println!();
    print!("Entre o valor do raio de uma circunferência: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut entra_raio).unwrap();
    raio_ent = entra_raio.trim().parse::<f32>().unwrap();
    calcarea(&Circunferencia{raio: raio_ent});
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, informe os valores solicitados, e as áreas do quadrado e da circunferência serão apresentadas.

O programa do projeto `c08ex15` é similar ao programa do projeto `c08ex14` ao definir o bloco `trait CalcArea` com a assinatura de uso da sub-rotina `area()`. A diferença está na definição das estruturas `Circunferencia` e `Quadrado`, que utilizam a mesma assinatura do `trait CalcArea` de formas diferentes (polimorfismo).

Nesta versão do código do programa, adotou-se uma maneira de indentação diferente para a escrita das estruturas e implementações, dando a impressão visual de que o conjunto *estrutura + implementação* se trata de uma única forma, como se fosse a definição de uma classe ao estilo típico da programação orientada a objetos.

Vale ressaltar que o uso do comando `return` é opcional, e este comando é utilizado para manter maior legibilidade do código do programa.

8.6 Exercícios de fixação

A. Elaborar um programa escolar que armazene o nome e as notas bimestrais de vinte alunos de um determinado curso, definir a estrutura de registro apropriada, o diagrama de blocos e a codificação de um

programa que, por meio de um menu de opções, execute as seguintes etapas:

1. Cadastrar os vinte registros (após o cadastro dos vinte registros, classificar imediatamente a tabela de dados pelo campo nome).
2. Pesquisar os vinte registros de cada vez, pelo campo nome (usar o método de pesquisa binária – nessa pesquisa o programa deve também apresentar a média do aluno e as mensagens “aprovado”, caso sua média seja maior ou igual a 5, ou “reprovado” para média abaixo de 5).
3. Apresentar todos os registros, médias e a mensagem de aprovação ou reprovação.
4. Sair do programa de cadastro.

B. Elaborar um programa que armazene o nome e a altura de quinze pessoas com o uso de registros. O programa deve usar um menu que execute as seguintes etapas:

1. Cadastrar os quinze registros.
2. Apresentar os registros (nome e altura) das pessoas com 1,5 m ou menores.
3. Apresentar os registros (nome e altura) das pessoas com mais de 1,5 m.
4. Apresentar os registros (nome e altura) das pessoas com mais de 1,5 m e menos de 2 m.
5. Apresentar todos os registros com a média extraída de todas as alturas armazenadas.
6. Sair do programa de cadastro.

C. Elaborar um programa que gerencie os registros de vinte funcionários, contendo os campos matrícula, nome e salário. O programa deve, por meio de um menu, executar as seguintes etapas:

1. Cadastrar os vinte empregados.

2. Classificar os registros por número de matrícula.
3. Pesquisar um determinado empregado pelo número de matrícula (método binário).
4. Apresentar de forma ordenada os registros dos empregados que recebem salários acima de \$ 1.000.
5. Apresentar de forma ordenada os registros dos empregados que recebem salários abaixo de \$ 1.000.
6. Apresentar de forma ordenada os registros dos empregados que recebem salários iguais a \$ 1.000.
7. Sair do programa de cadastro.

CAPÍTULO 9

Suplementos

Este capítulo mostra o uso de recursos suplementares e de apoio à linguagem Rust. Aborda o uso básico de programação genérica e funcional, além de apresentar detalhes sobre tratamento de erros e exceções, o uso de data e hora, a indicação do recurso de coleções, focando a coleção sequencial de vetor, a manipulação de *strings* e caracteres e a aplicação do princípio de criptografia.

9.1 Programação genérica

O paradigma de programação genérica caracteriza-se por permitir a escrita de algoritmos que se adaptem a qualquer tipo de dado primitivo ou mesmo derivado usado em um programa. Um algoritmo genérico, quando escrito, é normalmente definido junto a uma estrutura de dados ou na definição de sub-rotinas (procedimentos e funções), permitindo que os dados a serem estabelecidos junto à estrutura tenham seu tipo de uso definido no momento do uso.

Na linguagem Rust a definição do efeito de programação genérica pode ser aplicada junto a estruturas de dados `struct`, `enum` e sub-rotinas. Este recurso na linguagem Rust não consome tempo de processamento, uma vez que os tipos genéricos são definidos no programa executável no momento da compilação. Normalmente a definição de um tipo genérico é feita com a letra maiúscula `T` dentro do operador diamante (`<>`). No entanto pode-se fazer uso de qualquer outra letra maiúscula ou mesmo de uma palavra.

9.1.1 Funções

A partir da aplicação de programação genérica sobre funções, considere um programa que apresente a troca de valores inteiros, reais (de ponto flutuante) e caracteres. Para isso, crie o projeto `c09ex01`.

```
use std::io;
use std::io::prelude::*;

fn trocai32(a: &mut i32, b: &mut i32) {
    let x: i32;
    x = *a;
    *a = *b;
    *b = x;
}

fn trocaf32(a: &mut f32, b: &mut f32) {
    let x: f32;
    x = *a;
    *a = *b;
    *b = x;
}

fn trocachar(a: &mut char, b: &mut char) {
    let x: char;
    x = *a;
    *a = *b;
    *b = x;
}

fn main() {
    let mut valor_a = 1;
    let mut valor_b = 2;

    println!("Valor A antes da troca ...: {}", valor_a);
    println!("Valor B antes da troca ...: {}", valor_b);

    trocai32(&mut valor_a, &mut valor_b);

    println!();
    println!("Valor A depois da troca ...: {}", valor_a);
```

```

println!("Valor B depois da troca ..: {}", valor_b);

let mut valor_a = 3.5;
let mut valor_b = 4.5;
println!("\n");
println!("Valor A antes da troca ...: {}", valor_a);
println!("Valor B antes da troca ...: {}", valor_b);

trocaf32(&mut valor_a, &mut valor_b);

println!();
println!("Valor A depois da troca ..: {}", valor_a);
println!("Valor B depois da troca ..: {}", valor_b);

let mut valor_a = 'a';
let mut valor_b = 'b';

println!("\n");
println!("Valor A antes da troca ...: {}", valor_a);
println!("Valor B antes da troca ...: {}", valor_b);

trocachar(&mut valor_a, &mut valor_b);

println!();
println!("Valor A depois da troca ..: {}", valor_a);
println!("Valor B depois da troca ..: {}", valor_b);

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, são apresentadas as trocas de três tipos de valores diferentes. Observe que para cada troca foi definida uma sub-rotina em estilo procedimento para a execução da ação. Apesar de o programa fazer o que se requer, o trabalho para tratar cada tipo de dado usado torna-se cansativo e tedioso, pois é necessário definir três sub-rotinas como `trocai32()`, `trocaf32()` e `trocachar()`.

Uma solução para que se escreva apenas uma sub-rotina que se adapte a qualquer solução é o uso de uma sub-rotina genérica. No entanto, para o caso apresentado no projeto `c09ex01`, o qual faz uso de uma ação baseada na troca de pares de valores de tipos diferentes, é necessário considerar o uso de três novas bibliotecas: `std::mem::uninitialized`, `std::mem::forget` e `std::ptr`.

Dessa forma, considerando o programa do projeto `c09ex01` adaptado para uso de função genérica para a efetivação da troca de dados com tipos diferentes, execute a criação do projeto `c09ex02`.

```
use std::io;
use std::io::prelude::*;

fn troca<T>(a: &mut T, b: &mut T) {
    unsafe {
        let mut x: T = uninitialized();
        ptr::copy_nonoverlapping(&*a, &mut x, 1);
        ptr::copy_nonoverlapping(&*b, a, 1);
        ptr::copy_nonoverlapping(&x, b, 1);
        forget(x);
    }
}

fn main() {
    let mut valor_a = 1;
    let mut valor_b = 2;

    println!("Valor A antes da troca ...: {}", valor_a);
    println!("Valor B antes da troca ...: {}", valor_b);

    troca(&mut valor_a, &mut valor_b);

    println!();
    println!("Valor A depois da troca ..: {}", valor_a);
    println!("Valor B depois da troca ..: {}", valor_b);

    let mut valor_a = 3.5;
    let mut valor_b = 4.5;
```

```

println!("\n");
println!("Valor A antes da troca ...: {}", valor_a);
println!("Valor B antes da troca ...: {}", valor_b);

troca(&mut valor_a, &mut valor_b);
println!();
println!("Valor A depois da troca ..: {}", valor_a);
println!("Valor B depois da troca ..: {}", valor_b);

let mut valor_a = 'a';
let mut valor_b = 'b';

println!("\n");
println!("Valor A antes da troca ...: {}", valor_a);
println!("Valor B antes da troca ...: {}", valor_b);

troca(&mut valor_a, &mut valor_b);

println!();
println!("Valor A depois da troca ..: {}", valor_a);
println!("Valor B depois da troca ..: {}", valor_b);

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, o resultado da saída será idêntico ao do projeto `c09ex01`. Só que nesta versão foi escrita apenas uma sub-rotina que efetua a troca de pares de valores de tipos diferentes, proporcionando mais economia na escrita de código e melhor aproveitamento de recursos desenvolvidos. O trecho de código definido dentro da sub-rotina `main()` entre os projetos `c09ex01` e `c09ex02` é exatamente o mesmo.

A definição da sub-rotina `troca()` é feita com o trecho de código `troca<T>` (`a: &mut T`, `b: &mut T`), no qual é definido após o nome da sub-rotina o qualificador `<T>`, que indica que a função é um recurso genérico que

poderá ser usado para operar com qualquer tipo de dado. Dentro dos parênteses, os parâmetros também estão sendo definidos com o tipo `T`, lembrando que é possível fazer uso de outra letra maiúscula no lugar de `T`.

A biblioteca `std::mem::uninitialized` está sendo usada por causa da necessidade de definir manualmente a não inicialização da variável `x` do tipo `T` por meio da instrução `let mut x: T = uninitialized();`. Na linguagem Rust a memória definida para uso em tempo de execução do programa é criada automaticamente como não inicializada (*uninitialized*), o que pode ocasionar problemas para validar o estado de um tipo de dado não conhecido, como é o caso de uso da sub-rotina `troca()`. Neste caso, Rust fornece recursos para o tratamento de memória de forma manual quando em uso uma área não inicializada, que no projeto `c09ex02` é administrada junto ao bloco `unsafe {}`. Dessa forma, qualquer erro de não segurança no acesso à memória está sendo ajustado pela instrução `let mut x: T = uninitialized();`.

A operação de troca dos valores na memória, pelo fato de esses valores operarem com tipos de dados diferentes, exige que se use a função `copy_nonoverlapping()`, pertencente à biblioteca `ptr` colocada em uso com a instrução `use std::ptr;`. A função de cópia de dados na memória `copy_nonoverlapping()` opera com três parâmetros, sendo o primeiro a fonte de cópia, o segundo o destino da cópia e o terceiro a quantidade de bytes a serem copiados (`count * size_of<T>`). No caso de uso do projeto `c09ex02`, basta manter o terceiro parâmetro como 1.

A biblioteca `std::mem::forget` é definida em virtude da necessidade da operação da função `forget()`, que faz a limpeza da memória, retirando o valor da variável `x` e deixando essa área livre para outro uso.

O exemplo anterior, por requerer operações de movimentação de dados na memória com tipos de dados não numéricos, como o tipo `char`, precisou de um cuidado maior do que seria necessário com dados apenas dos tipos numéricos.

Com o objetivo de mostrar uma ação mais simples, considere um programa que apresente o menor valor numérico de dois valores

quaisquer fornecidos, que possam ser inteiros de 8, 16, 32, 64 bits ou reais de 32, 64 bits. Para isso, crie o projeto `c09ex03`.

```
use std::io;
use std::io::prelude::*;
use std::cmp::PartialOrd;

fn menorv<T: PartialOrd>(a: T, b: T) -> T {
    if a < b {return a;} else {return b;}
}

fn main() {
    let v1: u16 = 5;
    let v2: u16 = 4;

    let v3: u64 = 2;
    let v4: u64 = 9;

    let v5: f64 = 2.9;
    let v6: f64 = 5.8;

    let v7: f64 = 9.9;
    let v8: f64 = 6.7;

    println!("Entre {} e {} o menor valor é {}", v1, v2, menorv(&v1,
&v2));
    println!("Entre {} e {} o menor valor é {}", v3, v4, menorv(&v3,
&v4));
    println!("Entre {} e {} o menor valor é {}", v5, v6, menorv(&v5,
&v6));
    println!("Entre {} e {} o menor valor é {}", v7, v8, menorv(&v7,
&v8));

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, são apresentados os menores valores entre

dois valores indicados. Note que, para fazer uso da sub-rotina de função genérica, devem ser usados os pares de valores de mesmo tipo, e que o uso de `return` na sub-rotina é opcional, bem como os símbolos de ponto e vírgula junto a `{return a;}` e `{return b;}`.

Para o programa do projeto `c09ex02` está sendo implementada a biblioteca `std::cmp::PartialOrd`, necessária para que a ação executada pela instrução `if a < b {return a;} else {return b;}` possa ser validada a partir da configuração do tipo genérico da sub-rotina `menorvlr()` com `T: PartialOrd`, em que `PartialOrd` é a definição da característica de comportamento sendo estabelecida para o tipo `T`. A indicação `PartialOrd` especifica os limites operacionais da *trait* que o tipo genérico `T` irá representar e, assim, pode realizar ações de comparação sobre os valores em uso.

O tipo de dado genérico `T` é um mecanismo que requer certos tratamentos para validação de ações relacionadas a operações matemáticas e lógicas que se venham a executar.

O tipo de dado genérico `T` requer o estabelecimento de certos comportamentos operacionais para uso. Isso se faz necessário devido ao fato de o tipo `T` não ter ações de comportamento matemático e lógico associados. Sua finalidade é apenas representar genericamente um tipo de dado que venha a ser usado, e não estabelecer ações para o tipo em uso, ficando esta tarefa a cargo do programador.

No caso do projeto `c09ex02`, a ausência da biblioteca `std::cmp::PartialOrd` sem a definição de `PartialOrd` ocasionaria na compilação a apresentação da mensagem `error[E0369]: binary operation '<' cannot be applied to type 'T'` junto à instrução `if a < b {return a;} else {return b;}`, informando que o operador relacional `<` (menor que) não se aplica a ações binárias junto do tipo de dado `T`.

Para poder realizar a comparação de valores do tipo genérico `T` é necessário definir como método de operação a partir da biblioteca-padrão `std::cmp::PartialOrd` uma ação condicional para o operador relacional em uso quando se associa ao tipo genérico `T` a ação que deve ser executada como `T: PartialOrd`.

O programa seguinte apresenta o maior valor de uma lista de valores determinada em uma matriz interna (adaptado de <https://github.com/rust-lang/book/blob/master/second-edition/src/ch10-02-traits.md>) que precisará da indicação de uma ação de cópia para que o valor capturado da lista possa ser devolvido a sua chamada. Para isso, crie o projeto `c09ex04`.

```
use std::io;
use std::io::prelude::*;
use std::cmp::PartialOrd;

fn maiorvlr<T: PartialOrd + Copy>(vetor: &[T]) -> T {
    let mut maior = vetor[0];
    for &atual in vetor.iter() {
        if atual > maior {
            maior = atual;
        }
    }
    return maior;
}

fn main() {
    let numeros = vec![20, 25, 15, 98, 33, 78, 97, 10, 77, 45];
    let letras = vec!['F', 'A', 'X', 'T', 'W', 'C', 'G', 'H', 'U',
    'B'];

    println!("Maior valor numérico ...: {}", maiorvlr(&numeros));
    println!("Maior letra .....: {}", maiorvlr(&letras));

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, são apresentados os maiores valores da lista numérica e da lista alfabética.

Nesta versão do programa está em uso a característica operacional `Copy`,

concatenada à característica operacional `PartialOrd`, já comentada. A característica `Copy` se faz necessária porque não se pode sair de uma matriz genérica `[T]` sem que se tenha uma cópia do elemento apontado. Dados de matrizes com tipos conhecidos são alocados na pilha de memória, por se saber de antemão a quantidade de *bits* que cada dado ocupa. Neste caso, a cópia é efetuada automaticamente. Isso não ocorre com dados do tipo `T` em uma matriz, pois é preciso executar a ação de cópia manualmente.

9.1.2 Estruturas

Assim como as funções, as estruturas e as implementações também podem ser definidas de forma genérica na linguagem Rust. As estruturas genéricas, conhecidas como *containers*, podem ser usadas num grande conjunto de tipos de dados primitivos ou derivados. Uma estrutura genérica pode ser usada para definir diversas coleções específicas, sendo esta uma maneira de utilizar o recurso de reaproveitamento de código.

Para demonstrar o uso simples de estrutura genérica, considere um *container* que manipula um campo membro denominado `DADO` que deverá ser capaz de armazenar um dado de qualquer tipo fornecido no programa ao estilo de função membro construtor e apresentar os dados eventualmente informados. Para isso, crie o projeto `c09ex05`.

```
use std::io;
use std::io::prelude::*;

struct ClsGenerica<T> {
    dado: T
}

impl <T> ClsGenerica<T> {
    fn pegavalor(&self) -> &T {
        return &self.dado;
    }
}

fn main() {
```

```

let x1 = ClsGenerica {dado: 5u32};
let x2 = ClsGenerica {dado: 5.2f64};

println!("{}", x1.pegavalor());
println!("{}", x2.pegavalor());

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, os valores de tipos diferentes `u32` e `f64` definidos são apresentados como membros do mesmo campo (`DADO`) de uma mesma estrutura (`ClsGenerica`).

As instruções `let x1 = ClsGenerica {dado: 5u32};` e `let x2 = ClsGenerica {dado: 5.2f64};` são usadas para criar uma instância inicializada da estrutura `ClsGenerica` para os objetos (variáveis) `x1` e `x2`, da mesma maneira usada para inicializar um objeto por meio do método (função membro) construtor de uma classe.

A definição da estrutura genérica é feita com o uso do tipo `T` dentro de `<>` junto ao nome da estrutura no trecho da instrução `struct ClsGenerica<T>`. Na sequência é definida a implementação da estrutura com sua sub-rotina de acesso `pegavalor()` com o trecho `impl <T> ClsGenerica<T>`, em que se coloca, após o comando `impl`, antes do nome da estrutura, a definição de tipo `<T>`, para informar que a implementação da função membro `pegavalor()` junto à estrutura `ClsGenerica` se caracteriza por ser semelhante à definição de uma função membro estática.

Foi comentado anteriormente que o uso da letra `T` para representar um tipo de dado genérico é opcional. Com o objetivo de demonstrar essa possibilidade, considere um programa que aceite a entrada de um par de valores genéricos, sendo o primeiro valor a chave de identificação do par de dados e o segundo valor o conteúdo do par propriamente dito, utilizando-se como identificadores as palavras `CHAVE` e `VALOR`. Para isso, crie o projeto `c09ex06`.

```

use std::io;
use std::io::prelude::*;

struct Par<CHAVE, VALOR> {
    chv: CHAVE,
    vlr: VALOR
}

impl <CHAVE, VALOR> Par<CHAVE, VALOR> {
    fn pegachave(&self) -> &CHAVE {
        return &self.chv;
    }

    fn pegavalor(&self) -> &VALOR {
        return &self.vlr;
    }

    fn poechange(&mut self, chv: CHAVE) {
        self.chv = chv;
    }

    fn poevalor(&mut self, vlr: VALOR) {
        self.vlr = vlr;
    }
}

fn main() {
    let mut x1 = Par {chv: 0u8, vlr: "".to_string()};
    let mut x2 = Par {chv: 0u8, vlr: "".to_string()};
    let mut x3 = Par {chv: 0u8, vlr: 0};
    let mut x4 = Par {chv: 0u8, vlr: 0.0};

    let mut nome;

    nome = String::new();
    print!("Informe o 1o. nome: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut nome).expect("Entrada incorreta");

    if let Some('\n') = nome.chars().next_back() {

```

```

        nome.pop();
    }
    if let Some('\r') = nome.chars().next_back() {
        nome.pop();
    }
    x1.poechave(1);
    x1.poevalor(nome);

    nome = String::new();
    print!("Informe o 2o. nome: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut nome).expect("Entrada incorreta");

    if let Some('\n') = nome.chars().next_back() {
        nome.pop();
    }
    if let Some('\r') = nome.chars().next_back() {
        nome.pop();
    }

    x2.poechave(2);
    x2.poevalor(nome);

    x3.poechave(3);
    x3.poevalor(10u8);

    x4.poechave(4);
    x4.poevalor(9.99f64);

    println!();
    println!("Chave ... {} \n Nome ...: {}", x1.pegachave(),
x1.pegavalor());
    println!();
    println!("Chave ... {} \n Nome ...: {}", x2.pegachave(),
x2.pegavalor());
    println!();
    println!("Chave ... {} \n Valor ...: {}", x3.pegachave(),
x3.pegavalor());
    println!();

```

```

        println!("Chave ...: {} \n Valor ...: {}", x4.pegachave(),
x4.pegavalor());

        println!();
        print!("Tecle <Enter> para encerrar...");
        io::stdout().flush().unwrap();
        io::stdin().read(&mut [0u8]).unwrap();
    }
}

```

Ao ser executado o programa, informe os nomes solicitados e observe na apresentação a indicação dos nomes fornecidos e de dois valores, demonstrando o uso de uma estrutura genérica que aceite a entrada de dados de qualquer tipo.

Atente para a definição do trecho `struct Par<CHAVE, VALOR>`, que demonstra a utilização de tipos genéricos nomeados como `CHAVE` e `VALOR`.

9.2 Programação funcional

O paradigma de programação funcional opera as ações de processamento matemático e lógico de modo diferente do modelo imperativo usado pelos paradigmas estruturado e orientado a objetos, pois no paradigma funcional os programas são baseados em estruturas de função de modo semelhante ao do conceito de funções na ciência matemática. Assim, a função $f(x) = x + 1$ (que pode ser referenciada como $y = x + 1$) determina que, dado um parâmetro (argumento) x em $f(x)$, há sempre o retorno do valor sucessor de x , sendo este $x + 1$. Se fornecido para a função o parâmetro 5, o retorno será 6, pois $f(5) = 6$.

Uma função, do ponto de vista matemático, é uma forma de estabelecer uma relação entre dois conjuntos, sendo um o conjunto domínio (conjunto de saída), neste caso, $f(x)$, e o outro o conjunto contradomínio (conjunto de chegada), como $x + 1$. O resultado obtido a partir de uma função é a imagem da função, ou seja, para o domínio $f(x)$ e o contradomínio $x + 1$, a imagem será 6 se dado $f(5)$ e se no conjunto contradomínio forem encontrados os valores gerados pela função.

Na linguagem Rust é possível fazer uso do paradigma funcional de forma

isolada ou utilizá-lo em conjunto com os demais paradigmas suportados pela linguagem, o que torna o trabalho da programação bastante amplo. Para fazer uso do paradigma funcional é necessário levar em consideração cinco detalhes importantes a serem definidos em uma sub-rotina de função:

- Deve ter no mínimo um parâmetro (argumento) para a entrada do valor na função;
- Deve sempre devolver um valor como resposta de sua operação;
- Deve ter transparência referencial, ou seja, produzir o mesmo resultado quando usados os mesmos parâmetros;
- Deve fazer sua operação de processamento sem o uso de laços;
- Não deve acumular valores nas posições de memória utilizadas, daí o uso de variáveis imutáveis.

Uma sub-rotina que não atenda ao critério de retorno de valor não pode ser chamada de sub-rotina de função, apesar de isso ser muito comum entre programadores das linguagens C, C++, Java e C#, pois na prática esta é uma sub-rotina de procedimento. Considerando-se então uma sub-rotina do tipo função, isso leva a dois tipos de sub-rotinas: as comuns e as puras.

As sub-rotinas de função comum são, por exemplo, sub-rotinas que podem ou não receber algum parâmetro, que retornam um resultado e não atendem aos critérios de funcionalidade indicados. Já as sub-rotinas de função puras atendem plenamente aos critérios da programação funcional, tendo transparência referencial e sendo livres de efeito colateral.

Como exemplo simples e introdutório de aplicação de sub-rotina de função, considere a necessidade de efetuar o cálculo e retornar o resultado da fatorial de um número natural qualquer a partir da função $f(x) = x!$. O cálculo de uma fatorial só pode ser executado com valores naturais (inteiros e positivos).

A rotina aqui proposta deverá receber como parâmetro valores de 8 bits sinalizados e retornar como resposta um valor de 64 bits sinalizado. A

tentativa de calcular a fatorial de um valor negativo ou de um valor acima de 20 não é permitida.

Valores negativos como parâmetro de fatorial são matematicamente considerados erros, e o valor máximo 20 decorre da capacidade de ser o maior valor possível para a resposta de um resultado dentro da faixa de 64 bits. Neste casos, a rotina retorna zero como resposta.

O trecho de código seguinte demonstra o uso de uma sub-rotina de função `fatorial()` no estilo comum sem a preocupação de estar usando algum critério do paradigma da programação funcional.

```
fn fatorial(x: i8) -> i64 {  
    let mut f: i64 = 1;  
    if x == 0 || x == 1 {  
        return 1;  
    }  
    if x < 0 || x > 20 {  
        return 0;  
    } else {  
        for i in 1 .. x + 1 {  
            f *= i as i64;  
        }  
        return f;  
    }  
}
```

O código da função `fatorial()` indicado não pertence ao paradigma funcional porque utiliza um laço, o que exige o acúmulo de valores na posição de memória da variável `f`, além da definição da variável mutável `f`. Note que, dos cinco detalhes operacionais indicados ao uso do paradigma de programação funcional, a função `fatorial()` utiliza apenas três.

O trecho de código seguinte demonstra o uso de uma sub-rotina de função `fatorial()` no estilo puro, por isso, é uma sub-rotina de função pertencente ao paradigma da programação funcional.

```
fn fatorial(x: i8) -> i64 {  
    if x == 0 || x == 1 {  
        return 1;  
    }
```

```

    }
    if x < 0 || x > 20 {
        return 0;
    } else {
        return fatorial(x - 1) * x as i64;
    }
}

```

O código da função `fatorial()` indicado pertence ao paradigma funcional porque utiliza apenas a variável imutável `x` (toda variável em Rust sem o qualificador `mut` é, por padrão, uma variável imutável). Não usa internamente nenhuma outra variável para auxiliar o processamento do cálculo (não gera efeitos colaterais) e não faz uso de laço para o processamento. Observe que, nesta versão, os cinco detalhes apontados são plenamente consistentes com o código da sub-rotina de função.

A partir da visão básica e conceitual do paradigma da programação funcional, o próximo programa demonstra o uso das sub-rotinas de função comum e pura para a apresentação dos valores de algumas fatoriais. Para isso, crie o projeto `c09ex07`.

```

use std::io;
use std::io::prelude::*;

fn fatorialcomum(x: i8) -> i64 {
    let mut f: i64 = 1;
    if x == 0 || x == 1 {
        return 1;
    }
    if x < 0 || x > 20 {
        return 0;
    } else {
        for i in 1 .. x + 1 {
            f *= i as i64;
        }
        return f;
    }
}

```

```

fn fatorialpura(x: i8) -> i64 {
    if x == 0 || x == 1 {
        return 1;
    }
    if x < 0 || x > 20 {
        return 0;
    } else {
        return fatorialpura(x - 1) * x as i64;
    }
}

fn main() {

    // Uso da sub-rotina de função comum
    println!("Resultados da função comum\n");
    println!("-1! = {}", fatorialcomum(-1));
    println!("0! = {}", fatorialcomum(0));
    println!("1! = {}", fatorialcomum(1));
    println!("5! = {}", fatorialcomum(5));
    println!("20! = {}", fatorialcomum(20));
    println!("21! = {}", fatorialcomum(21));
    println!();

    // Uso da sub-rotina de função pura

    println!("Resultados da função pura\n");
    println!("-1! = {}", fatorialpura(-1));
    println!("0! = {}", fatorialpura(0));
    println!("1! = {}", fatorialpura(1));
    println!("5! = {}", fatorialpura(5));
    println!("20! = {}", fatorialpura(20));
    println!("21! = {}", fatorialpura(21));

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, são apresentados dois blocos com o mesmo conjunto de resultados obtidos a partir de dois estilos de programação diferentes (no que tange ao código das sub-rotinas de funções). O primeiro bloco mostra os resultados calculados a partir de uma função comum seguindo o paradigma imperativo (foco algorítmico), e o segundo bloco mostra os resultados calculados a partir de uma função pura seguindo o paradigma de programação funcional (foco na funcionalidade).

A partir da visão do que é uma função comum e uma função pura, cabe ressaltar os usos que se pode fazer das funções puras: como funções de primeira classe, funções de ordem superior e funções lambda. No entanto é pertinente salientar que não importa, para o uso, se uma função é comum ou pura, pois, a partir do momento que uma função é definida na linguagem Rust, esta poderá ser efetivamente usada. Mas é óbvio que, para produzir programas dentro do paradigma funcional, todo o conjunto de funções operacionalizadas e as demais ações devem também ser produzidos no mesmo paradigma.

As funções de ordem superior são sub-rotinas de função que podem receber como parâmetro outras funções e retornar como resposta um valor advindo de uma função. Por exemplo, dadas as funções $f(x) = x + 3$ e $g(x) = x * 2$, calcule a função $h(x) = f(g(x))$, em que será executada, primeiro, a função $g(x)$, e seu retorno será usado por $f(x)$, que efetuará o cálculo da função $h(x)$. Assim, a função $h(x)$ efetua como se a ela fosse definida a expressão $(x * 2) + 3$. Dessa forma, se dado o valor 5 para $h(x)$, o resultado obtido é 13 (ÁLVARES, 2017, p. 6).

O programa seguinte apresenta o resultado calculado a partir do uso de função de ordem superior com base no exemplo $h(x) = f(g(x))$, em que as funções $f(x)$ e $g(x)$ são respectivamente $f(x) = x + 3$ e $g(x) = x * 2$. Para isso, crie o projeto `c09ex08`.

```
use std::io;
use std::io::prelude::*;

fn f(x: i32) -> i32 {
```

```

        return x + 3;
    }

    fn g(x: i32) -> i32 {
        return x * 2;
    }

    fn h(x: i32) -> i32 {
        return f(g(x));
    }

    fn main() {
        println!("{}", h(5));

        println!();
        print!("Tecle <Enter> para encerrar...");
        io::stdout().flush().unwrap();
        io::stdin().read(&mut [0u8]).unwrap();
    }

```

Ao ser executado o programa, o valor 13 é apresentado para o parâmetro 5 de `h(5)` devido à multiplicação por 2 do valor 5 e à adição de 3 a esse valor. A partir de uma visão básica do uso de uma função como parâmetro de outra função, sendo esta uma ação de uso de função de ordem superior, considere um programa que apresente o resultado do somatório dos valores pares entre 0 e 100 obtidos a partir de uma ação imperativa e funcional com, obrigatoriamente, o passo de iteração definido como 1. Para isso, crie o projeto `c09ex09`.

```

use std::io;
use std::io::prelude::*;

fn checar(x: u32) -> bool {
    return x % 2 == 0;
}

fn main() {
    println!("Somatório dos valores pares entre 0 e 100\n");
}

```

```

// Forma interativa

let mut soma: u32 = 0;
for i in 0 .. 100 {
    if checapar(i) {
        soma += i;
    }
}
println!("Soma imperativa ....: {}", soma);

// Forma funcional

let soma: u32 = (0 ..)
    .map(|n| n)
    .take_while(|&n| n < 100)
    .filter(|&n| checapar(n))
    .fold(0, |s, i| s + i);
println!("Soma funcional .....: {}", soma);
println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa serão apresentados os valores 2450 como resposta às ações interativa e funcional definidas.

O primeiro detalhe a ser observado é a definição das variáveis *SOMA*, que no trecho iterativo requer a qualificação como *mut* junto ao comando *let* para poder ser executada, e no trecho funcional requer apenas o uso do comando *let*.

No trecho iterativo é necessária a definição de um laço para obter o resultado do somatório. Já na forma funcional bastou uma única instrução separada em cinco linhas para melhor distribuição visual, sendo definida a partir de `let soma: u32 = (0 ..).map(|n|n).take_while(|&n| n < 100).filter(|&n| checapar(n)).fold(0, |s, i| s + i);`, em que a sub-rotina de função *checapar()* é definida dentro do método *filter()* com o objetivo de validar o valor de *N*.

Após a definição da variável `SOMA` como `let soma: 32` são estabelecidos para esta variável a faixa de operação de `0` até `100`, em que o valor `0` é definido em `(0 ..)` e o valor final `100` é especificado no método `take_while()`, em que cada referência de `&N` é verificada para definir se abrange até `100` por meio da condição `N < 100`.

O método `fold()` é um recurso operacional que faz a iteração em uma faixa estabelecida de valores utilizando-se de dois parâmetros: o primeiro parâmetro representa o valor de base (valor inicial de uma iteração) e o segundo parâmetro é formado por um par estabelecido entre os símbolos de *pipe*, o qual estabelece que o primeiro valor do par é o acumulador de uma operação e o segundo, é o elemento iterado a ser acumulado na operação indicada à direita, fora dos símbolos de *pipe*; neste caso, como o somatório da variável interna `s` junto à variável interna `i` até o limite `100` especificado com o método `take_while()`. O método `fold()` está executando de forma funcional a mesma ação executada pelo comando `for` quando do uso do conceito de iteratividade.

O método `map()` define todo o conjunto de valores a ser usado informando que para todo o `N` (do trecho `|n|`) em uso considere o `N` (fora dos *pipes*) validado com `checapar()`.

De forma geral, observe que o programa demonstra em paralelo o uso de duas técnicas diferentes de programação que geram os mesmos resultados. No que tange ao modelo da programação funcional, a ação foi resolvida em apenas uma linha de instrução, utilizando-se uma série de métodos interligados para a efetivação do processamento, tais como `map()`, `take_while()`, `filter()` e `fold()`.

O método `map()` é usado para aplicar certa ação a cada valor de toda a coleção de valores. Neste caso, dos valores de `0` até o valor limitado pelo método `take_while(|&n| n < 100)`. Dessa forma, `talk()` é operacionalizado na faixa de `0` até `100`.

O método `take_while()` efetuará um ciclo de iterações enquanto a condição estabelecida for verdadeira. A variável `N` usada junto do método `take_while()` é uma variável interna para uso do próprio método,

auxiliando-o nas ações de interação executadas.

O método `filter()` é, de certa maneira, semelhante ao recurso `map()`, tendo como diferença o fato de retornar elementos que atendem a determinados critérios. Neste caso, `filter()` é limitado pela ação da sub-rotina de função `checapar()`.

O método `foldr()` é usado para aplicar uma função cujo objetivo é acumular todos os valores de cada iteração em um único valor. Neste caso, de 0 até o limite máximo definido junto a `filter()` a partir de `take_while()`.

A programação funcional é um tema extenso e digno de uma obra em separado. Neste tópico é feita uma breve introdução; há muito mais detalhes que devem ser estudados além do que foi apresentado aqui. O tema não foi aprofundado nesta obra porque foge de seu escopo introdutório.

9.3 Tratamento de erros e exceções

A linguagem Rust não tem efetivamente um comando especial de tratamento de exceção ao estilo `try ... catch` e nem algo parecido para a manipulação de erros, mas oferece recursos que podem ser utilizados para esse tipo de ação. O tratamento de exceções ou manipulação de erros é uma forma organizada de fazer o gerenciamento de ocorrências indesejadas que podem surgir em um programa por diversas razões.

Na linguagem Rust o tratamento de erros não é feito na forma de tratamento de exceções, mas na forma de retorno de valores com as enumerações `Option` e `Result`. É pertinente salientar que o gerenciamento de erros é executado ou na forma de tratamento de erros ou na forma de retorno de valor.

Outro detalhe importante é saber a diferença entre *erro* e *exceção*: *erro* é uma situação inesperada e incomum que pode ocorrer em um programa e pode ser resolvida corrigindo-se o código do programa, já a *exceção* é uma situação esperada e irregular que pode ocorrer em tempo de

execução de um programa (CAPUANO, 2016).

A enumeração `Option` da biblioteca `std::option` opera internamente com dois estados de comportamento: `Some(T)`, que envolve algum valor de um tipo genérico `T` quando a ação é válida para valor verificado, e `None`, que indica a ocorrência de alguma falha para o valor verificado, tendo como sintaxe geral:

```
Option<T> {  
    Some(T),  
    None,  
}
```

O programa seguinte demonstra uma aplicação que solicita a entrada de dois valores reais de 32 bits e mostra o resultado da divisão do primeiro valor pelo segundo valor. Nesta versão do programa não é feito nenhum tipo de controle sobre a entrada de valores para o divisor, tanto que, se for inserido o valor 0 (zero), o resultado será apresentado como `inf` (de infinito), mas poderia ser tradicionalmente erro. Para isso, crie o projeto `c09ex10`.

```
use std::io;  
use std::io::prelude::*;  
fn main() {  
    let mut dividendo = String::new();  
    let mut divisor = String::new();  
  
    let dvd: f32;  
    let dvr: f32;  
    let quo: f32;  
  
    print!("Entre o valor do dividendo ...: ");  
    io::stdout().flush().unwrap();  
    io::stdin().read_line(&mut dividendo).unwrap();  
    dvd = dividendo.trim().parse:::<f32>().unwrap();  
  
    print!("Entre o valor do divisor .....: ");  
    io::stdout().flush().unwrap();  
    io::stdin().read_line(&mut divisor).unwrap();
```

```

    dvr = divisor.trim().parse::<f32>().unwrap();

    quo = dvd / dvr;

    println!();
    println!("Quociente = {}", quo);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, forneça para o dividendo um valor diferente de zero e forneça para o divisor um valor zero e veja o resultado `inf`. O resultado `inf` (infinite) é o resultado de um valor qualquer dividido por zero. Tradicionalmente, calculadoras apresentam de forma incorreta esta operação como erro.

Depois, teste a entrada de valores zero para o dividendo e para o divisor e veja o resultado apresentado indicando `NaN` (Not a Number), que mostra que a resposta não é um número, ou seja, um erro. A apresentação das respostas `inf` e `NaN` é realizada em Rust e não oferece, dessa forma, nenhuma ocorrência de erro que interrompa a execução do programa.

No sentido de tratar as duas ocorrências de “erro”, o programa seguinte faz uso do recurso `Option`. Para isso, crie o projeto `c09ex11`.

```

use std::io;
use std::io::prelude::*;

fn divisao(dividendo: f32, divisor: f32) -> Option<f32> {
    if dividendo == 0. || divisor == 0. {
        return None;
    } else {
        return Some(dividendo / divisor);
    }
}

fn main() {

```

```

let mut dividendo = String::new();
let mut divisor = String::new();

let dvd: f32;
let dvr: f32;

print!("Entre o valor do dividendo ...: ");
io::stdout().flush().unwrap();
io::stdin().read_line(&mut dividendo).unwrap();
dvd = dividendo.trim().parse::<f32>().unwrap();

print!("Entre o valor do divisor .....: ");
io::stdout().flush().unwrap();
io::stdin().read_line(&mut divisor).unwrap();
dvr = divisor.trim().parse::<f32>().unwrap();

match divisao(dvd, dvr) {
    Some(quo) => {
        println!("\nQuociente = {}", quo)
    },
    None => println!("\nErro na operação"),
}

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Execute o programa, refaça os testes de entrada dos valores e observe que agora não são mais apresentados os valores `inf` e `NaN`, mas sim a mensagem `Erro na operação`.

Nesta versão está sendo definida a função `divisão()`, que implementa (sobrepõe) sua ação sobre a enumeração `Option`, definida como `f32` por ser um recurso genérico. Dentro dos limites do bloco da função `divisao()`, o trecho de código da instrução `if dividendo == 0. || divisor == 0.` verifica a ocorrência de validade ou não da ação se os valores dos parâmetros `DIVIDENDO` ou `DIVISOR` forem iguais a zero. Se a condição for herdeira, o

elemento `None` será acionado; caso contrário, o elemento `Some()` é executado.

Há para a enumeração `Option` uma série de métodos que podem ser usados, os quais se encontram documentados no manual online da linguagem em <https://doc.rust-lang.org/std/option/enum.Option.html>.

A enumeração `Option` cria a estrutura de verificação como se fosse um tratamento de erros que é efetuado com o comando `match`, em que `Some()` faz o papel semelhante a um `try` e `None` faz um papel semelhante a `catch`. No entanto há a enumeração `Result`, que tem um estilo de comportamento bem mais próximo a uma ação de `try ... catch` do que a enumeração `Option`.

A enumeração `Result` da biblioteca `std::result` opera internamente com dois estados de comportamento: `Ok(T)`, que envolve algum valor de um tipo genérico `T` quando a ação é válida para valor verificado, e `Err(E)`, que indica a ocorrência de alguma falha para o valor genérico `E`, tendo como sintaxe geral:

```
Result<T> {  
    Ok(T),  
    Err(E),  
}
```

O funcionamento de `Result` é semelhante ao funcionamento de `Option` com a substituição de `Some(T)` por `Ok(T)` e de `None` por `Err(E)`. A funcionalidade `Ok()` opera como a funcionalidade `Some()` ao capturar o valor genérico de uma ação válida. A funcionalidade `Err()`, diferentemente de `None`, que é tratado posteriormente, aceita a definição de uma *string* com uma mensagem de erro incluída.

O programa seguinte trabalha com os mesmos critérios do programa do projeto `c09ex11`. Para isso, crie o projeto `c09ex12`.

```
use std::io;  
use std::io::prelude::*;  
  
fn divisao(dividendo: f32, divisor: f32) -> Result<f32, String> {  
    if dividendo == 0. || divisor == 0. {
```

```

        return Err("\nErro na operação".to_string());
    } else {
        return Ok(dividendo / divisor);
    }
}

fn main() {
    let mut dividendo = String::new();
    let mut divisor = String::new();

    let dvd: f32;
    let dvr: f32;

    print!("Entre o valor do dividendo ...: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut dividendo).unwrap();
    dvd = dividendo.trim().parse::<f32>().unwrap();

    print!("Entre o valor do divisor .....: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut divisor).unwrap();
    dvr = divisor.trim().parse::<f32>().unwrap();

    match divisao(dvd, dvr) {
        Ok(quo) => println!("\nQuociente = {}", quo),
        Err(erro) => println!("{}", erro),
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Execute o programa, refaça os testes de entrada dos valores e observe as saídas apresentadas.

O programa do projeto c09ex12, a partir do uso da enumeração `Result`, proporciona uma dinâmica diferente de detecção de erro. O trecho

definido para o `Result<>` da sub-rotina `divisao()` funciona de forma semelhante ao comando `try`, detectando a ocorrência de erro, e o comando `match`, ao fazer o tratamento da ação, executa um procedimento semelhante ao do comando `catch`.

A sub-rotina `divisao()` recebe a passagem de dois parâmetros do tipo real de 32 bits (`f32`) e direciona sua saída como sendo do tipo `Result<f32, String>`, em que os argumentos `f32` e `String` especificam respectivamente os valores genéricos `T` (para a ação quando válida) e `E` (para a ação quando inválida). Note que o bloco da sub-rotina `divisao()` tem a indicação de uso do método `Ok()`, que efetua o cálculo indicado e devolve o resultado para o argumento `f32` de `Result`, e o método `Err()` retorna a mensagem `Erro` na operação.

Após a validação da ação da sub-rotina `divisao()` junto à instrução `match divisao(dvd, dvr) {}` ocorre a ação do método `Ok(quo)`, em que a variável `QUO` recebe o valor da divisão quando esta ação é válida, ou a ação do método `Err(erro)`, em que a variável `ERRO` apresenta a mensagem de erro.

Há para a enumeração `Result` uma série de métodos que podem ser usados, os quais se encontram documentados no manual online da linguagem em <https://doc.rust-lang.org/std/result/enum.Result.html>.

9.4 Data e hora

A linguagem Rust tem como recurso adicional para o uso de data e hora o módulo `chrono`. Este módulo atende à norma `ISO 8601`, e para ser usado é necessário, como de costume, estar configurado junto ao arquivo `Cargo.toml` a partir da seção `dependencies` com o valor `chrono = "0.4"`. Para mais detalhes sobre o módulo `Chrono`, sugere-se consultar o site <https://docs.rs/chrono/0.4.0/chrono/>.

Com o objetivo de demonstrar um programa que opere com informações de calendário, apresentando a hora e a data do sistema, crie o projeto `c09ex13`.

Abra o arquivo de programa `Cargo.tom` e proceda com a adição da linha

em negrito indicada a seguir:

```
[package]
name = "c09ex13"
version = "0.1.0"
authors = ["Fulano"]
```

```
[dependencies]
chrono = "0.4"
```

Na sequência é necessário, na janela do terminal, após gravar a alteração do arquivo `Cargo.toml`, fazer a construção do recurso. Para isso, escreva no terminal a instrução

```
cargo build
```

Após a construção e configuração do módulo e a obtenção dos arquivos de suporte necessários, abra o arquivo de programa `main.rs` da pasta/diretório `src` em estudo e informe o código de programa:

```
extern crate chrono;

use chrono::prelude::*;
use std::io;
use std::io::prelude::*;

fn main() {
    let horautc: DateTime<Utc> = Utc::now();
    let horalocal: DateTime<Local> = Local::now();

    println!("{}", horautc);
    println!("{}", horalocal);
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

O programa irá apresentar as informações

```
2017-10-18 21:45:27.333406400 UTC
2017-10-18 19:45:27.333406400 -02:00
```


para a hora UTC e a hora Local.

A hora UTC (Universal Coordinated Time) também é conhecida como hora mundial ou hora média de Greenwich (GMT – Greenwich Mean Time); e a hora local mostra o fuso horário do momento e sua relação com a hora UTC.

As variáveis `HORAUTC` e `HORALOCAL` são estabelecidas a partir da data e da hora correntes, detectadas respectivamente com `Utc::now()` e `Local::now()` por meio do tipo genérico `DateTime` ajustado com os formatos `<Utc>` ou `<Local>`.

O programa seguinte apresenta a data e a hora de forma separada em elementos distintos. Para isso, crie o projeto `c09ex14`.

Abra o arquivo de programa `Cargo.tom` e proceda com a adição da linha em negrito indicada a seguir:

```
[package]
name = "c09ex14"
version = "0.1.0"
authors = ["Fulano"]
```

```
[dependencies]
chrono = "0.4"
```

Na sequência é preciso, na janela do terminal, após gravar a alteração do arquivo `Cargo.toml`, fazer a construção do recurso. Para isso, escreva no terminal a instrução

```
cargo build
```

Após a construção do módulo e a obtenção dos arquivos de suporte, abra o arquivo de programa `main.rs` da pasta/diretório `src` em `estudo` e informe o código de programa:

```
extern crate chrono;

use chrono::prelude::*;
use std::io;
use std::io::prelude::*;
```

```

fn main() {
    let datahora: DateTime<Local> = Local::now();

    println!("Dia .....: {}", datahora.day());
    println!("Mês .....: {}", datahora.month());
    println!("Ano .....: {}", datahora.year());
    println!();
    println!("Hora .....: {}", datahora.hour());
    println!("Minuto .....: {}", datahora.minute());
    println!("Segundo .....: {}", datahora.second());
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, a data e a hora do sistema são apresentadas separadamente. Além dos métodos indicados `day()`, `month()`, `year()`, `hour()`, `minute()` e `second()` para a extração dos dados da data e da hora do sistema, é possível obter outras informações como `weekday().number_from_monday()` para mostrar o número do dia da semana (1 – segunda-feira; 2 – terça-feira; 3 – quarta-feira; 4 – quinta-feira; 5 – sexta-feira; 6 – sábado; 7 – domingo), `ordinal()` para mostrar o número de dias corridos no ano a partir de primeiro de janeiro, `num_days_from_ce()` para apresentar o número de dias corridos a partir de 1º de janeiro do primeiro ano do calendário cristão, entre outros.

O programa a seguir apresenta a data e a hora com minutos no formato do idioma português. Para isso, crie o projeto `c09ex15` com as seguintes instruções:

Abra o arquivo de programa `Cargo.tom` e proceda com a adição da linha em negrito indicada a seguir.

```

[package]
name = "c09ex15"
version = "0.1.0"
authors = ["Fulano"]

```

```
[dependencies]
chrono = "0.4"
```

Na sequência é preciso, na janela do terminal, após gravar a alteração do arquivo `Cargo.toml`, fazer a construção do recurso. Para isso, escreva no terminal a instrução

```
cargo build
```

Após a construção do módulo e a obtenção dos arquivos de suporte, abra o arquivo de programa `main.rs` da pasta/diretório `src` em estudo e informe o código de programa:

```
extern crate chrono;

use std::io;
use std::io::prelude::*;
use chrono::Local;

fn main() {
    let data = Local::now();

    println!("{}", data.format("Data: %d/%m/%Y | Hora: %Hh%Mmin"));

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado, o programa mostra a data e as horas formatadas por meio do método `format()`, que opera a partir de códigos de formatação para a representação dos elementos de data e hora. Os códigos de formatação para ação da função `format()` são apresentados na Tabela 9.1.

Tabela 9.1 – Códigos de formatação para a função `format()`

Código	Função
%a	Permite apresentar o dia da semana abreviado (em inglês).
%A	Permite apresentar o dia da semana por extenso (em inglês).
%b	Permite apresentar o mês abreviado (em inglês).

Código	Função
%B	Permite apresentar o mês por extenso (em inglês).
%c	Permite apresentar data e hora.
%d	Permite apresentar o dia do mês.
%H	Permite apresentar hora no formato 24 (de 0 a 23).
%I	Permite apresentar hora no formato 12 (de 0 a 11).
%j	Permite apresentar o dia do ano.
%m	Permite apresentar o mês no formato numérico.
%M	Permite apresentar o minuto da hora.
%p	Permite apresentar o indicador AM ou PM.
%S	Permite apresentar o segundo da hora.
%U	Permite apresentar o primeiro dia da semana (de 0 a 53) como domingo.
%w	Permite apresentar o dia da semana (de 0 a 6).
%W	Permite apresentar o primeiro dia da semana (de 0 a 53) como segunda-feira.
%x	Permite apresentar data completa.
%X	Permite apresentar hora completa.
%y	Permite apresentar ano com dois dígitos.
%Y	Permite apresentar ano com quatro dígitos.
%Z	Permite apresentar o fuso horário.

O programa seguinte faz uso dos recursos apresentados anteriormente neste tópico a partir da definição manual de uma data e uma hora específicas, como **26 de abril de 1965 | 17h35min**, mostrando a data, a hora e o nome do dia da semana em inglês e português. Para isso, crie o projeto **c09ex16**.

Abra o arquivo de programa **Cargo.tom** e proceda com a adição da linha em negrito indicada a seguir:

```
[package]
name = "c09ex16"
version = "0.1.0"
authors = ["Fulano"]
```

```
[dependencies]
chrono = "0.4"
```

Na sequência é preciso, na janela do terminal, após gravar a alteração do arquivo `Cargo.toml`, fazer a construção do recurso. Para isso, escreva no terminal a instrução

```
cargo build
```

Após a construção do módulo e a obtenção dos arquivos de suporte, abra o arquivo de programa `main.rs` da pasta/diretório `src` em estudo e informe o código de programa:

```
extern crate chrono;

use std::io;
use std::io::prelude::*;
use chrono::prelude::*;

fn main() {
    let datahora = Utc.ymd(1965, 4, 25).and_hms(17, 35, 0);

    println!("Dia .....: {}", datahora.day());
    println!("Mês .....: {}", datahora.month());
    println!("Ano .....: {}", datahora.year());
    println!();
    println!("Hora .....: {}", datahora.hour());
    println!("Minuto .....: {}", datahora.minute());
    println!("Segundo .....: {}", datahora.second());
    println!();
    println!("{}", data.format("Data: %d/%m/%Y | Hora: %Hh%Mmin"));
    println!();
    println!("Dia da semana (inglês) .....: {}",
datahora.format("%A"));
    print!("Dia da semana (português) .....: ");

    match datahora.weekday().number_from_monday() {
        1 => println!("Segunda-feira"),
        2 => println!("Terça-feira"),
        3 => println!("Quarta-feira"),
        4 => println!("Quinta-feira"),
```

```

        5 => println!("Sexta-feira"),
        6 => println!("Sábado"),
        7 => println!("Domingo"),
        _ => {}
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, os dados da data e da hora especificados são apresentados a partir da definição da data e da hora com a instrução `let datahora = Utc.ymd(1965, 4, 25).and_hms(17, 35, 0);`. Para especificar certa data e determinada hora está sendo usado o método `ymd()` para a data e seu complemento `and_hms()` para a hora associados ao formato `Utc`.

Além do módulo adicional de data e hora `chrono`, a linguagem Rust tem internamente a biblioteca `std::time` com a finalidade de operacionalizar operações relacionadas a cálculos centrados no controle de tempo que podem usados por um computador.

Para usar o recurso timer, o programa seguinte apresenta uma mensagem no monitor de vídeo a cada segundo de ciclo do relógio do computador por um tempo predefinido; neste caso, cinco segundos. Para isso, crie o projeto `c09ex17`.

Abra o arquivo de programa `Cargo.tom` e proceda com a adição da linha em negrito indicada a seguir:

```

use std::io;
use std::io::prelude::*;

use std::time::{Duration, SystemTime};
use std::thread::sleep;

fn main() {
    let iniciocrono = SystemTime::now();
    let mut i: u8 = 1;

```

```

loop {
    sleep(Duration::new(1, 0));
    match iniciocrono.elapsed() {
        Ok(elapsed) => {
            println!("Mensagem no segundo: {}", elapsed.as_secs());
        }
        Err(erro) => {
            println!("Erro na operação: {:?}", erro);
        }
    }
    i += 1;
    if i > 5 { break; }
}

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

A ser executado o programa, este mostra por cinco vezes a cada segundo uma mensagem ilustrativa. A cada apresentação o programa mostra um valor de 1 a 5 a cada segundo do tempo decorrido.

Para usar o programa, dois padrões de ação da biblioteca `std::time` estão sendo habilitados entre as chaves junto à instrução `use std::time::{Duration, SystemTime};`: `Duration` (tipo de dado de enumeração que representa um período de tempo, normalmente, usado para a definição de tempo-limite do sistema) e `SystemTime` (tipo de dado de enumeração usado para definir a medida do relógio do sistema).

Obviamente a instrução `use std::time::{Duration, SystemTime};` pode ser substituída pela instrução `use std::time::*;`. No caso da definição `{Duration, SystemTime}`, faz-se uso apenas dos recursos indicados, e com o uso do símbolo (*) asterisco faz-se a indicação de uso de todos os recursos da biblioteca, mesmo que estes não estejam em operação.

A instrução `let iniciocrono = SystemTime::now();` estabelece para a variável

INICIOCRONO a instância de início da contagem a partir do valor do timer atual indicado por `now()` por meio de `SystemTime`.

A instrução `sleep(Duration::new(1, 0));` estabelece o tempo de espera entre segundos a partir do valor atual do timer do computador. Para que esta instrução opere é necessário usar a biblioteca `std::thread::sleep`.

O tipo `Duration` é composto de um número inteiro de segundos e uma parte fracionada em nanossegundos. O método `new(segundos, nanossegundos)` define por meio de seus parâmetros para o tipo `Duration` um número especificado de segundos (valor inteiro positivo de 64 bits) e nanossegundos (valor inteiro positivo de fração de tempo de 32 bits) a partir do formato. Se o número de nanossegundos for superior a 1 bilhão (o número de nanossegundos em um segundo), ele transferirá o valor para os segundos fornecidos. Para mais detalhes sobre as funcionalidades do tipo `Duration`, visite o manual online no endereço <https://doc.rust-lang.org/std/time/struct.Duration.html>, além de outras informações em <https://doc.rust-lang.org/std/time/>.

O trecho de instrução `match iniciocrono.elapsed()` a partir do método `elapsed()` associado à variável `INICIOCRONO` permite verificar se a operação é corretamente realizada por `Ok(elapsed)`, que permite apresentar a mensagem do programa, ou, se ocorrido algum erro, executa a ação de `Err(erro)` mostrando seu valor de ocorrência.

Há outra forma de cronometrar o espaço de tempo em segundos, por meio das bibliotecas `use std::time::Duration;` e `use std::thread;` em conjunto com a instrução `thread::sleep(Duration::from_secs(N));`, em que `N` é o tempo definido em segundos.

O tratamento de dados relacionados ao uso de datas, horas e cronometragem, mesmo em nível básico e introdutório, não estaria completo se não fosse levado em conta o uso das questões de fuso horário com o recurso `TimeZone` do módulo `Chrono-TZ`. Para usar o módulo de fuso horário é necessário configurar o arquivo `Cargo.toml` a partir da seção `dependencies` com o valor `chrono = "0.4"`, mais o valor `chrono-tz = "0.4"`. Para mais detalhes sobre recursos do módulo `Chrono-TZ`, sugere-se

consultar o site do repositório em <https://github.com/chronotope/chrono-tz> ou o seguinte endereço: https://docs.rs/chrono-tz/0.4.1/chrono_tz/.

O programa a seguir apresenta o fuso horário entre a cidade de São Paulo (Brasil) e a cidade de Lisboa (Portugal). Para isso, crie o projeto `c09ex18`.

Abra o arquivo de programa `Cargo.tom` e proceda com a adição da linha em negrito indicada a seguir:

```
[package]
name = "c09ex18"
version = "0.1.0"
authors = ["Fulano"]
```

```
[dependencies]
chrono = "0.4"
chrono-tz = "0.4"
```

Na sequência é preciso fazer a construção do recurso na janela do terminal, após gravar a alteração do arquivo `Cargo.toml`. Para isso, escreva no terminal a instrução

```
cargo build
```

Após a construção do módulo e a obtenção dos arquivos de suporte, abra o arquivo de programa `main.rs` da pasta/diretório `src` em estudo e informe o código de programa:

```
extern crate chrono;
extern crate chrono_tz;

use std::io;
use std::io::prelude::*;

use chrono::*;
use chrono_tz::UTC;

use chrono_tz::America::Sao_Paulo;
use chrono_tz::Europe::Lisbon;

fn main() {
    let dh = Sao_Paulo.ymd(2017, 4, 26).and_hms(0, 0, 0);
```

```

    let h = dh.with_timezone(&UTC);

    println!("São Paulo ...: {}", dh.format("Data: %d/%m/%Y | Hora:
%Hh%Mmin"));
    println!("Hora UTC ....: {}", h.format("Data: %d/%m/%Y | Hora:
%Hh%Mmin"));
    println!();

    let dh = Lisbon.ymd(2017, 4, 26).and_hms(0, 0, 0);
    let h = dh.with_timezone(&UTC);

    println!("Lisboa .....: {}", dh.format("Data: %d/%m/%Y | Hora:
%Hh%Mmin"));
    println!("Hora UTC ....: {}", h.format("Data: %d/%m/%Y | Hora:
%Hh%Mmin"));

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Após a escrita do código em `src` de `c09ex18` em `estudo`, execute, a partir da pasta/diretório `estudo`, a instrução

```
cargo run
```

Ao ser executado o programa, será mostrada a hora indicada de São Paulo e de Portugal. Com base na coordenada de tempo universal (UTC), é apresentada a diferença de horas entre a capital indicada e o horário UTC. A hora `00h00min` de São Paulo para `26/04/2017` é indicada como `03h00min`, e a hora `00h00min` de Lisboa é indicada como `23h00min` com a data indicada como `25/04/2017`. O fuso horário em São Paulo para UTC é de + 3h, e para Lisboa é de - 1h.

Para o programa do projeto `c09ex18` estão sendo colocados em uso os módulos `chrono` e `chrono_tz`. Para a biblioteca `chrono_tz` são usados os complementos `America::Sao_Paulo` e `Europe::Lisbon` para uso dos recursos de fuso horário. Uma lista completa desses recursos pode ser obtida no endereço https://docs.rs/chrono-tz/0.4.1/chrono_tz/.

Após definir a zona de tempo, o fuso horário `Sao_Paulo.ymd(2017, 4, 26).and_hms(0, 0, 0)` e `Lisbon.ymd(2017, 4, 26).and_hms(0, 0, 0)`, é definido o horário UTC com o método `with_timezone(&UTC)`.

9.5 Coleções

As coleções em Rust são tipos de dados derivados internos que fornecem uma forma padronizada para o gerenciamento de estruturas de dados clássicas, como listas, mapas, pilhas, filas, entre outras, de forma dinâmica.

As coleções estão divididas em quatro categorias e atendem a um conjunto de operações essenciais.

- Sequenciais: `Vec`, `VecDeque`, `LinkedList`
- Mapas: `HashMap`, `BTreeMap`
- Indexados: `HashSet`, `BTreeSet`
- Comum: `BinaryHeap`

Os exemplos de código apresentados neste tópico serão definidos na forma mais simplista possível, focando a coleção `Vec`. Uma coleção `Vec` (ou mesmo um vetor) é uma estrutura de dados em estilo lista, que é operacionalizada a partir de dados de tipos genéricos. Uma coleção `Vec` é usada para o gerenciamento de uma lista de valores definidos em um vetor dinâmico atuando suas ações em estruturas do tipo pilha ou fila.

Em uma coleção `Vec` em estilo pilha é permitido fazer por meio do método `push()` a entrada de elementos no final do vetor. A retirada de elementos é efetuada pelo método `pop()`, que efetua sua ação no final do vetor. Dessa forma, o último elemento inserido no vetor é o primeiro elemento a ser retirado.

O programa seguinte efetua a inserção em um vetor nesta ordem: os elementos 9, 8, 7, 6, 5, 4, 3, 2, 1, 9, 8 e 0. Na sequência, apresenta os valores armazenados; depois, o programa retira três elementos do vetor e apresenta novamente o vetor com a posição após a ação de remoção dos dados. Para isso, crie o projeto `c09ex19`.

```
use std::io;
```

```

use std::io::prelude::*;

fn main() {
    let mut lista = Vec::new();

    lista.push(9);
    lista.push(8);
    lista.push(7);
    lista.push(6);
    lista.push(5);
    lista.push(4);
    lista.push(3);
    lista.push(2);
    lista.push(1);
    lista.push(0);
    lista.push(9);
    lista.push(8);

    for i in &lista {
        println!("{}", i);
    }
    println!();

    lista.pop();
    lista.pop();
    lista.pop();

    for i in &lista {
        println!("{}", i);
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, os valores definidos são apresentados antes e depois da remoção de alguns elementos. Observe no laço `for` a definição

da estratégia de acesso via iterador quando da definição `&LISTA`, que determina para a variável interna `i` do laço os limites operacionais que devem percorrer dentro do vetor.

Observe nas duas partes da apresentação o efeito de uso dos métodos `push()` – que empurra um valor para dentro do vetor – e `pop()` – que retira um valor do vetor. Note que `push()` empilha valores e `pop()` desempilha valores.

Em uma coleção `Vec` em estilo fila é permitido fazer, por meio do método `push()`, a entrada de elementos no final do vetor. A retirada de elementos é efetuada pelo método `remove()` com índice `0` (zero), para efetuar a retirada do elemento do início do vetor. Dessa forma, o primeiro elemento inserido no vetor é o primeiro elemento a ser retirado.

O programa seguinte efetua a inserção em um vetor nesta ordem: os elementos 1, 2, 3, 4, e 5; e na sequência efetua a apresentação dos valores armazenados. Depois o programa retira um elemento do início do vetor e apresenta novamente o vetor com a posição após a remoção dos dados. Para isso, crie o projeto `c09ex20`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut lista = Vec::new();

    lista.push(1);
    lista.push(2);
    lista.push(3);
    lista.push(4);
    lista.push(5);

    for i in &lista {
        println!("{}", i);
    }
    println();

    lista.remove(0);
```

```

    for i in &lista {
        println!("{}", i);
    }
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, os valores são apresentados após a inserção dos elementos no vetor e a retirada do primeiro elemento do vetor. O uso do método `remove()` faz com que todos os elementos posteriores a sua retirada sejam realocados nas posições liberadas.

Além dos métodos `push()` `pop()` e `remove()`, há um extenso conjunto de ações que podem ser implementadas junto a um vetor. É impossível descrever todas as funcionalidades em um livro, pois tornaria seu tamanho muito extenso. No entanto é possível demonstrar alguns métodos que auxiliam diversas ações sobre uma coleção vetor, tais como `clear()`, que remove todos os elementos do vetor, `len`, que informa o tamanho do vetor, `first()`, que informa o primeiro elemento do vetor, e `last()`, que informa o último elemento do vetor.

Para ver alguns desses métodos em uso, considere o programa a seguir com o gerenciamento básico de uma lista de valores. Para isso, crie o projeto `c09ex21`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let mut lista = Vec::new();

    lista.push(1);
    lista.push(2);
    lista.push(3);
    lista.push(4);
    lista.push(5);
}

```

```

    for i in &lista {
        println!("{}", i);
    }
    println();

    println!("Tamanho do vetor ...: { :?}", lista.len());
    println!("Primeiro elemento ..: { :?}", lista.first());
    println!("Último elemento ....: { :?}", lista.last());

    println();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, é apresentada a quantidade de elementos – o primeiro e o último valores da lista indicados em `Some()`.

O programa seguinte mostra a limpeza total de um vetor e verifica se está ou não com conteúdos armazenados. Para isso, crie o projeto `c09ex22`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let mut lista = Vec::new();

    lista.push(9);
    lista.push(8);
    lista.push(7);
    lista.push(6);
    lista.push(5);

    if lista.is_empty() {
        println!("Vetor sem conteúdo");
    } else {
        println!("Vetor com conteúdo");
    }
}

```

```

    for i in &lista {
        println!("{}", i);
    }
    println();

    lista.clear();

    if lista.is_empty() {
        println!("Vetor sem conteúdo");
    } else {
        println!("Vetor com conteúdo");
    }

    println();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, aparece uma notificação informando que o vetor na memória possui conteúdo armazenado. Assim que o método `clear()` é usado, o vetor tem seus elementos removidos, sendo então apresentada uma mensagem informando que o vetor não contém elementos.

O programa seguinte demonstra as operações de ordenação em uma lista de valores desordenada. O programa lança mão de duas formas de ordenação: crescente e decrescente. Para isso, crie o projeto `c09ex23`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let mut lista = Vec::new();

    lista.push(9);
    lista.push(7);
    lista.push(8);
    lista.push(5);
}

```



```

println!("Listagem sem ordenação");
println!();

for i in &lista {
    println!("{}", i);
}
println!();

lista.sort();

println!("Listagem com ordenação crescente");
println!();

for i in &lista {
    println!("{}", i);
}
println!();

lista.reverse();

println!("Listagem com ordenação decrescente");
println!();
for i in &lista {
    println!("{}", i);
}

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

O programa, ao ser executado, mostra a lista de valores como definida; depois, por meio do método `sort()`, apresenta a lista ordenada; e após o uso do método `reverse()`, a lista ordenada é apresentada na forma inversa, simulando ordenação decrescente.

A ordenação decrescente pode ser usada de forma mais simples por meio do método `sort_by()` com auxílio do método `cmp()`.

O programa seguinte demonstra a ordenação de uma lista de valores com o método `sort_by()`. Para isso, crie o projeto `c09ex24`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let mut lista = Vec::new();

    lista.push(9);
    lista.push(7);
    lista.push(8);
    lista.push(5);

    println!("Listagem sem ordenação");
    println!();

    for i in &lista {
        println!("{}", i);
    }
    println!();

    lista.sort_by(|a, b| a.cmp(b));

    println!("Listagem com ordenação crescente");
    println!();

    for i in &lista {
        println!("{}", i);
    }
    println!();

    lista.sort_by(|a, b| b.cmp(a));

    println!("Listagem com ordenação decrescente");
    println!();

    for i in &lista {
        println!("{}", i);
    }
}
```

```

    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

O programa, ao ser executado, mostra a lista de valores sem ordenação; em seguida, por meio do método `sort_by(|a, b| b.cmp(a))`, apresenta a lista ordenada; e após o uso do método `sort_by(|a, b| b.cmp(a))`, a lista ordenada é apresentada na forma inversa, simulando ordenação decrescente.

O método `sort_by()` recebe como primeiro parâmetro dois valores entre *pipes* a serem comparados, e o segundo parâmetro usa o método `cmp()` vinculado aos valores informados no primeiro parâmetro para comparar os valores.

O método `cmp()` usa dois valores, sendo o primeiro valor definido à frente e fora da função, separado por um ponto e um segundo valor definido como seu parâmetro. Desta forma, é realizada a comparação do valor externamente indicado (A) com o valor passado como parâmetro (B).

9.6 Cadeias e caracteres

No capítulo 2 foram apresentados os tipos de dados para o tratamento de sequências de caracteres (*strings*) e caracteres, sendo `char`, `str` para dados imutáveis, e para dados mutáveis, o tipo `String`. Neste tópico, esse assunto será aprofundado além dos usos realizados nos programas apresentados nesta obra. No entanto, será uma exposição preliminar e introdutória ao tema.

O tipo de dado para a definição de *strings* na linguagem Rust opera com o padrão UTF-8 tendo à disposição duas formas operacionais para a definição de cadeias de caracteres: uma estática e outra dinâmica. A forma estática é definida a partir de `&'static str` (que pode, por vezes, ser definido como `&str`) e a dinâmica a partir de `String`.

O tipo caractere na linguagem Rust opera com o padrão Unicode de 32 bits, sendo este o tipo com tamanho suficiente para representar tão somente um caractere.

O programa seguinte demonstra o uso de duas *strings* com definição estática `&'static str` em conjunto com um *string* dinâmico do tipo `String` o qual efetua a concatenação das duas sequências de caracteres estáticas em uma só sequência. O programa apresenta duas maneiras de estabelecer a concatenação de sequências de caracteres estáticas em uma sequência de caracteres dinâmica. Para isso, no diretório (pasta) **estudo** crie um projeto com o nome `c09ex25`.

```
#![allow(unused_assignments)]
use std::io;
use std::io::prelude::*;

fn main() {
    let alo: &'static str = "Alô,";
    let mundo: &'static str = " Mundo!";

    let mut cadeia1: String = String::new();
    let mut cadeia2: String = String::new();

    cadeia1 += alo;
    cadeia1 += mundo;

    println!("Cadeia 1 = {}", cadeia1);
    println!();

    cadeia2 = alo.to_string() + mundo;

    println!("Cadeia 2 = {}", cadeia2);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, as duas concatenações definidas são apresentadas nas formas de mensagem. A variável mutável `CADEIA1` faz a concatenação a partir das ações `cadeia1 += alo;` e `cadeia1 += mundo;` ao ser estabelecida a ação de acumulação. No caso da variável mutável `CADEIA2`, a ação de concatenação é realizada a partir da instrução `cadeia2 = alo.to_string() + mundo;`, que faz a junção do conteúdo da variável `ALO` com o conteúdo da variável `MUNDO`, sendo necessário, junto à variável `ALO`, fazer uso do método `to_string()`.

O método `to_string()` tem funcionalidade similar ao método `to_owned()`, que é encontrado em programas Rust mais antigos. O método `to_owned()` é considerado obsoleto, devendo ser usado em seu lugar o método `to_string()`, que tem por finalidade converter um *string* estático na forma de *string* dinâmico.

No caso da instrução `cadeia2 = alo.to_string() + mundo;`, a variável `CADEIA` é definida como mutável e pode receber a atribuição de outra variável. No entanto a variável `ALO` deve ser concatenada à variável `MUNDO`, por isso precisa ser convertida de imutável para mutável, e isso pode ser feito por meio do método `to_string()`.

A instrução inicial `#![allow(unused_assignments)]` desabilita a apresentação de mensagens de advertência relacionadas ao uso da variável `CADEIA2`.

O programa seguinte, a partir da *string* `Alô, Mundo!`, mostra a quantidade de *bytes* usados para a mensagem e a quantidade de caracteres que compõem a mensagem. Para isso, crie o projeto `c09ex26`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let cadeia = "Alô, Mundo!";

    println!("Número de caracteres ..: {}", cadeia.chars().count());
    println!("Número de bytes .....: {}", cadeia.len());

    println!();
    print!("Tecle <Enter> para encerrar...");
```

```

    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, são apresentados os valores 11 e 12, respectivamente, para a quantidade de caracteres e a quantidade de bytes.

A variável `CADEIA`, ao ser atribuída com a sequência de caracteres `Alô, Mundo!`, é feita de forma imutável. Assim, `CADEIA` é uma variável do tipo `&'static str`.

A *string* `Alô, Mundo!` é composta de onze caracteres detectados com o uso dos métodos `chars()` em conjunto com o método `count()` e ocupa 12 bytes detectados pelo método `len()`. Cada caractere ocupa um byte, e no final da sequência de caracteres há um caractere de controle que indica o final da *string*. Por isso é apresentado o valor 12 para a quantidade de bytes.

Além da definição de variável do tipo *string* mutável com a forma `String::new()` ou com o método `to_string()`, pode-se ter o mesmo efeito utilizando-se a forma `String::from()`, como demonstra o próximo programa. Para isso, crie o projeto `c09ex27`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let alo: &str = "Alô,";
    let mundo: &str = " Mundo!";

    let cadeia = String::from(alo.to_string() + mundo);

    println!("{}", cadeia);

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, a mensagem `Alô, Mundo!` é apresentada como os demais exemplos. Mas neste programa as variáveis `ALO` e `MUNDO`

imutáveis estão sendo definidas com o tipo `&str`, que tem o mesmo significado do tipo `&'static str`. A variável mutável `CADEIA` está sendo definida sem o uso do comando `mut`, por fazer uso do recurso `String::from()`, utilizando-se como parâmetro a concatenação `alo.to_string() + mundo`.

O programa seguinte demonstra uma ação de junção de *strings* sem o uso da ação de concatenação. Para isso, crie o projeto `c09ex28`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let alo = String::from("Alô");
    let mundo = String::from(" Mundo!");

    let mut cadeia = String::new();

    cadeia.push_str(&alo);
    cadeia.push(',');
    cadeia.push_str(mundo.as_str());

    println!("{}", cadeia);
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, a mensagem `Alô, Mundo!` é apresentada como os demais exemplos. Na situação exposta, a junção da mensagem ocorre com o uso dos métodos `push_str()` para uso com *string* e `push()` para uso com caractere.

O parâmetro de operação do método `push_str()` pode ser usado de duas formas: com o operador e-comercial (`&`) ou com o método `as_str()`. Tanto o uso do operador e-comercial como o uso do método `as_str()` orientam a ação da junção com os dados como sendo estes do tipo *string*.

A ação de concatenação pode ser realizada por meio do método `new()` de

String a partir da definição de diversas referências. Sendo assim, o programa a seguir mostra este efeito. Para isso, crie o projeto `c09ex29`.

```
use std::io;
use std::io::prelude::*;

fn main() {
    let alo: &str = "Alô,";
    let mundo: &str = " Mundo!";

    let cadeia: String = String::new() + alo + mundo;

    println!("{}", cadeia);
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, a mensagem `Alô, Mundo!` é apresentada como os demais exemplos. Mas nesta versão a concatenação é feita de forma simplificada a partir do trecho da instrução `String::new() + alo + mundo`.

Além das ações de concatenação, pode-se, por meio de *strings*, fazer a passagem destas, como parâmetros de funções. Assim, o programa seguinte apresenta a mensagem `Alô, Mundo!` passada por parâmetro. Para isso, crie o projeto `c09ex30`:

```
use std::io;
use std::io::prelude::*;

fn escreva1(cadeia: String) {
    println!("{}", cadeia);
}

fn escreva2(cadeia: &str) {
    println!("{}", cadeia);
}
```



```

fn main() {
    escreva1("Alô, Mundo!".to_string());
    println!();

    escreva2("Alô, Mundo!");
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, duas mensagens de saudação `Alô, Mundo!` são apresentadas. A primeira mensagem é tratada na sub-rotina de procedimento `escreva1()`, que recebe como parâmetro o conteúdo `"Alô, Mundo!".to_string()`, em que o método `to_string()` faz com que a mensagem seja definida como mutável. A segunda mensagem é tratada na sub-rotina `escreva2()`, que recebe como parâmetro o conteúdo `"Alô, Mundo!"`.

A diferença na definição dos parâmetros nas sub-rotinas decorre do uso dos tipos de dados `String` e `&str`, sendo que a passagem de parâmetros com *strings* é mais fácil de ser usada quando é utilizado o tipo `&str`.

Com *strings* há a possibilidade operacional de fazer uso de *sub strings* a partir da técnica de *slices* ou a partir do uso dos métodos `chars()`, `skip()`, `take()` e `collect()`.

O programa a seguir mostra a palavra `COMPUTADOR` separada nas partes `COM` `PUTA` `DOR` por meio da técnica de *slice*. Para isso, crie o projeto `c09ex31`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let palavra = String::from("COMPUTADOR");

    let parte1 = &palavra[0..3];
    let parte2 = &palavra[3..7];
    let parte3 = &palavra[7..10];
}

```

```

    print!("{}", parte1);
    print!("{}", parte2);
    print!("{}", parte3);
    println();
    println();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, a palavra `COMPUTADOR` é apresentada em partes separadas, como definido. Nesta versão, os *slices* definidos efetuam a separação da palavra utilizando-se as faixas de: `[0..3]`, que pega do primeiro ao quarto caractere; `[3..7]`, que pega do quarto ao oitavo caractere; e `[7..10]`, que pega do oitavo ao décimo primeiro caractere.

O programa a seguir mostra a palavra `COMPUTADOR` separada nas partes `COM` `PUTA` `DOR` por meio dos métodos `chars()`, `skip()`, `take()` e `collect()`. Para isso, crie o projeto `c09ex32`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let palavra = String::from("COMPUTADOR");

    let parte1: String = palavra.chars().skip(0).take(3).collect();
    let parte2: String = palavra.chars().skip(3).take(4).collect();
    let parte3: String = palavra.chars().skip(7).take(3).collect();

    print!("{}", parte1);
    print!("{}", parte2);
    print!("{}", parte3);
    println();
    println();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, a palavra `COMPUTADOR` é apresentada em partes separadas, como definido. Nesta versão a separação das partes da palavra é feita a partir da captura isolada de cada caractere pelo método `chars()`, que, ao fazer uso do método `skip()`, determina a posição da palavra em que um caractere será pego. O método `take()` especifica a quantidade de caracteres a ser pego com base na indicação do método `skip()`. O método `collect()` trata a palavra como uma coleção.

Apesar de a segunda versão do programa ser mais trabalhosa para a escrita, ela tem uma facilidade de definição maior que a primeira versão.

O uso de sequências de caracteres permite que a apresentação dessas sequências ocorra de maneira formatada por meio do uso da macro `format!()`. Parte dos códigos de formatação de que a macro `format!()` faz uso já foi explorada com a macro `print!()` e `println!()`, como: `{:}`, `{:tamanho:decimal}`, `{:?}` e `{:p}`. A macro `format!()` é um recurso usado na definição de variáveis do tipo *string*. A Tabela 9.2 apresenta os códigos de formatação disponíveis para as macros `print!()`, `println!()` e `format!()`.

Tabela 9.2 – Códigos de formatação para as macros: `print!()`, `println!()` e `format!()`

Código	Função
<code>{:?}</code>	Mostra os dados de uma estrutura operacionalizada com o traço de depuração.
<code>{:#?}</code>	Mostra os campos e os dados de uma estrutura que deriva do traço de depuração.
<code>{:<N}</code> , <code>{ }</code>	Formata e alinha o texto à esquerda com N espaços.
<code>{:>N}</code>	Formata e alinha o texto à direita com N espaços.
<code>{:0N}</code>	Define zeros à esquerda para a largura definida com N.
<code>{:.N}</code>	Define a precisão de um número para N casas decimais.
<code>{:.0N}</code>	Define a precisão de um número para N casas decimais com zeros à direita.
<code>{:e}</code> , <code>{:E}</code>	Mostra valor exponencial com código em minúsculo ou maiúsculo.
<code>{:x}</code> , <code>{:X}</code>	Mostra valor hexadecimal com código em minúsculo ou maiúsculo com prefixo <code>0x</code> ou <code>0X</code> .
<code>{:o}</code>	Mostra valor octal com prefixo <code>0o</code> .

Código	Função
{:b}	Mostra valor binário com prefixo 0b.
{:p}	Mostra a localização da memória de uma estrutura do tipo ponteiro.

O programa seguinte apresenta o efeito dos códigos de formatação não utilizados anteriormente. Para isso, crie o projeto c09ex33.

```

use std::io;
use std::io::prelude::*;

#[derive(Debug)]
struct Coordenada {
    x: u32,
    y: u32,
}

fn main() {
    let posicao = Coordenada { x: 5, y: 9 };
    let palavra = String::from("Teste");
    let numero1 = 26;
    let numero2 = 1.24;
    let numero3 = 1.984;
    let numero4 = 1e6;
    let numero5 = 255;

    println!("Posicao .....: {:#?}", posicao);
    println!("Palavra .....: {:>15}", palavra);
    println!("Palavra .....: {:<15}", palavra);
    println!("Número 1 .....: {:05}", numero1);
    println!("Número 2 .....: {:.3}", numero2);
    println!("Número 3 .....: {:.05}", numero3);
    println!("Número 4 e ...: {:e}", numero4);
    println!("Número 4 E ...: {:E}", numero4);
    println!("Número 5 h ...: {:x}", numero5);
    println!("Número 5 H ...: {:X}", numero5);
    println!("Número 5 o ...: {:o}", numero5);
    println!("Número 5 b ...: {:b}", numero5);

```

```

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, um conjunto de saídas, de acordo com os códigos de formatação, é apresentado por meio da macro `println!()`.

Os códigos de formatação para saída de dados usados com os símbolos "{}" aceitam internamente a definição de rótulos em vez de valores diretos, como normalmente são usados, permitindo que o estabelecimento de valores seja firmado aos rótulos externamente. Por exemplo, a instrução `println!("{vlr:>tamanho$}", vlr=10, tamanho=5);` determina a apresentação do valor 10 representado pelo rótulo `vlr` alinhado à direita (indicativo do símbolo `>`) 5 posições de acordo com o que se encontra definido no rótulo `tamanho`. O caractere cifrão (\$) usado no final da sequência é o código que permite a aplicação desse efeito. Uma variação para a instrução pode ser o estabelecimento do valor de zeros a ser apresentado à esquerda que pode ser feito com o código de formatação `{vlr:>0tamanho$}` colocando-se o valor 0 (zero) antes da definição do rótulo que controla o tamanho do campo.

O estilo de formatação para apresentação de valores com a instrução `println!("{}", 1, 2, 3);` pode ser definido com um estilo próximo ao usado na linguagem C# como `println!("{0} {1} {2}", 1, 2, 3);` para a apresentação dos valores 1, 2 e 3. No entanto o segundo estilo de formatação permite uma variação interessante na apresentação dos dados ao fazer uso da instrução `println!("{1} {0} {2}", 1, 2, 3);`, que gera como saída os valores 2, 1 e 3.

A partir da demonstração do conjunto de códigos de formatação e da demonstração de uso com a macro `println!()` é possível fazer uso desses códigos com a macro `format!()`.

O programa seguinte apresenta a mensagem de saudação *Alô, Mundo!* a partir da definição da macro `format!()`. Para isso, crie o projeto `c09ex34`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let palavra = "Mundo!";
    let mensagem = format!("Alô, {}!", palavra);

    println!("{}", mensagem);
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, a mensagem de saudação é apresentada. Note que a variável `MENSAGEM` é constituída pela ação do trecho de instrução `format!("Alô, {}!", palavra)`, em que a macro `format!()` é atribuída à variável `MENSAGEM`, tendo seu funcionamento similar ao da macro `print!()` / `println!()`, com a diferença de que a macro `format!()`, para surtir efeito, deve ser sempre atribuída a uma variável do tipo *string*.

Além da manipulação de *strings*, cabe ressaltar operações realizadas com os caracteres de uma sequência de dados alfabéticos ou alfanuméricos. Neste sentido, existem diversos métodos para essa finalidade, podendo-se destacar `as_byte()` e `chars()`, que já foram utilizados.

O programa seguinte apresenta a mensagem `Linguagem Rust` e, então, decompõe cada caractere da mensagem em sequências de códigos ASCII e na forma separada de cada um dos caracteres. Para isso, crie o projeto `c09ex35`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let mensagem = "Linguagem Rust".to_string();

    println!("{}", mensagem);
    println!();
}

```

```

    for i in mensagem.as_bytes() {
        print!("{}", i);
    }
    println!("\n");

    for i in mensagem.chars() {
        print!("{}", i);
    }
    println!("\n");

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, a mensagem é apresentada de forma completa e, depois, separada (decomposta) na forma de código ASCII e na forma de caracteres isolados.

O método `as_byte()`, quando em uso, faz com que cada caractere da mensagem capturado seja traduzido para seu código ASCII equivalente. Já o método `chars()` apenas separa o caractere na forma de caractere que contém.

O programa a seguir demonstra essa finalidade. Para isso, crie o projeto `c09ex36`.

```

use std::io;
use std::io::prelude::*;

fn main() {
    let cadeia = String::from("Alô, Mundo!");

    let cadeia_string = &cadeia;
    let cadeia_str: &str = &cadeia;

    println!("Cadeia do tipo String ...: {}", cadeia_string);
    println!("Cadeia do tipo &str .....: {}", cadeia_str);
}

```

```
println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, as mensagens são apresentadas de forma semelhante. No entanto cada uma das instruções de apresentação opera com uma variável diferente da outra. A variável `CADEIA_STRING` se caracteriza por ser a definição de uma variável mutável, e a variável `CADEIA_STR` se caracteriza por ser uma variável imutável. As formas indicadas são uma maneira de estabelecer a conversão de uma variável *string* mutável em imutável.

9.7 Criptografia

O termo *criptografia* tem sua origem na Grécia, sendo a junção das palavras *kryptós* (oculto) e *graphos* (escrita), dando origem ao termo *criptografia* (escrita oculta). A ação de criptografar um texto pode ser efetuada por *transposição* ou *substituição*. A criptografia por transposição ocorre quando os caracteres de um texto são trocados de posição entre si com base em alguma regra definida; a criptografia por substituição ocorre quando os caracteres de um texto são trocados por outros caracteres, ou mesmo símbolos. (SINGH, 2011, p. 25-26).

Este tópico apresenta a aplicação do módulo criptográfico `cipher-crypt`, que fornece algumas ações de uso de criptografia. Na versão usada para a confecção deste livro, o módulo tinha como funcionalidade as cifras de Caesar (César), Rot13, Affline, Rail-fence, Polybius Square, Autokey, Hill, Fractionated Morse e Vigenère – disponível no endereço <https://github.com/arossopope/cipher-crypt>. Nesta obra são apresentadas apenas as cifras Caesar e Vigenère.

O algoritmo Caesar, usado pelo imperador romano Júlio César para se comunicar com seus generais, tem sua origem no ano 50 a.C. Ele usa uma estratégia de cifragem muito simples, baseada em um único alfabeto

cifrante (monoalfabético), em que ocorre o deslocamento de uma letra do alfabeto três posições à frente (monográfica) para cifrar mensagens curtas, sendo ideal para a escrita de pequenos textos. Sua segurança é muito baixa; basta que seja aplicada uma simples análise de frequência estatística baseada nas características do idioma cifrado para decifrar a mensagem (VICKI, 2008). A Tabela 9.2 mostra a regra de cifragem do algoritmo Caesar contendo na linha superior (em minúsculo) o caractere a ser cifrado e na linha inferior (em maiúsculo) o caractere cifrado.

Tabela 9.3 – Regra de cifragem: Caesar

Alfabeto																									
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Na Tabela 9.3, a linha superior contém o alfabeto convencional, e a linha inferior, o alfabeto cifrado em relação ao alfabeto convencional.

A mensagem ATACAR BASE INIMIGA AO AMANHECER (atacar base inimiga ao amanhecer) é cifrada (criptografada) como DWDFDU EDVH LQLPLJD DR DPDQKHFHU.

O programa seguinte demonstra a cifragem de uma mensagem segundo as regras da cifra de Caesar a partir do módulo `cipher_crypt`. Para isso, crie o projeto `c09ex37`.

Abra o arquivo de programa `Cargo.toml` e proceda com a adição da linha em negrito indicada a seguir:

```
[package]
name = "c09ex37"
version = "0.1.0"
authors = ["Fulano"]

[dependencies]
cipher-crypt = "^0.8"
```

Na sequência é necessário, na janela do terminal, após gravar a alteração do arquivo `Cargo.toml`, fazer a construção do recurso. Para isso, escreva no

terminal a instrução

```
cargo build
```

Após a construção do módulo e a obtenção dos arquivos de suporte, abra o arquivo de programa `main.rs` da pasta/diretório `src` em `estudo` e informe o código de programa:

```
#![allow(unused_assignments)]
extern crate cipher_crypt;

use cipher_crypt::{Cipher, Caesar};

use std::io;
use std::io::prelude::*;

fn main() {
    let mut msgorig = String::new();
    let mut msgcifr = String::new();
    let mut msgdeci = String::new();

    let rotacao = Caesar::new(3).unwrap();

    println!("CIFRA DE CAESAR");
    println!();

    print!("Informe mensagem a ser cifrada ..: ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut msgorig).expect("Entrada incorreta");

    if let Some('\n') = msgorig.chars().next_back() {
        msgorig.pop();
    }
    if let Some('\r') = msgorig.chars().next_back() {
        msgorig.pop();
    }
    println!();

    msgcifr = rotacao.encrypt(&msgorig).unwrap();
    msgdeci = rotacao.decrypt(&msgcifr).unwrap();
}
```

```
println!("Mensagem original .....: {}", msgorig);
println!("Mensagem com cifragem ..: {}", msgcifr.to_uppercase());
println!("Mensagem sem cifragem ..: {}", msgdeci.to_uppercase());

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, insira uma mensagem escrita tanto com caracteres minúsculos como com caracteres maiúsculos. O programa fará a conversão da mensagem em caracteres maiúsculos apresentando tanto a mensagem criptografada quanto a mensagem descriptografada.

A instrução `let rotacao = Caesar::new(3).unwrap();` estabelece para o método `new()` o valor de rotação 3 (valor-padrão da cifra com deslocamento de três posições). Este valor poderá ser alterado na faixa de 1 até 26 para que a ordem de cifragem seja alterada.

Observe que a criptografia é executada pelo método `encrypt()`, e a descriptografia é feita pelo método `decrypt()`.

O algoritmo de Vigenère foi desenvolvido por Blaise de Vigenère em 1586. Usa uma estratégia de cifragem bem-elaborada, baseada em 26 alfabetos cifrantes (polialfabético) aplicados individualmente às letras da mensagem a ser cifrada (monográfica) que são combinadas com as letras de uma palavra-chave. Sua segurança é considerada baixa nos dias atuais (VICKI, 2006), mas foi imbatível até 1854, quando Charles Babbage e, depois, em 1863, Friedrich Wilhelm Kasiski a quebraram. A cifra de Vigenère é imune à análise de frequência (SINGH, 2011, p. 68). Kasiski é considerado em primeira instância quem quebrou essa criptografia por ter publicado o feito antes de Babbage. Charles Babbage não pôde publicar sua descoberta em virtude do trabalho sigiloso que executava para o governo britânico na área de quebra de códigos (SINGH, 2011).

A tabela de Vigenère tem, na primeira linha, um alfabeto normal seguido de 26 alfabetos cifrados que são deslocados uma letra para frente em

relação ao alfabeto anterior. Para seu efetivo uso, deve ser definido para a cifração o conceito de palavra-chave, no sentido de neutralizar a proximidade com a cifra de Caesar (SINGH, 2011, p. 66). A estrutura da cifra de Vigenère é indicada na Tabela 94.

Com o objetivo de demonstrar o uso da cifra de Vigenère, considere a mensagem ATACAR BASE INIMIGA AO AMANHECER (*atacar base inimiga ao amanhecer*) com a chave de cifração B0L0L0 que será cifrada como BHLQLF CODS TBJATUL OP OXOYVFQPF. Na Tabela 94, sem levar em consideração a linha e a coluna de títulos, a primeira linha (horizontal) refere-se aos caracteres da mensagem original, e a primeira coluna refere-se aos caracteres da palavra-chave.

Para criar mensagem cifrada de acordo com a tabela de Vigenère, pode-se fazê-lo a partir de qualquer uma das linhas, observando-se certo número de linhas para cima e para baixo, para, então, representar a mensagem cifrada. Dessa forma, uma mensagem pode ter sua primeira letra cifrada a partir da segunda linha, a segunda letra cifrada a partir da oitava linha, a terceira letra cifrada a partir da décima quinta linha, e assim sucessivamente.

Tabela 94 – Regra adaptada de cifração: Vigenère (Leiradell, 1998; GIL, 2002, p. 6 e VICKI, 2006)

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
a	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
b	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
c	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
d	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
e	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
f	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
g	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F

h	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
i	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
j	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
k	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
l	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
m	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
n	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
o	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
p	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
r	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
r	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
s	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
t	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
u	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
v	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
w	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
x	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

A partir da cifração pelo estilo Vigenère se faz necessário ter uma forma

combinada entre emissor e receptor para codificar e decodificar a mensagem com a palavra-chave.

Note que abaixo da mensagem ATACAR BASE INIMIGA AO AMANHECER é colocada em toda a extensão a palavra-chave BOLOLO, como indicado na Tabela 9.5. Com isso, para proceder a cifragem, pega-se a primeira letra da mensagem original, neste caso, a letra A, e, observando a tabela a partir da primeira linha (fundo preto texto em branco), percorrem-se na mesma coluna as linhas da tabela até localizar a letra B (linha 1) da palavra-chave, então anota-se a letra encontrada, que, neste caso, é a letra B. Na sequência, pega-se a segunda letra, neste caso, a letra T, e percorrem-se as linhas até localizar a letra O, que está na linha 14, então anota-se a letra encontrada, neste caso, H. Com base nesse raciocínio, percorrem-se as demais letras da mensagem original junto à primeira linha da tabela, e na coluna da letra localizada, percorre-se até encontrar na linha a letra da chave relacionada à letra da mensagem original, então localiza-se, no ponto de intersecção das letras da mensagem original e da chave, a letra correspondente à cifragem.

Para decifrar, basta localizar na linha da tabela a letra da chave e na coluna a letra da mensagem cifrada. No ponto de intersecção estará a letra da mensagem original.

Tabela 9.5 – Mecanismo de cifragem Vigenère

Cifragem: 1ª linha – mensagem 2ª linha – chave 3ª linha – mensagem cifrada																												
a	t	a	c	a	r	b	a	s	e	i	n	i	m	i	g	a	a	o	a	m	a	n	h	e	c	e	r	
b	o	l	o	l	o	b	o	l	o	b	o	l	o	b	o	l	o	b	o	l	o	b	o	l	o	b	o	l
B	H	L	Q	L	F	C	O	D	S	T	B	J	A	T	U	L	O	P	O	X	O	Y	V	F	Q	P	F	

O programa seguinte demonstra a cifragem de uma mensagem segundo as regras da cifra de Vigenère. Para isso, crie o projeto `c09ex38`.

```
cargo new c09ex38 --bin
cd c09ex38
```

Abra o arquivo do programa `Cargo.tom` e proceda com a adição da linha

em negrito indicada a seguir:

```
[package]
name = "c09ex38"
version = "0.1.0"
authors = ["Fulano"]

[dependencies]
cipher-crypt = "^0.8"
```

Na sequência é preciso, na janela do terminal, após gravar a alteração do arquivo `Cargo.toml`, fazer a construção do recurso. Para isso, escreva no terminal a instrução

```
cargo build
```

Após a construção do módulo e a obtenção dos arquivos de suporte, abra o arquivo de programa `main.rs` da pasta/diretório `src` em estudo e informe o código de programa:

```
#![allow(unused_assignments)]
extern crate cipher_crypt;

use cipher_crypt::{Cipher, Vigenere};

use std::io;
use std::io::prelude::*;

fn main() {
    let mut msgorig = String::new();
    let mut msgcifr = String::new();
    let mut msgdeci = String::new();
    let mut chave = String::new();

    let rotacao;

    println!("CIFRA DE VIGENÈRE");
    println!();

    print!("Informe mensagem a ser cifrada ... ");
    io::stdout().flush().unwrap();
```

```

io::stdin().read_line(&mut msgorig).expect("Entrada incorreta");

if let Some('\n') = msgorig.chars().next_back() {
    msgorig.pop();
}
if let Some('\r') = msgorig.chars().next_back() {
    msgorig.pop();
}
print!("Informe chave de cifragem .....: ");
io::stdout().flush().unwrap();
io::stdin().read_line(&mut chave).expect("Entrada incorreta");

if let Some('\n') = chave.chars().next_back() {
    chave.pop();
}
if let Some('\r') = chave.chars().next_back() {
    chave.pop();
}
println!();

rotacao = Vigenere::new(String::from(chave)).unwrap();

msgcifr = rotacao.encrypt(&msgorig).unwrap();
msgdeci = rotacao.decrypt(&msgcifr).unwrap();

println!("Mensagem original .....: {}", msgorig);
println!("Mensagem com cifragem ..: {}", msgcifr.to_uppercase());
println!("Mensagem sem cifragem ..: {}", msgdeci.to_uppercase());

println!();
print!("Tecle <Enter> para encerrar...");
io::stdout().flush().unwrap();
io::stdin().read(&mut [0u8]).unwrap();
}

```

Após a escrita do código em src de c09ex38 em estudo, execute a partir da pasta/diretório estudo a instrução:

```
cargo run
```

Ao ser executado o programa, insira uma mensagem escrita tanto com

caracteres minúsculos como com caracteres maiúsculos. Informe a palavra-chave seguindo as mesmas regras da entrada da mensagem. O programa fará a conversão da mensagem em caracteres maiúsculos.

Dentro do tema criptografia há o uso das chamadas funções criptográficas, que geram, após a cifragem, o valor de uma *hash*. Uma *hash* é a definição de um valor criptográfico que, em tese, não pode ser revertido à sua forma original, sendo útil na definição de senhas, assinaturas digitais, entre outras possibilidades de controle. Destacam-se funções *hash* como MD5, SHA1, Whirlpool, entre outras. Nesta temática para a linguagem Rust, pode ser usada a biblioteca `rust-crypto`, disponível no endereço <https://github.com/DaGenix/rust-crypto>.

O programa a seguir demonstra o uso das funções *hash* MD5 e SHA1 do módulo `rust-crypto`. Para isso, crie o projeto `c09ex39`.

Abra o arquivo de programa `Cargo.tom` e proceda com a adição da linha em negrito indicada a seguir:

```
[package]
name = "c09ex39"
version = "0.1.0"
authors = ["Fulano"]
```

```
[dependencies]
rust-crypto = "^0.2"
```

Na sequência é preciso, na janela do terminal, após gravar a alteração do arquivo `Cargo.toml`, fazer a construção do recurso. Para isso, escreva no terminal a instrução

```
cargo build
```

Após a construção do módulo e a obtenção dos arquivos de suporte, abra o arquivo de programa `main.rs` da pasta/diretório `src` em estudo e informe o código de programa:

```
extern crate crypto;

use std::io;
use std::io::prelude::*;
```

```

use crypto::digest::Digest;
use crypto::md5::Md5;
use crypto::sha1::Sha1;
use crypto::whirlpool::Whirlpool;

fn main() {
    let mut palavra = Md5::new();
    palavra.input_str("Linguagem Rust");
    println!("MD5 ....: {}", palavra.result_str());

    let mut palavra = Sha1::new();
    palavra.input_str("Linguagem Rust");
    println!("SHA1 ...: {}", palavra.result_str());

    let mut palavra = Whirlpool::new();
    palavra.input_str("Linguagem Rust");
    println!("WHIRLPOOL ...: {}", palavra.result_str());
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, os valores de *hash* da mensagem "Linguagem Rust" são apresentados de acordo com os algoritmos utilizados. Independentemente do tamanho da mensagem, o valor *hash* tem sempre o mesmo tamanho para o algoritmo utilizado.

O módulo `rust-crypto` contém uma coleção de funcionalidades para criptografia e *hashs*. Para mais detalhes sobre essas funcionalidades, visite o endereço <https://messense.github.io/otpauth-rs/crypto/index.html>.

Para efetivar a entrada do dado (a mensagem) e sua apresentação transformada em *hash* são usados os métodos de entrada `input_str()` e de saída `result_str()` por meio da instrução `use crypto::digest::Digest;`.

A ação *hash* ocorre, especificamente, por meio da ativação dos recursos `crypto::md5::Md5`, `crypto::sha1::Sha1` e `crypto::whirlpool::Whirlpool`.

As funções da ação dos algoritmos MD5, SHA1 e WHIRLPOOL geram, respectivamente, para a mensagem `Linguagem Rust`, os valores de *hash*:

- 7f7987969db31e1093ea4039cceeaa06e
- 3d338fe4c8ee54efdec40911bb240ed46ca3c352
- 37f5abbbf9f74c49724465dc5278274b368e5e8f716bcd6a6e179862f46a09266e364de
a8698d5ab29f76669a463915dd9dcd71a1cde7db4
6b9115192964a476f2

Os exemplos indicados com o uso dos métodos de criptografia mantiveram nas mensagens os espaços em branco. Na computação comercial é importante manter os espaços, pois, se forem excluídos, sua restauração será bem trabalhosa. No entanto o estilo da criptografia militar, pelo menos do passado, fazia a remoção dos espaços em branco (e não se preocupavam em restaurá-los *a posteriori*).

A ação executada por um programa como a remoção de caracteres em branco de uma mensagem pode ser feita na linguagem Rust de algumas formas diferentes, com técnicas diferentes. O exemplo seguinte usa o modo imperativo para a exemplificação de sua ação. Assim, segue demonstração simples de como isso pode ser feito por meio de uma sub-rotina de função imperativa. O programa seguinte demonstra a remoção de espaços em branco sobre uma mensagem informada. Para isso, crie o projeto `c09ex40`.

```
#![allow(unused_assignments)]
use std::io;
use std::io::prelude::*;

fn removespc(mencomspc: &str) -> String {
    let mut mensempc = String::new();

    for caractere in mencomspc.chars() {
        if caractere != ' ' {
            mensempc.push(caractere);
        }
    }
}
```

```

    }

    return mensemspc;

}

fn main() {
    let mut mencomspc = String::new();
    let mut mensemspc = String::new();

    print!("Informe uma mensagem ... ");
    io::stdout().flush().unwrap();
    io::stdin().read_line(&mut mencomspc).expect("Entrada incorreta");

    if let Some('\n') = mencomspc.chars().next_back() {
        mencomspc.pop();
    }
    if let Some('\r') = mencomspc.chars().next_back() {
        mencomspc.pop();
    }
    mensemspc = removespc(&mencomspc);

    print!("Mensagem sem espaços ... {}", mensemspc.to_uppercase());

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Ao ser executado o programa, informe a mensagem "atacar base inimiga ao amanhecer" e, na sequência, será apresentada a mensagem sem espaço como ATACARBASEINIMIGAAOAMANHECER.

O programa usa na sub-rotina de função `removespc()` a definição das variáveis locais `MENCOMSPC` (mensagem com espaço) como passagem de parâmetro para a retirada dos espaços e `MENSEMSPC` (mensagem sem espaço) como retorno da função sem espaços. A instrução `let mut mensem`spc = String::new(); estabelece o espaço de memória suficiente para

a variável `MENSEMSPC`. O trecho definido no laço `for` detecta e pega cada um dos caracteres da mensagem e verifica por meio do comando `if` se o caractere em foco é diferente de espaço. Se o caractere for diferente de espaço, será empurrado por meio do método `push(caractere)` para a variável `MENSEMSPC`.

Os detalhes apresentados neste capítulo podem ser definidos manualmente em programação quando o programador tem tal conhecimento. No entanto a comunidade Rust colabora com a disponibilização de recursos que facilitam o desenvolvimento de programas até mesmo por programadores menos experientes. Antes de proceder ao desenvolvimento de algum recurso, é interessante verificar se não há disponível algum módulo ou uma biblioteca para aquilo que se deseja fazer; caso não haja, então se faz o seu desenvolvimento.

9.8 Armazenamento básico de dados

O armazenamento de dados é efetivado na forma de arquivos em disco que são uma maneira de extensão de armazenamento de bytes similar à dos arranjos e vetores, ou seja, as matrizes. Para Backes (2014, p. 248), “um arquivo, de modo abstrato, nada mais é do que uma coleção de bytes armazenados em um dispositivo de armazenamento secundário”.

A principal vantagem no uso de arquivos é a capacidade dos dados de ficarem armazenados para uso a qualquer momento. Outra vantagem dos arquivos é a capacidade de armazenar um número maior de bytes do que em uma tabela em memória na forma de uma matriz (arranjos e vetores), sendo limitante nessa armazenagem apenas o tamanho do meio físico utilizado para sua gravação.

As operações com arquivos feitas com as funcionalidades da linguagem Rust utilizam, em média, o conceito e o uso do paradigma da programação funcional, o que faz com que muitas das rotinas de acesso a arquivos sejam escritas de forma muito simples, tendo alto poder de processamento.

Este tópico foca o uso de duas situações com aplicações de arquivos na

forma mais simples possível, deixando-se uma apresentação mais elaborada para outro trabalho. Neste sentido, os exemplos a seguir operam arquivos do tipo texto, que consiste no armazenamento de uma sequência de caracteres gravados em uma mídia de massa em um formato conhecido como ASCII. O formato ASCII usado em um arquivo-texto representa a gravação de um dado com seu valor numérico referente à tabela ASCII que representa um dos 256 símbolos que um computador pode utilizar.

A escrita de dados num arquivo-texto é efetuada no final do arquivo, e sua leitura é executada sequencialmente a partir do início do arquivo. Este formato não é bom para o armazenamento de dados que requeiram atualizações constantes, como um arquivo para o cadastro de clientes de uma empresa. Nesse caso, a melhor alternativa é o uso de arquivo binário apresentado mais à frente.

Um arquivo-texto, para ser usado pelo computador, dá ao sistema operacional um grau de trabalho maior do que o uso de arquivo binário, pois todos os dados fornecidos para gravação precisam ser traduzidos para o formato ASCII para ser gravados no arquivo. No momento da leitura, esses dados precisam ser convertidos para sua forma binária para que o sistema operacional possa direcioná-los para o uso devido. Por exemplo, imagine gravar em um arquivo-texto os caracteres maiúsculos ABC, que serão armazenados com seus valores ASCII, sendo 065, 066 e 067 contiguamente, ou seja, 065066067. No momento da leitura, o sistema operacional terá de pegar cada grupo de valores ASCII, sendo 065, 066 e 067, e transformar cada grupo em seu respectivo valor binário para este ser usado. Um detalhe: se for para ver esses valores no monitor de vídeo, os valores serão lidos do arquivo, convertidos em binário, levados ao buffer do monitor de vídeo e convertidos novamente em formato ASCII para ser apresentados ao usuário do sistema, que, normalmente, não conhece nada sobre esses detalhes.

Os exemplos deste tópico demonstram o uso básico de acesso e manipulação de arquivos do tipo texto para a manipulação de caracteres

alfanuméricos na forma de palavras ou frases. Para facilitar a identificação desses arquivos será usada a extensão `DAD`, indicando arquivo do tipo texto. Vale salientar que para a linguagem Rust o uso de extensão no nome do arquivo é uma mera informação ilustrativa, podendo-se usar qualquer nome para identificação para uma extensão de arquivo. Os programas exemplos utilizam para tratamento do arquivo-texto alguns recursos que serão indicados em separado.

O programa seguinte cria, por meio do método `create()`, um arquivo de dados do tipo texto de nome `ARQUIV.DAD`, caso o arquivo não exista. Para isso, crie o projeto `c09ex41`.

```
#![allow(unused_variables, unused_assignments)]

use std::io;
use std::io::prelude::*;

use std::fs::File;
use std::fs;

fn main() {
    let arquivo;
    let acesso: &'static str = "../..../arquiv.dad";

    match fs::metadata(acesso) {
        Ok(_) => {
            println!("Arquivo existente.");
        },
        Err(_) => {
            arquivo = File::create(acesso).unwrap();
            println!("Arquivo criado com sucesso.");
        }
    }

    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}
```

Ao ser executado o programa, o arquivo `ARQUIV.DAD` poderá ser criado na pasta/diretório `estudo`, caso este não exista, e a mensagem informando a ação de criação do arquivo será apresentada. Caso o arquivo exista uma mensagem informando que o arquivo não pode ser criado é apresentada.

A instrução `use std::fs::File;` habilita para uso a biblioteca de manipulação de arquivos. Na sequência, a instrução `arquivo = File::create(acesso).unwrap();` faz a criação do arquivo dois níveis acima da pasta/diretório `src`. O método `create()` é usado para criar um arquivo colocando-o em modo de escrita, e toda vez que o método `create()` é utilizado, efetivamente, faz a criação do arquivo ao método indicado. Caso o arquivo exista, o método o recria apagando todo o seu conteúdo. Por essa razão, o programa faz uso do padrão `match` com a definição do método `metadata()`, que, ao fazer o acesso ao nome do arquivo, obtém um de dois valores para a variável usada, neste caso, `ACESSO`, podendo ser um valor verdadeiro para `Ok()` e, assim, apresentar a mensagem `Arquivo existente`, ou ser falso para `Err()` e proceder a criação do arquivo e a apresentação da mensagem `"Arquivo criado com sucesso"`. O método `metadata()` usa por critério a biblioteca `std::fs`. O controle de verificação da existência ou não de arquivo deve ser considerado com o intuito de evitar ocorrência de ações de pânico no programa, ocasionando a interrupção de sua execução. Após a criação de um arquivo, é possível fazer uso deste para escrita ou leitura de dados.

O programa seguinte efetua a leitura de uma linha de texto e grava o texto informado no arquivo `ARQUIV.DAD` caso o arquivo esteja em forma sequencial. Se o programa for executado mais de uma vez, fará o cadastramento de cada entrada uma após a outra. Para isso, crie o projeto `c09ex42`.

```
use std::io;
use std::io::prelude::*;

use std::fs;
use std::fs::OpenOptions;
```



```

fn main() {
    let mut arquivo;
    let acesso: &'static str = "../..../arquiv.dad";
    let mut conteudo = String::new();

    match fs::metadata(acesso) {
        Ok(_) => {
            arquivo = OpenOptions::new().write(true).append(true)
                .open(acesso).unwrap();
            println!("Informe uma palavra:");
            print!("--> ");
            io::stdout().flush().unwrap();
            io::stdin().read_line(&mut conteudo).expect("Entrada
incorreta");
            arquivo.write_all(conteudo.as_bytes()).unwrap();
        },
        Err(_) => {
            println!("Arquivo inexistente.");
        }
    };
    println!();
    print!("Tecle <Enter> para encerrar...");
    io::stdout().flush().unwrap();
    io::stdin().read(&mut [0u8]).unwrap();
}

```

Após executar o programa e informar a palavra ou a frase, abra o arquivo ARQUIV.DAD com um editor de texto e veja o conteúdo registrado. Na sequência, execute o programa mais algumas vezes e volte a abrir o arquivo. Repare que as entradas efetuadas no arquivo são inseridas uma após a outra, gerando uma lista de entradas.

A gravação das palavras ou das frases no arquivo ARQUIV01.DAD é efetivada pelo método `write_all()`, que recebe um parâmetro como conteúdo de escrita definido no formato byte, que, neste caso, é realizado pelo método `as_byte()`.

Para poder fazer a ação de acréscimo de dados no arquivo, o programa faz uso dos métodos `append()`, além do auxílio do método `write()`, que

mantém o arquivo em operação de escrita, compatibilizando esta ação com o método `open()`, que abre o arquivo para escrita. Os métodos `append()` e `write()` são postos em operação com o valor de parâmetro `true` para definir suas ações a partir da biblioteca `std::fs::OpenOptions`.

A seguir, é exibido o programa que vai ler o conteúdo de um arquivo `ARQUIV.DAD`, apresentando no vídeo as palavras ou frases que foram gravadas pelo programa anterior. Para isso, crie o projeto `c09ex43`.

```
#![allow(unused_must_use)]

use std::io;
use std::io::prelude::*;

use std::fs::File;
use std::fs;

fn main() {
    let mut arquivo;
    let acesso: &'static str = "../..../arquiv.dad";
    let mut conteudo = String::new();

    match fs::metadata(acesso) {
        Ok(_) => {
            arquivo = File::open(acesso).unwrap();
            arquivo.read_to_string(&mut conteudo);
            if let Some('\n') = conteudo.chars().next_back() {
                conteudo.pop();
            }
            if let Some('\r') = conteudo.chars().next_back() {
                conteudo.pop();
            }
            println!("{}", conteudo);
        },
        Err(_) => {
            println!("Arquivo inexistente.");
        }
    };
}
```

```
println!();  
print!("Tecle <Enter> para encerrar...");  
io::stdout().flush().unwrap();  
io::stdin().read(&mut [0u8]).unwrap();  
}
```

Ao ser executado o programa, a lista de palavras cadastradas no programa do projeto anterior é apresentada. Neste programa a instrução `arquivo.read_to_string(&mut palavra);` efetua a leitura do arquivo indicado junto à instrução `let mut arquivo = File::open("../..../arqtxt.tex").unwrap();`, que faz uso do método `open()` definido para ações de leitura em arquivo.

Referências bibliográficas

ÁLVARES, A. R. *Linguagens de Programação: Programação Funcional (Haskell)*. Belo Horizonte: CEFET-MG. Disponível em: <<http://homepages.dcc.ufmg.br/~rimsa/documents/decom009/lessons/Aula10.pdf>>.

BACKES, A. R. *Linguagem C: Completa e Descomplicada*. Rio de Janeiro: Campus, 2014.

BALBAERT, I. *Rust Essentials*. Birmingham: Packt Publishing, 2015.

BLANDY, J, & ORENDORFF, J. *Programming Rust: Fast, Safe Systems Development* (Early Release, Raw & Unedited). Sebastopol: O'Reilly Media Inc., 2016.

CAPUANO, G. *Errors and Exceptions (in Rust)*. Developer Careers, Honeypot GmbH. Disponível em: <<http://blog.honeypot.io/errors-and-exceptions-in-rust/>>.

DAN_ATILIO. *Conhecendo a Linguagem de Programação Rust*. 2015. Disponível em: <<https://terminaldeinformacao.com/2015/05/28/conhecendo-a-linguagem-de-programacao-rust/>>.

DHINAKARAN, V. *Rust Cookbook*. Birmingham: Packt Publishing, 2017.

GOMES, D. *Conheça a Rust, a Nova Linguagem de Programação da Mozilla*. 2010. Disponível em: <<http://webholic.com.br/mozilla-esta-desenvolvendo-uma-nova-linguagem-de-programacao-a-rust/>>.

GOMEZ, G. *Tutorial Rust*. Disponível em: <<https://blog.guillaume-gomez.fr/Rust/>>.

GIL, P. C. *Introducción a la Criptografía*. 2. ed. Madrid: Ra-Ma, 2002.

KAYHLAVIRTA, V. *Mastering Rust*. Birmingham: Packt Publishing, 2017.

LEIRADELL, A. de. *Introdução à criptografia: Suplemento - Revista Conecta*, São Paulo: Block, nº 21, abr. 1998.

MAFRA, D. *Rust, a linguagem do futuro*. 2014. Disponível em: <<http://www.diogomafra.com.br/2014/01/rust-linguagem-do-futuro.html>>.

MOZ://A. *Rust language*. 2017. Disponível em: <<https://research.mozilla.org/rust/>>.

PRESSMAN, R. S. *Engenharia de Software*. São Paulo: Makron, 1995.

RADTKE III, H. J. *Effectively Using Iterators In Rust*. Disponível em: <<http://hermanradtke.com/2015/06/22/effectively-using-iterators-in-rust.html>>.

RAVICHANDRAN, D. *Programming in C*. New Deli: New Age International (p) Ltd., Publishers, 1996.

RUNGE, F, *Bachelorarbeit: Die Programmiersprache Rust - Einsatz und modern entwurfsmuster in der systemnahen programmierung dargestellt und auf einer plaform zentralisiert*. Hamburg: Fakultät Technik und Informatik, 2016.

RUST. *A Linguagem de Programação Rust*. 2017. Disponível em: <<https://www.rust-lang.org/pt-BR/>>.

SEBESTA, R. W. *Conceitos de Linguagens de Programação*, Rio Grande do Sul: Record, 2003.

SINGH, S. *O livro dos códigos: A ciência do sigilo, do antigo Egito à criptografia quântica*. São Paulo: Record, 2011.

TAVARES, B. *Rust, uma linguagem interessante*. 2017. Disponível em: <<https://presentations.bltavares.com/tdc-introducao-ao-rust/>>.

THE RUST TEAM. *The Rust Programming Language*. California: The Rust Team / Mozilla Corporation, 2015.

VICKI, V. *Arte e Cultura: A cifra de Vigenère*. 2006. Disponível em: <http://www.numaboa.com.br/index.php?option=com_content&view=article&id=506&Itemid=134>.

_____. *O Código de César*. 2008. Disponível em: <<http://www.numaboa.com.br/criptografia/124-substituicao-simples/165-codigo-de-cesar>>.

WIRTH, W. *Algoritmos e Estruturas de Dados*. Rio de Janeiro: Prentice-Hall

Brasil, 1989.